

计算理论-预备知识

- 基本数学对象
- 定义, 定理, 证明
- 常用证明方法

1 基本数学对象

基本数学对象-集合

定义(集合). 把具有共同性质的对象汇集成一个整体, 称为一个集合. 构成集合的对象称为集合的元素.

说明.

- 通常用大写字母 A, B, C 等表示集合, 用小写字母 a, b, c 等表示元素.
- 给定集合 A 以及对象 a . 若 a 在 A 中, 则称 a 属于 A , 用 $a \in A$ 表示. 否则称 a 不属于 A , 用 $a \notin A$ 表示.

基本数学对象-集合

定义(集合的大小). 给定集合 A , 用 $|A|$ 表示 A 的大小, 即 A 中元素的个数. 若 $|A|$ 为有限数, 则称 A 是有限集. 否则称 A 是无限集.

例 1: 大小为0的集合称为空集, 用 $\emptyset$ 表示.

例 2: 几个特殊的无限集.

$$\mathbb{N} = \{1, 2, 3, \dots\}$$

自然数集

$$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$$

整数集

$$\mathbb{Q} = \left\{ \frac{q}{p} \mid p, q \in \mathbb{Z}, p \neq 0 \right\}$$

有理数集

$$\mathbb{R}$$

实数集

$$\mathbb{C} = \{a + bi \mid a, b \in \mathbb{R}\}$$

复数集

基本数学对象-集合

定义(集合的包含关系). 设有集合 A, B .

- 若 A 的任何元素都是 B 的元素, 则称 A 包含于 B , 记作 $A \subseteq B$.
- 若 $A \subseteq B$ 且存在 $b \in B$ 但 $b \notin A$, 则称 A 真包含于 B , 记作 $A \subset B$.
- 若 $A \subseteq B$ 且 $B \subseteq A$, 则称 A 与 B 相等, 记作 $A = B$.

集合的运算: 交, 差, 并, 补.

- $A \cap B = \{x | x \in A \text{ 且 } x \in B\}$.
- $A \cup B = \{x | x \in A \text{ 或 } x \in B\}$.
- $A \setminus B = \{x | x \in A \text{ 且 } x \notin B\}$.
- $\bar{A} = \{x | x \notin A\}$ (相对于所考虑的对象全体).

基本数学对象-有序组

定义(有序组). 将 k 个对象按固定次序排列并用 $()$ 加以界定称为一个 k -元有序组. 2-元有序组通常称为有序对.

例 1: $(1, 3, 5, 7, 9)$ 是一个5-元有序组.

例 2: 平面上任何点的坐标 (x, y) 是一个有序对.

一个常用的构造有序组的方法是利用集合的笛卡尔积.

定义(集合的笛卡尔积). 给定集合 A, B , A 与 B 的笛卡尔积为

$$A \times B = \{(a, b) | a \in A, b \in B\}.$$

$$|A \times B| = ?$$

说明. k 个集合的笛卡尔积 $A_1 \times A_2 \times \cdots \times A_k$ 可以类似定义.

基本数学对象-映射

定义(映射). 给定非空集合 A 与 B , 若对 A 中任意的元素 a 都有 B 中唯一的元素 b 与之对应, 则称这种对应规则为 A 到 B 的一个**映射**, 记作 $f: A \rightarrow B$. 元素 b 称为 a 在 f 下的**像**, 记作 $b = f(a)$. 元素 a 称为 b 在 f 下的**原像**.

例 1: 学生 $\rightarrow$ 学号



例 2: 学生 $\rightarrow$ 课程



基本数学对象-映射

定义(单射, 满射, 双射). 设 $f: A \rightarrow B$ 是一个映射.

- 若任何 $a_1, a_2 \in A$ 且 $a_1 \neq a_2$, 都有 $f(a_1) \neq f(a_2)$, 则称 f 为单射.
(不同的元素有不同的像)
- 若任给 $b \in B$, 都存在 $a \in A$ 使得 $b = f(a)$, 则称 f 为满射.
- 若 f 既是单射又是满射, 则称 f 为双射(一一对应).

例 1: $f(x) = x$ 的父亲 既非单射又非满射

例 2: 设 S 为集合. 任给 $A \subseteq S$, $f(A) = \bar{A}$ 是一个 S 的幂集 $\mathcal{P}(S)$ 到自身的双射.

基本数学对象-映射

练习.

$$f: \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$$

$$x \mapsto x^2$$

是否为单射？ 满射？

$$f: \mathbb{R} \rightarrow \mathbb{R}$$

$$x \mapsto x^2$$

是否为单射？ 满射？

基本数学对象-映射

定理(映射的性质). 设 A 与 B 为有限集合且 $f: A \rightarrow B$ 是一个映射. 则

- 若 f 为单射, 则 $|A| \leq |B|$.
- 若 f 为满射, 则 $|A| \geq |B|$.
- 若 f 为双射, 则 $|A| = |B|$.

基本数学对象-关系

定义(关系). 设 A 是有限集合. 若 R 是一个 k -元有序组的集合, 其中 R 的每个 k -元有序组的分量都是 A 的元素, 即 $R \subseteq A^k$, 则称 R 是 A 上的一个 k -元关系.

若 $k = 2$, 则 R 称为二元关系.

例: “朋友”关系是一个定义在所有人的集合上的二元关系.

基本数学对象-关系

定义(等价关系). 二元关系 R 是一个等价关系, 如果它满足

- 自反性: $(x, x) \in R, \forall x \in A$.
- 对称性: 若 $(x, y) \in R$, 则 $(y, x) \in R$.
- 传递性: 若 $(x, y), (y, z) \in R$, 则 $(x, z) \in R$.

例 1: 朋友关系. 

例 2: 定义 $\mathbb{N}$ 上的二元关系 $\sim$: $i \sim j$ 当且仅当 $7|j - i$. 

基本数学对象-关系

定义(等价类). 设 R 是集合 A 上的等价关系. 任给 $a \in A$, 称

$$C(a) = \{b \in A \mid (a, b) \in R\}$$

为 a 所在的等价类.

性质. 任给 $a, b \in A$, $C(a) \cap C(b) = \emptyset$ 或 $C(a) = C(b)$.

意义. 集合 A 是等价类的不交并.

例: $\mathbb{N}$ 上的二元关系 $\sim$: $i \sim j$ 当且仅当 $7 \mid j - i$.

$$C(i) = \{7k + i \mid k = 0, 1, \dots\}, \quad i = 1, 2, \dots, 7.$$



星期 i

基本数学对象-关系

练习.

“相同的性别”是一个等价关系？若是给出其等价类；若不是请说明理由.

基本数学对象-字符串和语言

定义(字母表). 任何非空有限集合称为一个字母表. 字母表中的元素称为字符.

说明. 字母表通常用 Γ, Σ 等大写希腊字母表示.

定义(字符串). 字母表 Σ 上的字符组成的有限序列 s 称为 Σ 上的字符串. 用 $|s|$ 表示 s 的长度, 即 s 中所包含字符的个数.

例: 长度为0的字符串称为空字符串, 用 ε 表示.

定义(语言). 字母表 Σ 上的若干字符串组成的集合称为语言.

例: $\Sigma = \{0,1\}$. 则 $L = \{00,01,10,11\}$ 是 Σ 上的一个语言.

基本数学对象-布尔代数

定义(布尔变量). 取值要么是0要么是1的变量称为布尔变量.

定义(布尔变量的运算). 给定布尔变量 x, y ,

- $x \wedge y$ 表示 x 和 y 的合取.
- $x \vee y$ 表示 x 和 y 的析取.
- $\neg x$ 表示 x 的否定.

运算律.

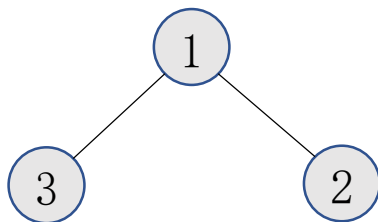
- $x \wedge y = 1 \Leftrightarrow x = 1 \text{ 且 } y = 1.$
- $x \vee y = 1 \Leftrightarrow x = 1 \text{ 或 } y = 1.$
- $\neg x = 1 \Leftrightarrow x = 0.$

基本数学对象-图

定义(无向图). 一个无向图 G 是一个二元组 $(V(G), E(G))$,

- $V(G)$ 称为 G 的顶点集.
- $E(G)$ 是若干 $V(G)$ 的二元子集组成的集合, 称为 G 的边集.

例: $G = (\{1,2,3\}, \{\{1,2\}, \{1,3\}\})$ 是一个图.



说明. 连接顶点之间的线的形状可以是任意的.

基本数学对象-图

定义(度). 给定无向图 G . 对于 $v \in V(G)$, 与 v 关联的边的数目称为 v 的度, 记作 $d_G(v)$.

握手引理. 给定无向图 $G = (V(G), E(G))$,

$$\sum_{v \in V(G)} d_G(v) = 2|E(G)|.$$

证明. 每条边 $\{x, y\} \in E(G)$ 在左边求和式中被计算了两次. ■

说明. 这是一个组合学上常用的证明技巧: double counting.

基本数学对象-图

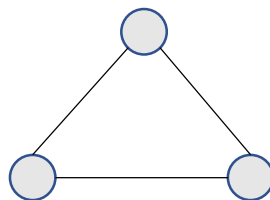
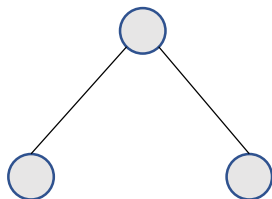
定义(子图). 给定图 $G = (V(G), E(G))$, $H = (V(H), E(H))$. 若 $V(H) \subseteq V(G), E(H) \subseteq E(G)$, 则称 H 是 G 的子图.

定义(迹, 闭迹, 路, 圈). 给定图 $G = (V(G), E(G))$,

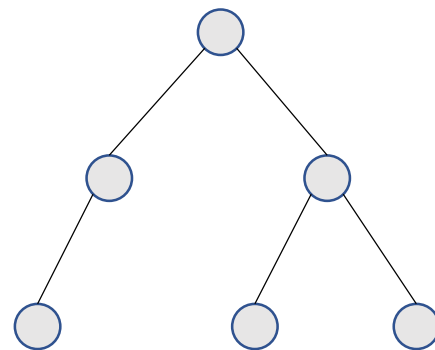
- 顶点序列 $W = v_0 v_1 \cdots v_k$ 是 G 的迹, 如果 $v_i v_{i+1} \in E(G), i = 0, 1, \dots, k - 1$. 数 k 称为 W 的长度.
- 若迹 $W = v_0 v_1 \cdots v_k$ 满足 $v_0 = v_k$, 则称 W 为闭迹.
- 顶点不重复的迹称为路.
- 除起点和终点外顶点不重复的闭迹称为圈.

基本数学对象-图

定义(连通性). 给定图 $G = (V(G), E(G))$, 若 G 的任何两点之间都存在一条路, 则称 G 是连通的.



定义(树). 不含圈的连通图称为树.



基本数学对象-图

定义(树). 不含圈的连通图称为树.

定理. 令 T 是一棵树, 则 $|E(T)| = |V(T)| - 1$. ■

定理(树的等价定义). 令 G 是一个 n 点图, 则下面的命题等价.

- G 是一棵树.
- G 是连通的且有 $n - 1$ 条边.
- G 是无圈的且有 $n - 1$ 条边. ■

2 定义, 定理, 证明

定义，定理，证明

定义：描述研究的对象和概念的陈述.

若整数 $p \geq 2$ 且没有非平凡的因子，则称 p 为素数.

证明：逻辑论证.

定理：被证明为真的数学命题.

欧几里得：素数有无穷多.

3 常用证明方法

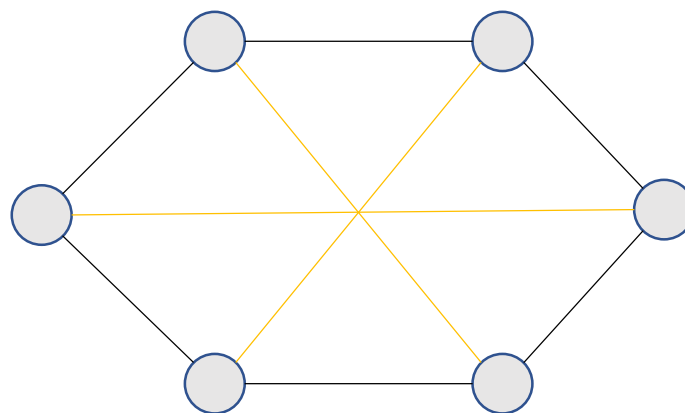
常用证明方法-构造法

定义(正则图). 若图 G 的每个顶点度为 k , 则称 G 为 k -正则图.

定理. 若 $n > 2$ 为偶数, 则存在 n 个顶点的3-正则图.

证明. 构造 n 个顶点的3-正则图 G_n 如下:

- $V(G_n) = \{1, 2, \dots, n\}$.
- 将 $1, 2, \dots, n$ 排成一个圆圈, 连接相邻两个顶点.
- 再将相对的顶点连接.



显然, G_n 是3-正则图.

常用证明方法-反证法

定理(欧几里得). 素数有无穷多.

证明. 假设素数只有有限多个, 令 $p_1, p_2, \dots, p_n$ 是所有的素数.

$$N = p_1 p_2 \cdots p_n + 1.$$

- 由**算术基本定理**, N 有一个素因子 p_i .
- 则 $p_i | N$ 且 $p_i | p_1 p_2 \cdots p_n$.
- 从而 $p_i | 1$.

这是一个矛盾.



常用证明方法-反证法

练习. 用反证法证明 $\sqrt{2}$ 不是有理数.

常用证明方法-归纳法

定理. $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$.

证明.

- 基本情况. 当 $n = 1$ 时等式显然成立.
- 归纳步. 假设 $n \geq 2$ 且等式对 $n - 1$ 成立. 则

$$1 + 2 + \cdots + n = 1 + 2 + \cdots + n - 1 + n$$

$$= \frac{n(n-1)}{2} + n$$

$$= \frac{n(n+1)}{2}.$$



常用证明方法-归纳法

定理. 令 T 是一棵树, 则 $|E(T)| = |V(T)| - 1$.

证明. 当 $n = 1$ 时定理显然成立.

假设 $|V(T)| \geq 2$ 且定理对 $|V(T)| - 1$ 个顶点的树都成立.

- 因 T 无圈, T 有一个1度顶点 v (why?).
- 则 $T' = T - v$ 是一棵树.
- 由归纳假设,

$$|E(T')| = |V(T')| - 1.$$

- 从而 $|E(T)| = |V(T)| - 1$. ■

常用证明方法-归纳法

说明. 归纳法的优点是适用范围广, 缺点是对发现定理帮助有限.

$$1^4 + 2^4 + \cdots + n^4 = ?$$

预备知识总结

预备知识总结

- 基本数学对象
 - 集合
 - 有序组
 - 函数
 - 关系
 - 字符串和语言
 - 布尔代数
 - 图
- 常用证明方法
 - 构造法
 - 反证法
 - 归纳法

练习

1. 写出字母表 Σ 上不含任何字符串的语言以及只含空字符串的语言.
2. 给定图 $G = (V(G), E(G))$, 定义关系 $R = \{(u, v) | \text{存在从 } u \text{ 到 } v \text{ 的路}\}$. 证明 R 是等价关系. 这个等价关系的等价类是什么?
3. 找出下列证明中的错误.

命题: 任何 h 匹马都有相同的颜色.

“证明”: 对 h 用归纳法. 显然命题对 $h = 1$ 成立.

现假设 $h \geq 2$ 且命题对 $h - 1$ 成立. 令 H 是有 h 匹马的集合.

- 选择 $a \in H$, 则 $H \setminus \{a\}$ 是一个有 $h - 1$ 匹马的集合. 由归纳假设, $H \setminus \{a\}$ 中的马颜色相同.
- 选择 $b \in H$ 且 $b \neq a$, 则 $H \setminus \{b\}$ 是一个有 $h - 1$ 匹马的集合. 由归纳假设, $H \setminus \{b\}$ 中的马颜色相同.

从而 H 中的马颜色相同. ■

4. 若 G 是无圈图, 则称 G 是一个森林. 给出一个森林边数和点数的关系.

计算理论-有限自动机

- 有限自动机
- 非确定性有限自动机
- 正则语言
- 非正则语言

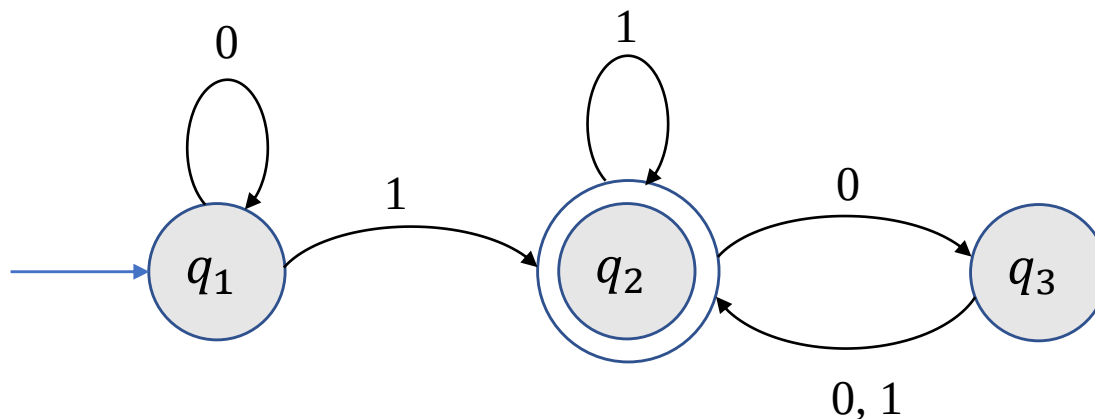
1 有限自动机

有限自动机

自动门控制

- 开和关两个状态
- 事件触发状态的改变

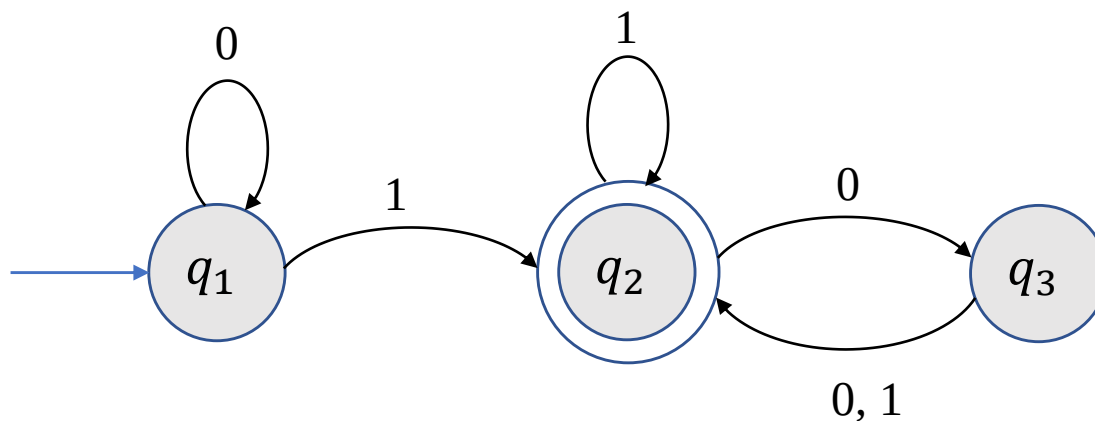
有限自动机-引例



一个有限自动机 M_1

- 圆圈代表状态
 - 蓝色箭头指向的状态称为初始状态
 - 两个圆圈代表接受状态
- 黑色箭头代表转移规则

有限自动机-引例



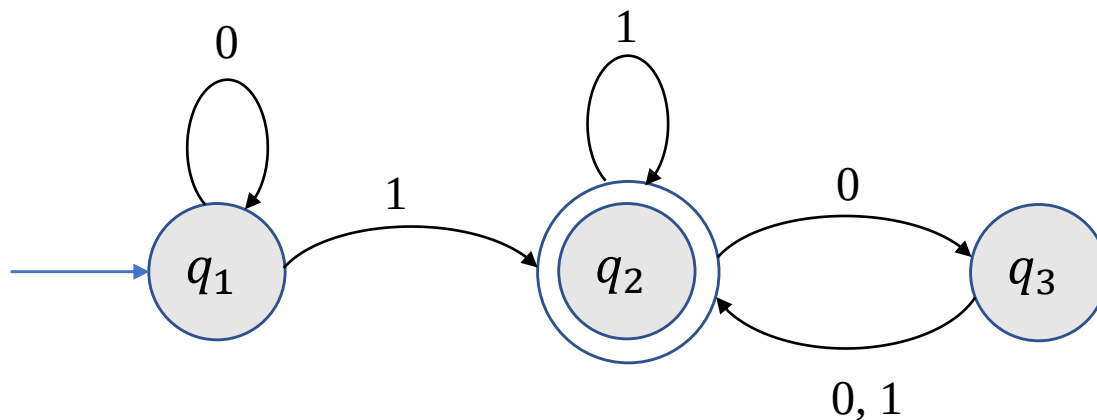
输入: $\{0,1\}$ -字符串.

输出: 接受或拒绝.

计算

- 从左向右依次读取输入字符串中的字符, 根据转移规则从一个状态跳转到下一个状态.
- 若终止状态为接受状态, 输出接受; 否则输出拒绝.

有限自动机-引例



计算

- 从左向右依次读取输入字符串中的字符, 根据转移规则从一个状态跳转到下一个状态.
- 若终止状态为接受状态, 输出接受; 否则输出拒绝.

例: $w = 1101$.

$$q_1 \xrightarrow{1} q_2 \xrightarrow{1} q_2 \xrightarrow{0} q_3 \xrightarrow{1} q_2$$

M_1 接受 w .

M_1 接受 10100?

有限自动机-正式定义

定义(有限自动机). 一个有限自动机是一个5-元有序组 $(Q, \Sigma, \delta, q_0, F)$, 其中

- Q 是一个有限集, 称为状态集.
- Σ 是一个字母表.
- $\delta: Q \times \Sigma \rightarrow Q$ 是转移函数.
- $q_0 \in Q$ 称为初始状态.
- $F \subseteq Q$ 是接受状态集.

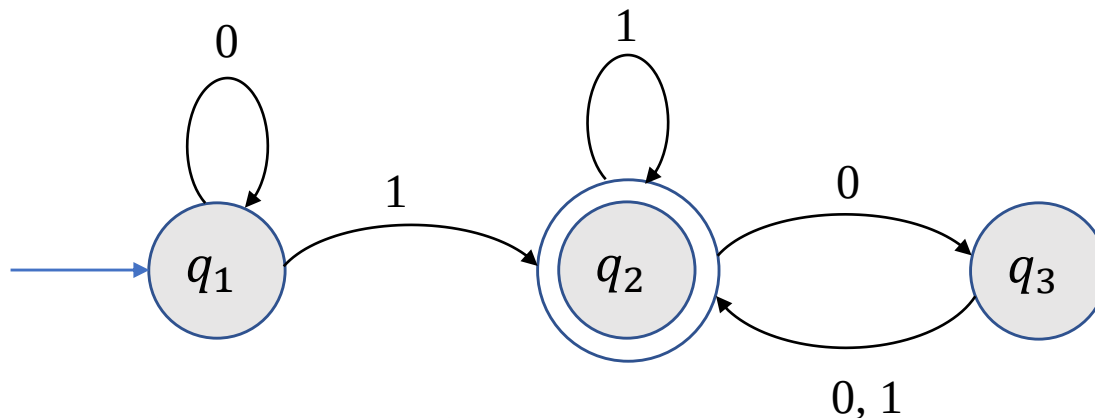
定义(有限自动机的语言). 给定有限自动机 M , 所有 M 接受的字符串的集合 $L(M)$ 称为 M 的语言, 即

$$L(M) = \{w | M \text{ 接受 } w\}.$$

此时也称 M 识别 $L(M)$.

有限自动机-正式定义

例 1.



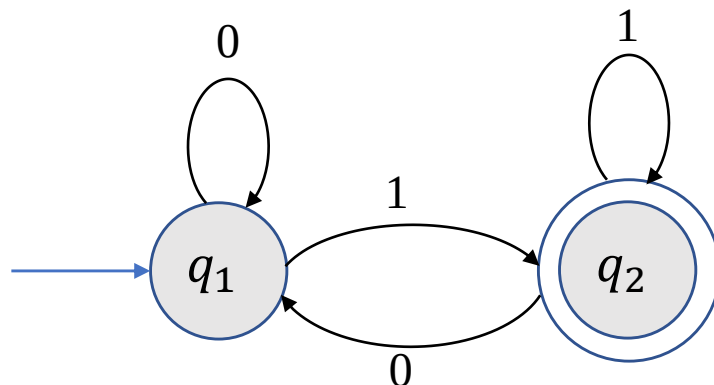
$$M_1 = (Q, \Sigma, \delta, q_1, F)$$

- $Q = \{q_1, q_2, q_3\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :
- q_1 是初始状态.
- 接受状态集 $F = \{q_2\}$.

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

有限自动机-正式定义

例 2.



$$M_2 = (Q, \Sigma, \delta, q_1, \{q_2\})$$

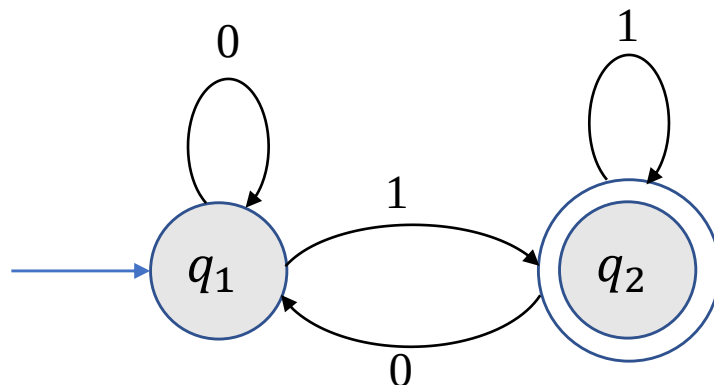
- $Q = \{q_1, q_2\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :

	0	1
q_1	q_1	q_2
q_2	q_1	q_2

$$L(M_2) = ?$$

有限自动机-正式定义

例 2.



$$M_2 = (Q, \Sigma, \delta, q_1, \{q_2\})$$

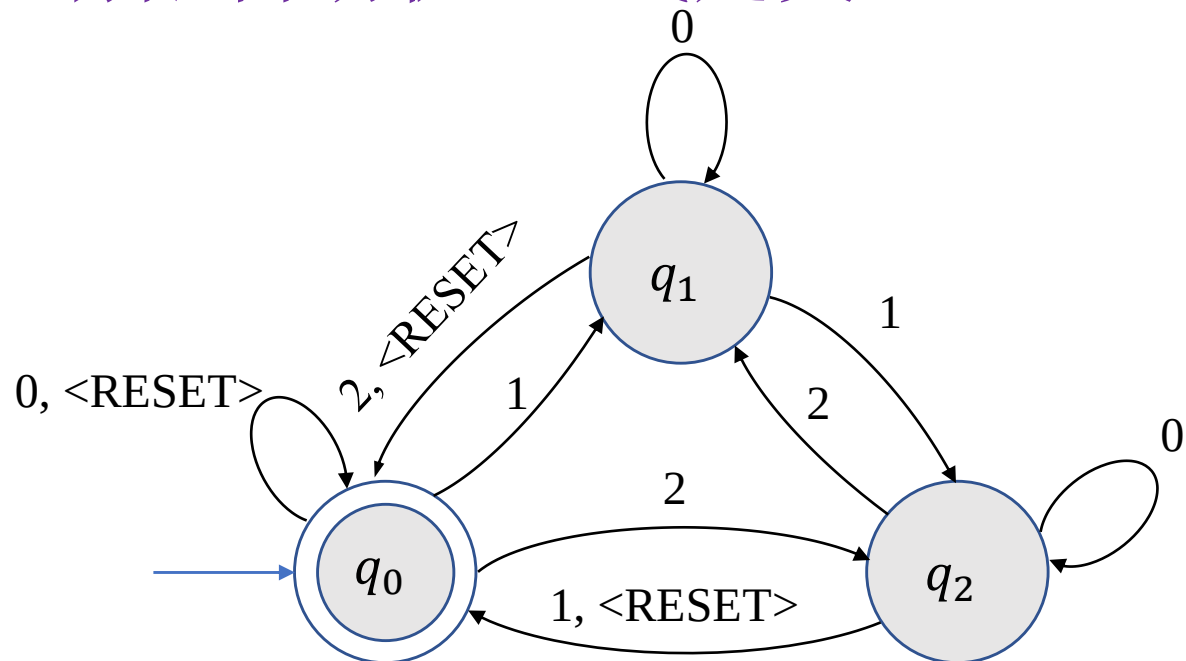
- $Q = \{q_1, q_2\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :

	0	1
q_1	q_1	q_2
q_2	q_1	q_2

$$L(M_2) = \{w \mid w \text{ 以 } 1 \text{ 结尾}\}.$$

有限自动机-正式定义

例 3.



$$M_3 = (\{q_0, q_1, q_2\}, \Sigma, \delta, q_0, \{q_0\})$$

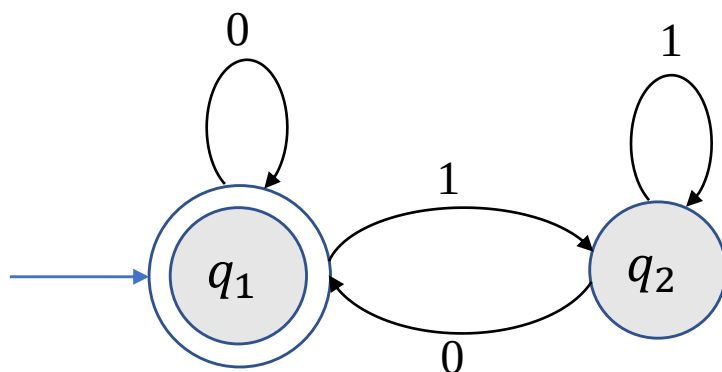
- $\Sigma = \{0, 1, 2, < \text{RESET} >\}$.
- δ : 记录输入字符串数字之和模3后的值.

一旦遇到< RESET > 字符将数值归0.

$L(M_3) = \{w | w \text{中最后一个 } < \text{RESET} > \text{ 字符后所有数字之和被3整除}\}.$

有限自动机-正式定义

练习. 给出下列有限自动机的正式定义并确定其识别的语言.



有限自动机-正式定义

问题：为什么需要形式化的定义？

- 精确描述：给定5-元有序组可以通过定义判定哪些是有限自动机.
- 有时有限自动机的状态图过于庞大，形式化定义更加简洁.
- 有限自动机可以和某个参数相关，每一个参数的取值对应一个有限自动机，因此无法画出无穷多个状态图.

有限自动机-正式定义

例：给定 $\Sigma = \{0,1,2, < \text{RESET} >\}$.

$L_i = \{w | w \text{中最后一个 } < \text{RESET} > \text{ 字符后所有数字之和被 } i \text{ 整除}\}$.

给出一个有限自动机 M_i 识别 L_i .

解： $M_i = (\{q_0, q_1, \dots, q_{i-1}\}, \Sigma, \delta_i, q_0, \{q_0\})$, 其中 δ_i 满足如果 M_i 处于状态 q_j , 则已读数字之和模 i 等于 j .

- $\delta_i(q_j, < \text{RESET} >) = q_0$.
- $\delta_i(q_j, 0) = q_j$.
- $\delta_i(q_j, 1) = q_k$, 其中 $k \equiv j + 1 \pmod{i}$.
- $\delta_i(q_j, 2) = q_k$, 其中 $k \equiv j + 2 \pmod{i}$.

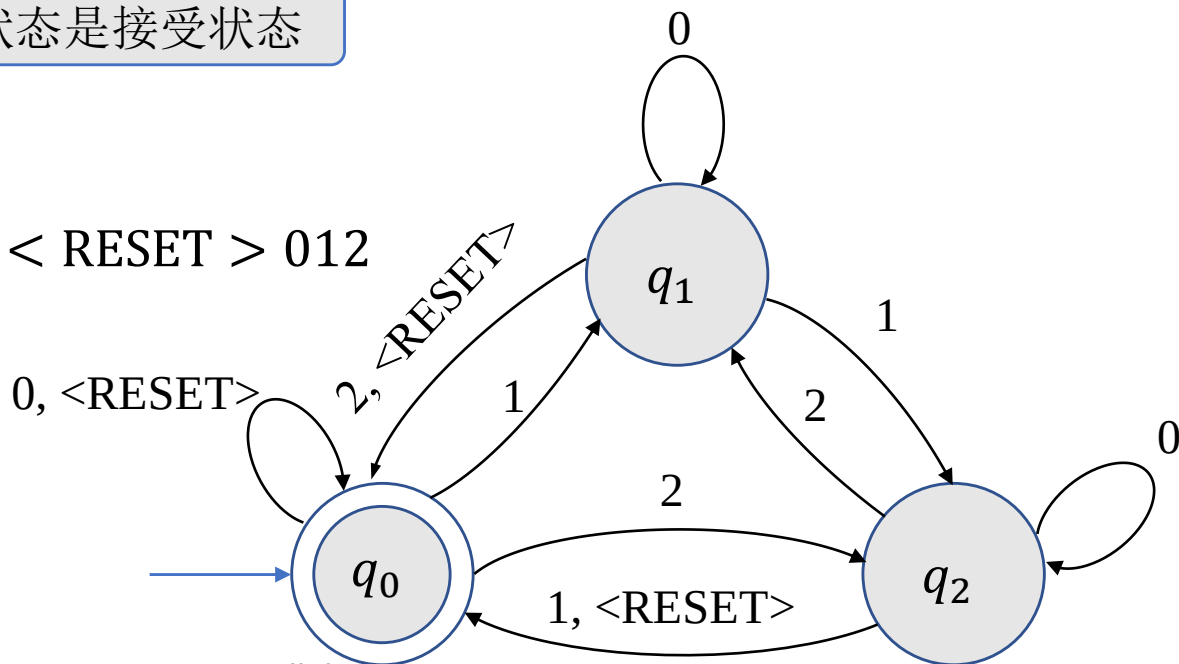
有限自动机-计算的定义

定义(计算). 设 $M = (Q, \Sigma, \delta, q_0, F)$ 是一个有限自动机, $w = w_1w_2 \cdots w_n$ 是 Σ 上的字符串. 如果存在状态序列 $r_0, r_1, \dots, r_n \in Q$ 使得

- $r_0 = q_0$; 从初始状态开始
- $\delta(r_i, w_{i+1}) = r_{i+1}, i = 0, 1, \dots, n-1$; 状态根据转移函数跳转
- $r_n \in F$, 终止状态是接受状态

则称 M 接受 w .

例. 给出 M_3 计算 $w = 10 \langle \text{RESET} \rangle 012$ 时进入的状态序列.



有限自动机-设计

问题：如何设计识别给定语言 L 的有限自动机？

回答：将自己想象成有限自动机.

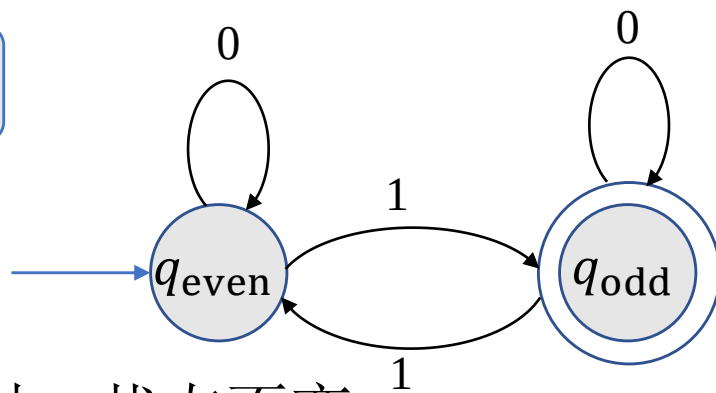
有限自动机-设计

例 1：设计识别 $\Sigma = \{0,1\}$ 上的语言 $L = \{w|w \text{ 包含奇数个} 1\}$ 的有限自动机 E_1 .

解：当读取输入字符串时，需要记录什么信息？

不是1的数目，而是1的数目的奇偶性！

- 因此 E_1 有两个状态 q_{odd} 和 q_{even} .
- 当读到1时，状态翻转；当读到0时，状态不变.
- 接受状态为 q_{odd} ，初始状态为 q_{even} (对应空字符串).

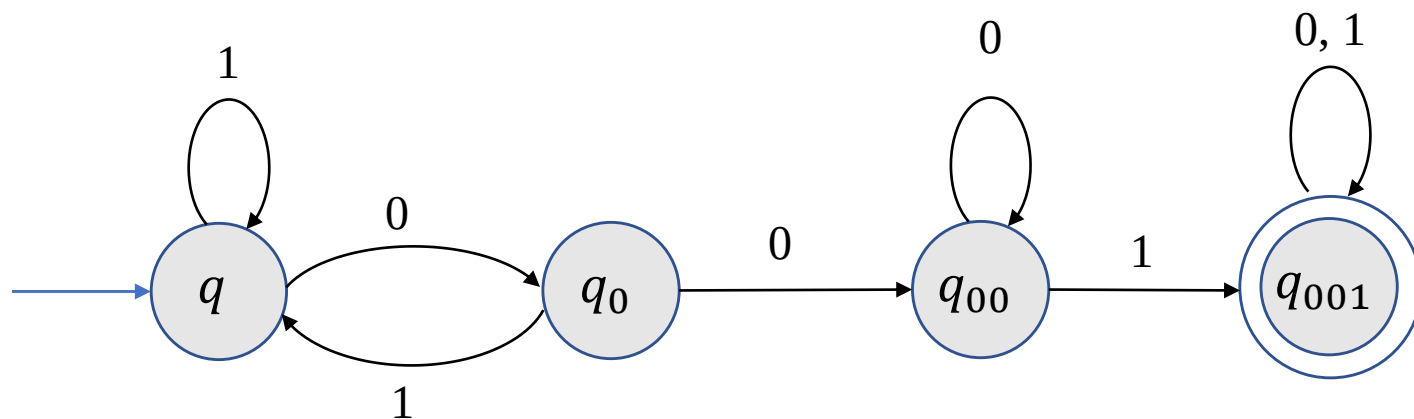


有限自动机-设计

例 2: 设计识别 $\Sigma = \{0,1\}$ 上的语言

$$L = \{w \mid w \text{ 包含 } 001 \text{ 作为子串}\}$$

的有限自动机 E_2 .



有限自动机-正式定义

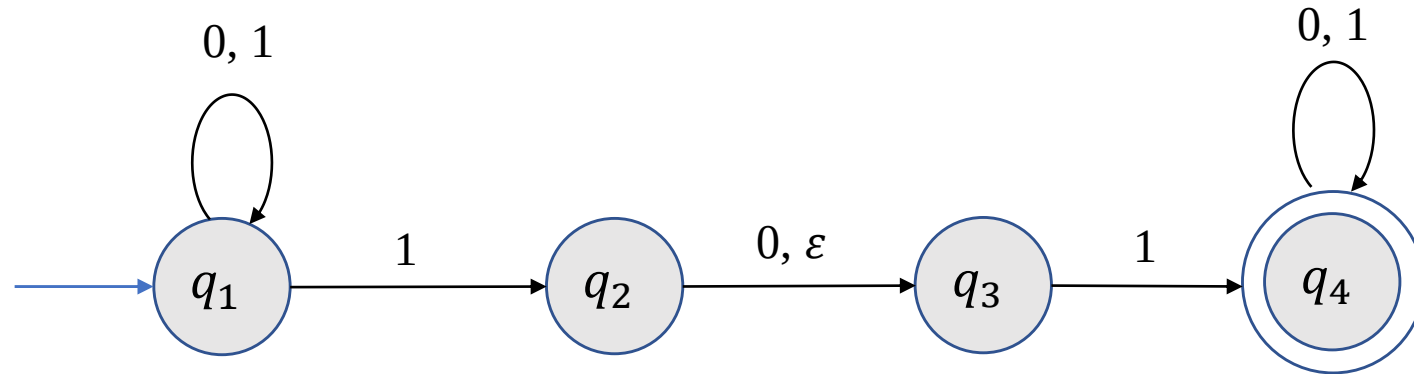
练习. 设计识别 $\Sigma = \{0,1\}$ 上的语言

$$L = \{w | w \text{ 包含至少三个 } 1\}$$

的有限自动机 E_3 .

2 非确定性有限自动机

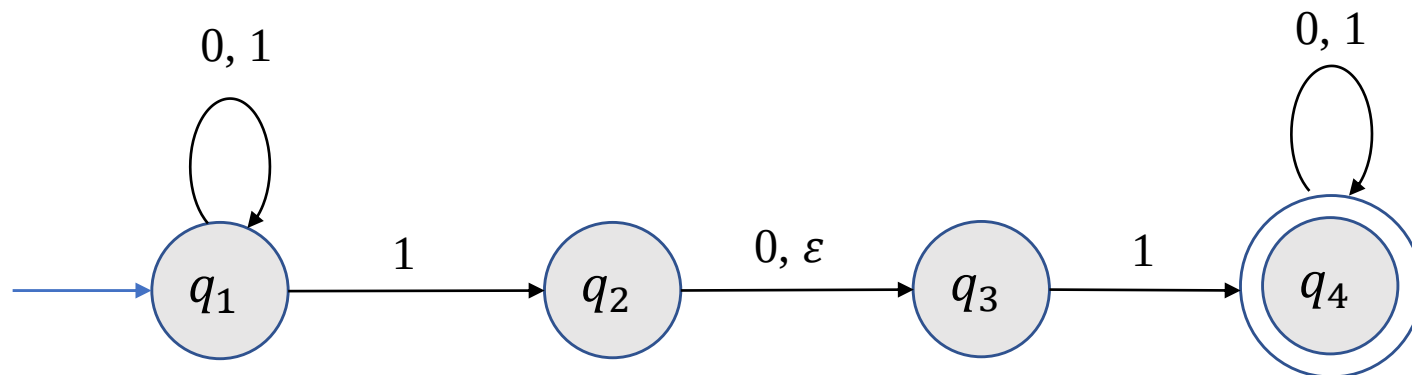
非确定性有限自动机-引例



非确定性的特点.

- 转移规则：一入多出.
- ϵ 箭头：无需读入字符的状态转移.

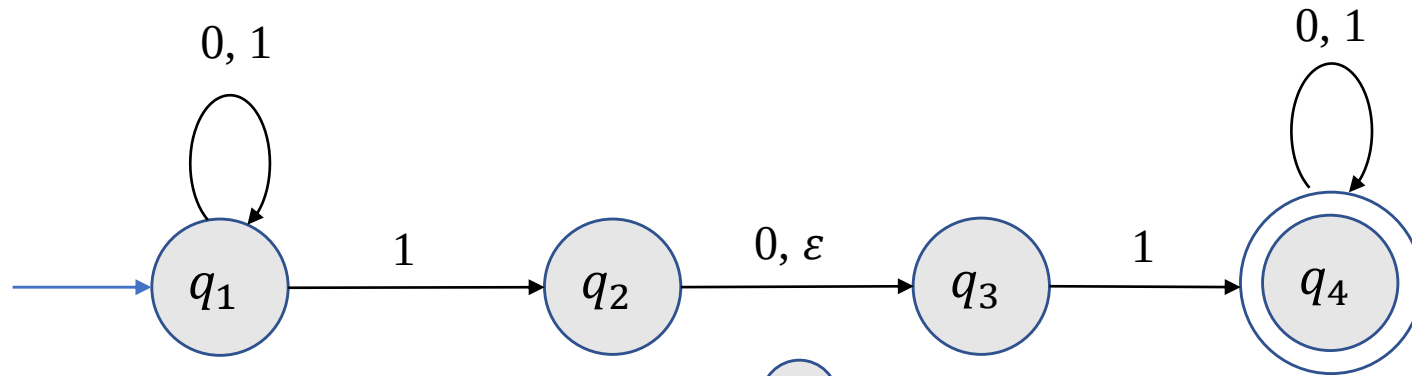
非确定性有限自动机-引例



计算.

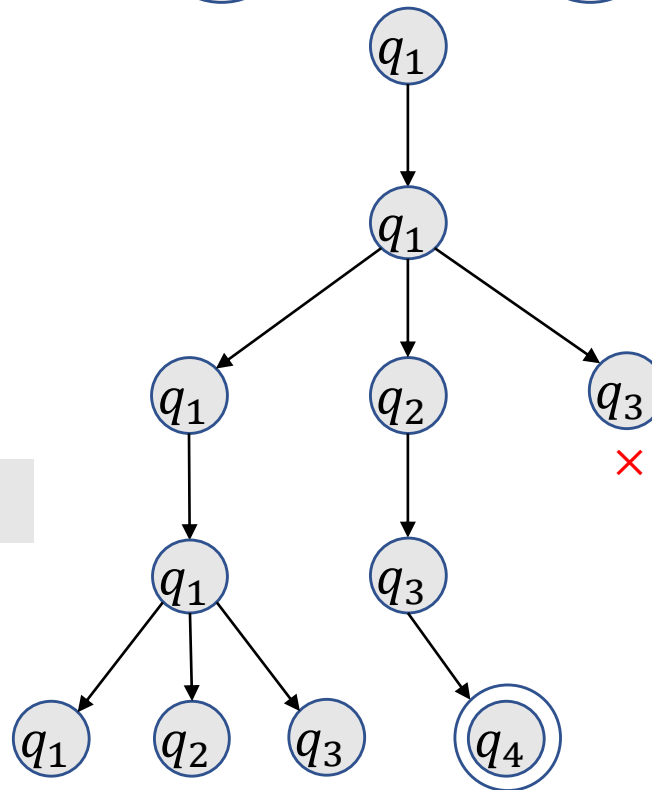
- 如果机器处于给定状态读入给定字符有多个出去的箭头，则机器复制成多份，每一份对应一个可能的状态.
- 如果机器处于给定状态且出去的箭头上有 ϵ 符号，则机器复制一份，其状态为箭头所指状态.
- 如果有一个计算分支接受，则机器接受.

非确定性有限自动机-引例



例: $w = 0101$.

N_1 接受 1001?



----- 0

----- 1

----- 0

----- 1

非确定性有限自动机-正式定义

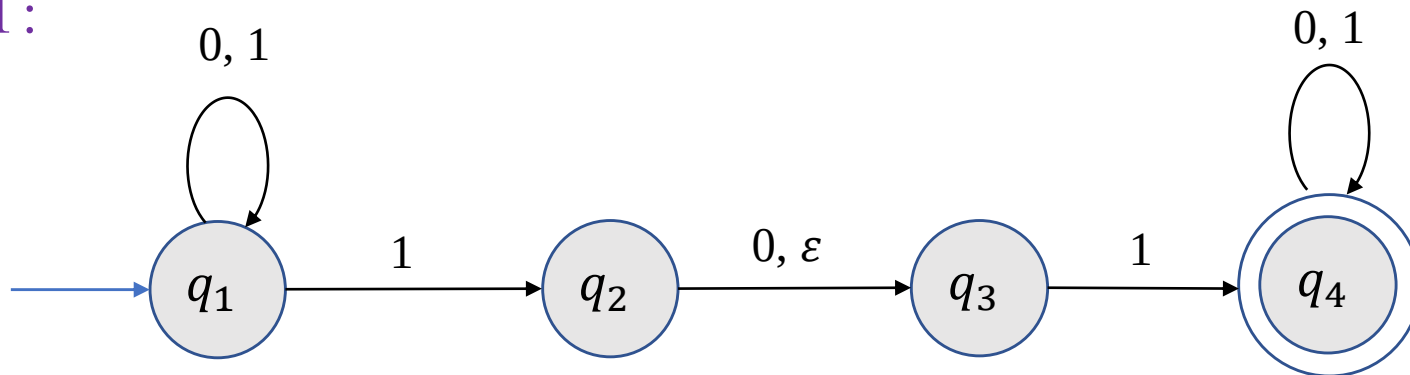
给定字母表 Σ , 定义 $\Sigma_\epsilon \triangleq \Sigma \cup \{\epsilon\}$.

定义(非确定性有限自动机). 一个非确定性有限自动机是一个5-元有序组 $(Q, \Sigma, \delta, q_0, F)$, 其中

- Q 是一个有限集, 称为状态集.
- Σ 是一个字母表.
- $\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ 是转移函数.
- $q_0 \in Q$ 称为初始状态.
- $F \subseteq Q$ 是接受状态集.

非确定性有限自动机-正式定义

例 1:



$$N_1 = (Q, \Sigma, \delta, q_1, F),$$

- $Q = \{q_1, q_2, q_3, q_4\}$;
- $\Sigma = \{0, 1\}$;
- q_1 是初始状态;
- $F = \{q_4\}$;
- 转移函数 δ :

	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

非确定性有限自动机-计算的定义

定义(计算). 设 $N = (Q, \Sigma, \delta, q_0, F)$ 是一个非确定性有限自动机, w 是 Σ 上的字符串. 如果能将 w 写成 $w = y_1 y_2 \cdots y_m$, 其中每个 $y_i \in \Sigma_\varepsilon$ 且存在状态序列 $r_0, r_1, \dots, r_m \in Q$ 使得

- $r_0 = q_0$; 可能的状态之一
- $r_{i+1} \in \delta(r_i, y_{i+1}), i = 0, 1, \dots, m - 1$;
- $r_m \in F$,

则称 N 接受 w .

例 1(续): 给出 N_1 接受 $w = 11$ 时 Σ_ε 上的序列以及相应的状态序列.

解: 当 w 写成 $w = 1 \varepsilon 1$, 相应的状态序列为 q_1, q_2, q_3, q_4 .

计算理论-有限自动机

- 有限自动机
- 非确定性有限自动机
- 正则语言
- 非正则语言

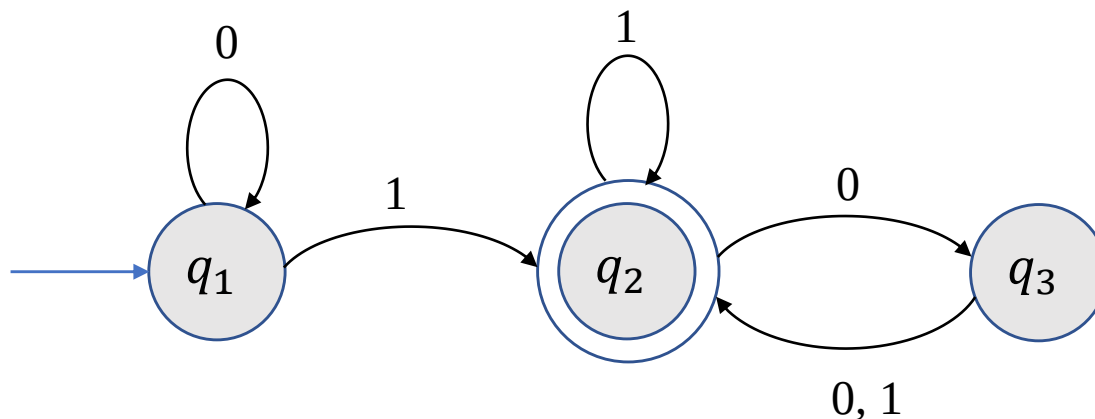
1 有限自动机

有限自动机

自动门控制

- 开和关两个状态
- 事件触发状态的改变

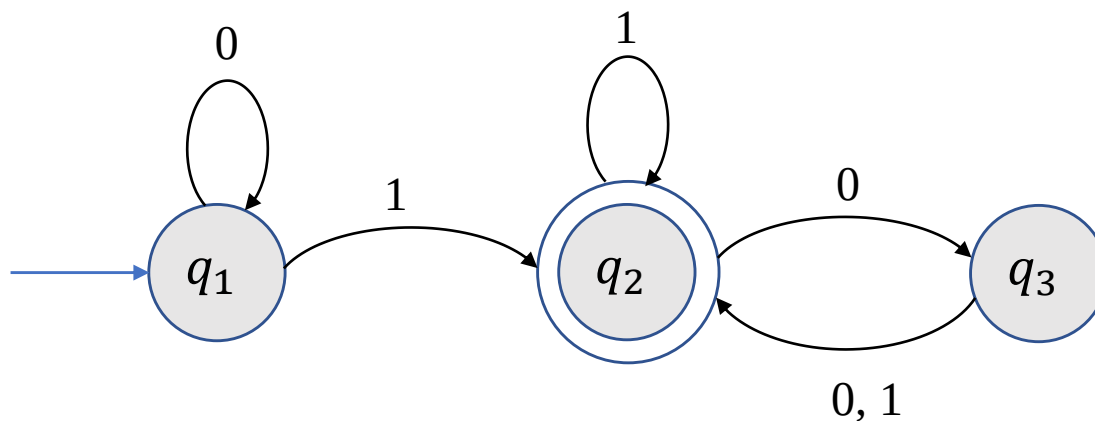
有限自动机-引例



一个有限自动机 M_1

- 圆圈代表状态
 - 蓝色箭头指向的状态称为初始状态
 - 两个圆圈代表接受状态
- 黑色箭头代表转移规则

有限自动机-引例



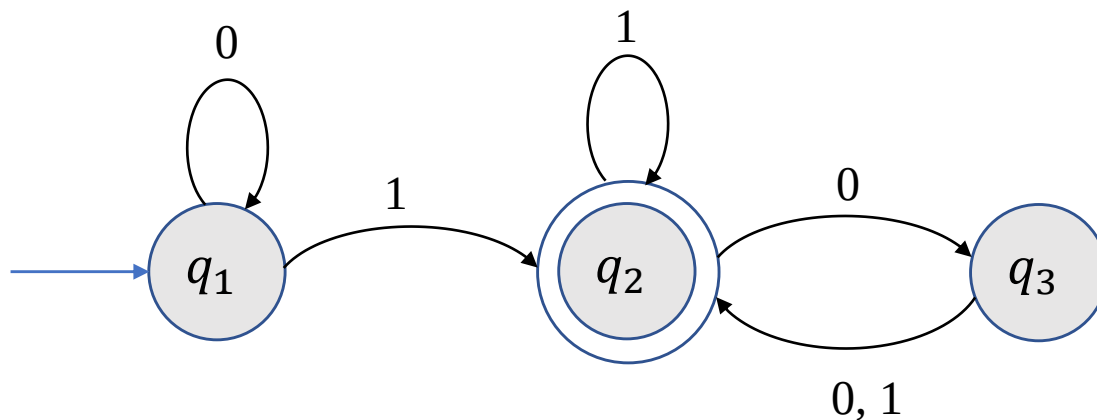
输入: $\{0,1\}$ -字符串.

输出: 接受或拒绝.

计算

- 从左向右依次读取输入字符串中的字符, 根据转移规则从一个状态跳转到下一个状态.
- 若终止状态为接受状态, 输出接受; 否则输出拒绝.

有限自动机-引例



计算

- 从左向右依次读取输入字符串中的字符, 根据转移规则从一个状态跳转到下一个状态.
- 若终止状态为接受状态, 输出接受; 否则输出拒绝.

例: $w = 1101$.

$$q_1 \xrightarrow{1} q_2 \xrightarrow{1} q_2 \xrightarrow{0} q_3 \xrightarrow{1} q_2$$

M_1 接受 w .

M_1 接受 10100?

有限自动机-正式定义

定义(有限自动机). 一个有限自动机是一个5-元有序组 $(Q, \Sigma, \delta, q_0, F)$, 其中

- Q 是一个有限集, 称为状态集.
- Σ 是一个字母表.
- $\delta: Q \times \Sigma \rightarrow Q$ 是转移函数.
- $q_0 \in Q$ 称为初始状态.
- $F \subseteq Q$ 是接受状态集.

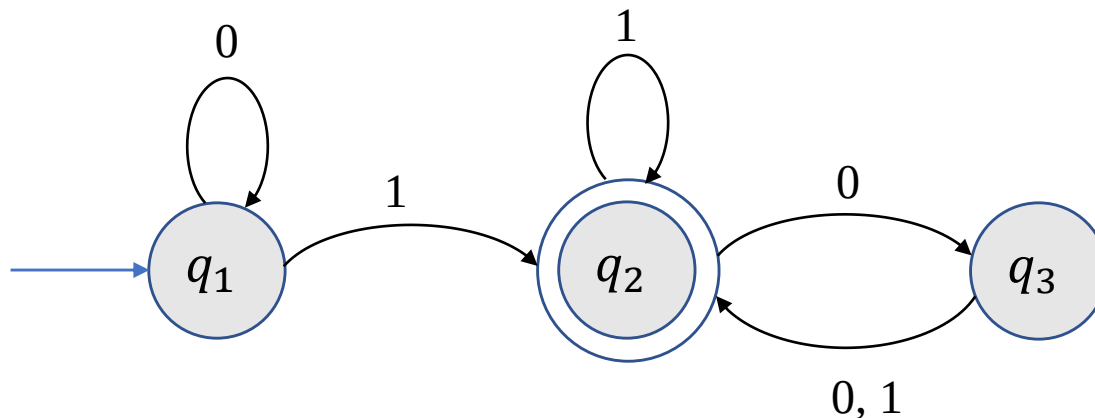
定义(有限自动机的语言). 给定有限自动机 M , 所有 M 接受的字符串的集合 $L(M)$ 称为 M 的语言, 即

$$L(M) = \{w | M \text{ 接受 } w\}.$$

此时也称 M 识别 $L(M)$.

有限自动机-正式定义

例 1.



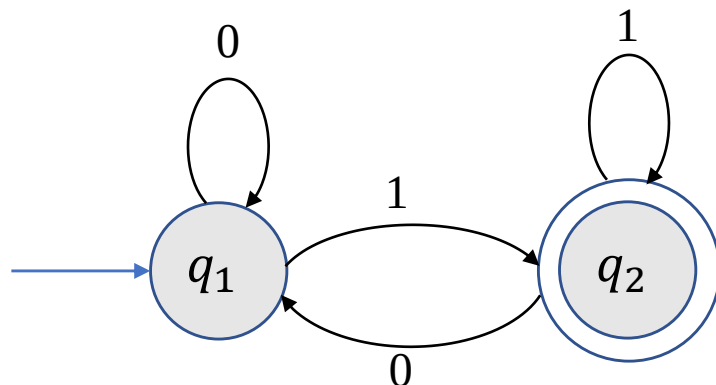
$$M_1 = (Q, \Sigma, \delta, q_1, F)$$

- $Q = \{q_1, q_2, q_3\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :
- q_1 是初始状态.
- 接受状态集 $F = \{q_2\}$.

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

有限自动机-正式定义

例 2.



$$M_2 = (Q, \Sigma, \delta, q_1, \{q_2\})$$

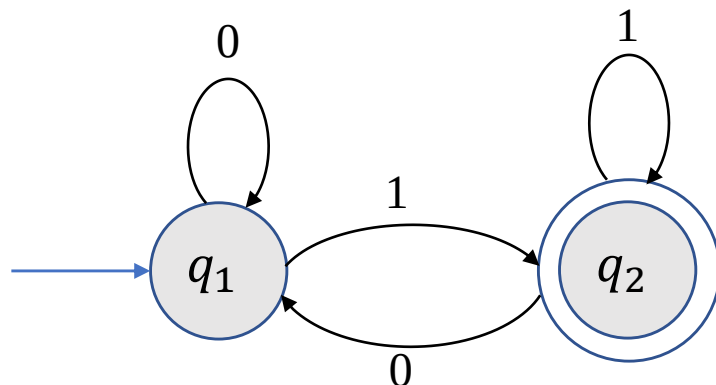
- $Q = \{q_1, q_2\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :

	0	1
q_1	q_1	q_2
q_2	q_1	q_2

$$L(M_2) = ?$$

有限自动机-正式定义

例 2.



$$M_2 = (Q, \Sigma, \delta, q_1, \{q_2\})$$

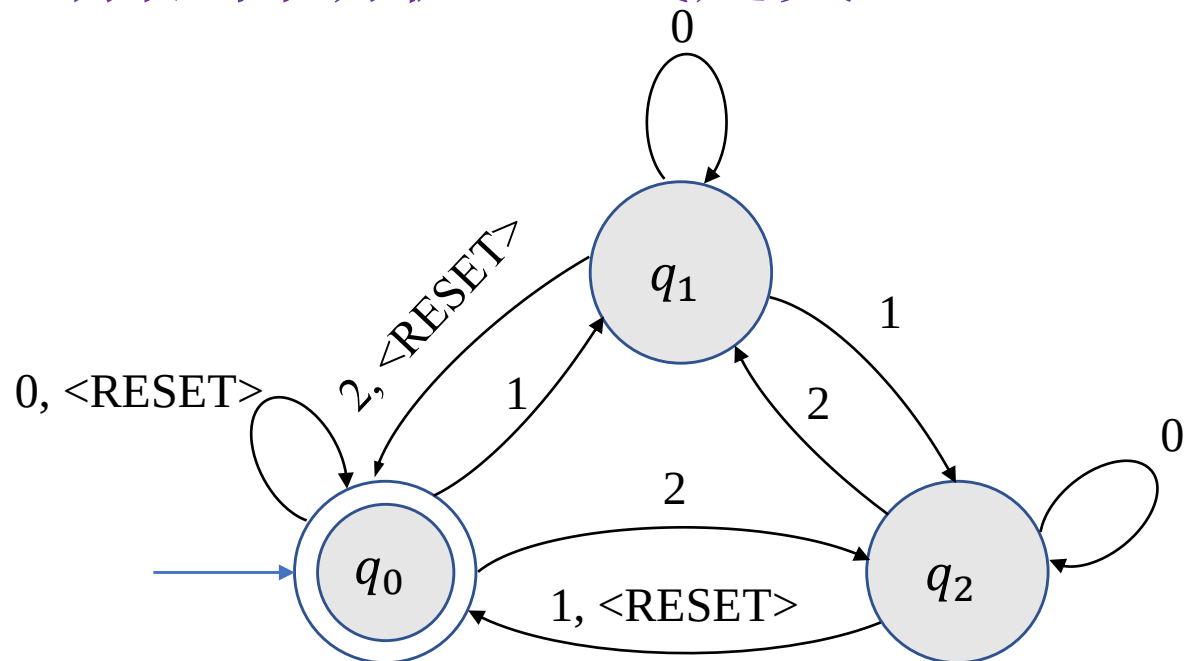
- $Q = \{q_1, q_2\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :

	0	1
q_1	q_1	q_2
q_2	q_1	q_2

$$L(M_2) = \{w \mid w \text{ 以 } 1 \text{ 结尾}\}.$$

有限自动机-正式定义

例 3.



$$M_3 = (\{q_0, q_1, q_2\}, \Sigma, \delta, q_0, \{q_0\})$$

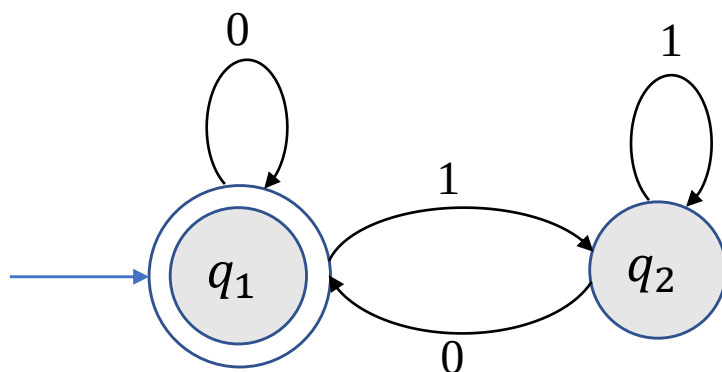
- $\Sigma = \{0, 1, 2, < \text{RESET} >\}$.
- δ : 记录输入字符串数字之和模3后的值.

一旦遇到< RESET > 字符将数值归0.

$L(M_3) = \{w | w \text{中最后一个 } < \text{RESET} > \text{ 字符后所有数字之和被3整除}\}.$

有限自动机-正式定义

练习. 给出下列有限自动机的正式定义并确定其识别的语言.



有限自动机-正式定义

问题：为什么需要形式化的定义？

- 精确描述：给定5-元有序组可以通过定义判定哪些是有限自动机.
- 有时有限自动机的状态图过于庞大，形式化定义更加简洁.
- 有限自动机可以和某个参数相关，每一个参数的取值对应一个有限自动机，因此无法画出无穷多个状态图.

有限自动机-正式定义

例：给定 $\Sigma = \{0,1,2, < \text{RESET} >\}$.

$L_i = \{w | w \text{中最后一个 } < \text{RESET} > \text{ 字符后所有数字之和被 } i \text{ 整除}\}.$

给出一个有限自动机 M_i 识别 L_i .

解： $M_i = (\{q_0, q_1, \dots, q_{i-1}\}, \Sigma, \delta_i, q_0, \{q_0\})$, 其中 δ_i 满足如果 M_i 处于状态 q_j , 则已读数字之和模 i 等于 j .

- $\delta_i(q_j, < \text{RESET} >) = q_0.$
- $\delta_i(q_j, 0) = q_j.$
- $\delta_i(q_j, 1) = q_k$, 其中 $k \equiv j + 1 \pmod{i}.$
- $\delta_i(q_j, 2) = q_k$, 其中 $k \equiv j + 2 \pmod{i}.$

有限自动机-计算的定义

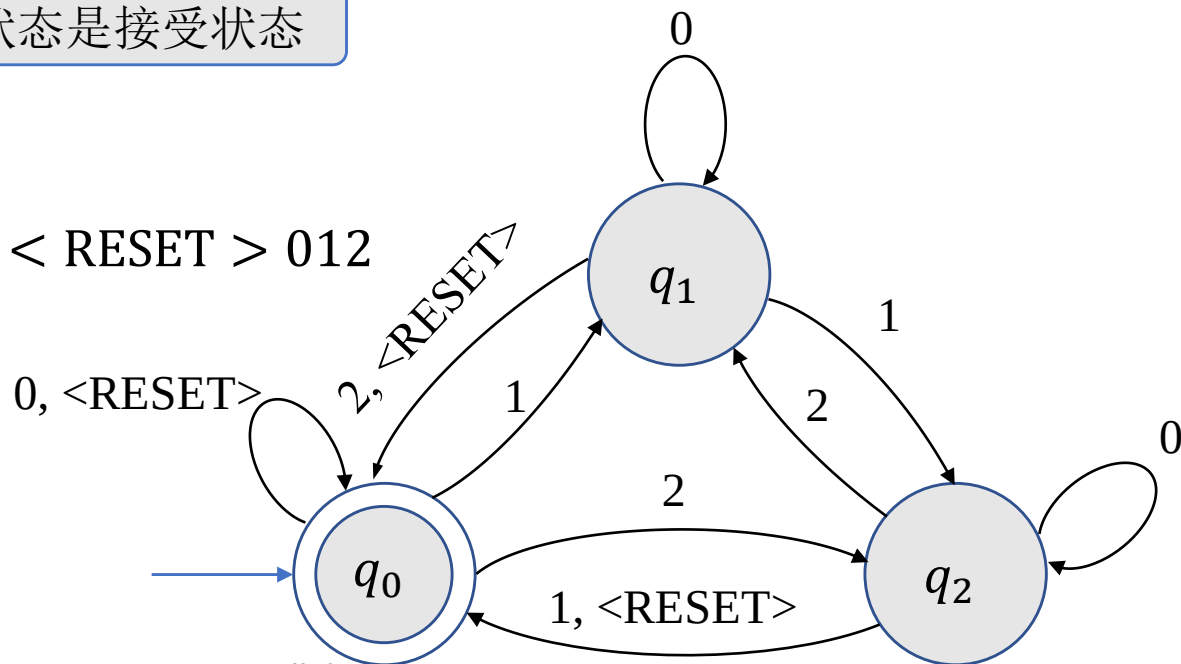
定义(计算). 设 $M = (Q, \Sigma, \delta, q_0, F)$ 是一个有限自动机, $w = w_1w_2 \cdots w_n$ 是 Σ 上的字符串. 如果存在状态序列 $r_0, r_1, \dots, r_n \in Q$ 使得

- $r_0 = q_0$; 从初始状态开始
- $\delta(r_i, w_{i+1}) = r_{i+1}, i = 0, 1, \dots, n-1$; 状态根据转移函数跳转
- $r_n \in F$, 终止状态是接受状态

则称 M 接受 w .

例. 给出 M_3 计算 $w = 10 \langle \text{RESET} \rangle 012$

时进入的状态序列.



有限自动机-设计

问题：如何设计识别给定语言 L 的有限自动机？

回答：将自己想象成有限自动机.

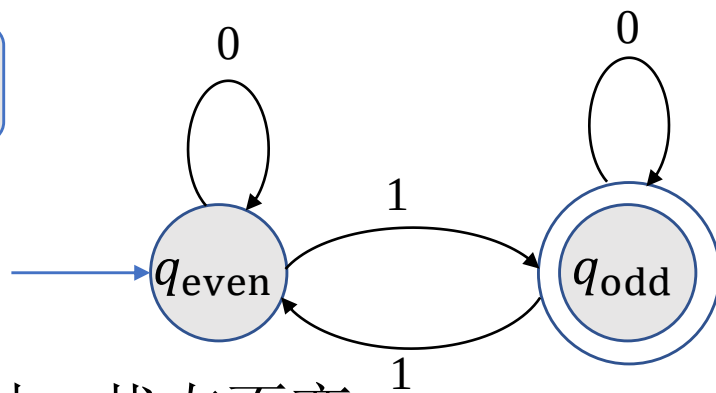
有限自动机-设计

例 1：设计识别 $\Sigma = \{0,1\}$ 上的语言 $L = \{w|w \text{ 包含奇数个} 1\}$ 的有限自动机 E_1 .

解：当读取输入字符串时，需要记录什么信息？

不是1的数目，而是1的数目的奇偶性！

- 因此 E_1 有两个状态 q_{odd} 和 q_{even} .
- 当读到1时，状态翻转；当读到0时，状态不变.
- 接受状态为 q_{odd} ，初始状态为 q_{even} (对应空字符串).

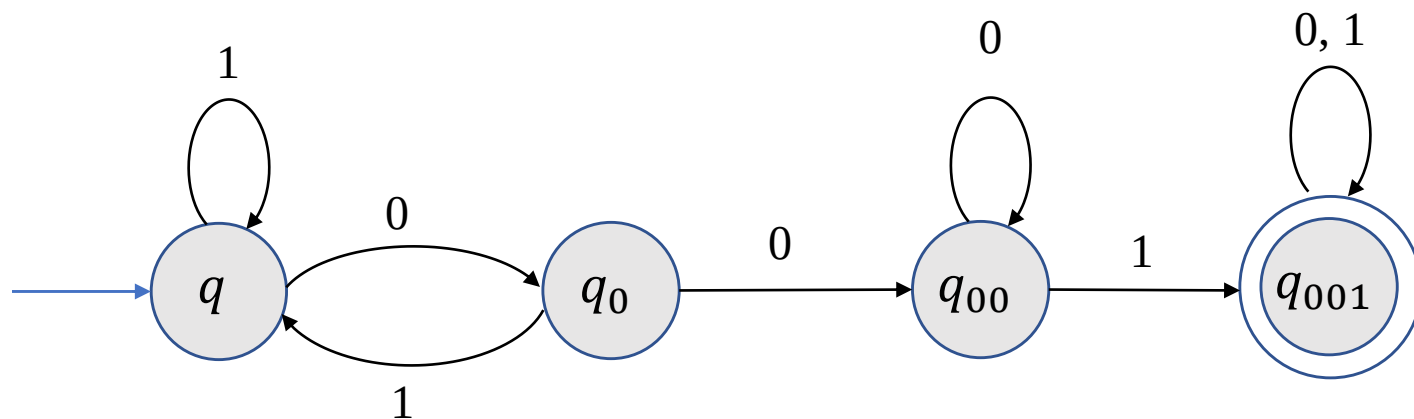


有限自动机-设计

例 2: 设计识别 $\Sigma = \{0,1\}$ 上的语言

$$L = \{w \mid w \text{ 包含 } 001 \text{ 作为子串}\}$$

的有限自动机 E_2 .



有限自动机-正式定义

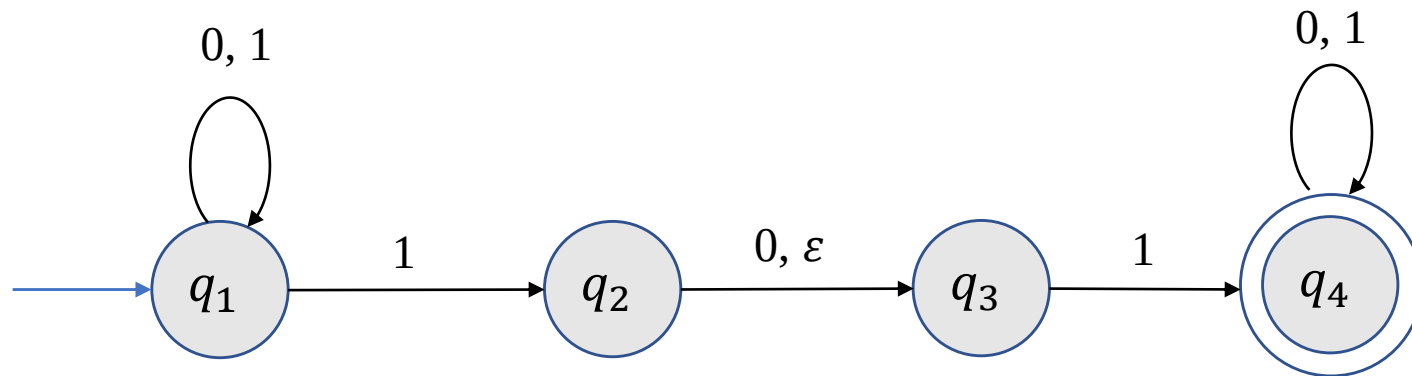
练习. 设计识别 $\Sigma = \{0,1\}$ 上的语言

$$L = \{w | w \text{ 包含至少三个 } 1\}$$

的有限自动机 E_3 .

2 非确定性有限自动机

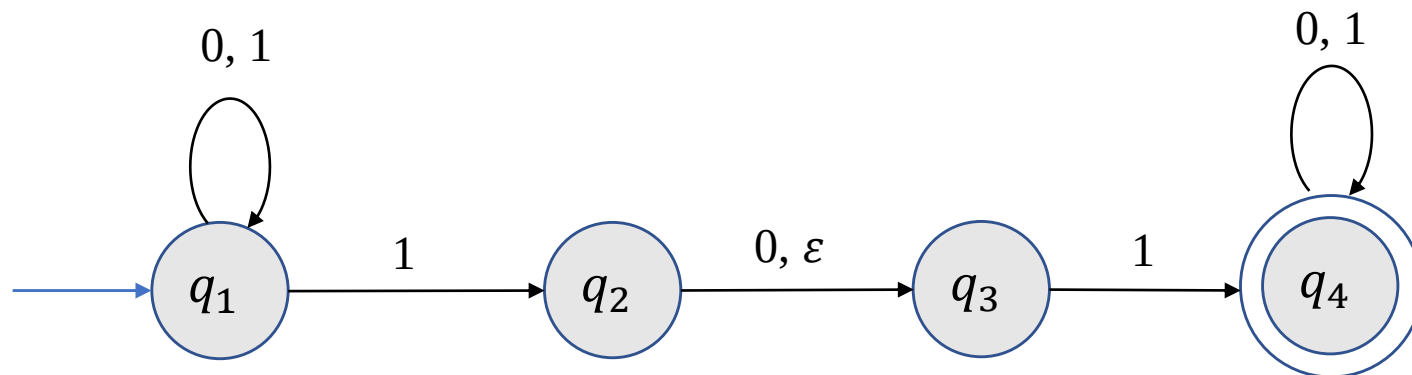
非确定性有限自动机-引例



非确定性的特点.

- 转移规则：一入多出.
- ϵ 箭头：无需读入字符的状态转移.

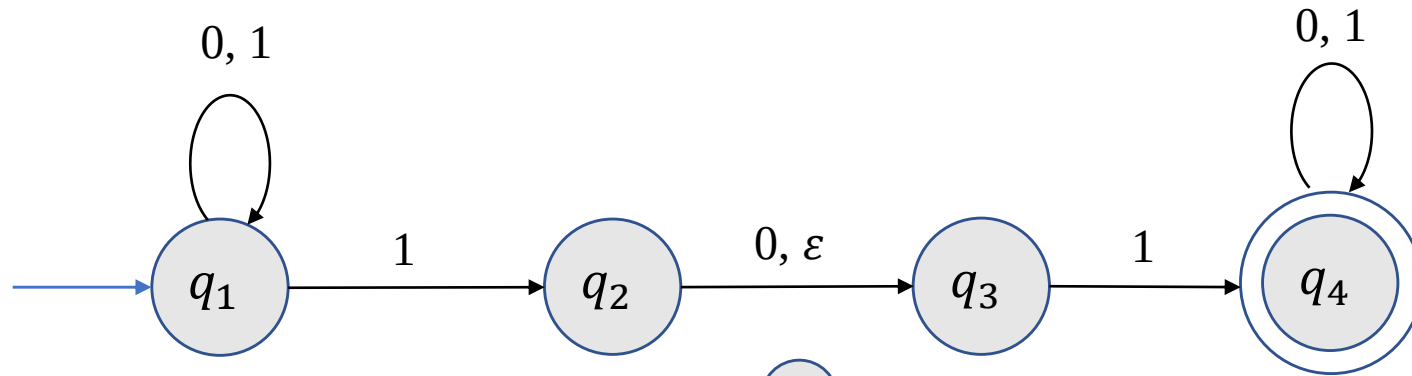
非确定性有限自动机-引例



计算.

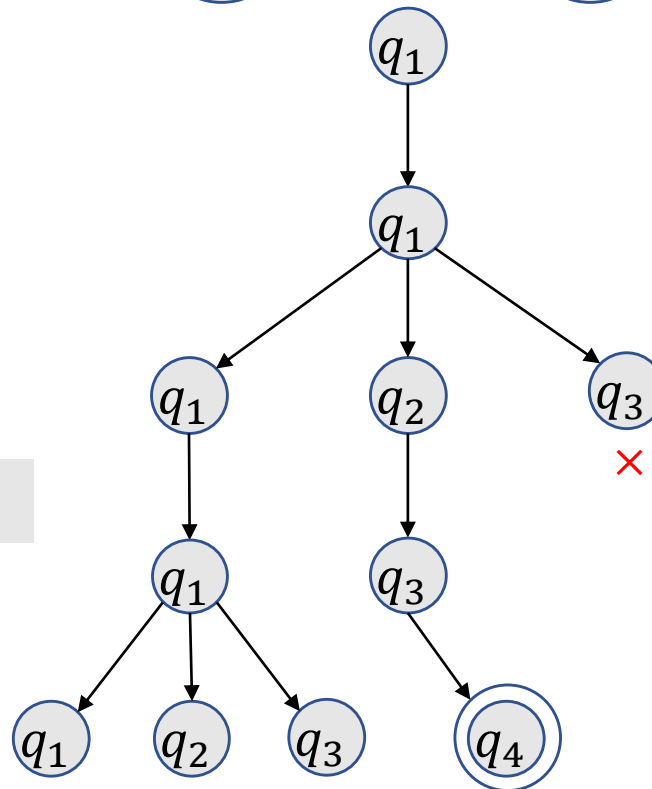
- 如果机器处于给定状态读入给定字符有多个出去的箭头，则机器复制成多份，每一份对应一个可能的状态.
- 如果机器处于给定状态且出去的箭头上有 ϵ 符号，则机器复制一份，其状态为箭头所指状态.
- 如果有一个计算分支接受，则机器接受.

非确定性有限自动机-引例



例: $w = 0101$.

N_1 接受 1001?



----- 0

----- 1

----- 0

----- 1

非确定性有限自动机-正式定义

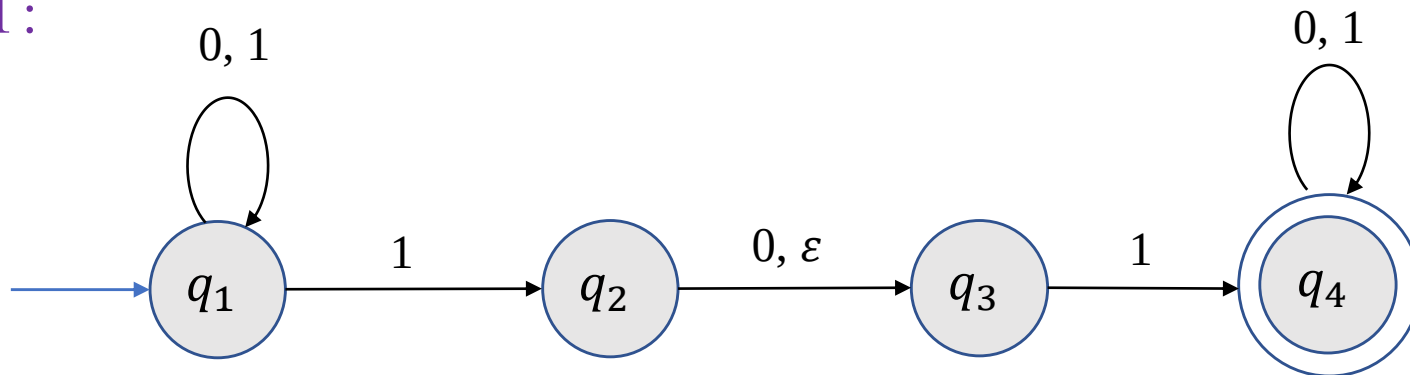
给定字母表 Σ , 定义 $\Sigma_\epsilon \triangleq \Sigma \cup \{\epsilon\}$.

定义(非确定性有限自动机). 一个非确定性有限自动机是一个5-元有序组 $(Q, \Sigma, \delta, q_0, F)$, 其中

- Q 是一个有限集, 称为状态集.
- Σ 是一个字母表.
- $\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ 是转移函数.
- $q_0 \in Q$ 称为初始状态.
- $F \subseteq Q$ 是接受状态集.

非确定性有限自动机-正式定义

例 1:



$$N_1 = (Q, \Sigma, \delta, q_1, F),$$

- $Q = \{q_1, q_2, q_3, q_4\}$;
- $\Sigma = \{0, 1\}$;
- q_1 是初始状态;
- $F = \{q_4\}$;
- 转移函数 δ :

	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

非确定性有限自动机-计算的定义

定义(计算). 设 $N = (Q, \Sigma, \delta, q_0, F)$ 是一个非确定性有限自动机, w 是 Σ 上的字符串. 如果能将 w 写成 $w = y_1 y_2 \cdots y_m$, 其中每个 $y_i \in \Sigma_\varepsilon$ 且存在状态序列 $r_0, r_1, \dots, r_m \in Q$ 使得

- $r_0 = q_0$; 可能的状态之一
- $r_{i+1} \in \delta(r_i, y_{i+1}), i = 0, 1, \dots, m - 1$;
- $r_m \in F$,

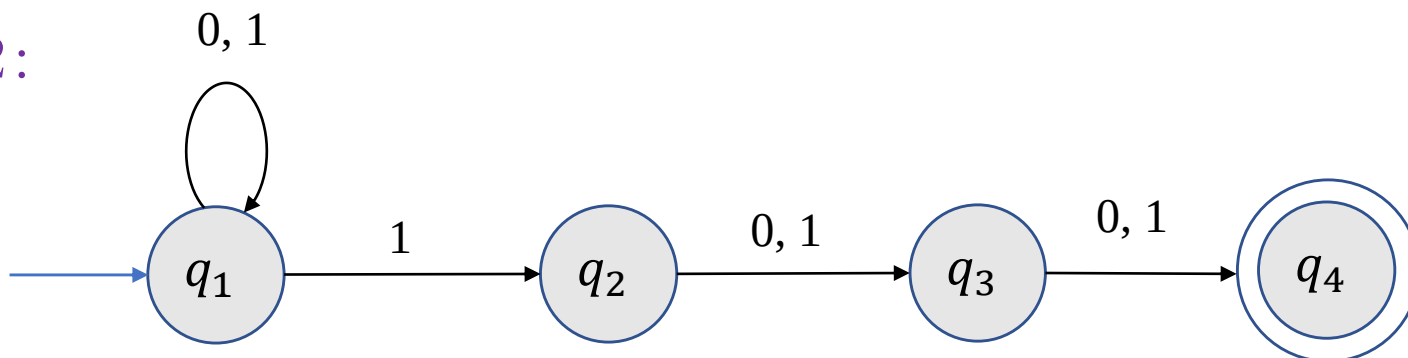
则称 N 接受 w .

例 1(续): 给出 N_1 接受 $w = 11$ 时 Σ_ε 上的序列以及相应的状态序列.

解: 当 w 写成 $w = 1 \varepsilon 1$, 相应的状态序列为 q_1, q_2, q_3, q_4 .

非确定性有限自动机-正式定义

例 2:



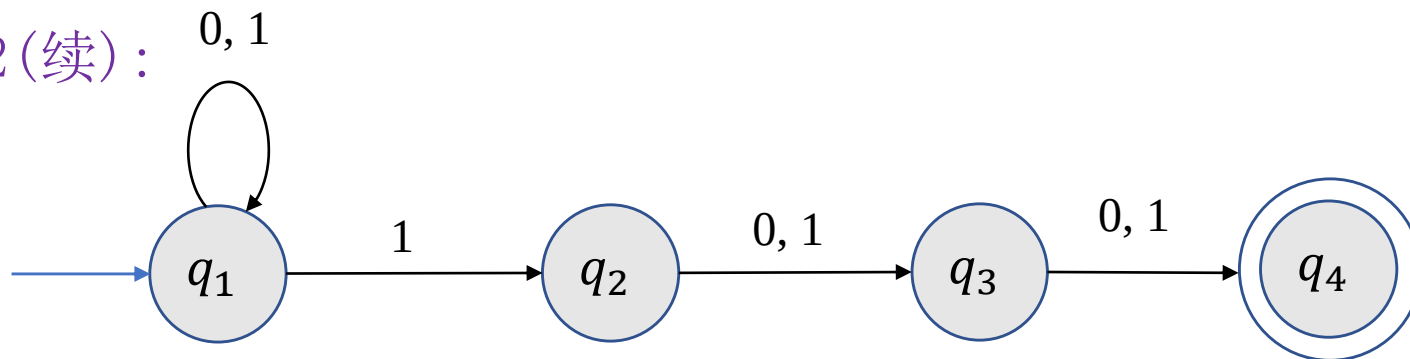
$$N_2 = (Q, \Sigma, \delta, q_1, \{q_4\}),$$

- $Q = \{q_1, q_2, q_3, q_4\}$;
- $\Sigma = \{0, 1\}$;
- 转移函数 δ :

	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\{q_3\}$	$\emptyset$
q_3	$\{q_4\}$	$\{q_4\}$	$\emptyset$
q_4	$\emptyset$	$\emptyset$	$\emptyset$

非确定性有限自动机-正式定义

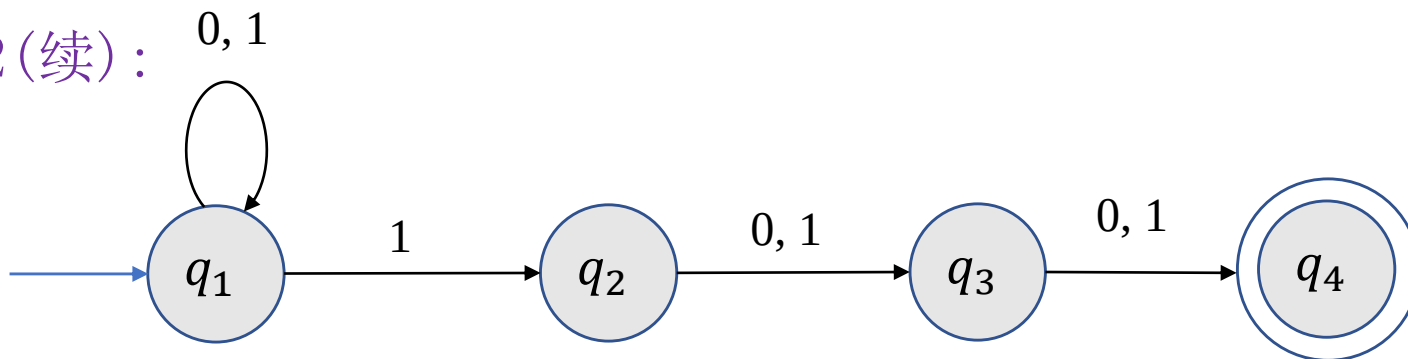
例 2(续):



$$L(N_2) = ?$$

非确定性有限自动机-正式定义

例 2(续):



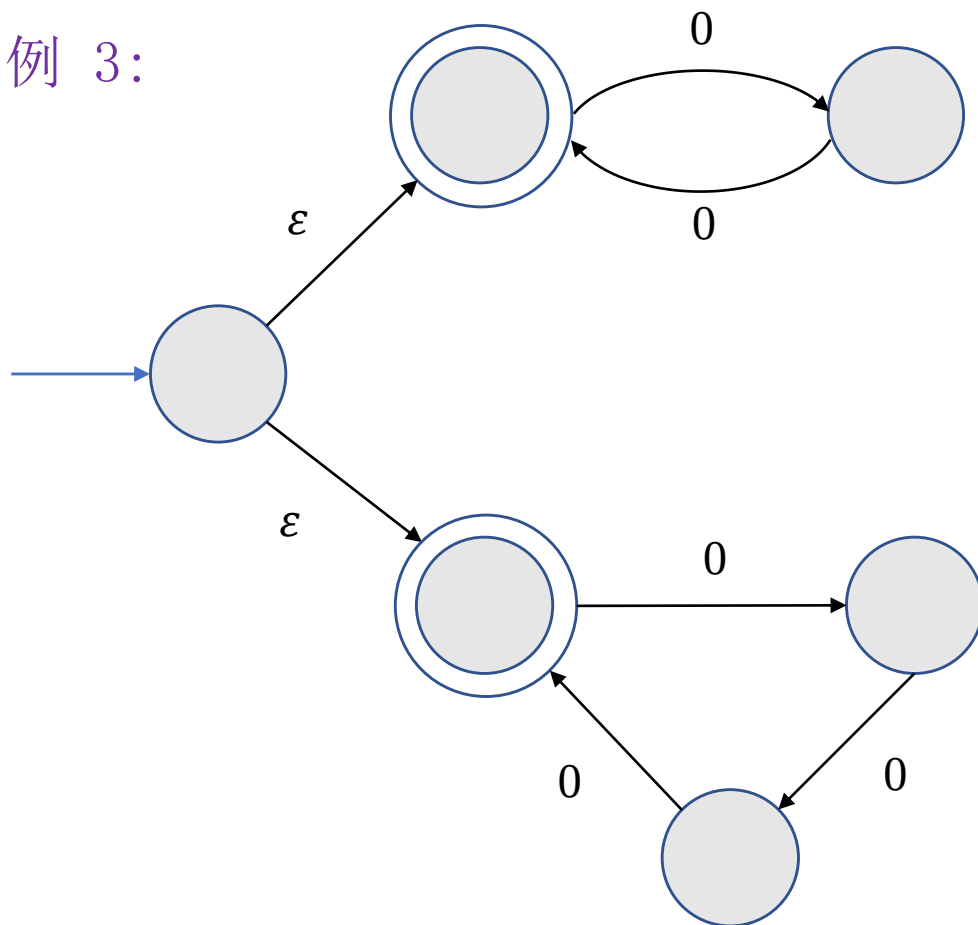
$$L(N_2) = \{w | w \text{ 倒数第3个位置上是1}\}.$$

- 机器一直处于 q_1 状态并“猜测”读到字符1是倒数第3个字符. 此时, 复制一份机器并处于 q_2 状态.
- 用状态 q_3 和 q_4 检查“猜测”是否正确.

若将 q_2 至 q_3 以及 q_3 至 q_4 的箭头上增加 ϵ 符号, 这样修改后的机器识别的语言是什么?

非确定性有限自动机-正式定义

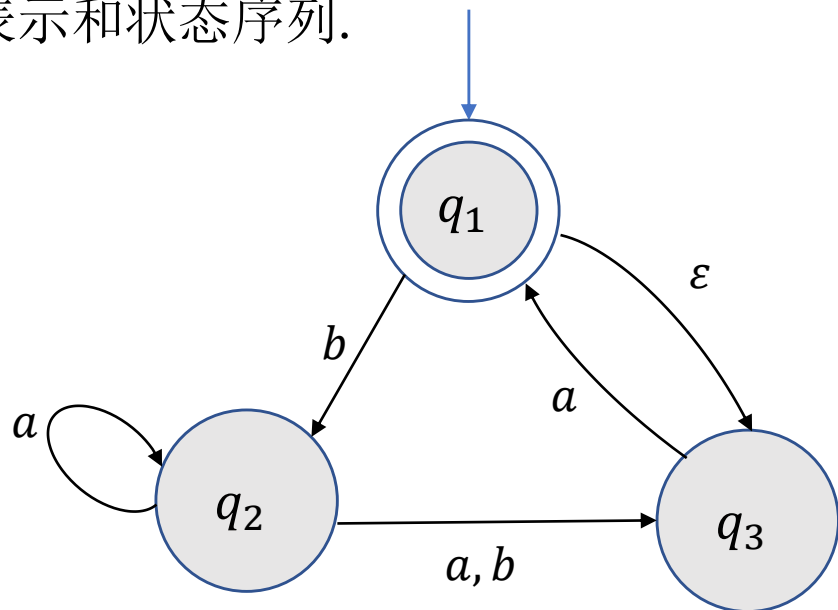
例 3:



$$L(N_3) = \{0^k \mid k \text{ 能被 } 2 \text{ 或 } 3 \text{ 整除}\}.$$

非确定性有限自动机-正式定义

例 4: 下列非确定性有限自动机 N_4 是否接受 $baba, a, bb$? 若接受, 给出相应的字符表示和状态序列.



- N_4 接受 a :

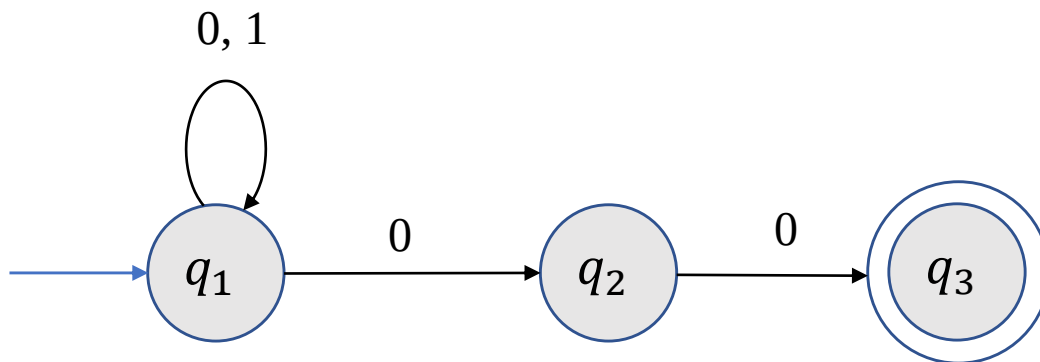
$\varepsilon a \rightarrow q_1, q_3, q_1.$

- N_4 不接受 bb .

- N_4 接受 $baba$.

非确定性有限自动机-正式定义

练习.



- 给出上述非确定性有限自动机 N 的形式化定义.
- N 是否接受 $\varepsilon, 001, 100$? 若接受, 给出相应字符串表示以及状态序列.
- N 识别的是什么语言?

非确定性有限自动机-与DFA的等价性

定义. 若有限自动机 M 和非确定性有限自动机 N 识别相同的语言, 则称 M 和 N 是等价的.

定理. 每个非确定性有限自动机 N 等价于一个确定性有限自动机 M .

证明思路. 想象自己是一个有限自动机, 需要记录哪些信息?

- N 计算时需要记录每一步机器所有可能处于的状态.
- 因此 M 需要记录 N 的状态集的子集.

非确定性有限自动机-与DFA的等价性

定理. 每个非确定性有限自动机 N 等价于一个确定性有限自动机 M .

证明. 令 $N = (Q, \Sigma, \delta, q_0, F)$ 是一个非确定性有限自动机, 构造有限自动机 $M = (Q', \Sigma, \delta', q_0', F')$ 如下.

先考虑 N 没有 ϵ 箭头的情况.

- $Q' = \mathcal{P}(Q)$.
- 对任何 $R \in Q', a \in \Sigma$,

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).$$

- $q_0' = \{q_0\}$.
- $F' = \{R \in Q' \mid R \text{ 包含 } F \text{ 中的某个状态}\}.$

非确定性有限自动机-与DFA的等价性

定理. 每个非确定性有限自动机 N 等价于一个确定性有限自动机 M .

证明(续). 再考虑 N 有 ε 箭头的情况.

对任何 $R \in Q'$, 定义 $E(R)$ 是所有能够通过 ε 箭头到达的状态集合包括 R 本身, 即

$$E(R) = \{q \mid \text{从} R \text{出发沿着} 0 \text{或者多个} \varepsilon \text{箭头可以到达} q\}.$$

- $Q' = \mathcal{P}(Q)$.
- 对任何 $R \in Q', a \in \Sigma$,

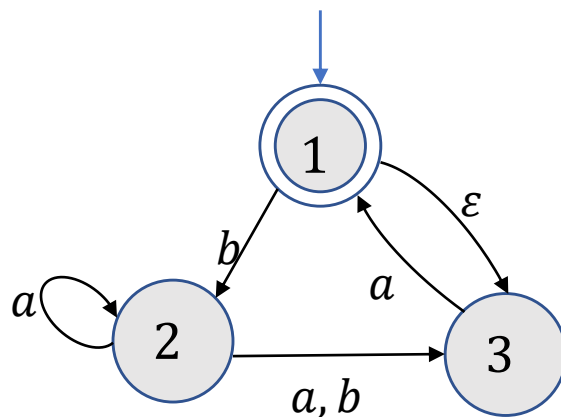
$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a) \quad \rightarrow \delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a)).$$

- $q'_0 = \{q_0\}$. $\rightarrow q'_0 = E(\{q_0\})$.
- $F' = \{R \in Q' \mid R \text{包含} F \text{中的某个状态}\}.$



非确定性有限自动机-与DFA的等价性

例. 将 N_4 按定理证明的方式转换成一个等价的确定性有限自动机.



	a	b
$\{1\}$	$\emptyset$	$\{2\}$
$\{2\}$	$\{2,3\}$	$\{3\}$
$\{3\}$	$\{1,3\}$	$\emptyset$

3 正则语言

正则语言

定义(正则语言). 对字母表上的语言 L , 若存在(非确定性)有限自动机 M 使得 $L = L(M)$, 则称 L 是正则语言.

定义(正则操作). 给定语言 A, B , 定义正则运算并, 连接和星号如下:

- 并: $A \cup B = \{s | s \in A \text{ 或 } s \in B\}.$
- 连接: $A \circ B = \{xy | x \in A, y \in B\}.$
- 星号: $A^* = \{x_1 x_2 \cdots x_k | k \geq 0 \text{ 且每个 } x_i \in A\}.$

例: 设 $\Sigma = \{a, b, \dots, z\}$. 令 $A = \{\text{good}, \text{bad}\}, B = \{\text{boy}, \text{girl}\}$, 求 $A \cup B, A \circ B, A^*$.

解: $A \cup B = \{\text{good}, \text{bad}, \text{boy}, \text{girl}\}.$

$A \circ B = \{\text{goodboy}, \text{goodgirl}, \text{badboy}, \text{badgirl}\}.$

$A^* = \{\epsilon, \text{good}, \text{bad}, \text{goodgood}, \text{goodbad}, \text{badgood}, \text{badbad}, \dots\}.$

正则语言

定理. 正则语言对于正则运算封闭.

正则语言

定理. 正则语言对于并运算封闭.

证明. 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \cup A_2$ 也是正则语言.

假设 $M_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的DFA. 构造有限自动机 M 识别 $A_1 \cup A_2$.

- 自然的想法是将给定字符串 s 串输入 M_1 , 判定 M_1 是否接受 s . 如果接受, 则 M 接受 s ; 否则将 s 输入 M_2 , 若 M_2 接受 s , 则 M 接受 s ; 否则 M 拒绝 s .
- 问题在于只能读取输入一遍!
- 所以当 M 读取字符串时, 需要同时记录 M_1 和 M_2 所处的状态.

M 的状态是一个有序对, 分别记录 M_1 和 M_2 所处的状态.

正则语言

定理. 正则语言对于并运算封闭.

证明(续). 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \cup A_2$ 也是正则语言.

假设 $M_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的DFA. 构造有限自动机 M 识别 $A_1 \cup A_2$.

M 的状态是一个有序对, 分别记录 M_1 和 M_2 所处的状态.

$M = (Q, \Sigma, \delta, q_0, F)$:

- $Q = Q_1 \times Q_2$.
- 任给 $(r_1, r_2) \in Q, a \in \Sigma$,

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a)).$$

- $q_0 = (q_1, q_2)$.
- $F = (Q_1 \times F_2) \cup (F_1 \times Q_2)$.



正则语言

思考：如何修改上述证明推出正则语言对交运算封闭？

正则语言

定理. 正则语言对于连接运算封闭.

证明. 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \circ A_2$ 也是正则语言.

假设 $M_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的DFA. 构造有限自动机 M 识别 $A_1 \circ A_2$.

- M 需要识别的字符串 s 可以分割成两段 $s = s_1 s_2$ 使得 M_1 接受 s_1 且 M_2 接受 s_2 .
- 问题在于预先不知道在哪个位置分割!

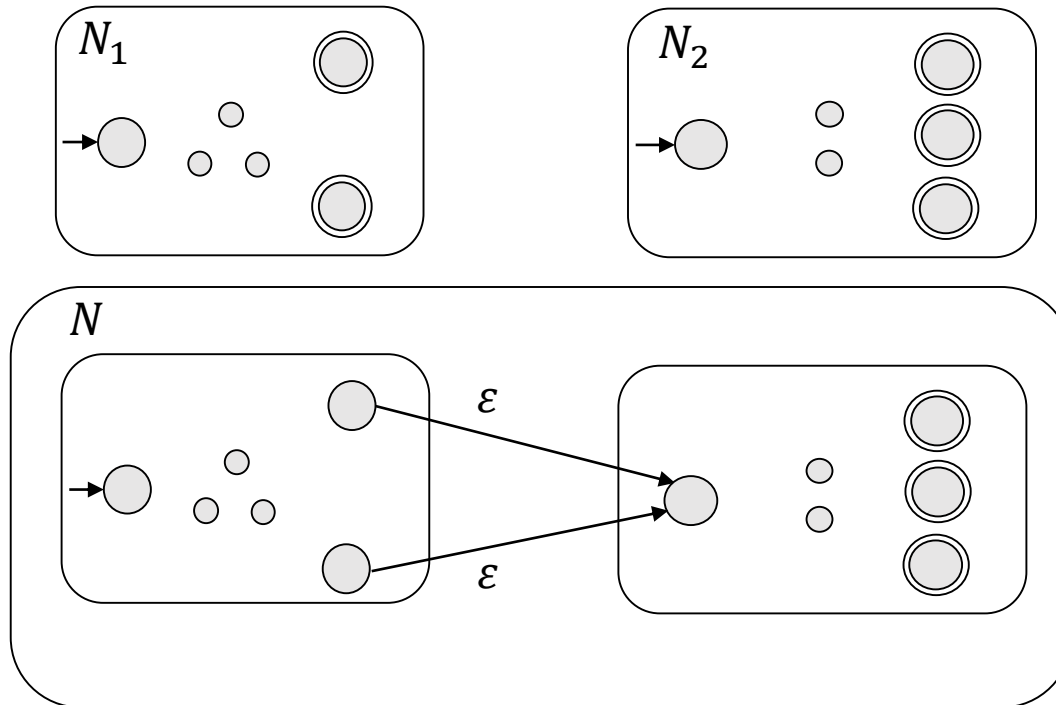
需要非确定性!

正则语言

定理. 正则语言对于连接运算封闭.

证明. 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \circ A_2$ 也是正则语言.

假设 $N_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的NFA. 构造有限自动机 $N = (Q, \Sigma, \delta, q, F)$ 识别 $A_1 \circ A_2$.



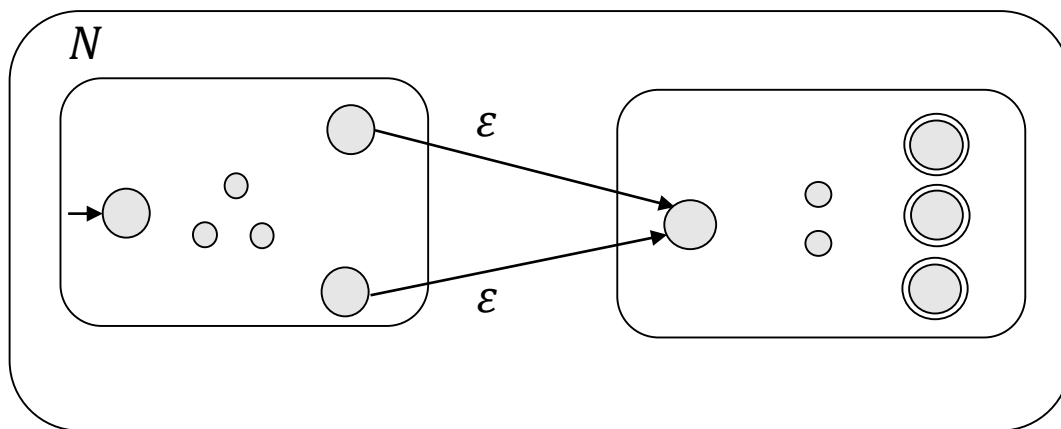
正则语言

定理. 正则语言对于连接运算封闭.

证明. 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \circ A_2$ 也是正则语言.

假设 $N_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的NFA. 构造有限自动机 $N = (Q, \Sigma, \delta, q, F)$ 识别 $A_1 \circ A_2$.

- $Q = Q_1 \cup Q_2$.
- $q_0 = q_1$.
- $F = F_2$.
- 任给 $a \in \Sigma_\varepsilon, q \in Q$,



$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ 且 } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ 且 } a \notin \varepsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ 且 } a = \varepsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

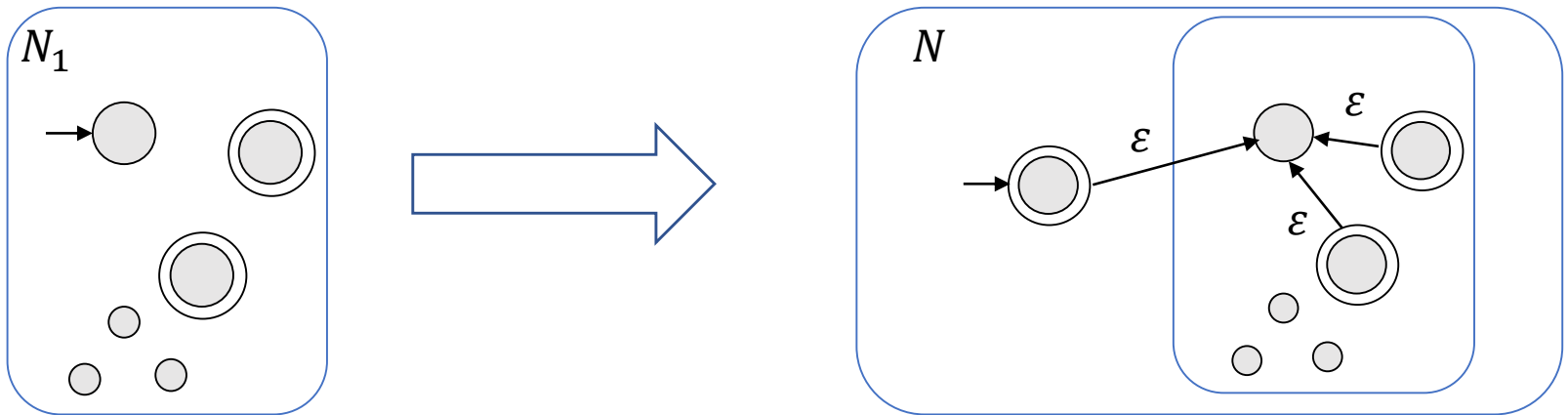
显然 $N = (Q, \Sigma, \delta, q, F)$ 识别 $A_1 \circ A_2$.

正则语言

定理. 正则语言对于星号运算封闭.

证明. 设 A_1 是正则语言, 需要证明 A_1^* 也是正则语言.

假设 $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ 是识别 A_1 的NFA. 构造非确定性有限自动机 $N = (Q, \Sigma, \delta, q_0, F)$ 识别 A_1^* .



正则语言

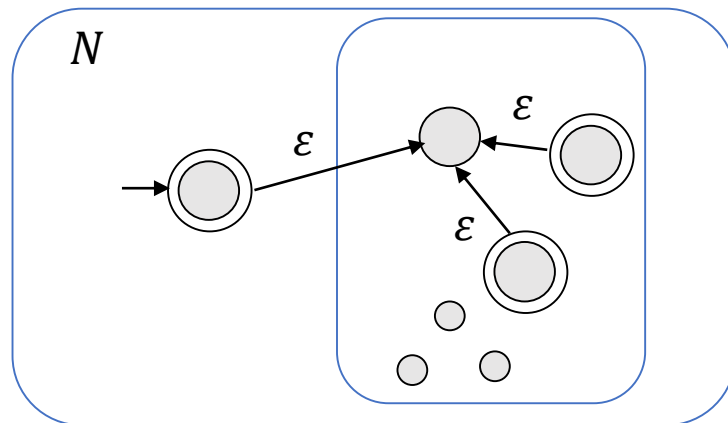
定理. 正则语言对于星号运算封闭.

证明. 设 A_1 是正则语言, 需要证明 A_1^* 也是正则语言.

假设 $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ 是识别 A_1 的NFA. 构造非确定性有限自动机 $N = (Q, \Sigma, \delta, q_0, F)$ 识别 A_1^* .

- $Q = Q_1 \cup \{q_0\}$.
- q_0 是新的初始状态.
- $F = F_1 \cup \{q_0\}$.
- 转移函数 δ 定义如下: 对任何 $a \in \Sigma_\varepsilon, q \in Q$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ 且 } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ 且 } a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ 且 } a = \varepsilon \\ \{q_1\} & q = q_0 \text{ 且 } a = \varepsilon \\ \emptyset & q = q_0 \text{ 且 } a \neq \varepsilon \end{cases}$$



■

正则语言

思考：如何使用非确定性证明正则语言对并运算封闭？

4 正则表达式

正则表达式-引例

- 用 $+$, $-$, $\times$, $\div$ 组成算数表达式

$$(5 + 3) \times 4.$$

- 用并，连接和星号运算组成正则表达式

$$(0 \cup 1) \circ 0^*.$$



$\{0,1\}$

任意多个0

所有以0或1开头后面有任意多个0的字符串.

正则表达式-引例

- $(0 \cup 1)^*$: 所有 $\{0,1\}$ -字符串.
- Σ^* : 字母表 Σ 上的任意字符串.
- Σ^*1 : 所有以1结尾的字符串.

正则表达式-定义

定义(正则表达式). 称 R 是一个正则表达式, 如果 R 是

1. 字母表 Σ 中的某个字符 a ,
2. 空字符串 ε ,
3. 空语言 $\emptyset$,
4. 表达式 $(R_1 \cup R_2)$, 其中 R_1 和 R_2 都是正则表达式,
5. 表达式 $(R_1 \circ R_2)$, 其中 R_1 和 R_2 都是正则表达式,
6. 表达式 R_1^* , 其中 R_1 是正则表达式.

记号.

• $L(R)$ 表示由 R 描述的语言.

• $R^+ \triangleq RR^*$. 

$$R^+ \cup \varepsilon = R^*$$

正则表达式-定义

例. 下面我们假设字母表 $\Sigma = \{0,1\}$.

1. $0^*10^* = \{w|w \text{恰好包含一个} 1\}$.
2. $\Sigma^*1\Sigma^* = \{w|w \text{至少包含一个} 1\}$.
3. $\Sigma^*001\Sigma^* = \{w|w \text{包含} 001 \text{作为子串}\}$.
4. $(01^+)^* = \{w|w \text{中每个} 0 \text{后面至少有一个} 1\}$.
5. $(\Sigma\Sigma)^* = \{w|w \text{是偶长的}\}$.
6. $(\Sigma\Sigma\Sigma)^* = \{w|w \text{的长度是} 3 \text{的倍数}\}$.
7. $01 \cup 10 = \{01, 10\}$.
8. $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1 = \{w|w \text{的首字符和最后一个字符相同}\}$.
9. $(0 \cup \varepsilon)1^* = 01^* \cup 1^*$.
10. $(0 \cup \varepsilon)(1 \cup \varepsilon) = \{\varepsilon, 0, 1, 01\}$.

正则表达式-定义

例(续). 下面我们假设字母表 $\Sigma = \{0,1\}$.

- $1^*\emptyset = \emptyset$.



在任何表达式后面连接空语言得到的是空语言.

- $\emptyset^* = \{\varepsilon\}$.

练习.

- $R \cup \emptyset = ?$ $R \circ \varepsilon = ?$

- $R \cup \varepsilon = ?$ $R \circ \emptyset = ?$

与有限自动机等价

问题：正则表达式和正则语言有什么联系吗？

定理. 一个语言是正则语言当且仅当存在一个正则表达式描述它.

与有限自动机等价

引理. 若一个语言可由一个正则表达式描述, 则它是正则的.

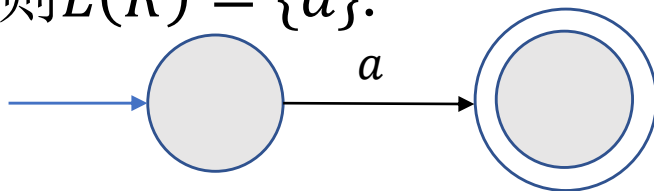
证明思路. 假设语言 A 可由正则表达式 R 来描述. 则将 R 转化为一个能够识别 A 的NFA.

与有限自动机等价

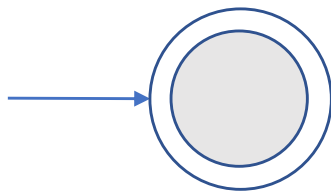
引理. 若一个语言可由一个正则表达式描述, 则它是正则的.

证明. 考虑正则表达式中的6种情况.

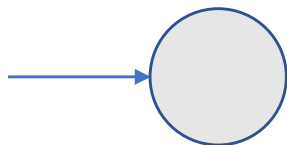
- 若 $R = a$, 则 $L(R) = \{a\}$.



- 若 $R = \varepsilon$, 则 $L(R) = \{\varepsilon\}$.



- 若 $R = \emptyset$, 则 $L(R) = \emptyset$.



与有限自动机等价

引理. 若一个语言可由一个正则表达式描述, 则它是正则的.

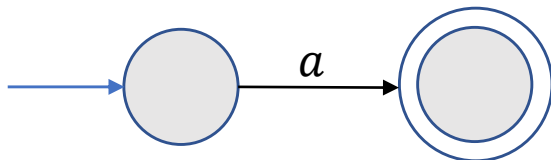
证明(续).

- 若 $R = R_1 \cup R_2$, $R = R_1 \circ R_2$, 或 $R = R_1^*$, 则可从识别 $L(R_1)$ 和 $L(R_2)$ 的NFA构造识别 $L(R)$ 的NFA. ■

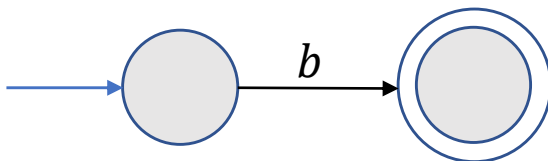
与有限自动机等价

例. 将 $(ab \cup a)^*$ 转化为一个等价的NFA.

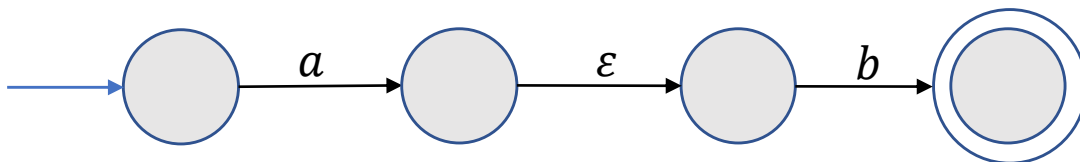
a



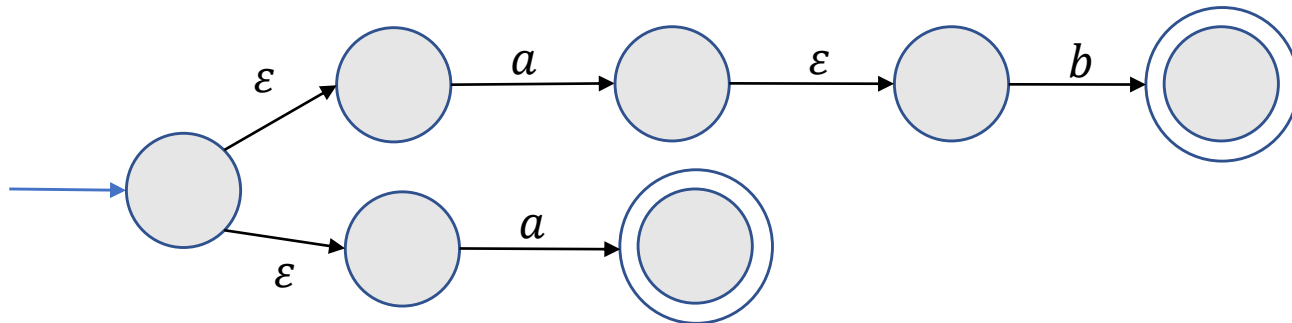
b



ab

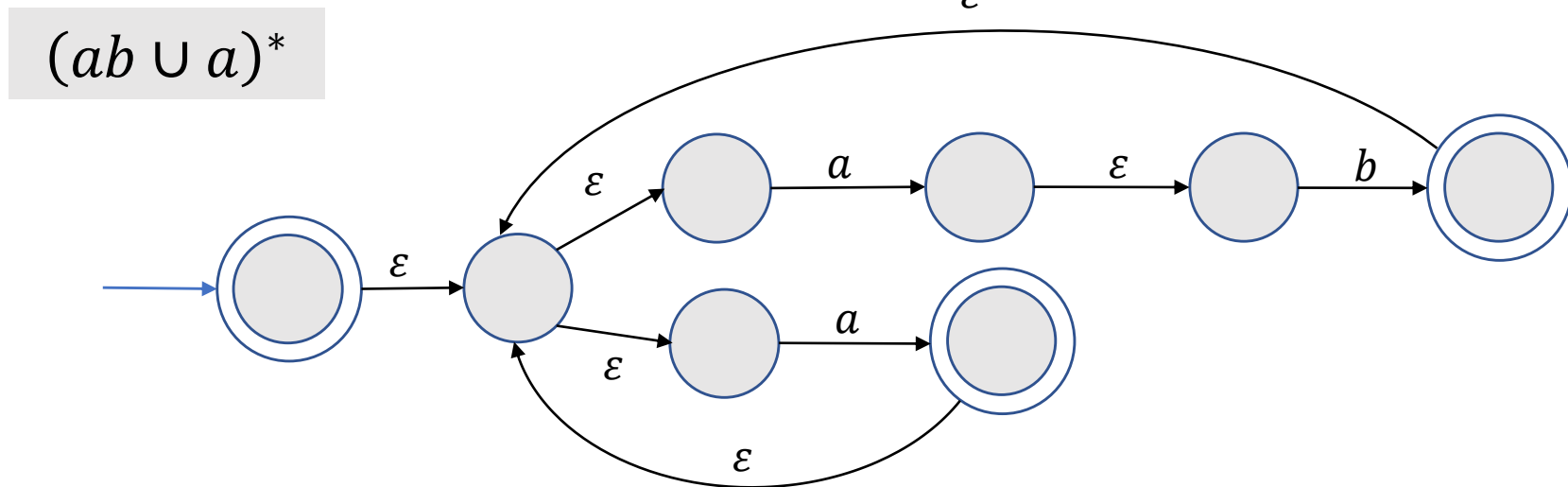
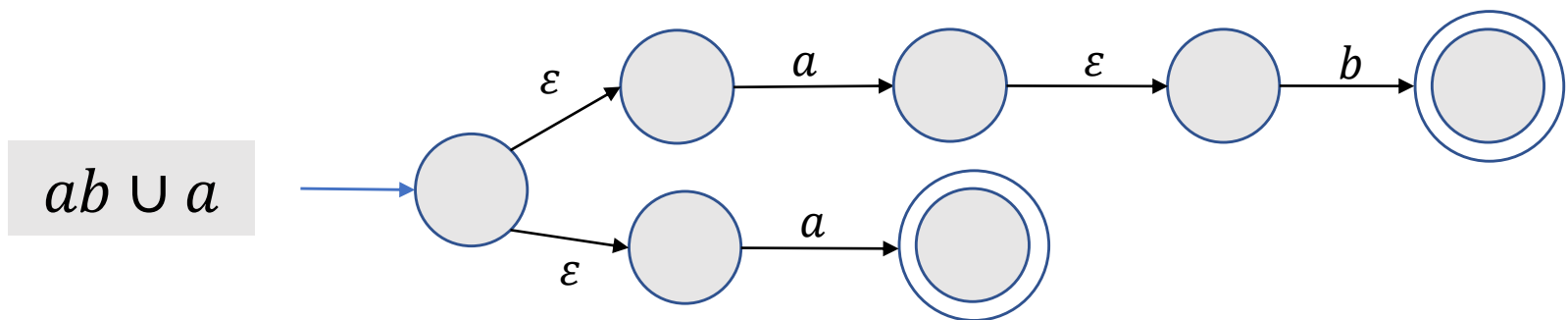


$ab \cup a$



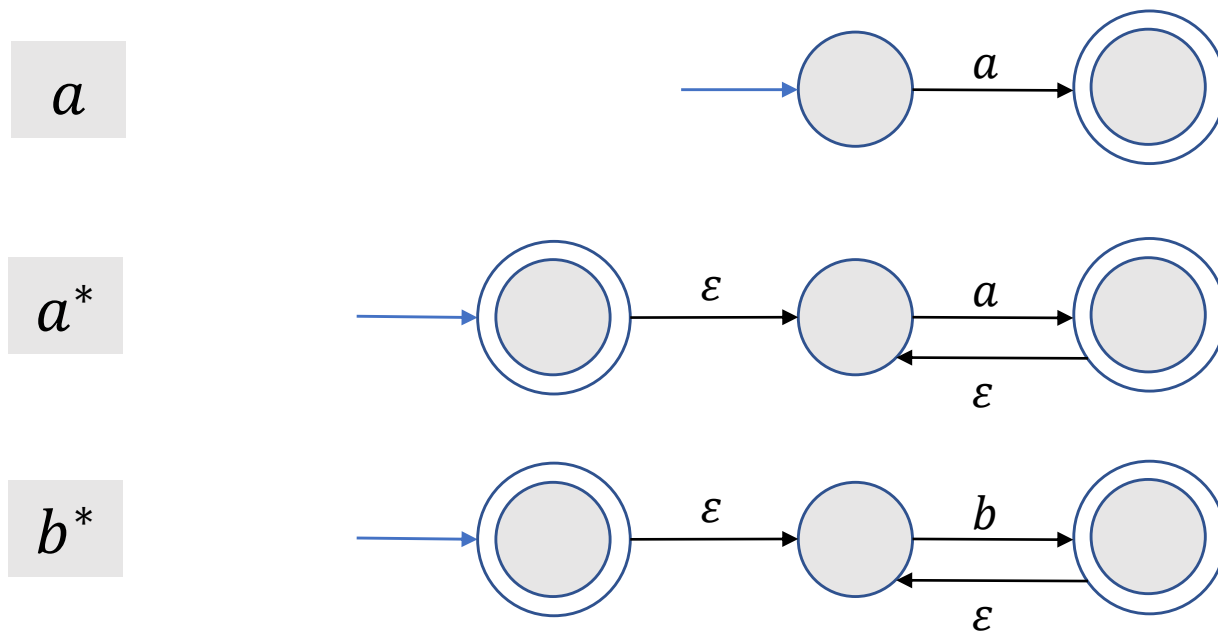
与有限自动机等价

例(续). 将 $(ab \cup a)^*$ 转化为一个等价的NFA.



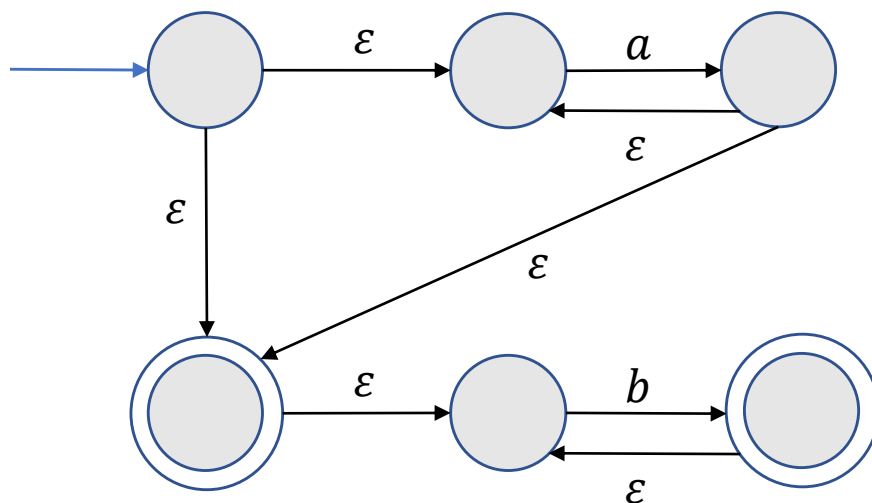
与有限自动机等价

练习. 将 a^*b^* 转化为一个等价的NFA.



与有限自动机等价

练习. 将 a^*b^* 转化为一个等价的NFA.

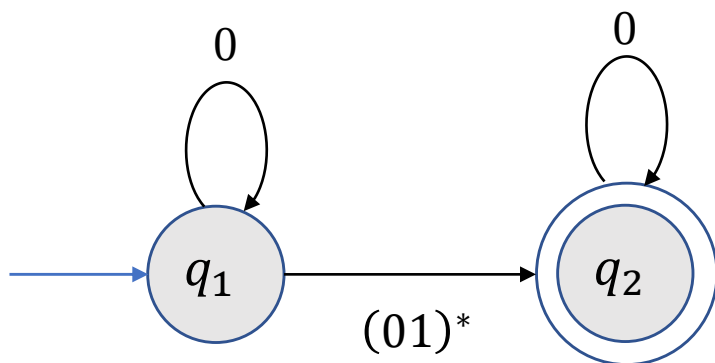


a^*b^*

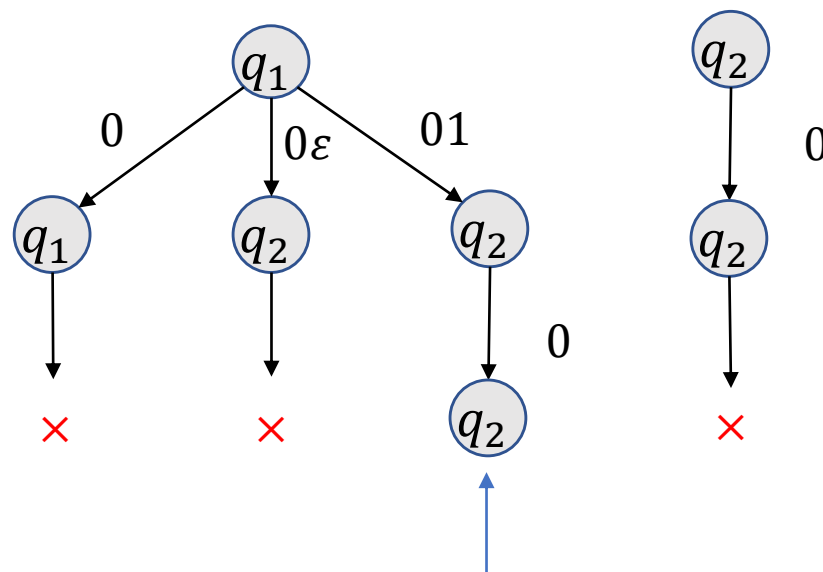
与有限自动机等价

引理. 任何正则语言都能由一个正则表达式描述.

广义非确定性有限自动机



例. $w = 010$.



是否接受011?

接受

广义非确定性有限自动机

特殊形式.

- 只有一个接受状态且不同于初始状态
- 任意两个状态之间有一条边, 除了
 - 初始状态只有出去的边
 - 接受状态只有进入的边

转化步骤.

- 添加一个新的初始状态并添加一个 ϵ 箭头指向原来的初始状态, 添加一个新的接受状态, 并且添加从原来的接受状态到新接受状态的 ϵ 箭头.
- 若没有从 q_i 指向 q_j 的箭头, 添加这样一个箭头并且其上的标签设为 $\emptyset$.
- 若在 q_i 指向 q_j 的箭头上有多个标签, 则用这些标签的并替换原标签.

广义非确定性有限自动机

定义(广义非确定性有限自动机). 一个广义非确定性有限自动机是一个5-元组 $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$, 其中

- Q 是状态集,
- Σ 是字母表,
- $\delta: (Q - \{q_{\text{accept}}\}) \times (Q - \{q_{\text{start}}\}) \rightarrow \mathcal{R}$ 是转移函数, 其中 $\mathcal{R}$ 是 Σ 上所有正则表达式的集合.
- q_{start} 是起始状态,
- q_{accept} 是接受状态.

理解.

- 箭头上的符号是正则表达式.
- 满足特殊形式.

广义非确定性有限自动机

定义(广义非确定性有限自动机的计算). 一个广义非确定性有限自动机接受字符串 $w = w_1 w_2 \dots w_k$, 其中 $w_i \in \Sigma^*$, 如果存在状态 $q_0, q_1, \dots, q_k$ 使得

- $q_0 = q_{\text{start}}$ 是起始状态,
- $q_k = q_{\text{accept}}$ 是接受状态,
- 且对任何 $1 \leq i \leq k$ 有 $w_i \in L(R_i)$, 其中 $R_i = \delta(q_{i-1}, q_i)$.

理解.

- 箭头上的符号是正则表达式.
- 机器接受到输入后, 读取子字符串 (不仅仅是单个字符). 若该子串可由箭头上的正则表达式描述, 则机器进行相应的跳转.

广义非确定性有限自动机

练习. 画出一个有3个状态的特殊GNFA并给出一个接受字符串及其接受分支的状态序列.

与有限自动机等价

引理. 任何正则语言都能由一个正则表达式描述.

证明思路. 给定正则语言 A , 则 A 可被一个DFA M 识别.

- 首先将 M 转化为一个GNFA G .
- 其次将 G 转化为一个正则表达式. 假设 G 有 k 个状态.
 - 若 $k = 2$, 则返回从初始状态到接受状态的箭头上的表达式.
 - 若 $k > 2$, 则构造一个与 G 等价的GNFA G' , 其中 G' 有 $k - 1$ 个状态.

与有限自动机等价

引理. 任何正则语言都能由一个正则表达式描述.

证明. 给定正则语言 A , 则 A 可被一个DFA M 识别.

Step 1. 将 M 转化为一个GNFA G .

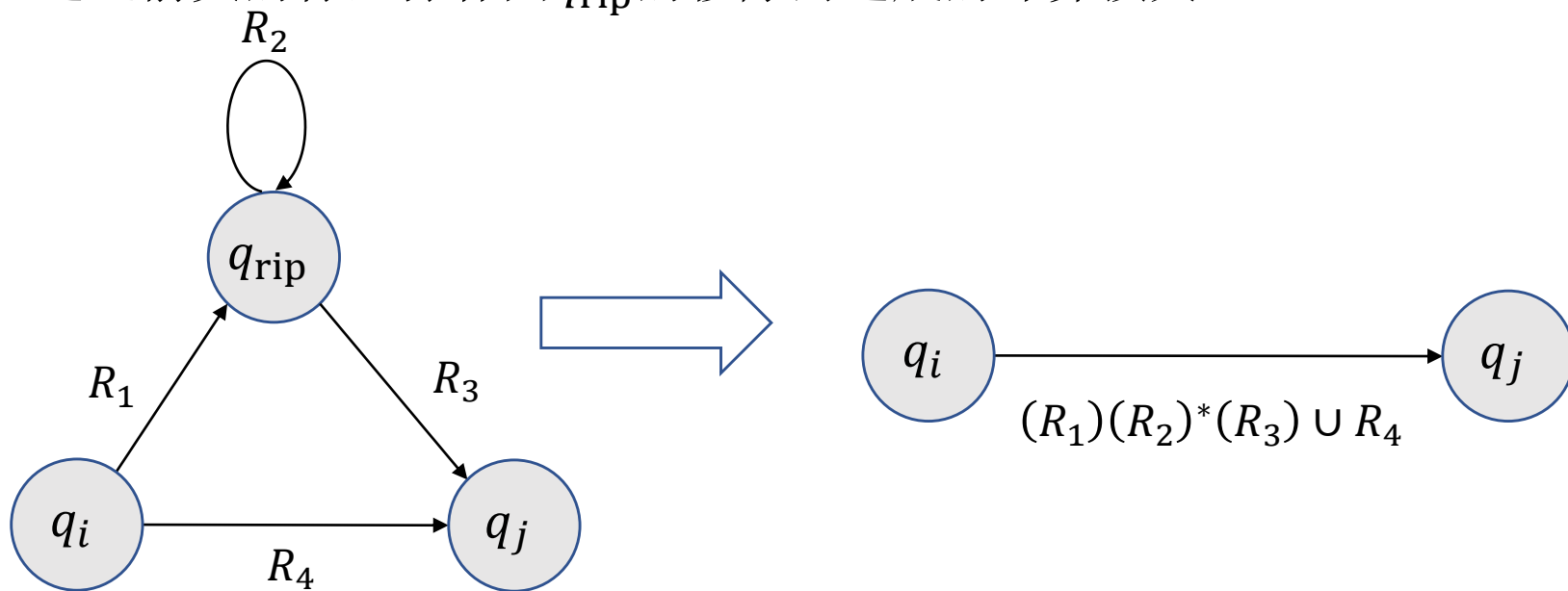
- 添加一个新的初始状态并添加一个 ε 箭头指向原来的初始状态.
- 添加一个新的接受状态, 并且添加从原来的接受状态到新接受状态的 ε 箭头.
- 若没有从 q_i 指向 q_j 的箭头, 添加这样一个箭头并且其上的标签设为 $\emptyset$.
- 若在 q_i 指向 q_j 的箭头上有多个标签, 则用这些标签的并替换原标签.

与有限自动机等价

证明(续). 给定正则语言 A , 则 A 可被一个DFA M 识别.

Step 2. 将 G 转化为一个正则表达式.

- 假设 $k \geq 3$ 且选取一个不同于初始状态和接受状态的状态 q_{rip} .
- 构造一个与 G 等价的GNFA G' , 其中 G' 的状态集为 $Q - \{q_{\text{rip}}\}$.
- 通过箭头的标签弥补因 q_{rip} 的移除而造成的计算损失.



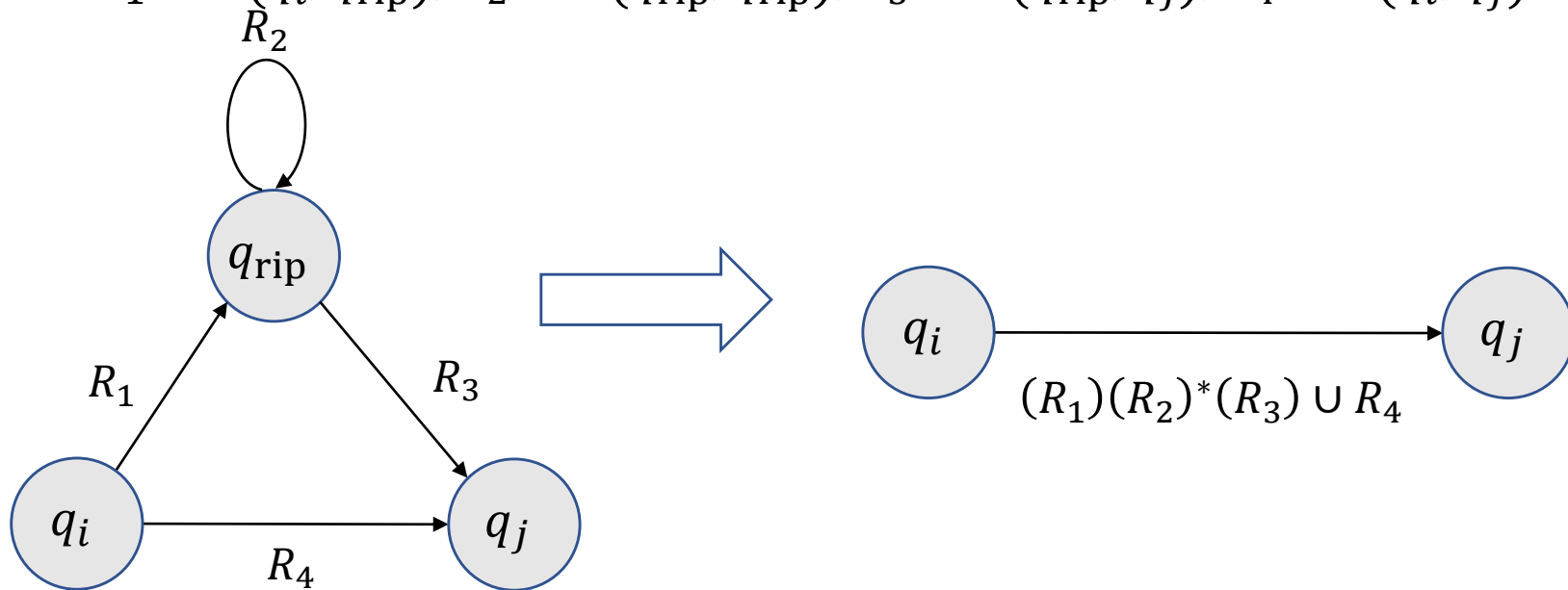
与有限自动机等价

Step 2. 将 G 转化为一个正则表达式.

令 $G' = (Q', \Sigma, \delta', q_{\text{start}}, q_{\text{accept}})$, 其中 $Q' = Q - \{q_{\text{rip}}\}$ 且对任何 $q_i \in Q' - \{q_{\text{accept}}\}$, $q_j \in Q' - \{q_{\text{start}}\}$, 令

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup R_4,$$

其中 $R_1 = \delta(q_i, q_{\text{rip}})$, $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$, $R_3 = \delta(q_{\text{rip}}, q_j)$, $R_4 = \delta(q_i, q_j)$.



与有限自动机等价

Claim. G 与 G' 识别相同的语言.

Proof of the claim. 若 G 接受 w , 令 $q_{\text{start}}, q_1, q_2, \dots, q_{\text{accept}}$ 是某个接受计算分支的状态序列.

- 若 q_{rip} 没有出现, 则因为 G' 的标签包含了原来的标签, 故 G' 接受 w .
- 若 q_{rip} 出现了, 则将 q_{rip} 移除对应了 G' 接受 w 的状态序列.

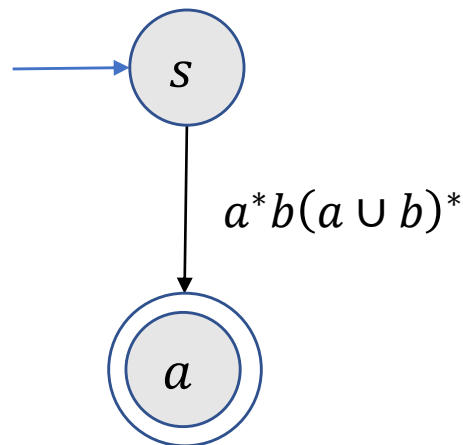
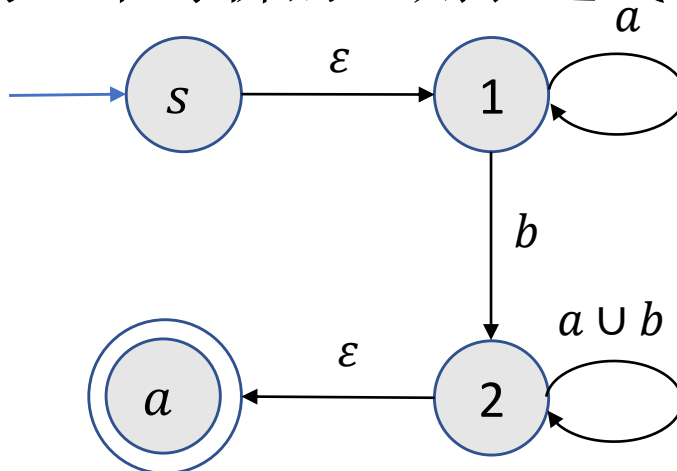
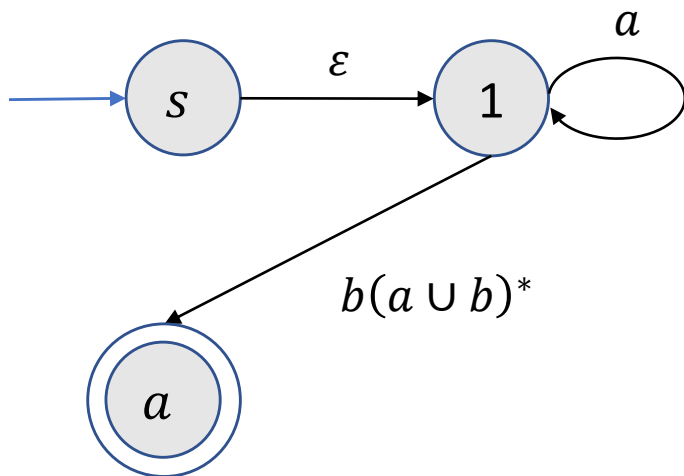
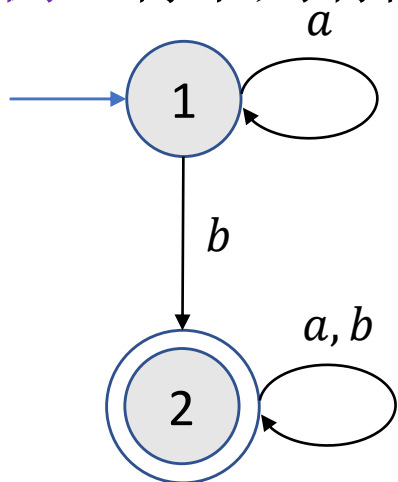
$$q_i, q_{\text{rip}}, q_{\text{rip}}, q_{\text{rip}}, q_j \rightarrow q_i, q_j$$

若 G' 接受 w , 则因为 G' 中从 q_i 到 q_j 的箭头描述了所有能让 q_i 要么直接要么经过 q_{rip} 跳转到 q_j 的字符串, G 接受 w . ■

重复上述步骤直到 G' 只有2个状态, 从而唯一箭头的标签即为等价的正则表达式. ■

与有限自动机等价

例. 将下列有限自动机转化为一个等价的正则表达式.



与有限自动机等价

练习.

- 设计一个识别 $\Sigma = \{0,1\}$ 上的语言 0^* 的 NFA N .
- 按照定理的证明将 N 转化为一个等价的正则表达式.

计算理论-上下文无关语法

- 上下文无关语法
- 下推自动机
- 上下文无关语法与下推自动机的等价性
- 非上下文无关语言

1 上下文无关语法

上下文无关语法-引例

一个上下文无关的语法 G_1 :

$$S \rightarrow 0S1$$

$$S \rightarrow T$$

$$T \rightarrow \varepsilon$$

- 每一行代表一个替换规则.
- 箭头左边的符号称为变量.
- 箭头右边除变量之外的符号称为终止符.
- 最上方替换规则左边的变量称为起始变量.

S, T

$0, 1$

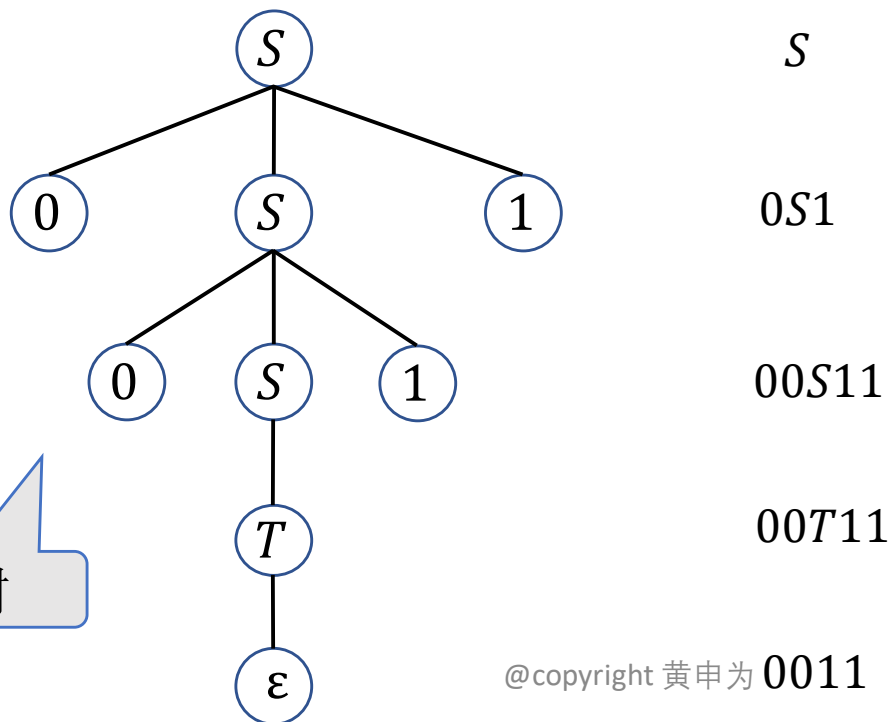
S

上下文无关语法-引例

上下文无关的语法产生字符串：

- 写下初始变量.
- 取一个已写下的变量，并找到以该变量开始的规则，把这个变量替换成规则右边的字符串.
- 重复上述步骤直到字符串中没有任何变量.

例.



G_1 :

$S \rightarrow 0S1$

$S \rightarrow T$

$T \rightarrow \epsilon$

解析树

上下文无关语法-引例

上下文无关的语法产生字符串：

- 写下初始变量.
- 取一个已写下的变量，并找到以该变量开始的规则，把这个变量替换成规则右边的字符串.
- 重复上述步骤直到字符串中没有任何变量.

练习. 下列哪些字符串可由 G_2 生成?

(a) 001101

(b) 000111

(c) 1010

(d) 空字符串 ε

G_2 :

$S \rightarrow RR$

$R \rightarrow 0R1$

$R \rightarrow \varepsilon$

上下文无关语法-正式定义

定义(上下文无关语法). 一个上下文无关的语法是一个4-元组 (V, Σ, R, S) , 其中

- V 是一个有限集, 称为变量集.
- Σ 是与 V 不交的有限集, 称为终止符集合.
- R 是一个有限替换规则的集合(规则形式: $V \rightarrow (V \cup \Sigma)^*$).
- $S \in V$ 是起始变量.

定义(生成和派生).

- 给定 $u, v \in (V \cup \Sigma)^*$ 以及替换规则 $A \rightarrow w$, 称 uAv 生成 uwv , 记作 $uAv \Rightarrow uwv$.
- 若存在 $u_0, u_1, \dots, u_k$ ($k \geq 0$)使得

$$u = u_0 \Rightarrow u_1 \Rightarrow \dots \Rightarrow u_k = v$$

则称 u 派生 v , 记作 $u \Rightarrow^* v$.

给定上下文无关语法 G , 所有由 G 生成的语言为

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}.$$

上下文无关语法-正式定义

例 1:

G_1 :

$S \rightarrow 0S1$

$S \rightarrow T$

$T \rightarrow \varepsilon$

$$G_1 = (V, \Sigma, R, S)$$

- $V = \{S, T\}$.
- $\Sigma = \{0, 1\}$.
- G_1 有三条替换规则.
- S 是初始变量.

$$0S1 \xRightarrow{*} 01$$

$$L(G_1) = \{0^k 1^k \mid k \geq 0\}$$

上下文无关语法-正式定义

例 2: $G_2 = (\{S\}, \{a, b\}, R, S)$, 其中替换规则为

$$S \rightarrow aSb \mid SS \mid \varepsilon.$$

该语法生成字符串 $\varepsilon, ab, aabb, abab, aababb$ 等等.

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb.$$

$$S \Rightarrow aSb \Rightarrow aSSb \overset{*}{\Rightarrow} aaSbaSbb \overset{*}{\Rightarrow} aababb.$$

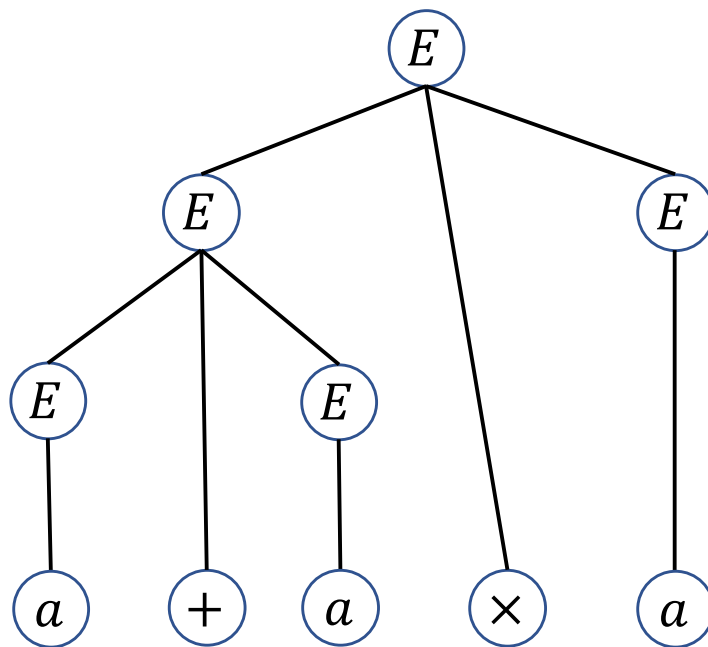
若把 a 看成 $($, b 看成 $)$, 则 $L(G_2)$ 是所有括号正常嵌套的字符串构成的语言.

上下文无关语法-正式定义

例 3: $G_3 = (\{E\}, \{a, +, \times, (,)\}, R, E)$, 其中替换规则为

$$E \rightarrow E + E | E \times E | (E) | a.$$

给出生成字符串 $a + a \times a$ 的解析树.



上下文无关语法-正式定义

练习. 确定由下列语法生成的语言.

$$S \rightarrow T1$$

$$T \rightarrow T0|T1|\varepsilon$$

$$L = \{w|w \text{以} 1 \text{结尾}\}.$$

上下文无关语法-设计

设计CFG的常用方法.

- 考察子串
- 化繁为简
- 利用正则

上下文无关语法-设计

考察字符串：语言中的字符串有两个子串是相互“关联”的，需要记住第一个子串的信息(需要无限内存)去验证另一个子串确实与之“关联”。

例： $L = \{0^k 1^k \mid k \geq 0\}$.

- 需要记住字符0的个数去验证1的个数与之相等.
- 使用替换规则 $S \rightarrow 0S1$ 来验证0的个数与1的个数相等.

$$S \rightarrow 0S1 \mid \varepsilon$$

上下文无关语法-设计

化繁为简：所要生成的语言是一些简单语言的并，则可以分别设计这些简单语言的语法，再通过替换规则

$$S \rightarrow S_1 | S_2 | \cdots | S_k,$$

就可生成给定语言.

例： $L = \{0^k 1^k \mid k \geq 0\} \cup \{1^k 0^k \mid k \geq 0\}$.

- $S_1 \rightarrow 0S_1 1 \mid \varepsilon$

- $S_2 \rightarrow 1S_2 0 \mid \varepsilon$



$$S \rightarrow S_1 | S_2$$

$$S_1 \rightarrow 0S_1 1 \mid \varepsilon$$

$$S_2 \rightarrow 1S_2 0 \mid \varepsilon$$

上下文无关语法-设计

利用正则. 任何有限自动机都能够转化为一个等价的上下文无关语法.

给定有限自动机 $M = (Q, \Sigma, \delta, q_0, F)$, 设计语法 G 如下:

- 令 R_i 是对应状态 q_i 的变量.
- R_0 是初始变量.
- 若 $\delta(q_i, a) = q_j$, 则有相应的替换规则

$$R_i \rightarrow aR_j.$$

- 若 q_i 是一个接受状态, 则添加替换规则 $R_i \rightarrow \varepsilon$.

则 $L(G) = L(M)$.

上下文无关语法-设计

练习. 设计生成 $L = \{w|w\text{至少包含三个}1\}$ 的语法.

$$S \rightarrow TTT$$

$$T \rightarrow TR|RT|1$$

$$R \rightarrow 0|1|\varepsilon$$

上下文无关语法-歧义性

定义(最左派生). 对于语法 G 中的字符串 w 的派生, 如果每一步都是替换最左边的变量, 则称这个派生为最左派生.

例: 考虑 $G_3 = (\{E\}, \{a, +, \times, (,)\}, R, E)$ 中的字符串 $a + a \times a$, 其中替换规则为

$$E \rightarrow E + E \mid E \times E \mid (E) \mid a.$$

- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a.$
- $E \Rightarrow E + E \Rightarrow E + E \times E \Rightarrow \dots \Rightarrow a + a \times a.$

否

是

上下文无关语法-歧义性

定义(歧义性). 如果字符串 w 在上下文无关语法 G 中有两个或两个以上不同的最左派生, 则称 G 歧义地产生字符串 w . 如果文法 G 歧义地产生某个字符串, 则称 G 是歧义的.

例: $G_3 = (\{E\}, \{a, +, \times, (,)\}, E \rightarrow E + E | E \times E | (E) | a, E)$ 歧义地产生字符串 $a + a \times a$, 从而 G_3 是歧义的.

- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a.$
- $E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a.$

理解. 第一种方式是先做加法再做乘法, 而第二种方式是先做乘法再做加法.

上下文无关语法-乔姆斯基范式

定义(乔姆斯基范式). 称一个上下文无关语法为乔姆斯基范式 (Chomsky normal form), 如果它的每一个规则具有如下形式:

$$A \rightarrow BC$$

$$A \rightarrow a$$

其中 a 是任意的终结符, A, B, C 是任意的变量, 且 B 和 C 不能为起始变量.

此外允许规则 $S \rightarrow \varepsilon$, 其中 S 是起始变量.

上下文无关语法-乔姆斯基范式

定理(乔姆斯基范式). 任一上下文无关语言都可以用一个乔姆斯基范式的上下文无关语法产生.

证明. 增加一个新的起始变量 S_0 以及替换规则 $S_0 \rightarrow S$. 这样保证了起始变量不会出现在任何替换规则的右边.

根据右端字符串长度 ℓ 分类.

- $\ell = 0$ 移除 ε -规则 $A \rightarrow \varepsilon$: 对于每条右端出现 A 的规则, 增加一条相应的规则, 该规则将 A 从右端中移除.

$$R \rightarrow uAvAw \xrightarrow{\text{增加}} R \rightarrow uvAw, R \rightarrow uAvw, R \rightarrow uvw$$

- $\ell = 1$ 移除单一生产式 $A \rightarrow B$: 对于每条形如 $B \rightarrow u$ 的规则, 增加规则 $A \rightarrow u$.

上下文无关语法-乔姆斯基范式

定理(乔姆斯基范式). 任一上下文无关语言都可以用一个乔姆斯基范式的上下文无关语法产生.

证明(续). 增加一个新的起始变量 S_0 以及替换规则 $S_0 \rightarrow S$. 这样保证了起始变量不会出现在任何替换规则的右边.

根据右端字符串长度 ℓ 分类.

- $\ell \geq 3$ 移除 $A \rightarrow u_1 u_2 \cdots u_\ell$, u_i 是变量或者终止符.

$$A \rightarrow u_1 u_2 \cdots u_\ell \xrightarrow{\text{替换为}} A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{\ell-2} \rightarrow u_{\ell-1} u_\ell$$

- $\ell = 2$ 移除规则 $A \rightarrow u_1 u_2$:

$$A \rightarrow u_1 u_2 \xrightarrow{\text{替换为}} A \rightarrow U_1 U_2, \quad U_1 \rightarrow u_1, \quad U_2 \rightarrow u_2$$

证毕. ■

上下文无关语法-乔姆斯基范式

例：考虑语法 $G_4 = (\{S, A, B\}, \{a, b\}, R, S)$:

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\varepsilon$$

将其转化成乔姆斯基范式.

第 1 步. 增加新的起始变量.

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\varepsilon$$



$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\varepsilon$$

上下文无关语法-乔姆斯基范式

第2步. 移除 ϵ 规则.

$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\epsilon$$



$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a$$

$$A \rightarrow B|S|\epsilon$$

$$B \rightarrow b|\epsilon$$



$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a|SA|AS|S$$

$$A \rightarrow B|S|\epsilon$$

$$B \rightarrow b$$

上下文无关语法-乔姆斯基范式

第3步. 移除单一生产式.

$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a|SA|AS|S$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$



$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a|SA|AS|S$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$



$$S_0 \rightarrow S|ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$

上下文无关语法-乔姆斯基范式

第3步. 移除单一生产式.

$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$



$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow \textcolor{gray}{B}|S|\textcolor{black}{b}$$

$$B \rightarrow b$$



$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow \textcolor{gray}{S}|b|\textcolor{black}{ASA|aB|a|SA|AS}$$

$$B \rightarrow b$$

上下文无关语法-乔姆斯基范式

第 4 步. 移除其他不符合条件的规则.

$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow b|ASA|aB|a|SA|AS$$

$$B \rightarrow b$$



$$S_0 \rightarrow AA_1|UB|a|SA|AS$$

$$S \rightarrow AA_1|UB|a|SA|AS$$

$$A \rightarrow b|AA_1|UB|a|SA|AS$$

$$B \rightarrow b$$

$$A_1 \rightarrow SA$$

$$U \rightarrow a$$

计算理论-有限自动机

- 有限自动机
- 非确定性有限自动机
- 正则语言
- 非正则语言

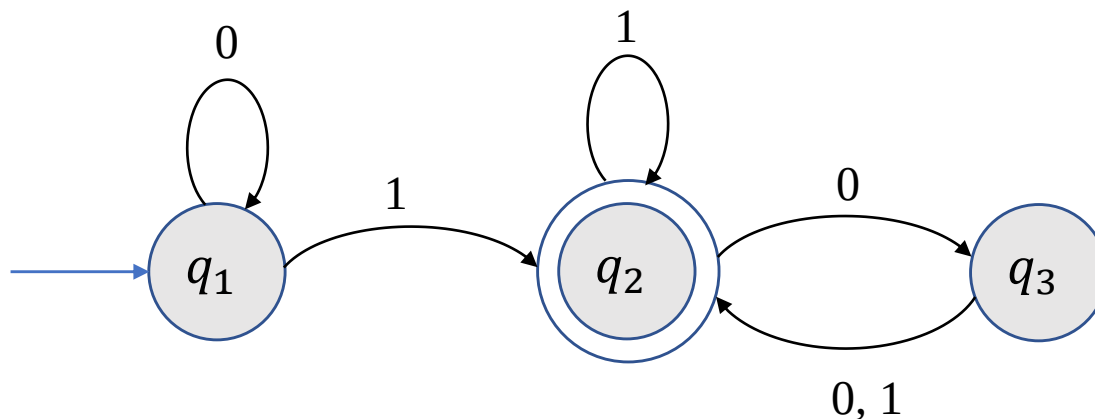
1 有限自动机

有限自动机

自动门控制

- 开和关两个状态
- 事件触发状态的改变

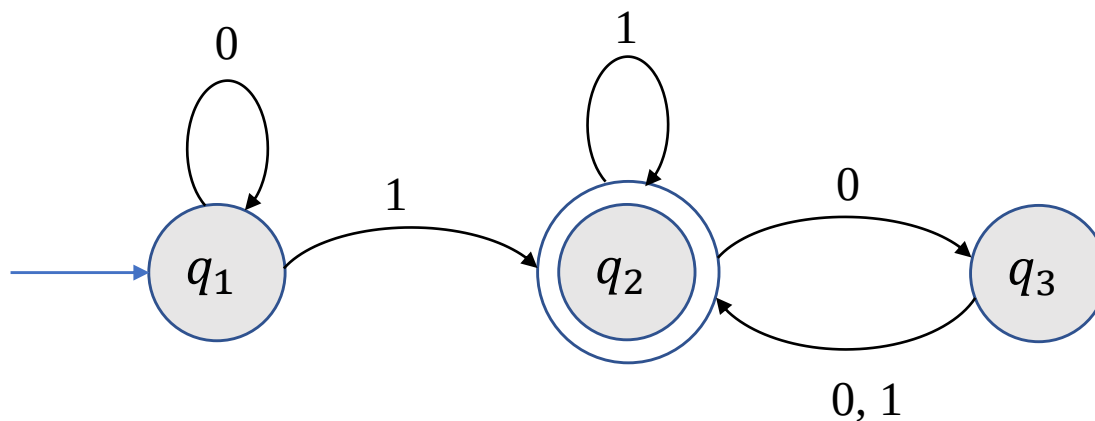
有限自动机-引例



一个有限自动机 M_1

- 圆圈代表状态
 - 蓝色箭头指向的状态称为初始状态
 - 两个圆圈代表接受状态
- 黑色箭头代表转移规则

有限自动机-引例



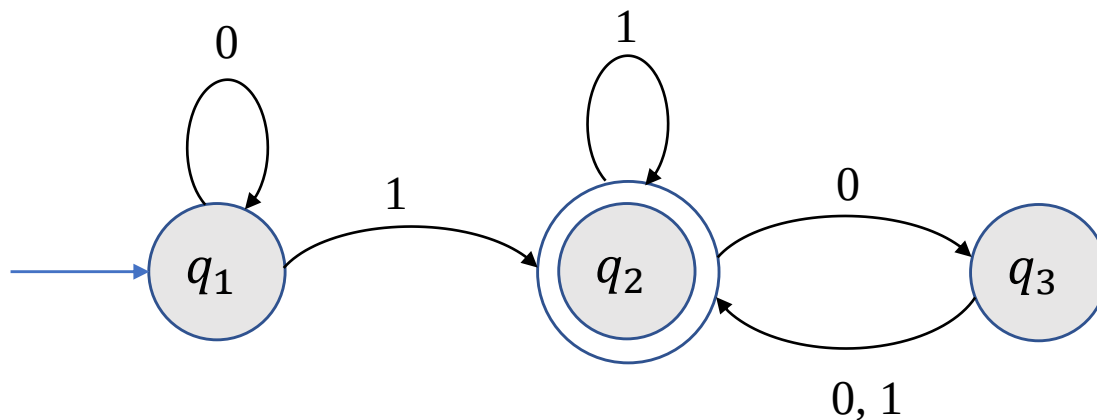
输入: $\{0,1\}$ -字符串.

输出: 接受或拒绝.

计算

- 从左向右依次读取输入字符串中的字符, 根据转移规则从一个状态跳转到下一个状态.
- 若终止状态为接受状态, 输出接受; 否则输出拒绝.

有限自动机-引例



计算

- 从左向右依次读取输入字符串中的字符, 根据转移规则从一个状态跳转到下一个状态.
- 若终止状态为接受状态, 输出接受; 否则输出拒绝.

例: $w = 1101$.

$$q_1 \xrightarrow{1} q_2 \xrightarrow{1} q_2 \xrightarrow{0} q_3 \xrightarrow{1} q_2$$

M_1 接受 w .

M_1 接受 10100?

有限自动机-正式定义

定义(有限自动机). 一个有限自动机是一个5-元有序组 $(Q, \Sigma, \delta, q_0, F)$, 其中

- Q 是一个有限集, 称为状态集.
- Σ 是一个字母表.
- $\delta: Q \times \Sigma \rightarrow Q$ 是转移函数.
- $q_0 \in Q$ 称为初始状态.
- $F \subseteq Q$ 是接受状态集.

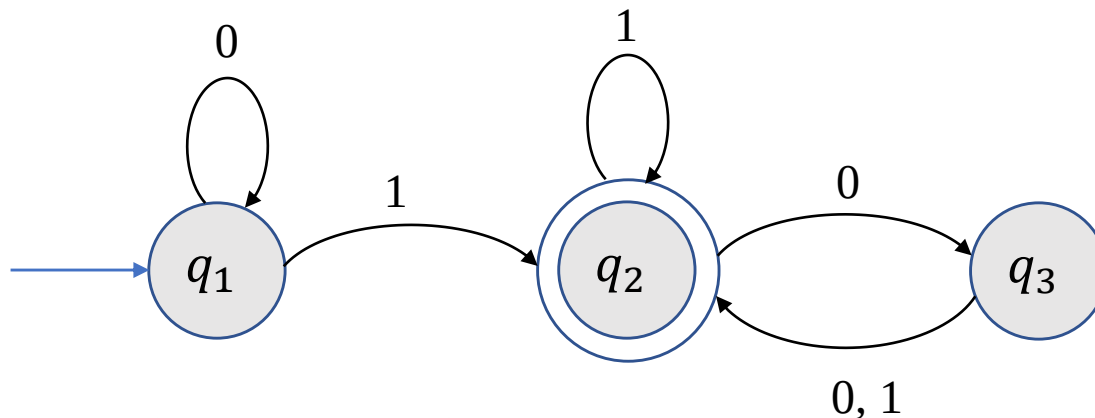
定义(有限自动机的语言). 给定有限自动机 M , 所有 M 接受的字符串的集合 $L(M)$ 称为 M 的语言, 即

$$L(M) = \{w | M \text{ 接受 } w\}.$$

此时也称 M 识别 $L(M)$.

有限自动机-正式定义

例 1.



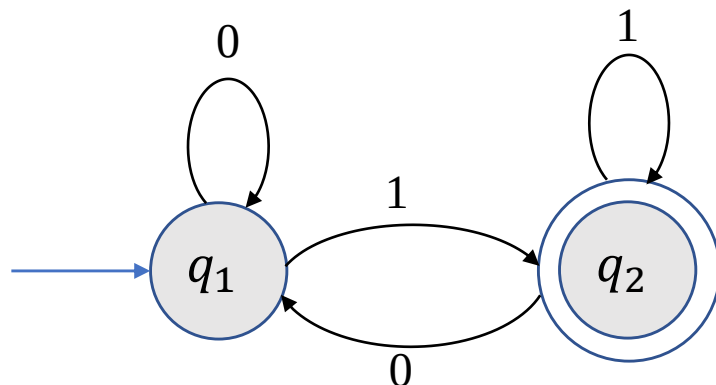
$$M_1 = (Q, \Sigma, \delta, q_1, F)$$

- $Q = \{q_1, q_2, q_3\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :
- q_1 是初始状态.
- 接受状态集 $F = \{q_2\}$.

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

有限自动机-正式定义

例 2.



$$M_2 = (Q, \Sigma, \delta, q_1, \{q_2\})$$

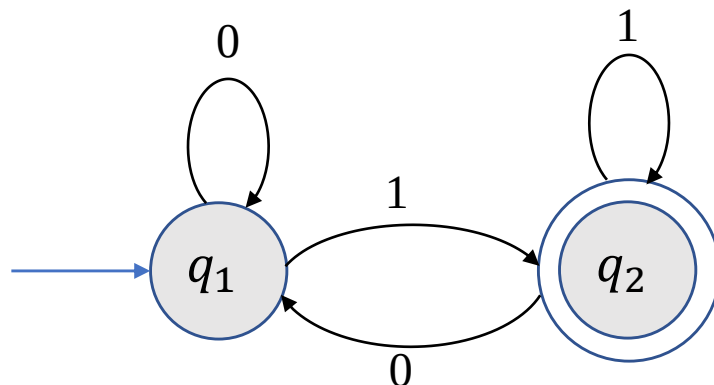
- $Q = \{q_1, q_2\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :

	0	1
q_1	q_1	q_2
q_2	q_1	q_2

$$L(M_2) = ?$$

有限自动机-正式定义

例 2.



$$M_2 = (Q, \Sigma, \delta, q_1, \{q_2\})$$

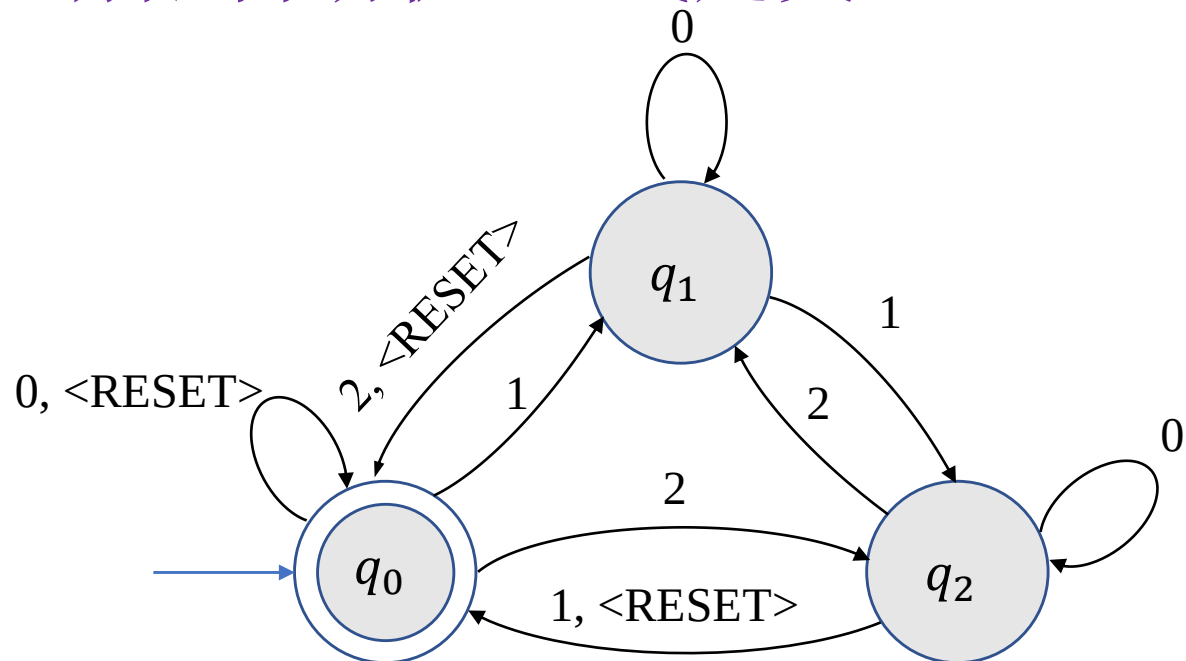
- $Q = \{q_1, q_2\}$.
- $\Sigma = \{0, 1\}$.
- 转移函数 δ :

	0	1
q_1	q_1	q_2
q_2	q_1	q_2

$$L(M_2) = \{w \mid w \text{ 以 } 1 \text{ 结尾}\}.$$

有限自动机-正式定义

例 3.



$$M_3 = (\{q_0, q_1, q_2\}, \Sigma, \delta, q_0, \{q_0\})$$

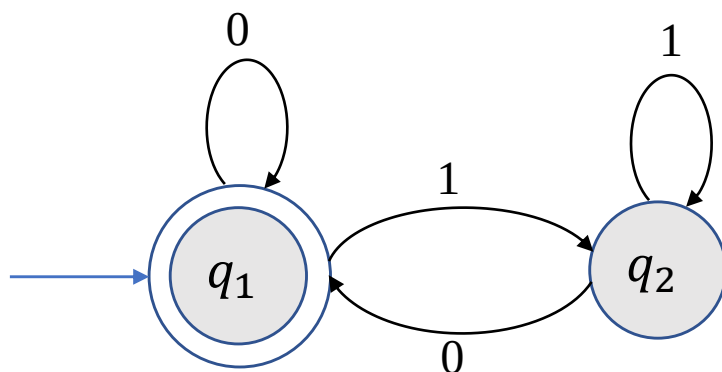
- $\Sigma = \{0, 1, 2, < \text{RESET} >\}$.
- δ : 记录输入字符串数字之和模3后的值.

一旦遇到< RESET > 字符将数值归0.

$L(M_3) = \{w | w \text{中最后一个 } < \text{RESET} > \text{ 字符后所有数字之和被3整除}\}.$

有限自动机-正式定义

练习. 给出下列有限自动机的正式定义并确定其识别的语言.



有限自动机-正式定义

问题：为什么需要形式化的定义？

- 精确描述：给定5-元有序组可以通过定义判定哪些是有限自动机.
- 有时有限自动机的状态图过于庞大，形式化定义更加简洁.
- 有限自动机可以和某个参数相关，每一个参数的取值对应一个有限自动机，因此无法画出无穷多个状态图.

有限自动机-正式定义

例：给定 $\Sigma = \{0,1,2, < \text{RESET} >\}$.

$L_i = \{w | w \text{中最后一个 } < \text{RESET} > \text{ 字符后所有数字之和被 } i \text{ 整除}\}.$

给出一个有限自动机 M_i 识别 L_i .

解： $M_i = (\{q_0, q_1, \dots, q_{i-1}\}, \Sigma, \delta_i, q_0, \{q_0\})$, 其中 δ_i 满足如果 M_i 处于状态 q_j , 则已读数字之和模 i 等于 j .

- $\delta_i(q_j, < \text{RESET} >) = q_0.$
- $\delta_i(q_j, 0) = q_j.$
- $\delta_i(q_j, 1) = q_k$, 其中 $k \equiv j + 1 \pmod{i}.$
- $\delta_i(q_j, 2) = q_k$, 其中 $k \equiv j + 2 \pmod{i}.$

有限自动机-计算的定义

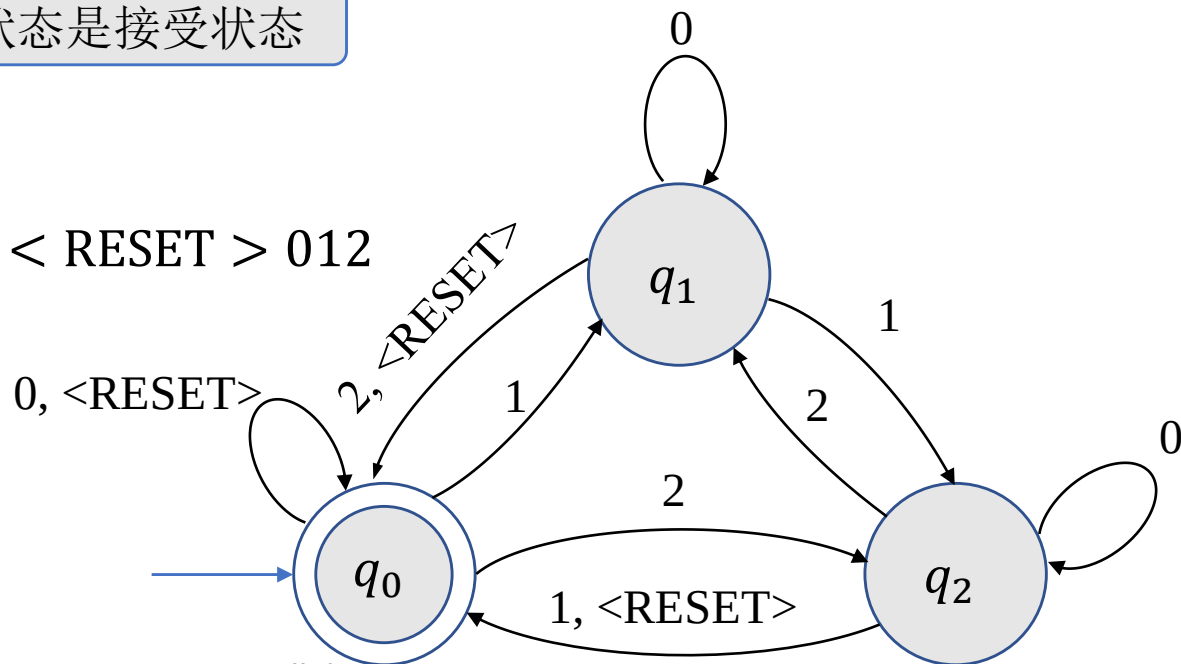
定义(计算). 设 $M = (Q, \Sigma, \delta, q_0, F)$ 是一个有限自动机, $w = w_1w_2 \cdots w_n$ 是 Σ 上的字符串. 如果存在状态序列 $r_0, r_1, \dots, r_n \in Q$ 使得

- $r_0 = q_0$; 从初始状态开始
- $\delta(r_i, w_{i+1}) = r_{i+1}, i = 0, 1, \dots, n-1$; 状态根据转移函数跳转
- $r_n \in F$, 终止状态是接受状态

则称 M 接受 w .

例. 给出 M_3 计算 $w = 10 \langle \text{RESET} \rangle 012$

时进入的状态序列.



有限自动机-设计

问题：如何设计识别给定语言 L 的有限自动机？

回答：将自己想象成有限自动机.

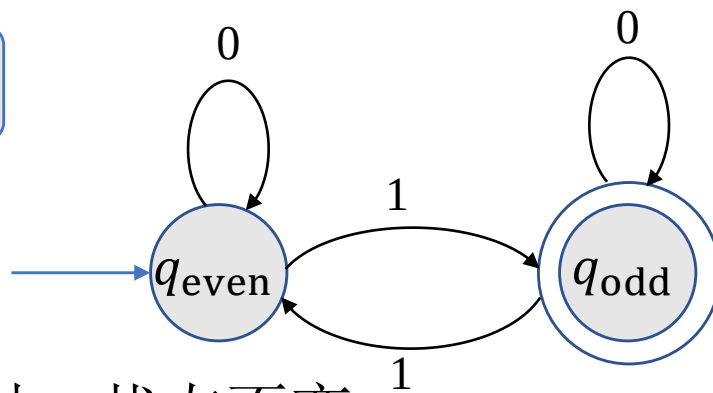
有限自动机-设计

例 1：设计识别 $\Sigma = \{0,1\}$ 上的语言 $L = \{w|w\text{包含奇数个}1\}$ 的有限自动机 E_1 .

解：当读取输入字符串时，需要记录什么信息？

不是1的数目，而是1的
数目的奇偶性！

- 因此 E_1 有两个状态 q_{odd} 和 q_{even} .
- 当读到1时，状态翻转；当读到0时，状态不变.
- 接受状态为 q_{odd} ，初始状态为 q_{even} (对应空字符串).

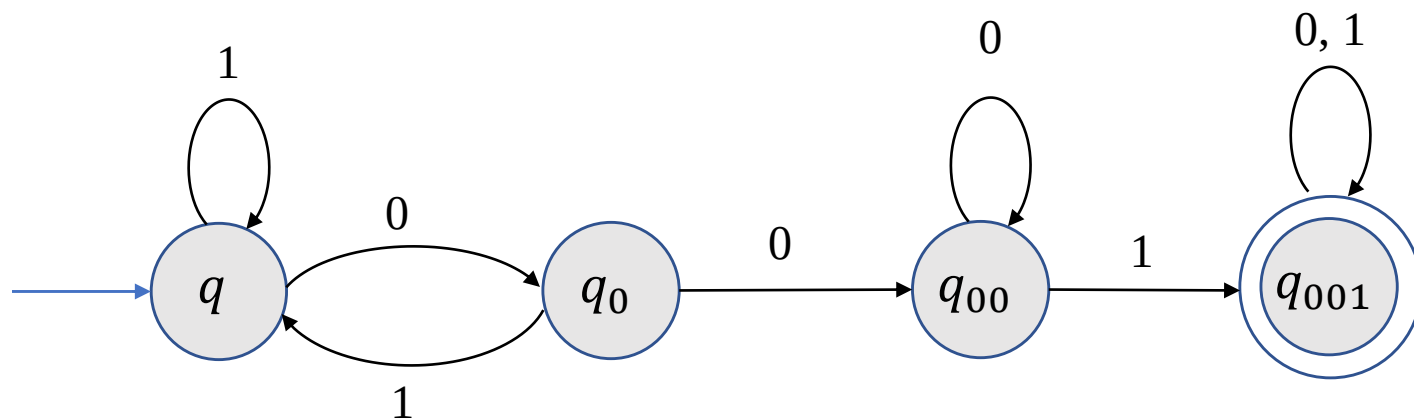


有限自动机-设计

例 2: 设计识别 $\Sigma = \{0,1\}$ 上的语言

$$L = \{w \mid w \text{ 包含 } 001 \text{ 作为子串}\}$$

的有限自动机 E_2 .



有限自动机-正式定义

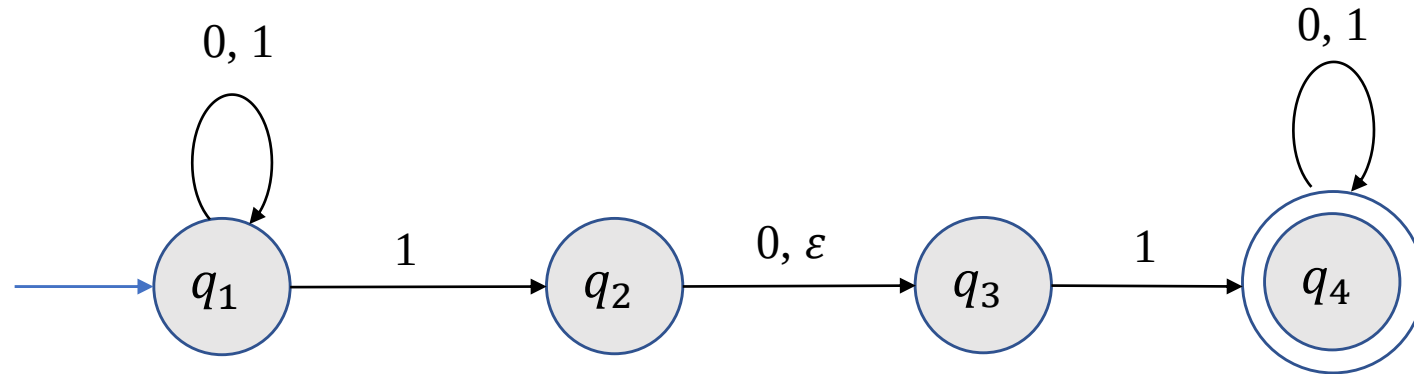
练习. 设计识别 $\Sigma = \{0,1\}$ 上的语言

$$L = \{w | w \text{ 包含至少三个 } 1\}$$

的有限自动机 E_3 .

2 非确定性有限自动机

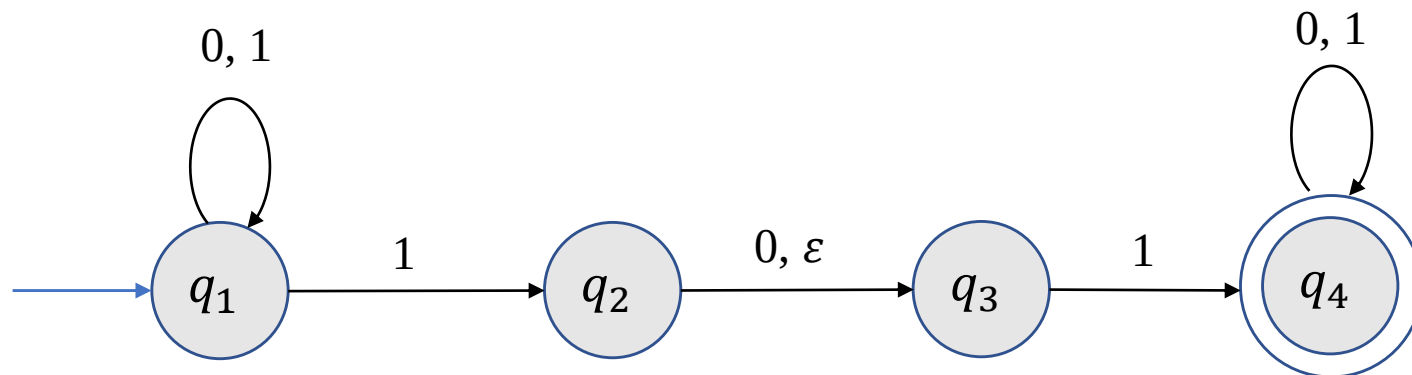
非确定性有限自动机-引例



非确定性的特点.

- 转移规则：一入多出.
- ϵ 箭头：无需读入字符的状态转移.

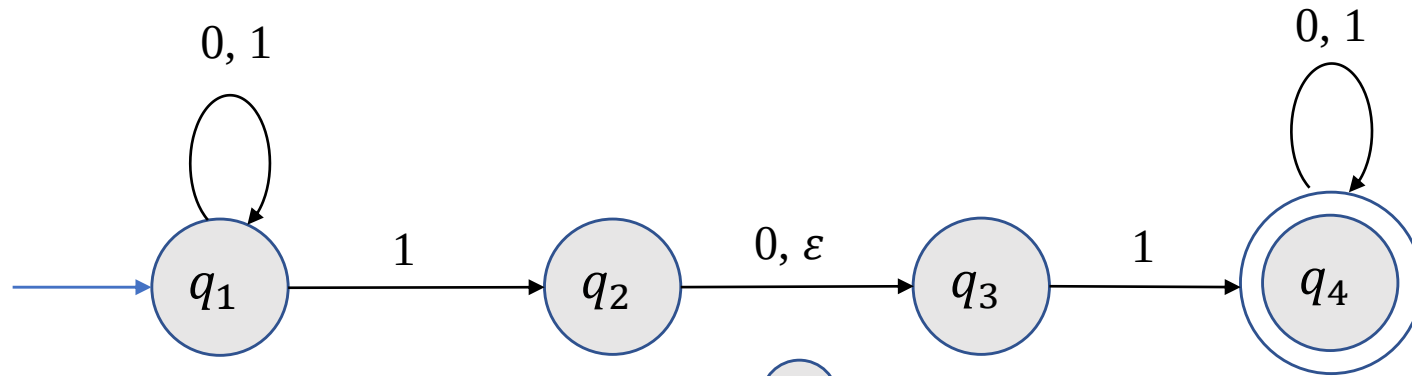
非确定性有限自动机-引例



计算.

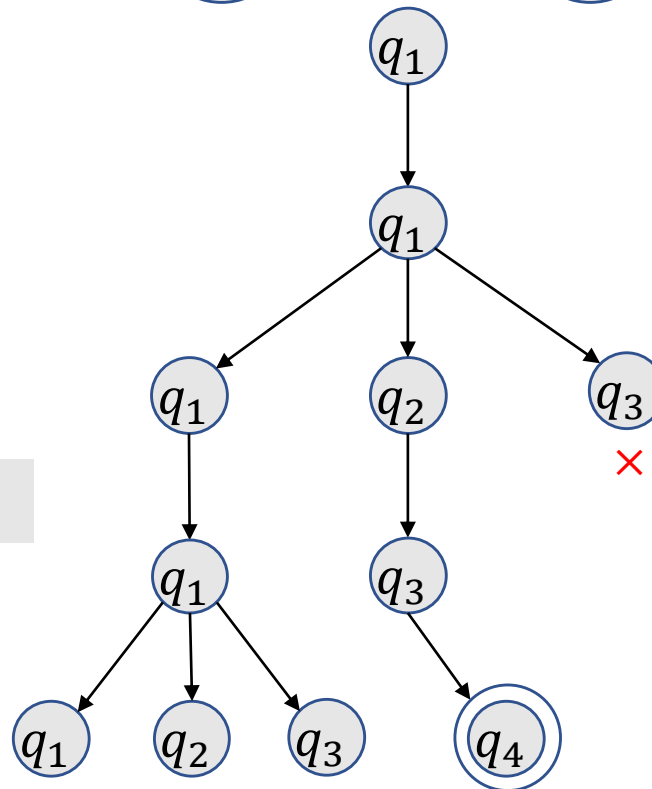
- 如果机器处于给定状态读入给定字符有多个出去的箭头，则机器复制成多份，每一份对应一个可能的状态.
- 如果机器处于给定状态且出去的箭头上有 ε 符号，则机器复制一份，其状态为箭头所指状态.
- 如果有一个计算分支接受，则机器接受.

非确定性有限自动机-引例



例: $w = 0101$.

N_1 接受 1001?



----- 0

----- 1

----- 0

----- 1

非确定性有限自动机-正式定义

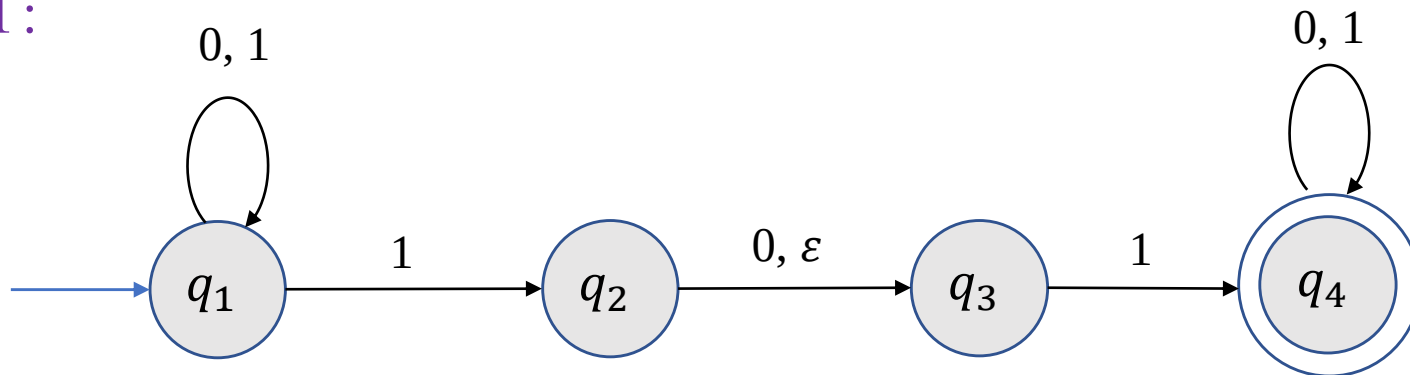
给定字母表 Σ , 定义 $\Sigma_\epsilon \triangleq \Sigma \cup \{\epsilon\}$.

定义(非确定性有限自动机). 一个非确定性有限自动机是一个5-元有序组 $(Q, \Sigma, \delta, q_0, F)$, 其中

- Q 是一个有限集, 称为状态集.
- Σ 是一个字母表.
- $\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ 是转移函数.
- $q_0 \in Q$ 称为初始状态.
- $F \subseteq Q$ 是接受状态集.

非确定性有限自动机-正式定义

例 1:



$$N_1 = (Q, \Sigma, \delta, q_1, F),$$

- $Q = \{q_1, q_2, q_3, q_4\}$;
- $\Sigma = \{0, 1\}$;
- q_1 是初始状态;
- $F = \{q_4\}$;
- 转移函数 δ :

	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

非确定性有限自动机-计算的定义

定义(计算). 设 $N = (Q, \Sigma, \delta, q_0, F)$ 是一个非确定性有限自动机, w 是 Σ 上的字符串. 如果能将 w 写成 $w = y_1 y_2 \cdots y_m$, 其中每个 $y_i \in \Sigma_\varepsilon$ 且存在状态序列 $r_0, r_1, \dots, r_m \in Q$ 使得

- $r_0 = q_0$; 可能的状态之一
- $r_{i+1} \in \delta(r_i, y_{i+1}), i = 0, 1, \dots, m - 1$;
- $r_m \in F$,

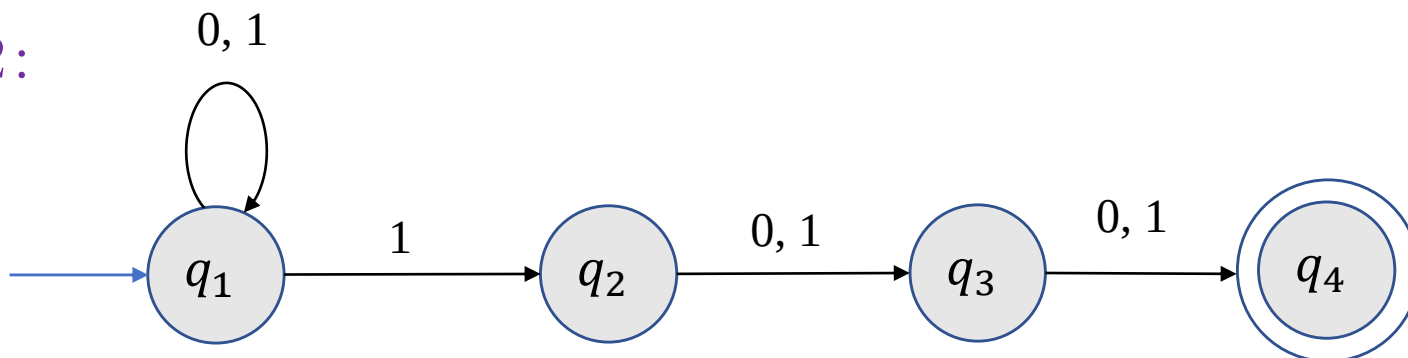
则称 N 接受 w .

例 1(续): 给出 N_1 接受 $w = 11$ 时 Σ_ε 上的序列以及相应的状态序列.

解: 当 w 写成 $w = 1 \varepsilon 1$, 相应的状态序列为 q_1, q_2, q_3, q_4 .

非确定性有限自动机-正式定义

例 2:



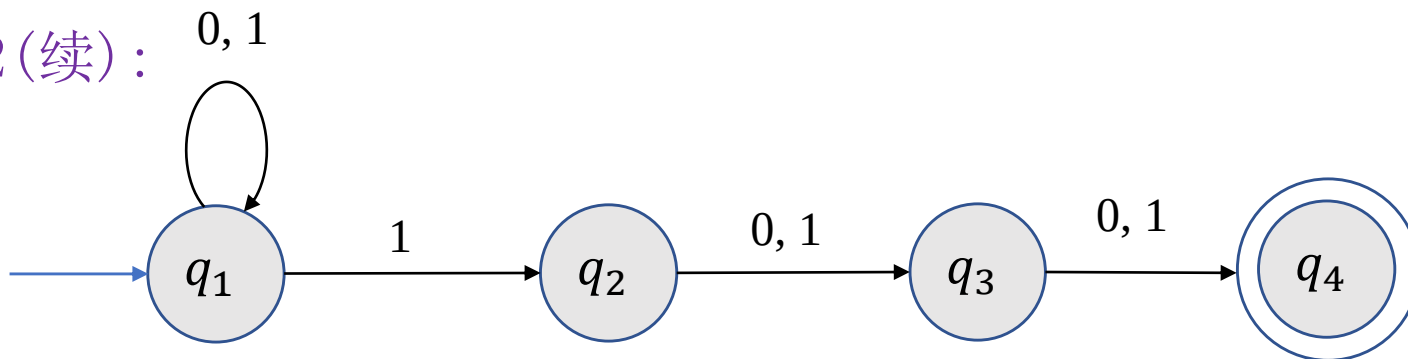
$$N_2 = (Q, \Sigma, \delta, q_1, \{q_4\}),$$

- $Q = \{q_1, q_2, q_3, q_4\}$;
- $\Sigma = \{0, 1\}$;
- 转移函数 δ :

	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\{q_3\}$	$\emptyset$
q_3	$\{q_4\}$	$\{q_4\}$	$\emptyset$
q_4	$\emptyset$	$\emptyset$	$\emptyset$

非确定性有限自动机-正式定义

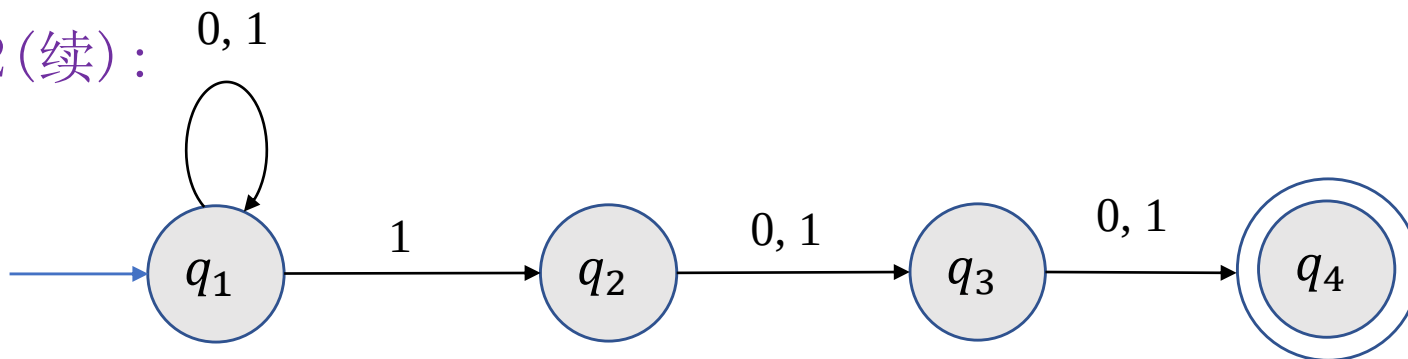
例 2(续):



$$L(N_2) = ?$$

非确定性有限自动机-正式定义

例 2(续):



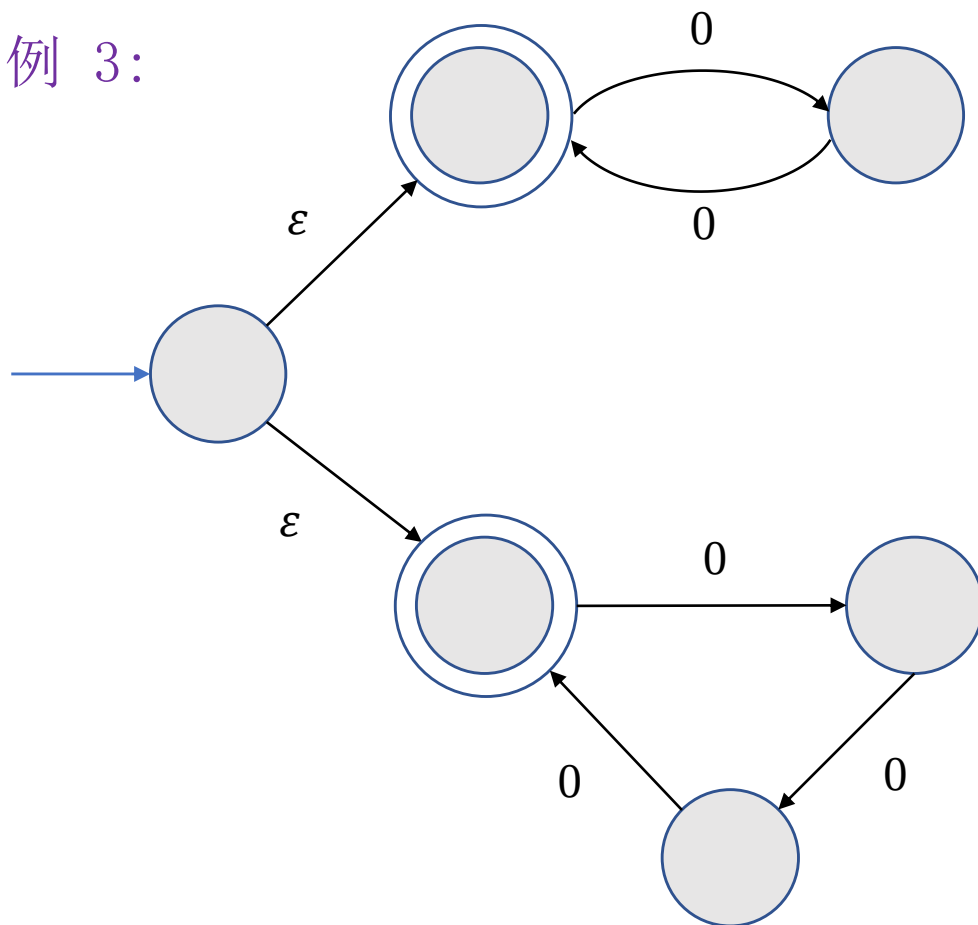
$$L(N_2) = \{w | w \text{ 倒数第3个位置上是1}\}.$$

- 机器一直处于 q_1 状态并“猜测”读到字符1是倒数第3个字符. 此时, 复制一份机器并处于 q_2 状态.
- 用状态 q_3 和 q_4 检查“猜测”是否正确.

若将 q_2 至 q_3 以及 q_3 至 q_4 的箭头上增加 ϵ 符号, 这样修改后的机器识别的语言是什么?

非确定性有限自动机-正式定义

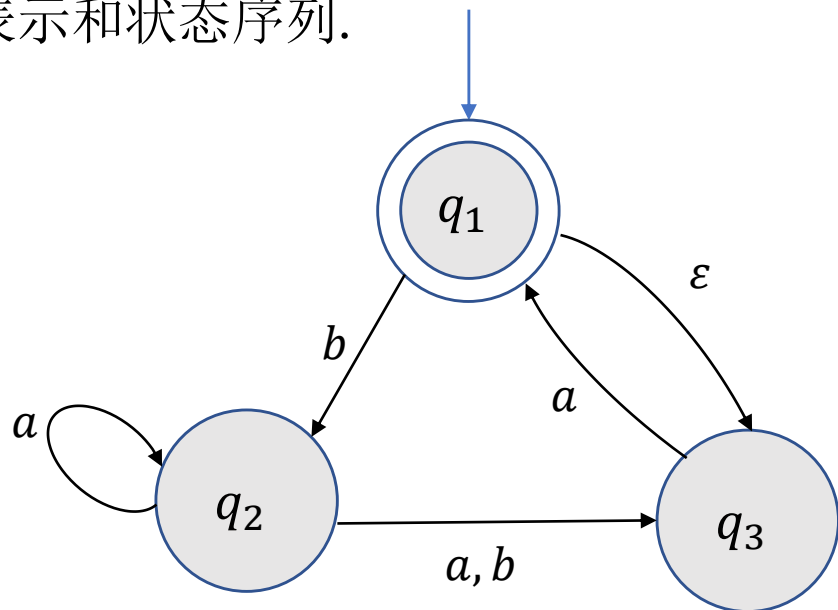
例 3:



$$L(N_3) = \{0^k \mid k \text{ 能被 } 2 \text{ 或 } 3 \text{ 整除}\}.$$

非确定性有限自动机-正式定义

例 4: 下列非确定性有限自动机 N_4 是否接受 $baba, a, bb$? 若接受, 给出相应的字符表示和状态序列.



- N_4 接受 a :

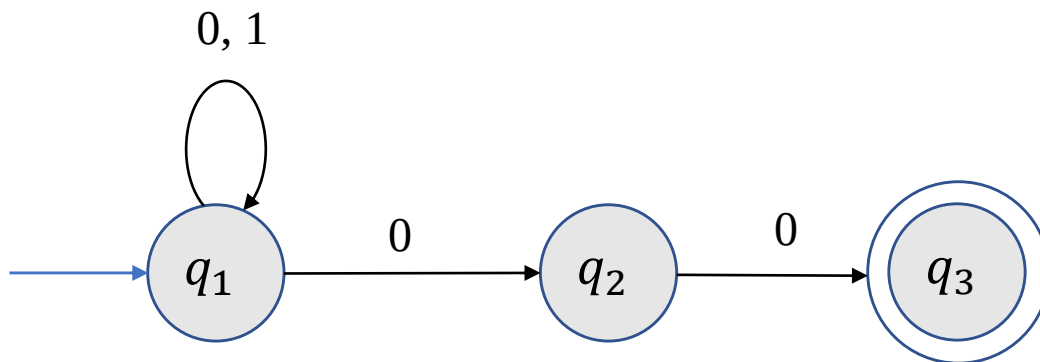
$$\epsilon a \rightarrow q_1, q_3, q_1.$$

- N_4 不接受 bb .

- N_4 接受 $baba$.

非确定性有限自动机-正式定义

练习.



- 给出上述非确定性有限自动机 N 的形式化定义.
- N 是否接受 $\varepsilon, 001, 100$? 若接受, 给出相应字符串表示以及状态序列.
- N 识别的是什么语言?

非确定性有限自动机-与DFA的等价性

定义. 若有限自动机 M 和非确定性有限自动机 N 识别相同的语言, 则称 M 和 N 是等价的.

定理. 每个非确定性有限自动机 N 等价于一个确定性有限自动机 M .

证明思路. 想象自己是一个有限自动机, 需要记录哪些信息?

- N 计算时需要记录每一步机器所有可能处于的状态.
- 因此 M 需要记录 N 的状态集的子集.

非确定性有限自动机-与DFA的等价性

定理. 每个非确定性有限自动机 N 等价于一个确定性有限自动机 M .

证明. 令 $N = (Q, \Sigma, \delta, q_0, F)$ 是一个非确定性有限自动机, 构造有限自动机 $M = (Q', \Sigma, \delta', q_0', F')$ 如下.

先考虑 N 没有 ϵ 箭头的情况.

- $Q' = \mathcal{P}(Q)$.
- 对任何 $R \in Q', a \in \Sigma$,

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).$$

- $q_0' = \{q_0\}$.
- $F' = \{R \in Q' \mid R \text{ 包含 } F \text{ 中的某个状态}\}.$

非确定性有限自动机-与DFA的等价性

定理. 每个非确定性有限自动机 N 等价于一个确定性有限自动机 M .

证明(续). 再考虑 N 有 ε 箭头的情况.

对任何 $R \in Q'$, 定义 $E(R)$ 是所有能够通过 ε 箭头到达的状态集合包括 R 本身, 即

$$E(R) = \{q \mid \text{从} R \text{出发沿着} 0 \text{或者多个} \varepsilon \text{箭头可以到达} q\}.$$

- $Q' = \mathcal{P}(Q)$.
- 对任何 $R \in Q', a \in \Sigma$,

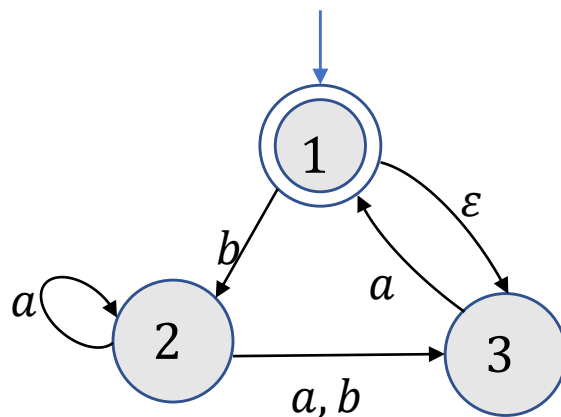
$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a) \quad \rightarrow \delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a)).$$

- $q'_0 = \{q_0\}$. $\rightarrow q'_0 = E(\{q_0\})$.
- $F' = \{R \in Q' \mid R \text{包含} F \text{中的某个状态}\}.$



非确定性有限自动机-与DFA的等价性

例. 将 N_4 按定理证明的方式转换成一个等价的确定性有限自动机.



	a	b
$\{1\}$	$\emptyset$	$\{2\}$
$\{2\}$	$\{2,3\}$	$\{3\}$
$\{3\}$	$\{1,3\}$	$\emptyset$

3 正则语言

正则语言

定义(正则语言). 对字母表上的语言 L , 若存在(非确定性)有限自动机 M 使得 $L = L(M)$, 则称 L 是正则语言.

定义(正则操作). 给定语言 A, B , 定义正则运算并, 连接和星号如下:

- 并: $A \cup B = \{s | s \in A \text{ 或 } s \in B\}.$
- 连接: $A \circ B = \{xy | x \in A, y \in B\}.$
- 星号: $A^* = \{x_1 x_2 \cdots x_k | k \geq 0 \text{ 且每个 } x_i \in A\}.$

例: 设 $\Sigma = \{a, b, \dots, z\}$. 令 $A = \{\text{good}, \text{bad}\}, B = \{\text{boy}, \text{girl}\}$, 求 $A \cup B, A \circ B, A^*$.

解: $A \cup B = \{\text{good}, \text{bad}, \text{boy}, \text{girl}\}.$

$A \circ B = \{\text{goodboy}, \text{goodgirl}, \text{badboy}, \text{badgirl}\}.$

$A^* = \{\epsilon, \text{good}, \text{bad}, \text{goodgood}, \text{goodbad}, \text{badgood}, \text{badbad}, \dots\}.$

正则语言

定理. 正则语言对于正则运算封闭.

正则语言

定理. 正则语言对于并运算封闭.

证明. 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \cup A_2$ 也是正则语言.

假设 $M_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的DFA. 构造有限自动机 M 识别 $A_1 \cup A_2$.

- 自然的想法是将给定字符 s 串输入 M_1 , 判定 M_1 是否接受 s . 如果接受, 则 M 接受 s ; 否则将 s 输入 M_2 , 若 M_2 接受 s , 则 M 接受 s ; 否则 M 拒绝 s .
- 问题在于只能读取输入一遍!
- 所以当 M 读取字符串时, 需要同时记录 M_1 和 M_2 所处的状态.

M 的状态是一个有序对, 分别记录 M_1 和 M_2 所处的状态.

正则语言

定理. 正则语言对于并运算封闭.

证明(续). 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \cup A_2$ 也是正则语言.

假设 $M_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的DFA. 构造有限自动机 M 识别 $A_1 \cup A_2$.

M 的状态是一个有序对, 分别记录 M_1 和 M_2 所处的状态.

$M = (Q, \Sigma, \delta, q_0, F)$:

- $Q = Q_1 \times Q_2$.
- 任给 $(r_1, r_2) \in Q, a \in \Sigma$,

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a)).$$

- $q_0 = (q_1, q_2)$.
- $F = (Q_1 \times F_2) \cup (F_1 \times Q_2)$.



正则语言

思考：如何修改上述证明推出正则语言对交运算封闭？

正则语言

定理. 正则语言对于连接运算封闭.

证明. 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \circ A_2$ 也是正则语言.

假设 $M_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的DFA. 构造有限自动机 M 识别 $A_1 \circ A_2$.

- M 需要识别的字符串 s 可以分割成两段 $s = s_1 s_2$ 使得 M_1 接受 s_1 且 M_2 接受 s_2 .
- 问题在于预先不知道在哪个位置分割!

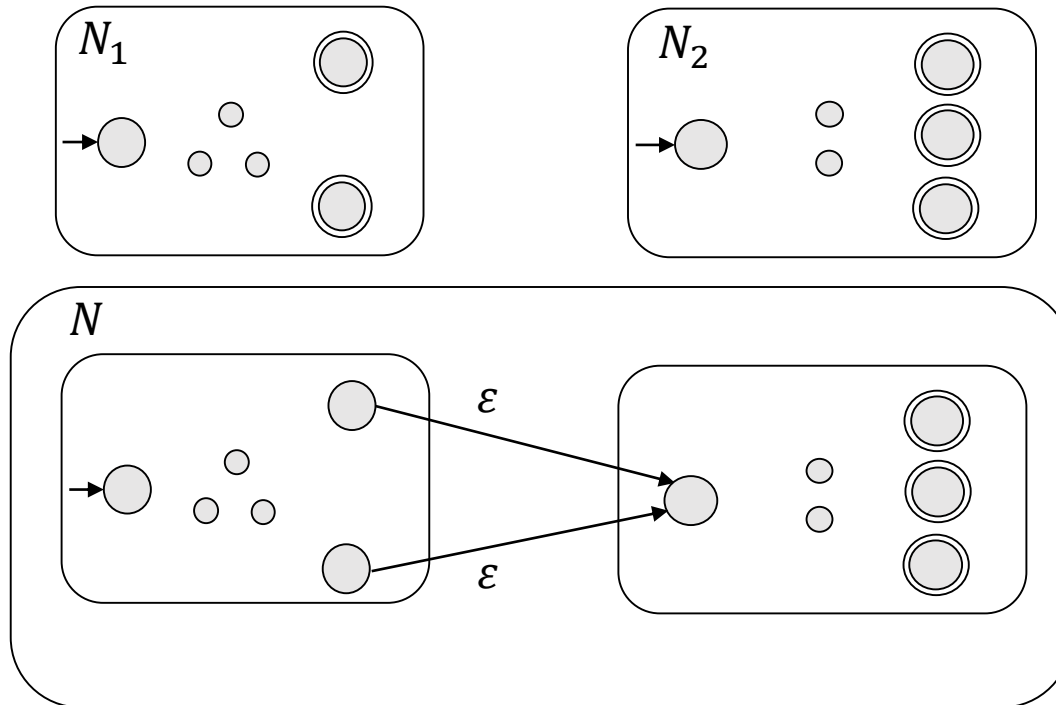
需要非确定性!

正则语言

定理. 正则语言对于连接运算封闭.

证明. 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \circ A_2$ 也是正则语言.

假设 $N_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的NFA. 构造有限自动机 $N = (Q, \Sigma, \delta, q, F)$ 识别 $A_1 \circ A_2$.



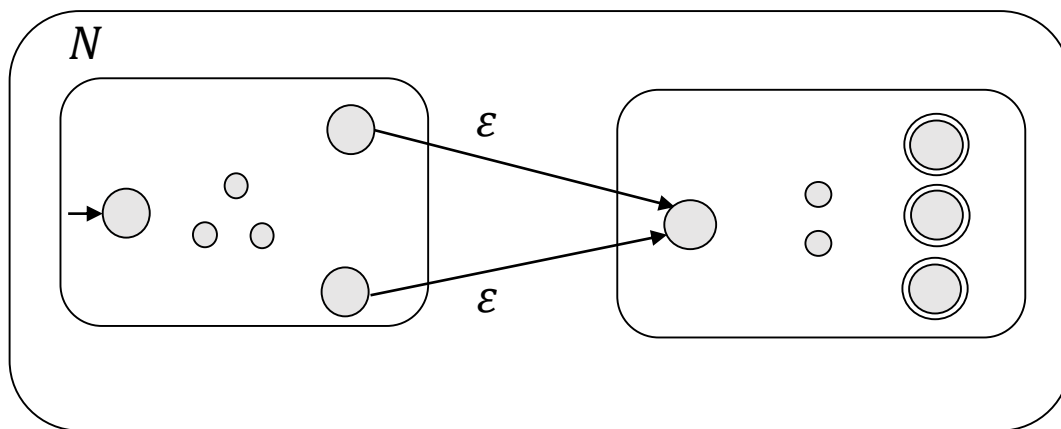
正则语言

定理. 正则语言对于连接运算封闭.

证明. 设 A_1, A_2 是两个正则语言, 需要证明 $A_1 \circ A_2$ 也是正则语言.

假设 $N_i = (Q_i, \Sigma, \delta_i, q_i, F_i), i = 1, 2$ 是识别 A_i 的NFA. 构造有限自动机 $N = (Q, \Sigma, \delta, q, F)$ 识别 $A_1 \circ A_2$.

- $Q = Q_1 \cup Q_2$.
- $q_0 = q_1$.
- $F = F_2$.
- 任给 $a \in \Sigma_\varepsilon, q \in Q$,



$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ 且 } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ 且 } a \notin \varepsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ 且 } a = \varepsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

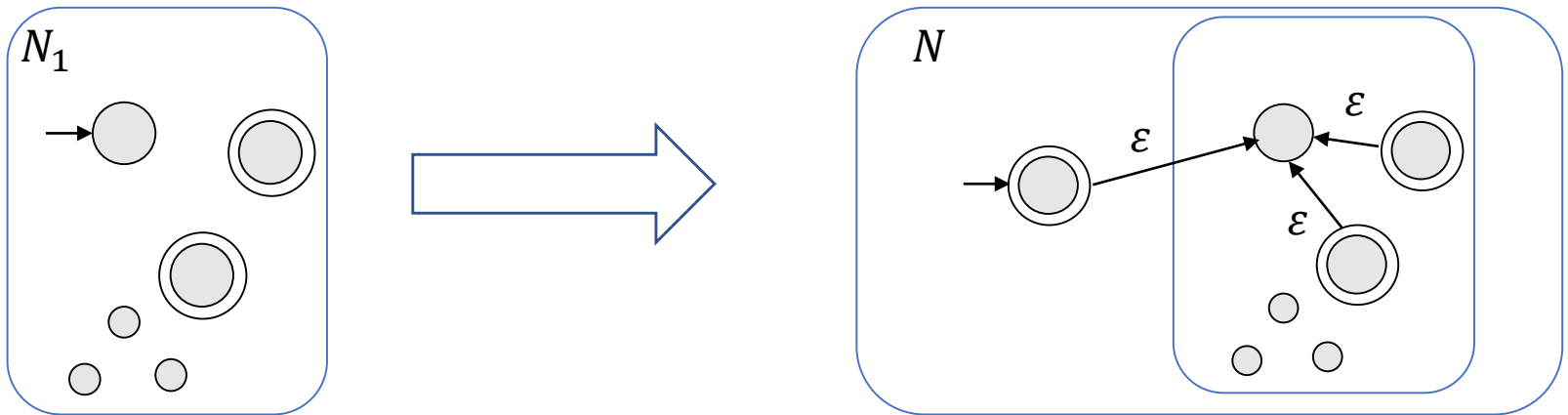
显然 $N = (Q, \Sigma, \delta, q, F)$ 识别 $A_1 \circ A_2$.

正则语言

定理. 正则语言对于星号运算封闭.

证明. 设 A_1 是正则语言, 需要证明 A_1^* 也是正则语言.

假设 $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ 是识别 A_1 的NFA. 构造非确定性有限自动机 $N = (Q, \Sigma, \delta, q_0, F)$ 识别 A_1^* .



正则语言

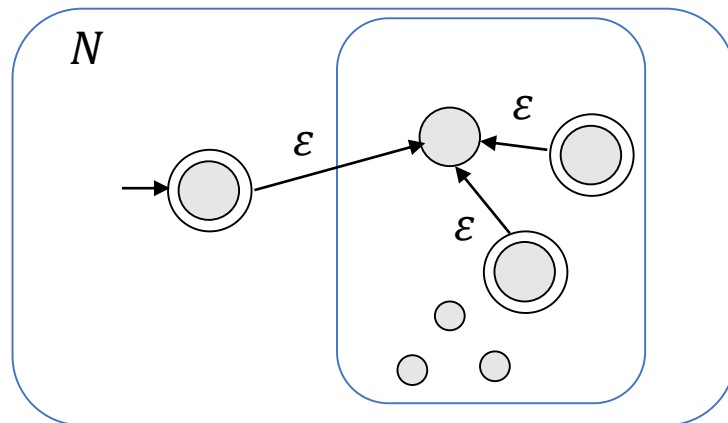
定理. 正则语言对于星号运算封闭.

证明. 设 A_1 是正则语言, 需要证明 A_1^* 也是正则语言.

假设 $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ 是识别 A_1 的NFA. 构造非确定性有限自动机 $N = (Q, \Sigma, \delta, q_0, F)$ 识别 A_1^* .

- $Q = Q_1 \cup \{q_0\}$.
- q_0 是新的初始状态.
- $F = F_1 \cup \{q_0\}$.
- 转移函数 δ 定义如下: 对任何 $a \in \Sigma_\varepsilon, q \in Q$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ 且 } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ 且 } a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ 且 } a = \varepsilon \\ \{q_1\} & q = q_0 \text{ 且 } a = \varepsilon \\ \emptyset & q = q_0 \text{ 且 } a \neq \varepsilon \end{cases}$$



■

正则语言

思考：如何使用非确定性证明正则语言对并运算封闭？

4 正则表达式

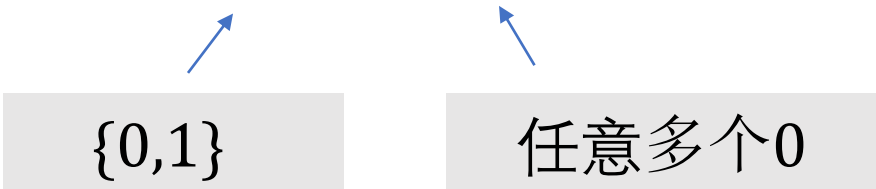
正则表达式-引例

- 用 $+$, $-$, $\times$, $\div$ 组成算数表达式

$$(5 + 3) \times 4.$$

- 用并，连接和星号运算组成正则表达式

$$(0 \cup 1) \circ 0^*.$$



$\{0,1\}$

任意多个0

所有以0或1开头后面有任意多个0的字符串.

正则表达式-引例

- $(0 \cup 1)^*$: 所有 $\{0,1\}$ -字符串.
- Σ^* : 字母表 Σ 上的任意字符串.
- Σ^*1 : 所有以1结尾的字符串.

正则表达式-定义

定义(正则表达式). 称 R 是一个正则表达式, 如果 R 是

1. 字母表 Σ 中的某个字符 a ,
2. 空字符串 ε ,
3. 空语言 $\emptyset$,
4. 表达式 $(R_1 \cup R_2)$, 其中 R_1 和 R_2 都是正则表达式,
5. 表达式 $(R_1 \circ R_2)$, 其中 R_1 和 R_2 都是正则表达式,
6. 表达式 R_1^* , 其中 R_1 是正则表达式.

记号.

• $L(R)$ 表示由 R 描述的语言.

• $R^+ \triangleq RR^*$. 

$$R^+ \cup \varepsilon = R^*$$

正则表达式-定义

例. 下面我们假设字母表 $\Sigma = \{0,1\}$.

1. $0^*10^* = \{w|w \text{恰好包含一个} 1\}$.
2. $\Sigma^*1\Sigma^* = \{w|w \text{至少包含一个} 1\}$.
3. $\Sigma^*001\Sigma^* = \{w|w \text{包含} 001 \text{作为子串}\}$.
4. $(01^+)^* = \{w|w \text{中每个} 0 \text{后面至少有一个} 1\}$.
5. $(\Sigma\Sigma)^* = \{w|w \text{是偶长的}\}$.
6. $(\Sigma\Sigma\Sigma)^* = \{w|w \text{的长度是} 3 \text{的倍数}\}$.
7. $01 \cup 10 = \{01, 10\}$.
8. $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1 = \{w|w \text{的首字符和最后一个字符相同}\}$.
9. $(0 \cup \varepsilon)1^* = 01^* \cup 1^*$.
10. $(0 \cup \varepsilon)(1 \cup \varepsilon) = \{\varepsilon, 0, 1, 01\}$.

正则表达式-定义

例(续). 下面我们假设字母表 $\Sigma = \{0,1\}$.

- $1^*\emptyset = \emptyset$.



在任何表达式后面连接空语言得到的是空语言.

- $\emptyset^* = \{\varepsilon\}$.

练习.

- $R \cup \emptyset = ?$ $R \circ \varepsilon = ?$

- $R \cup \varepsilon = ?$ $R \circ \emptyset = ?$

与有限自动机等价

问题：正则表达式和正则语言有什么联系吗？

定理. 一个语言是正则语言当且仅当存在一个正则表达式描述它.

与有限自动机等价

引理. 若一个语言可由一个正则表达式描述, 则它是正则的.

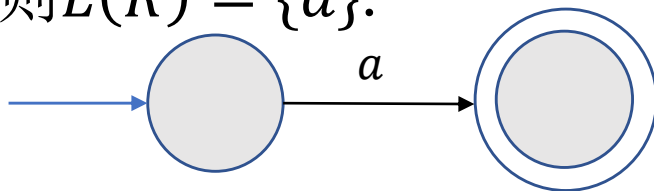
证明思路. 假设语言 A 可由正则表达式 R 来描述. 则将 R 转化为一个能够识别 A 的NFA.

与有限自动机等价

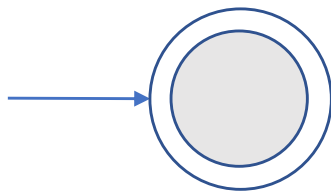
引理. 若一个语言可由一个正则表达式描述, 则它是正则的.

证明. 考虑正则表达式中的6种情况.

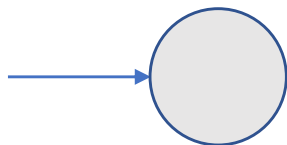
- 若 $R = a$, 则 $L(R) = \{a\}$.



- 若 $R = \varepsilon$, 则 $L(R) = \{\varepsilon\}$.



- 若 $R = \emptyset$, 则 $L(R) = \emptyset$.



与有限自动机等价

引理. 若一个语言可由一个正则表达式描述, 则它是正则的.

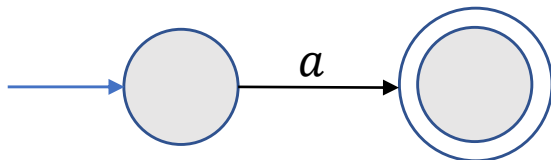
证明(续).

- 若 $R = R_1 \cup R_2$, $R = R_1 \circ R_2$, 或 $R = R_1^*$, 则可从识别 $L(R_1)$ 和 $L(R_2)$ 的NFA构造识别 $L(R)$ 的NFA. ■

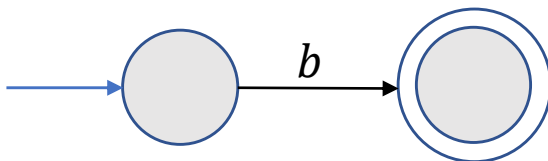
与有限自动机等价

例. 将 $(ab \cup a)^*$ 转化为一个等价的NFA.

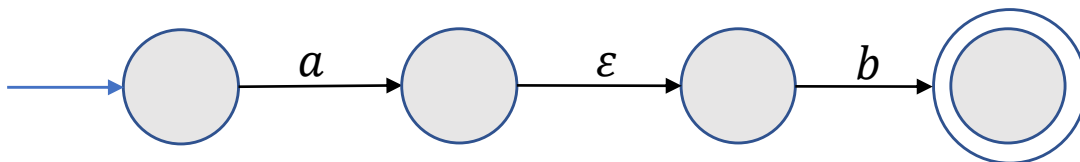
a



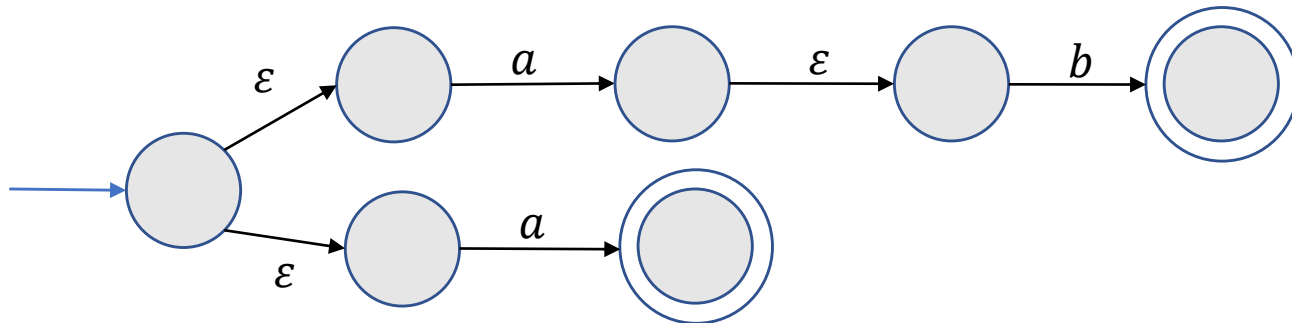
b



ab

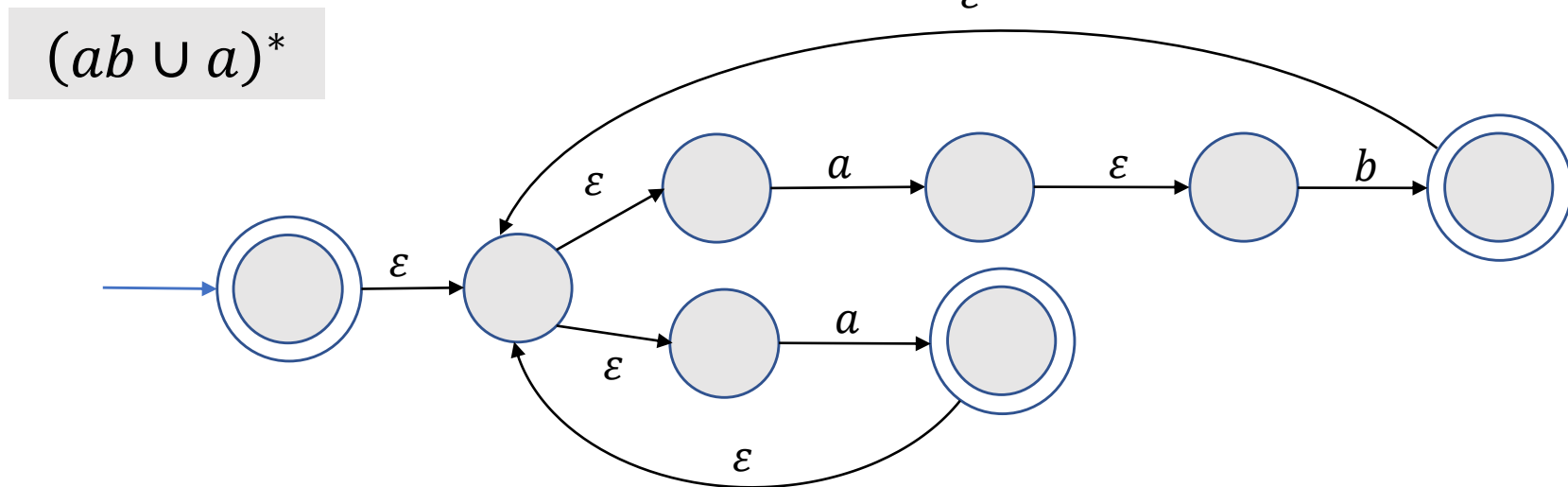
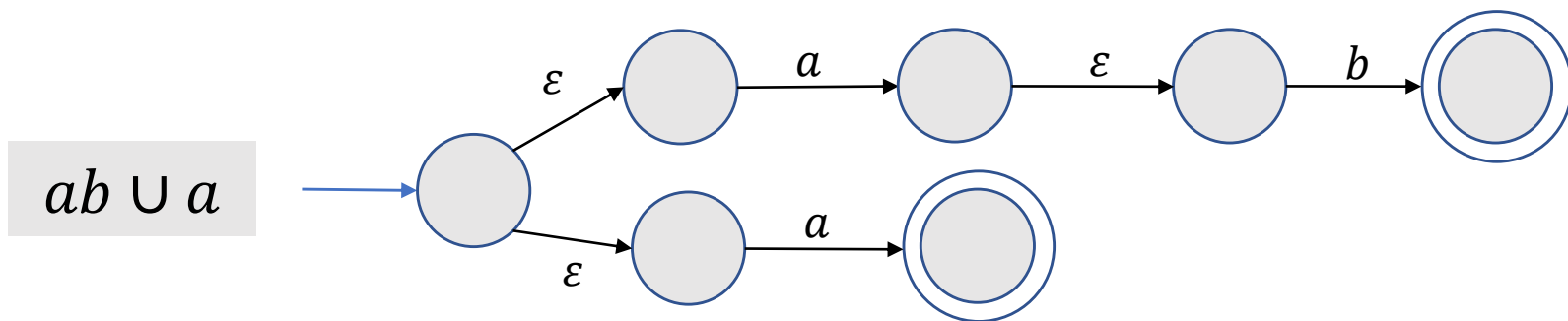


$ab \cup a$



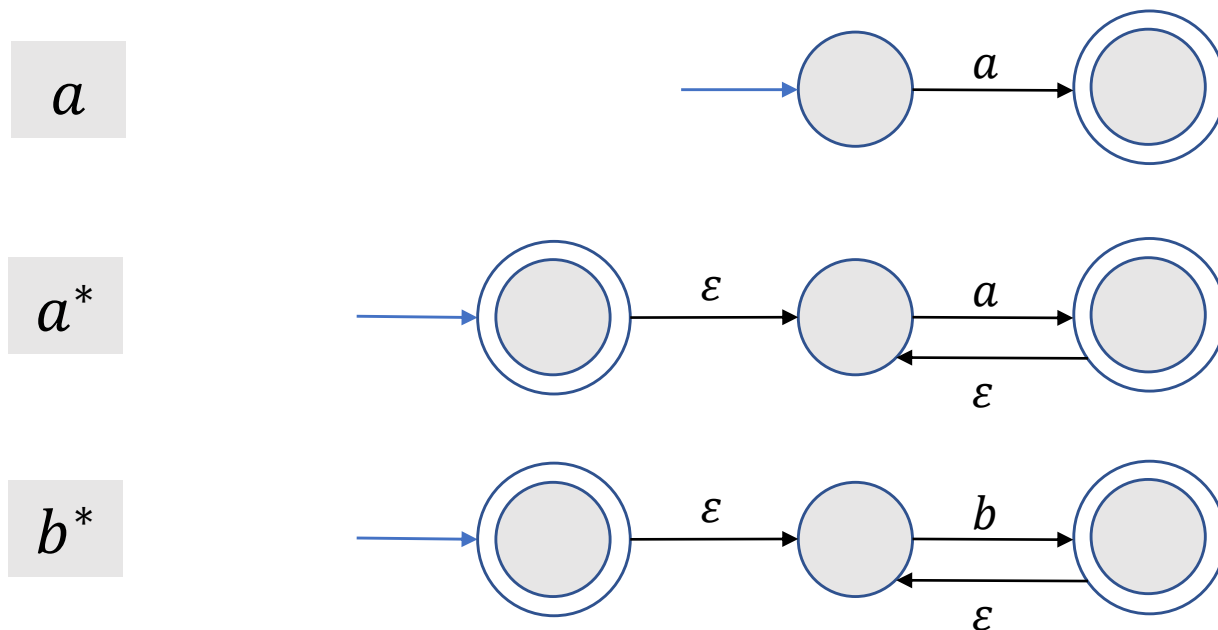
与有限自动机等价

例(续). 将 $(ab \cup a)^*$ 转化为一个等价的NFA.



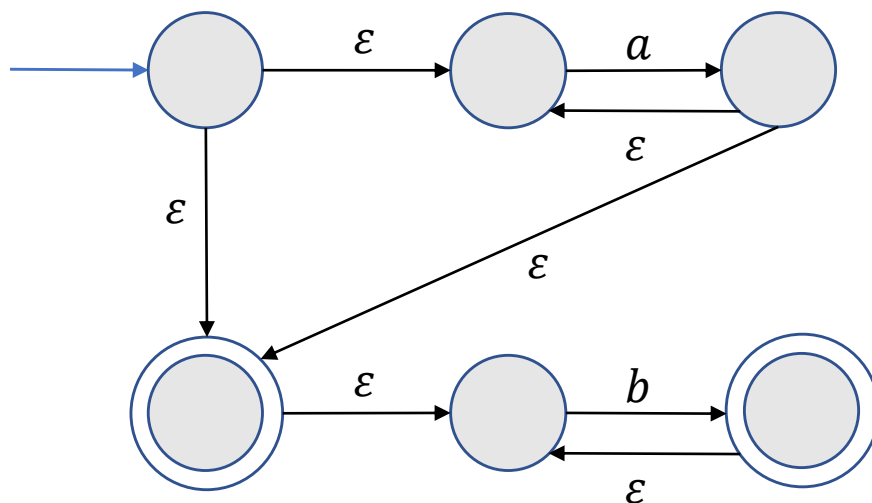
与有限自动机等价

练习. 将 a^*b^* 转化为一个等价的NFA.



与有限自动机等价

练习. 将 a^*b^* 转化为一个等价的NFA.

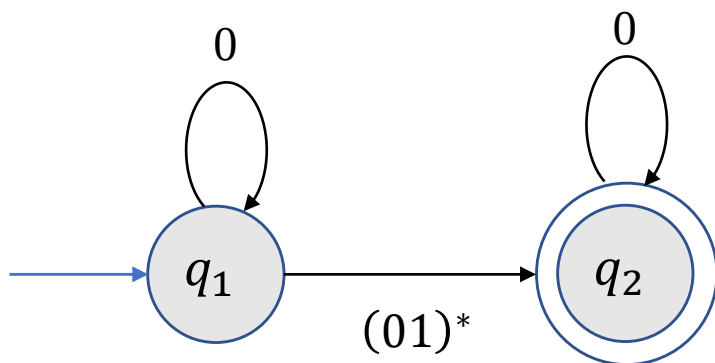


a^*b^*

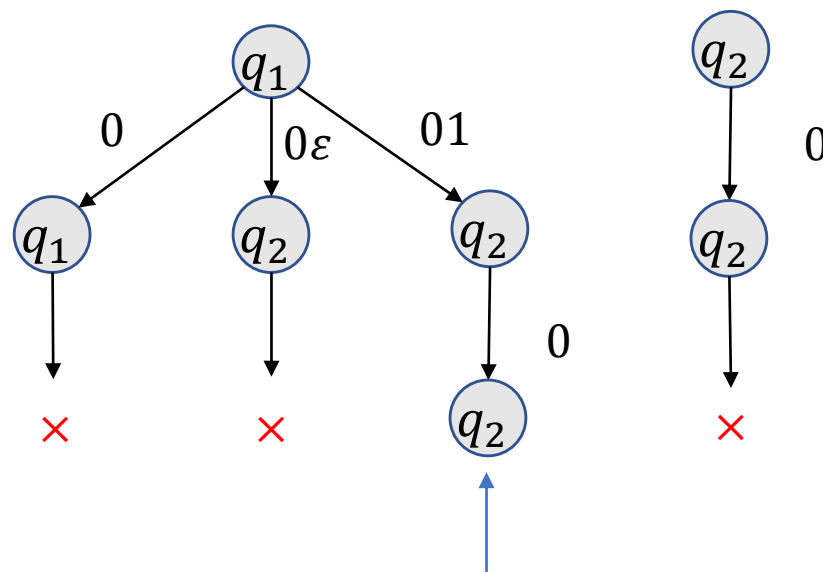
与有限自动机等价

引理. 任何正则语言都能由一个正则表达式描述.

广义非确定性有限自动机



例. $w = 010$.



是否接受011?

接受

广义非确定性有限自动机

特殊形式.

- 只有一个接受状态且不同于初始状态
- 任意两个状态之间有一条边, 除了
 - 初始状态只有出去的边
 - 接受状态只有进入的边

转化步骤.

- 添加一个新的初始状态并添加一个 ϵ 箭头指向原来的初始状态, 添加一个新的接受状态, 并且添加从原来的接受状态到新接受状态的 ϵ 箭头.
- 若没有从 q_i 指向 q_j 的箭头, 添加这样一个箭头并且其上的标签设为 $\emptyset$.
- 若在 q_i 指向 q_j 的箭头上有多个标签, 则用这些标签的并替换原标签.

广义非确定性有限自动机

定义(广义非确定性有限自动机). 一个广义非确定性有限自动机是一个5-元组 $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$, 其中

- Q 是状态集,
- Σ 是字母表,
- $\delta: (Q - \{q_{\text{accept}}\}) \times (Q - \{q_{\text{start}}\}) \rightarrow \mathcal{R}$ 是转移函数, 其中 $\mathcal{R}$ 是 Σ 上所有正则表达式的集合.
- q_{start} 是起始状态,
- q_{accept} 是接受状态.

理解.

- 箭头上的符号是正则表达式.
- 满足特殊形式.

广义非确定性有限自动机

定义(广义非确定性有限自动机的计算). 一个广义非确定性有限自动机接受字符串 $w = w_1 w_2 \dots w_k$, 其中 $w_i \in \Sigma^*$, 如果存在状态 $q_0, q_1, \dots, q_k$ 使得

- $q_0 = q_{\text{start}}$ 是起始状态,
- $q_k = q_{\text{accept}}$ 是接受状态,
- 且对任何 $1 \leq i \leq k$ 有 $w_i \in L(R_i)$, 其中 $R_i = \delta(q_{i-1}, q_i)$.

理解.

- 箭头上的符号是正则表达式.
- 机器接受到输入后, 读取子字符串 (不仅仅是单个字符). 若该子串可由箭头上的正则表达式描述, 则机器进行相应的跳转.

广义非确定性有限自动机

练习. 画出一个有3个状态的特殊GNFA并给出一个接受字符串及其接受分支的状态序列.

与有限自动机等价

引理. 任何正则语言都能由一个正则表达式描述.

证明思路. 给定正则语言 A , 则 A 可被一个DFA M 识别.

- 首先将 M 转化为一个GNFA G .
- 其次将 G 转化为一个正则表达式. 假设 G 有 k 个状态.
 - 若 $k = 2$, 则返回从初始状态到接受状态的箭头上的表达式.
 - 若 $k > 2$, 则构造一个与 G 等价的GNFA G' , 其中 G' 有 $k - 1$ 个状态.

与有限自动机等价

引理. 任何正则语言都能由一个正则表达式描述.

证明. 给定正则语言 A , 则 A 可被一个DFA M 识别.

Step 1. 将 M 转化为一个GNFA G .

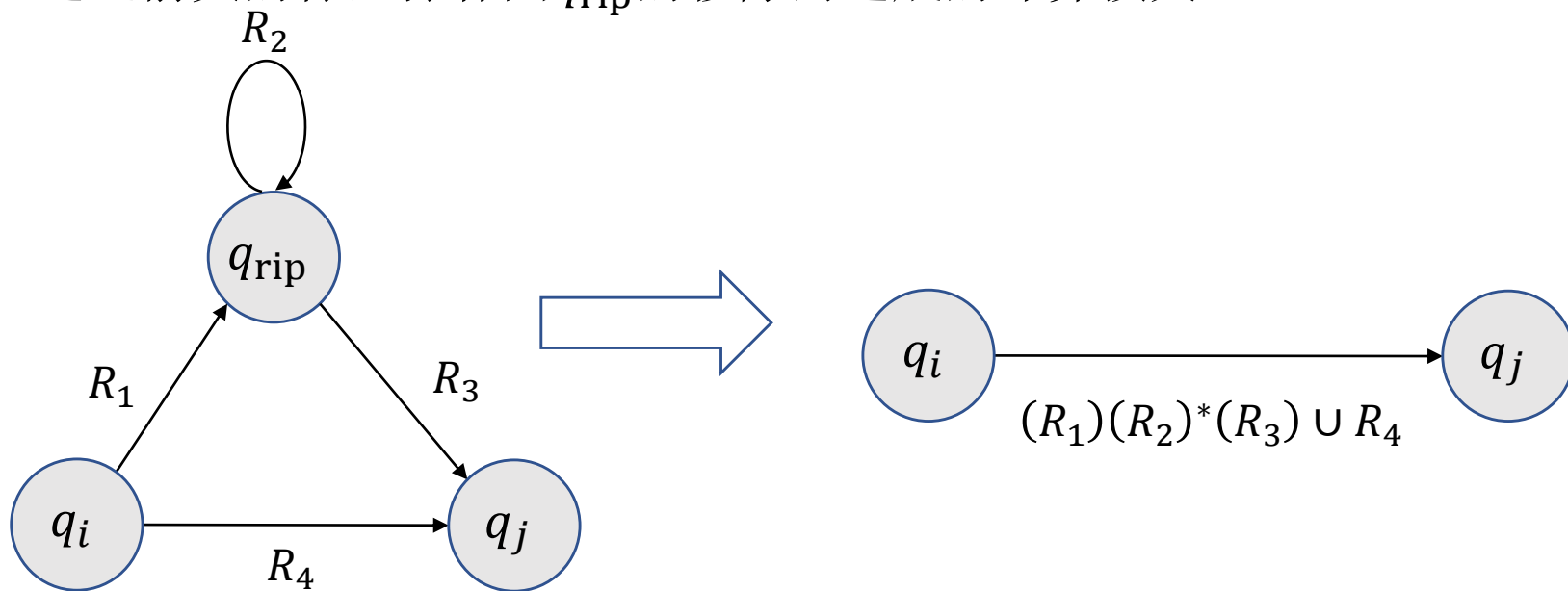
- 添加一个新的初始状态并添加一个 ε 箭头指向原来的初始状态.
- 添加一个新的接受状态, 并且添加从原来的接受状态到新接受状态的 ε 箭头.
- 若没有从 q_i 指向 q_j 的箭头, 添加这样一个箭头并且其上的标签设为 $\emptyset$.
- 若在 q_i 指向 q_j 的箭头上有多个标签, 则用这些标签的并替换原标签.

与有限自动机等价

证明(续). 给定正则语言 A , 则 A 可被一个DFA M 识别.

Step 2. 将 G 转化为一个正则表达式.

- 假设 $k \geq 3$ 且选取一个不同于初始状态和接受状态的状态 q_{rip} .
- 构造一个与 G 等价的GNFA G' , 其中 G' 的状态集为 $Q - \{q_{\text{rip}}\}$.
- 通过箭头的标签弥补因 q_{rip} 的移除而造成的计算损失.



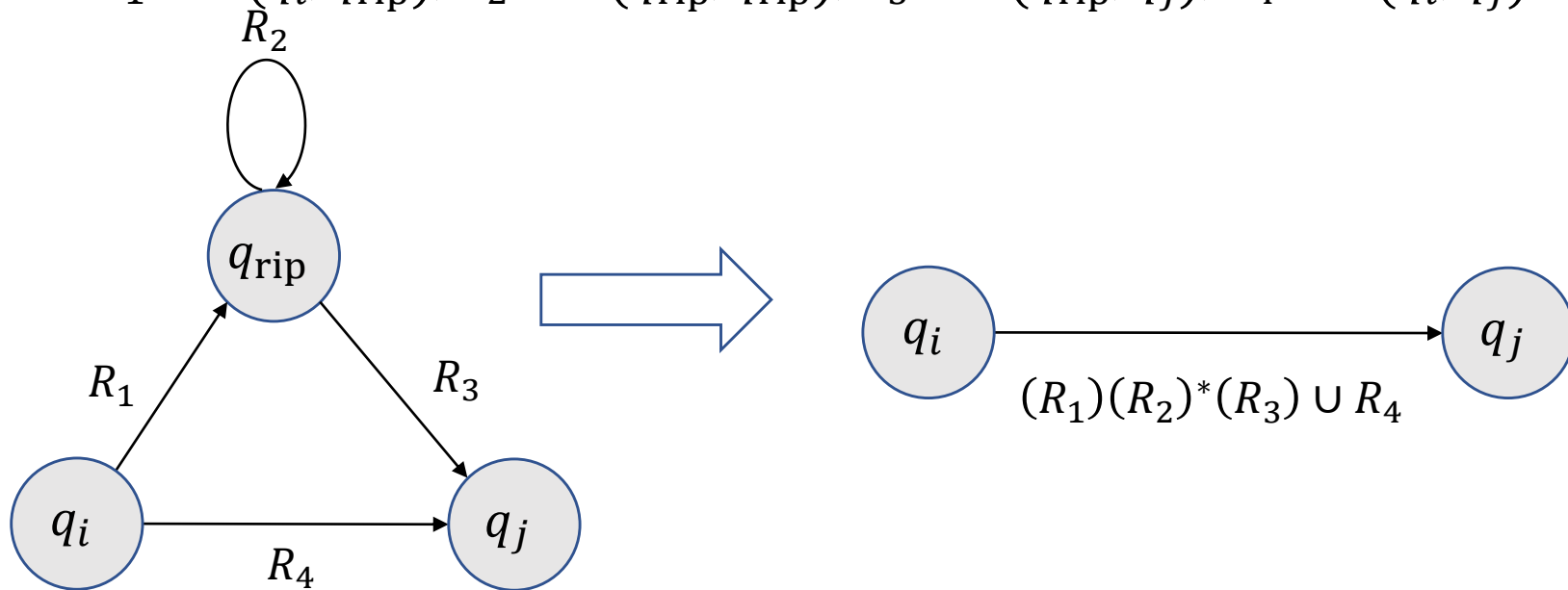
与有限自动机等价

Step 2. 将 G 转化为一个正则表达式.

令 $G' = (Q', \Sigma, \delta', q_{\text{start}}, q_{\text{accept}})$, 其中 $Q' = Q - \{q_{\text{rip}}\}$ 且对任何 $q_i \in Q' - \{q_{\text{accept}}\}$, $q_j \in Q' - \{q_{\text{start}}\}$, 令

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup R_4,$$

其中 $R_1 = \delta(q_i, q_{\text{rip}})$, $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$, $R_3 = \delta(q_{\text{rip}}, q_j)$, $R_4 = \delta(q_i, q_j)$.



与有限自动机等价

Claim. G 与 G' 识别相同的语言.

Proof of the claim. 若 G 接受 w , 令 $q_{\text{start}}, q_1, q_2, \dots, q_{\text{accept}}$ 是某个接受计算分支的状态序列.

- 若 q_{rip} 没有出现, 则因为 G' 的标签包含了原来的标签, 故 G' 接受 w .
- 若 q_{rip} 出现了, 则将 q_{rip} 移除对应了 G' 接受 w 的状态序列.

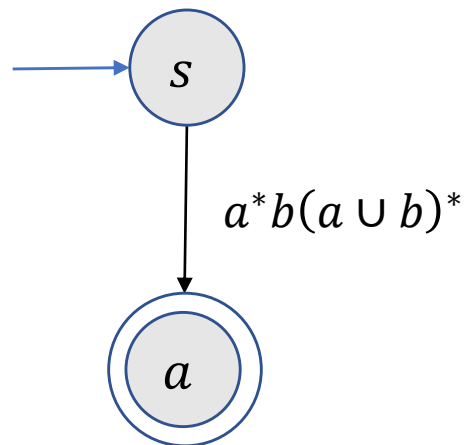
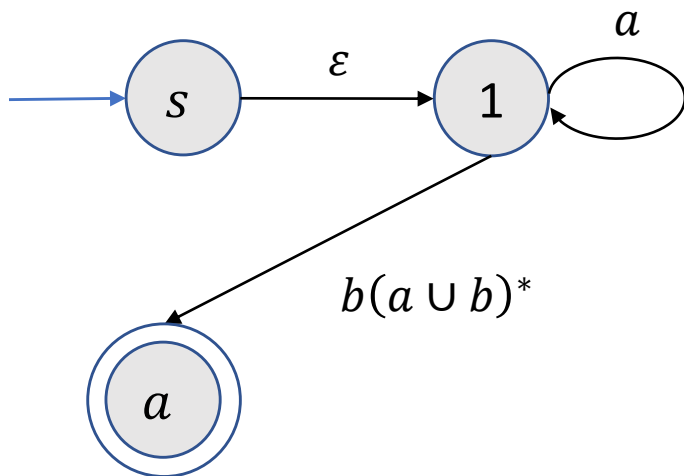
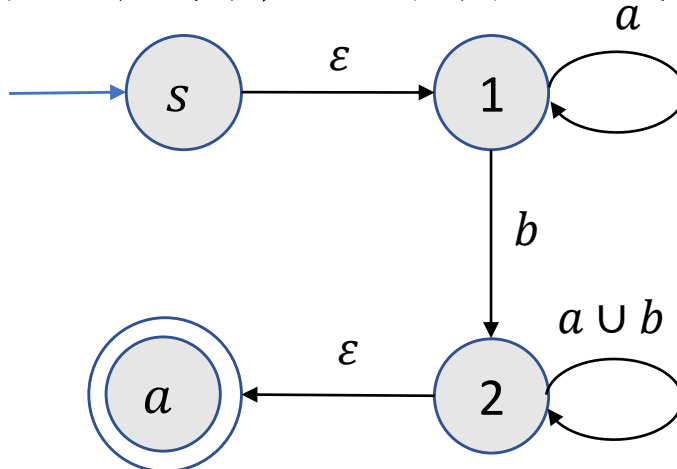
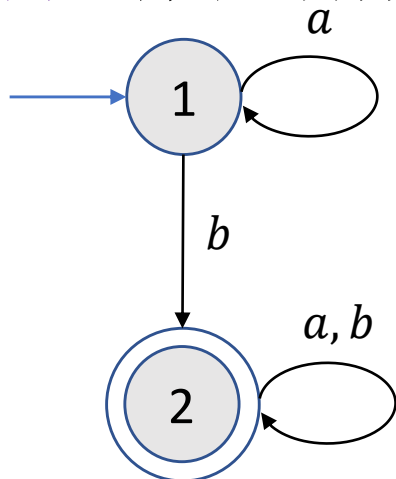
$$q_i, q_{\text{rip}}, q_{\text{rip}}, q_{\text{rip}}, q_j \rightarrow q_i, q_j$$

若 G' 接受 w , 则因为 G' 中从 q_i 到 q_j 的箭头描述了所有能让 q_i 要么直接要么经过 q_{rip} 跳转到 q_j 的字符串, G 接受 w . ■

重复上述步骤直到 G' 只有2个状态, 从而唯一箭头的标签即为等价的正则表达式. ■

与有限自动机等价

例. 将下列有限自动机转化为一个等价的正则表达式.



与有限自动机等价

练习.

- 设计一个识别 $\Sigma = \{0,1\}$ 上的语言 0^* 的NFA N .
- 按照定理的证明将 N 转化为一个等价的正则表达式.

4 非正则语言

非正则语言

问题：什么语言不能被任何有限自动机识别？

非正则语言

定义(非正则语言). 若语言 L 不能被任何有限自动机识别, 则称 L 为非正则语言.

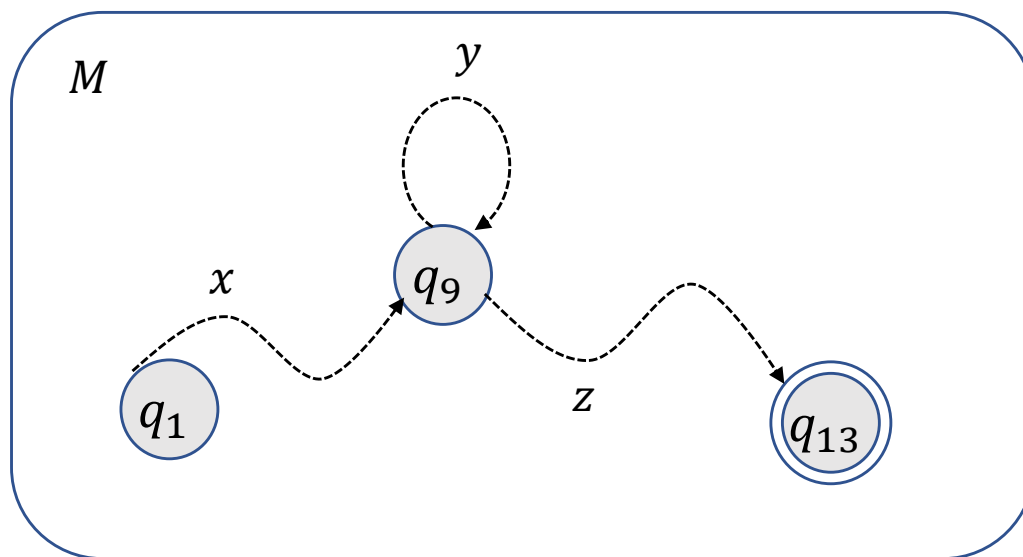
如何证明给定语言不是正则语言?

- 给出正则语言所满足的必要条件.
- 说明给定语言不满足该必要条件.

非正则语言

泵引理. 若 A 是正则语言, 则存在正整数 p (**泵长度**)使得 A 中长度至少为 p 的字符串 s 都能写成三个字符串的连接 $s = xyz$, 其中

- 对每个整数 $i \geq 0$, $xy^iz \in A$.
- $|y| > 0$.
- $|xy| \leq p$.



非正则语言

泵引理. 若 A 是正则语言, 则存在正整数 p (**泵长度**)使得 A 中长度至少为 p 的字符串 s 都能写成三个字符串的连接 $s = xyz$, 其中

- 对每个整数 $i \geq 0$, $xy^iz \in A$.
- $|y| > 0$.
- $|xy| \leq p$.

证明.

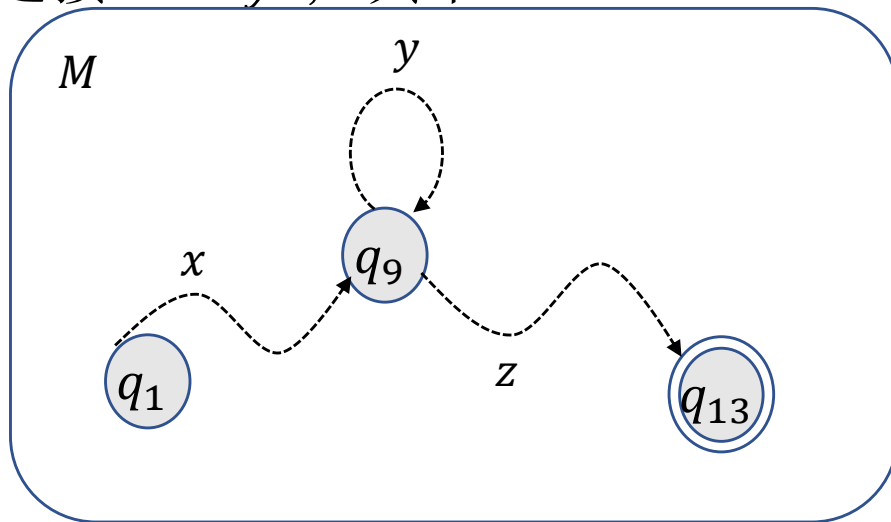
- 假设 $M = (Q, \Sigma, \delta, q_1, F)$ 是识别 A 的有限自动机. 令 $p = |Q|$.
- 设 $s = s_1s_2 \cdots s_n \in A$ 且 $n \geq p$. 令 $r_1, r_2, \dots, r_{n+1}$ 是 M 接受 s 时的状态序列.
- 因为 $n \geq p$, $r_1, r_2, \dots, r_{n+1}$ 至少有 $p + 1$ 个状态. 由**鸽笼原理**, 前 $p + 1$ 个状态中存在两个相同的状态 $r_j = r_k$ ($1 \leq j < k \leq p + 1$).

非正则语言

泵引理. 若 A 是正则语言, 则存在正整数 p 使得 A 中长度至少为 p 的字符串 s 都能写成三个字符串的连接 $s = xyz$, 其中

- 对每个整数 $i \geq 0$, $xy^iz \in A$.
- $|y| > 0$.
- $|xy| \leq p$.

证明(续).



- 令 $x = s_1 \cdots s_{j-1}$, $y = s_j \cdots s_{k-1}$, $z = s_k \cdots s_n$.
- 因为 $1 \leq j < k \leq p + 1$, $|y| > 0$ 且 $|xy| \leq p$.
- 由于 x 将 M 从状态 r_1 跳转至 r_j , y 将 M 从状态 r_j 跳转至 r_j , z 将 M 从状态 r_j 跳转至 r_{n+1} , 这是一个接受状态. 从而 M 接受 xy^iz . ■

非正则语言

例 1: 说明 $B = \{0^n 1^n | n \geq 0\}$ 不是正则语言.

解: 假设 B 是正则语言, 令 p 是泵长度.

则 $s = 0^p 1^p$ 是 B 中长度至少为 p 的字符串.

由泵引理, s 可表示成 $s = xyz$ 使得 $xy^i z \in B, \forall i \geq 0$.

- 因为 $|xy| \leq p$, y 只含0.
- 故 $xyyz$ 中0的数目大于1的数目, 矛盾.

非正则语言

例 2: 说明 $C = \{w | w \text{ 中 } 0 \text{ 和 } 1 \text{ 的数目相等}\}$ 不是正则语言.

解: 假设 C 是正则语言, 令 p 是泵长度.

则 $s = 0^p 1^p$ 是 C 中长度至少为 p 的字符串.

由泵引理, s 可表示成 $s = xyz$ 使得 $xy^i z \in C, \forall i \geq 0$.

- 因为 $|xy| \leq p$, y 只含 0.
- 故 $xyyz \notin C$, 矛盾.

非正则语言-利用运算封闭性

例 2(续): 说明 $C = \{w | w \text{ 中 } 0 \text{ 和 } 1 \text{ 的数目相等}\}$ 不是正则语言.

解法 2 (正则运算的封闭性):

- 0^*1^* 是正则语言.
- 若 C 是正则语言, 则 $C \cap 0^*1^* = B$ 是正则语言, 矛盾.

非正则语言

例 3: 说明 $D = \{1^{n^2} | n \geq 0\}$ 不是正则语言.

解: 假设 D 是正则语言, 令 p 是泵长度.

则 $s = 1^{p^2}$ 是 C 中长度至少为 p 的字符串.

由泵引理, s 可表示成 $s = xyz$ 使得 $xy^iz \in D, \forall i \geq 0$.

因为 $|y| > 0, |xy| \leq p, |xyz| = p^2$,

$$\begin{aligned} p^2 &< |xyyz| \\ &= |xyz| + |y| = p^2 + |y| \\ &\leq p^2 + p < (p + 1)^2. \end{aligned}$$

故 $xyyz \notin D$, 矛盾.

非正则语言

练习. 说明 $E = \{0^i 1^j \mid i > j\}$ 不是正则语言.

非正则语言

例 4: 说明 $F = \{ww | w \in \{0,1\}^*\}$ 不是正则语言.

解: 假设 F 是正则语言, 令 p 是泵长度.

First try: $s = 0^p 0^p$.

Second try: $s = 0^p 10^p 1$.

说明. 如何选择 s 需要创新性.

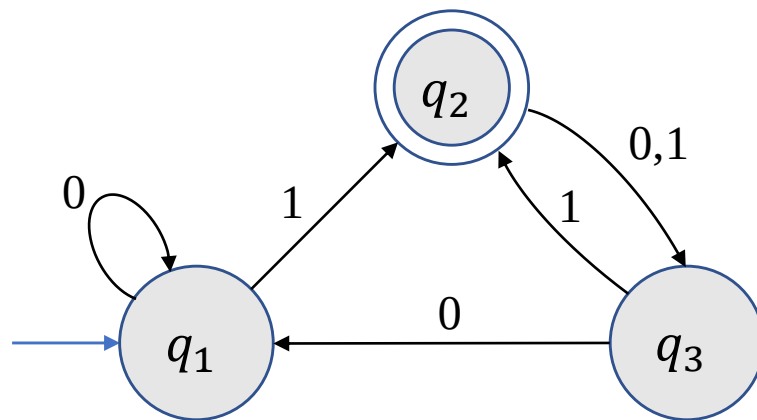
有限自动机总结

有限自动机总结

- 有限自动机
 - 状态图
 - 形式化定义
- 非确定性有限自动机
 - 状态图
 - 形式化定义
 - 计算的定义： ϵ 箭头的理解.
 - 与确定性有限自动机等价.
- 正则语言对正则运算封闭
- 正则语言与正则表达式的等价性
- 正则语言的泵引理

习题

1. (a) 给出下列有限自动机的形式化定义；(b) 给出该自动机运行0011时的状态序列并判定是否接受0011?



2. 证明 $L = \{w | w \text{ 包含相同个数的01和10作为子字符串}\}$ 是正则语言.
3. 设计一个有两个状态的NFA识别 $\{0\}$.
4. 假设非确定性有限自动机 N_1 识别 A_1 . 说明将 N_1 的初始状态 q_0 添加为接受状态并对每个 N_1 的接受状态添加 ϵ 箭头到 q_0 所得到的自动机 N 并不一定能识别 A_1^* .
5. 证明每个NFA都能转化成一个等价的只有一个接受状态的NFA.
6. 证明 $A = \{www | w \in \{0,1\}^*\}$ 是非正则语言.

计算理论-上下文无关语法

- 上下文无关语法
- 下推自动机
- 上下文无关语法与下推自动机的等价性
- 非上下文无关语言

1 上下文无关语法

上下文无关语法-引例

一个上下文无关的语法 G_1 :

$$S \rightarrow 0S1$$

$$S \rightarrow T$$

$$T \rightarrow \varepsilon$$

- 每一行代表一个替换规则.
- 箭头左边的符号称为变量.
- 箭头右边除变量之外的符号称为终止符.
- 最上方替换规则左边的变量称为起始变量.

S, T

$0, 1$

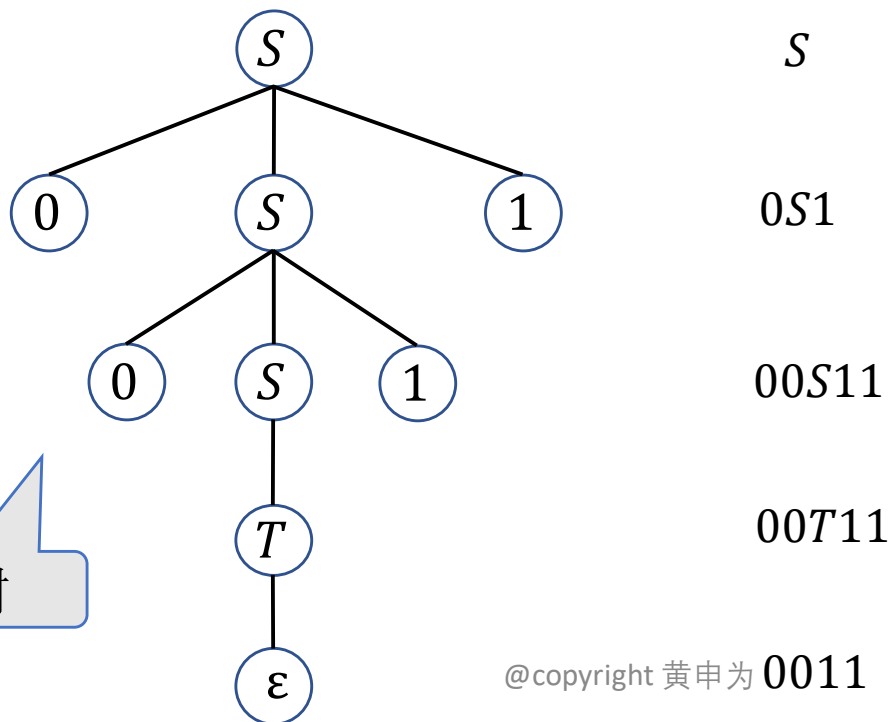
S

上下文无关语法-引例

上下文无关的语法产生字符串：

- 写下初始变量.
- 取一个已写下的变量，并找到以该变量开始的规则，把这个变量替换成规则右边的字符串.
- 重复上述步骤直到字符串中没有任何变量.

例.



$G_1:$

$S \rightarrow 0S1$

$S \rightarrow T$

$T \rightarrow \epsilon$

解析树

上下文无关语法-引例

上下文无关的语法产生字符串：

- 写下初始变量.
- 取一个已写下的变量，并找到以该变量开始的规则，把这个变量替换成规则右边的字符串.
- 重复上述步骤直到字符串中没有任何变量.

练习. 下列哪些字符串可由 G_2 生成?

(a) 001101

(b) 000111

(c) 1010

(d) 空字符串 ε

G_2 :

$S \rightarrow RR$

$R \rightarrow 0R1$

$R \rightarrow \varepsilon$

上下文无关语法-正式定义

定义(上下文无关语法). 一个上下文无关的语法是一个4-元组 (V, Σ, R, S) , 其中

- V 是一个有限集, 称为变量集.
- Σ 是与 V 不交的有限集, 称为终止符集合.
- R 是一个有限替换规则的集合(规则形式: $V \rightarrow (V \cup \Sigma)^*$).
- $S \in V$ 是起始变量.

定义(生成和派生).

- 给定 $u, v \in (V \cup \Sigma)^*$ 以及替换规则 $A \rightarrow w$, 称 uAv 生成 uwv , 记作 $uAv \Rightarrow uwv$.
- 若存在 $u_0, u_1, \dots, u_k$ ($k \geq 0$)使得

$$u = u_0 \Rightarrow u_1 \Rightarrow \dots \Rightarrow u_k = v$$

则称 u 派生 v , 记作 $u \Rightarrow^* v$.

给定上下文无关语法 G , 所有由 G 生成的语言为

$$L(G) = \{w \in \Sigma^* \mid S \xRightarrow{*} w\}.$$

上下文无关语法-正式定义

例 1:

G_1 :

$S \rightarrow 0S1$

$S \rightarrow T$

$T \rightarrow \varepsilon$

$$G_1 = (V, \Sigma, R, S)$$

- $V = \{S, T\}$.
- $\Sigma = \{0, 1\}$.
- G_1 有三条替换规则.
- S 是初始变量.

$$0S1 \xRightarrow{*} 01$$

$$L(G_1) = \{0^k 1^k \mid k \geq 0\}$$

上下文无关语法-正式定义

例 2: $G_2 = (\{S\}, \{a, b\}, R, S)$, 其中替换规则为

$$S \rightarrow aSb \mid SS \mid \varepsilon.$$

该语法生成字符串 $\varepsilon, ab, aabb, abab, aababb$ 等等.

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb.$$

$$S \Rightarrow aSb \Rightarrow aSSb \xRightarrow{*} aaSbaSbb \xRightarrow{*} aababb.$$

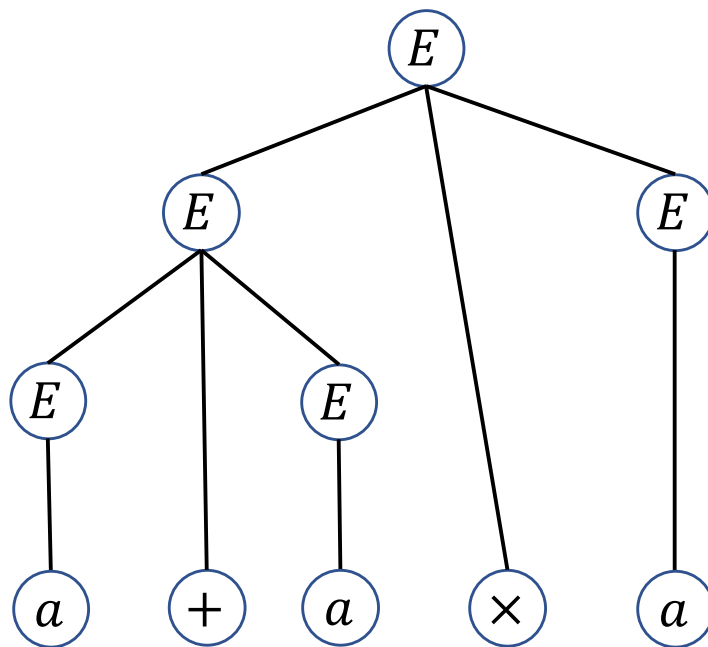
若把 a 看成 $($, b 看成 $)$, 则 $L(G_2)$ 是所有括号正常嵌套的字符串构成的语言.

上下文无关语法-正式定义

例 3: $G_3 = (\{E\}, \{a, +, \times, (,)\}, R, E)$, 其中替换规则为

$$E \rightarrow E + E | E \times E | (E) | a.$$

给出生成字符串 $a + a \times a$ 的解析树.



上下文无关语法-正式定义

练习. 确定由下列语法生成的语言.

$$S \rightarrow T1$$

$$T \rightarrow T0|T1|\varepsilon$$

$$L = \{w|w \text{以} 1 \text{结尾}\}.$$

上下文无关语法-设计

设计CFG的常用方法.

- 考察子串
- 化繁为简
- 利用正则

上下文无关语法-设计

考察字符串：语言中的字符串有两个子串是相互“关联”的，需要记住第一个子串的信息(需要无限内存)去验证另一个子串确实与之“关联”。

例： $L = \{0^k 1^k \mid k \geq 0\}$.

- 需要记住字符0的个数去验证1的个数与之相等.
- 使用替换规则 $S \rightarrow 0S1$ 来验证0的个数与1的个数相等.

$$S \rightarrow 0S1 \mid \varepsilon$$

上下文无关语法-设计

化繁为简：所要生成的语言是一些简单语言的并，则可以分别设计这些简单语言的语法，再通过替换规则

$$S \rightarrow S_1 | S_2 | \cdots | S_k,$$

就可生成给定语言.

例： $L = \{0^k 1^k \mid k \geq 0\} \cup \{1^k 0^k \mid k \geq 0\}$.

- $S_1 \rightarrow 0S_1 1 \mid \varepsilon$
- $S_2 \rightarrow 1S_2 0 \mid \varepsilon$



$$S \rightarrow S_1 | S_2$$

$$S_1 \rightarrow 0S_1 1 \mid \varepsilon$$

$$S_2 \rightarrow 1S_2 0 \mid \varepsilon$$

上下文无关语法-设计

利用正则. 任何有限自动机都能够转化为一个等价的上下文无关语法.

给定有限自动机 $M = (Q, \Sigma, \delta, q_0, F)$, 设计语法 G 如下:

- 令 R_i 是对应状态 q_i 的变量.
- R_0 是初始变量.
- 若 $\delta(q_i, a) = q_j$, 则有相应的替换规则

$$R_i \rightarrow aR_j.$$

- 若 q_i 是一个接受状态, 则添加替换规则 $R_i \rightarrow \varepsilon$.

则 $L(G) = L(M)$.

上下文无关语法-设计

练习. 设计生成 $L = \{w | w \text{至少包含三个} 1\}$ 的语法.

$$S \rightarrow TTT$$

$$T \rightarrow TR | RT | 1$$

$$R \rightarrow 0 | 1 | \varepsilon$$

上下文无关语法-歧义性

定义(最左派生). 对于语法 G 中的字符串 w 的派生, 如果每一步都是替换最左边的变量, 则称这个派生为最左派生.

例: 考虑 $G_3 = (\{E\}, \{a, +, \times, (,)\}, R, E)$ 中的字符串 $a + a \times a$, 其中替换规则为

$$E \rightarrow E + E \mid E \times E \mid (E) \mid a.$$

- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a.$
- $E \Rightarrow E + E \Rightarrow E + E \times E \Rightarrow \dots \Rightarrow a + a \times a.$

否

是

上下文无关语法-歧义性

定义(歧义性). 如果字符串 w 在上下文无关语法 G 中有两个或两个以上不同的最左派生, 则称 G 歧义地产生字符串 w . 如果文法 G 歧义地产生某个字符串, 则称 G 是歧义的.

例: $G_3 = (\{E\}, \{a, +, \times, (,)\}, E \rightarrow E + E | E \times E | (E) | a, E)$ 歧义地产生字符串 $a + a \times a$, 从而 G_3 是歧义的.

- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a.$
- $E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a.$

理解. 第一种方式是先做加法再做乘法, 而第二种方式是先做乘法再做加法.

上下文无关语法-乔姆斯基范式

定义(乔姆斯基范式). 称一个上下文无关语法为乔姆斯基范式 (Chomsky normal form), 如果它的每一个规则具有如下形式:

$$A \rightarrow BC$$

$$A \rightarrow a$$

其中 a 是任意的终结符, A, B, C 是任意的变量, 且 B 和 C 不能为起始变量.

此外允许规则 $S \rightarrow \varepsilon$, 其中 S 是起始变量.

上下文无关语法-乔姆斯基范式

定理(乔姆斯基范式). 任一上下文无关语言都可以用一个乔姆斯基范式的上下文无关语法产生.

证明. 增加一个新的起始变量 S_0 以及替换规则 $S_0 \rightarrow S$. 这样保证了起始变量不会出现在任何替换规则的右边.

根据右端字符串长度 ℓ 分类.

- $\ell = 0$ 移除 ε -规则 $A \rightarrow \varepsilon$: 对于每条右端出现 A 的规则, 增加一条相应的规则, 该规则将 A 从右端中移除.

$$R \rightarrow uAvAw \xrightarrow{\text{增加}} R \rightarrow uvAw, R \rightarrow uAvw, R \rightarrow uvw$$

- $\ell = 1$ 移除单一生产式 $A \rightarrow B$: 对于每条形如 $B \rightarrow u$ 的规则, 增加规则 $A \rightarrow u$.

上下文无关语法-乔姆斯基范式

定理(乔姆斯基范式). 任一上下文无关语言都可以用一个乔姆斯基范式的上下文无关语法产生.

证明(续). 增加一个新的起始变量 S_0 以及替换规则 $S_0 \rightarrow S$. 这样保证了起始变量不会出现在任何替换规则的右边.

根据右端字符串长度 ℓ 分类.

- $\ell \geq 3$ 移除 $A \rightarrow u_1 u_2 \cdots u_\ell$, u_i 是变量或者终止符.

$$A \rightarrow u_1 u_2 \cdots u_\ell \xrightarrow{\text{替换为}} A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{\ell-2} \rightarrow u_{\ell-1} u_\ell$$

- $\ell = 2$ 移除规则 $A \rightarrow u_1 u_2$:

$$A \rightarrow u_1 u_2 \xrightarrow{\text{替换为}} A \rightarrow U_1 U_2, \quad U_1 \rightarrow u_1, \quad U_2 \rightarrow u_2$$

证毕. ■

上下文无关语法-乔姆斯基范式

例：考虑语法 $G_4 = (\{S, A, B\}, \{a, b\}, R, S)$:

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\varepsilon$$

将其转化成乔姆斯基范式.

第 1 步. 增加新的起始变量.

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\varepsilon$$



$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\varepsilon$$

上下文无关语法-乔姆斯基范式

第2步. 移除 ϵ 规则.

$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\epsilon$$



$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a$$

$$A \rightarrow B|S|\epsilon$$

$$B \rightarrow b|\epsilon$$



$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a|SA|AS|S$$

$$A \rightarrow B|S|\epsilon$$

$$B \rightarrow b$$

上下文无关语法-乔姆斯基范式

第3步. 移除单一生产式.

$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a|SA|AS|S$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$



$$S_0 \rightarrow S$$

$$S \rightarrow ASA|aB|a|SA|AS|S$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$



$$S_0 \rightarrow S|ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$

上下文无关语法-乔姆斯基范式

第3步. 移除单一生产式.

$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$



$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow \textcolor{gray}{B}|S|\textcolor{black}{b}$$

$$B \rightarrow b$$



$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow \textcolor{gray}{S}|b|\textcolor{black}{ASA|aB|a|SA|AS}$$

$$B \rightarrow b$$

上下文无关语法-乔姆斯基范式

第 4 步. 移除其他不符合条件的规则.

$$S_0 \rightarrow ASA|aB|a|SA|AS$$

$$S \rightarrow ASA|aB|a|SA|AS$$

$$A \rightarrow b|ASA|aB|a|SA|AS$$

$$B \rightarrow b$$



$$S_0 \rightarrow AA_1|UB|a|SA|AS$$

$$S \rightarrow AA_1|UB|a|SA|AS$$

$$A \rightarrow b|AA_1|UB|a|SA|AS$$

$$B \rightarrow b$$

$$A_1 \rightarrow SA$$

$$U \rightarrow a$$

习题

考虑语法 $G = (\{A, B\}, \{0\}, R, A)$:

$$A \rightarrow BAB|B|\varepsilon$$

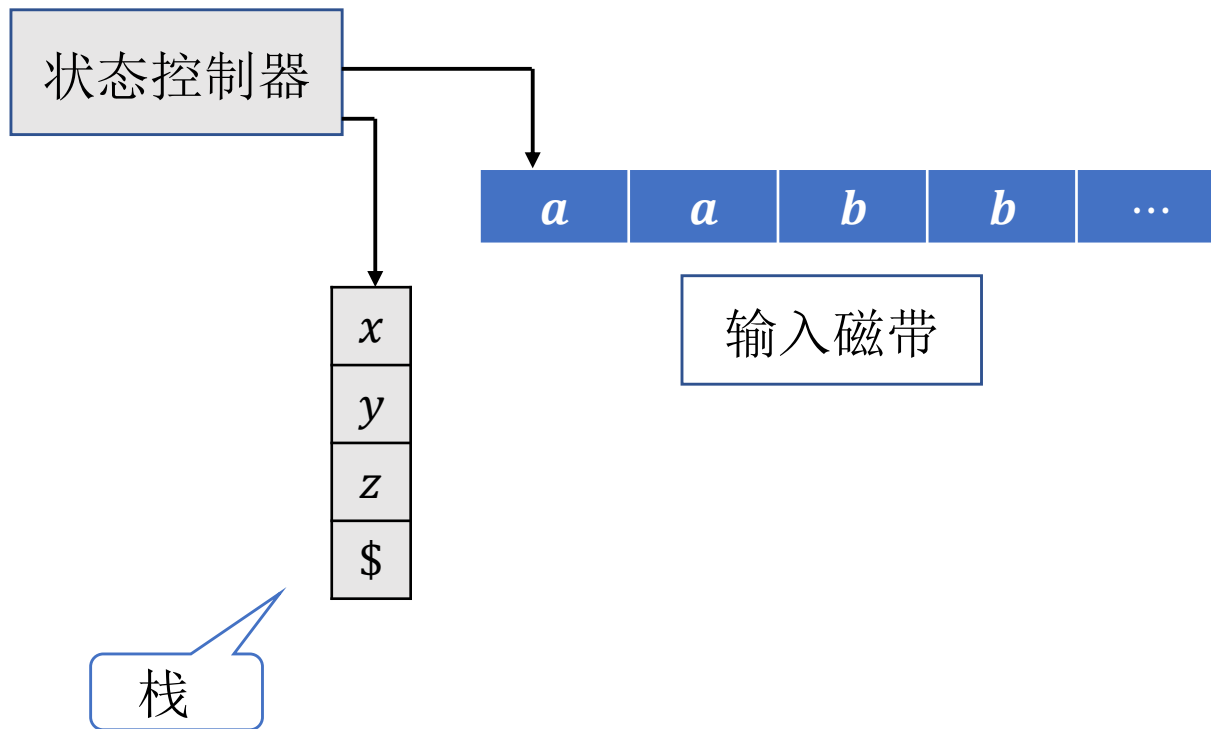
$$B \rightarrow 00|\varepsilon$$

将其转化成乔姆斯基范式.

2 下推自动机

下推自动机

抽象示意图.



PDA = NFA + stack with unlimited memory

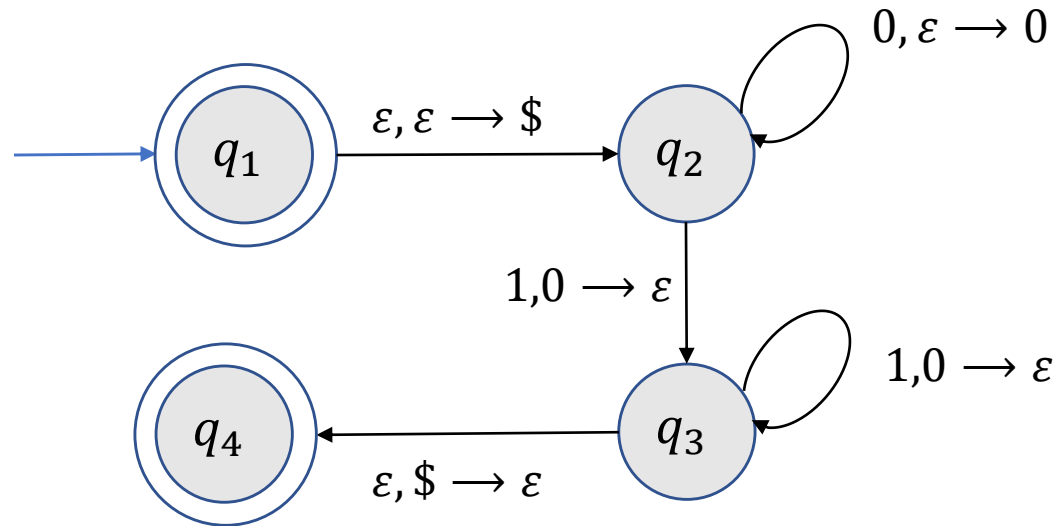
下推自动机-引例

设计识别 $L = \{0^k 1^k | k \geq 0\}$ 的下推自动机.

- 每读到0，压入栈中，直到读到1.
- 每读到一个1，从栈中移除一个0.
- 如果输入全部读完时，栈恰好为空，则接受；否则拒绝.

下推自动机-引例

设计识别 $L = \{0^k 1^k \mid k \geq 0\}$ 的下推自动机.

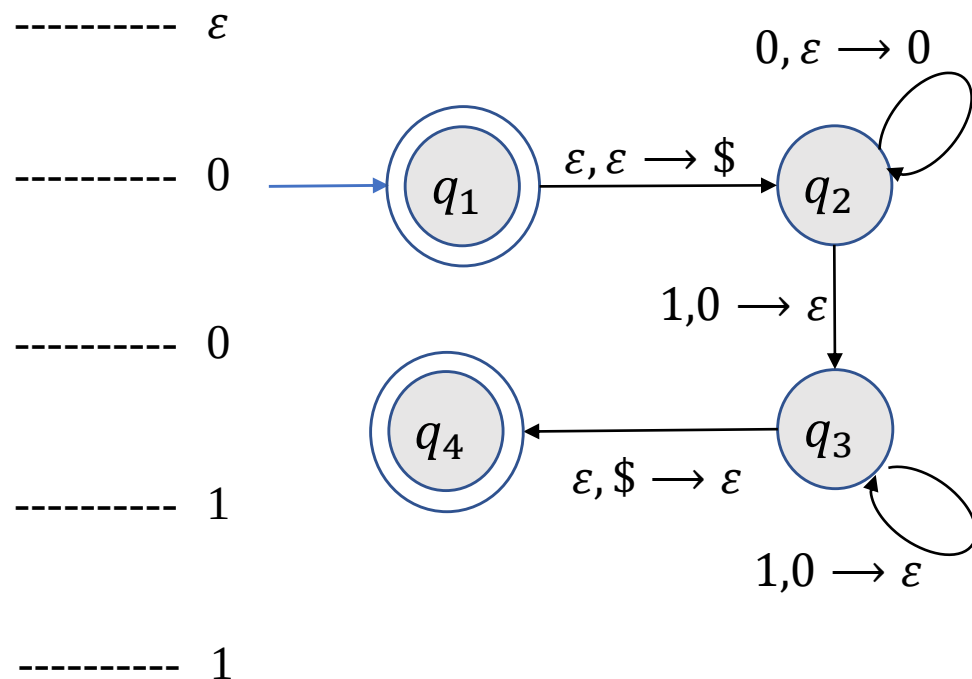
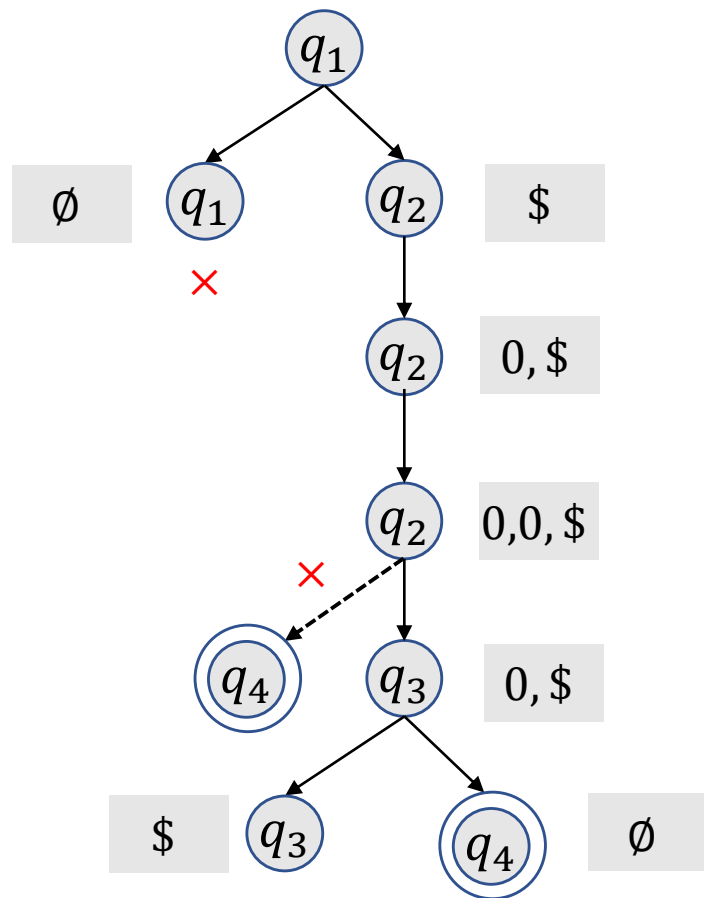


记号. $a, b \rightarrow c$ 表示读取输入字符 a , 将栈顶端字符 b 替换成 c .

- 若 $a = \varepsilon$, 则表示该操作不需要读取任何字符串.
- 若 $b = \varepsilon$, 则表示直接将 c 压入栈顶.
- 若 $c = \varepsilon$, 则表示将栈顶元素 b 移除.

下推自动机-引例

例: $w = 0011$.



下推自动机-正式定义

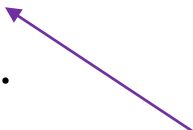
定义(下推自动机). 一个下推自动机是一个6-元组 $(Q, \Sigma, \Gamma, \delta, q_0, F)$ 满足

- Q 是有限状态集.
- Σ 是输入字母表.
- Γ 是栈字母表.
- q_0 是起始状态.
- F 是接受状态集.
- $\delta: Q \times \Sigma_{\varepsilon} \times \Gamma_{\varepsilon} \rightarrow \mathcal{P}(Q \times \Gamma_{\varepsilon})$ 是转移函数.

下推自动机-正式定义

定义(下推自动机的计算). 一个下推自动机 $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ 接受输入 w , 如果 w 可写成 $w = w_1 w_2 \cdots w_m$, 其中 $w_i \in \Sigma_\varepsilon$ 且存在状态序列 $r_0, r_1, \dots, r_m$ 和字符串 $s_0, s_1, \dots, s_m \in \Gamma^*$ 使得

- $r_0 = q_0, s_0 = \varepsilon$.
- 对 $i = 0, \dots, m - 1$, 存在 $a, b \in \Gamma_\varepsilon, t \in \Gamma^*$ 使得
 - $s_i = at, s_{i+1} = bt$.
 - $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$.
- $r_m \in F$.

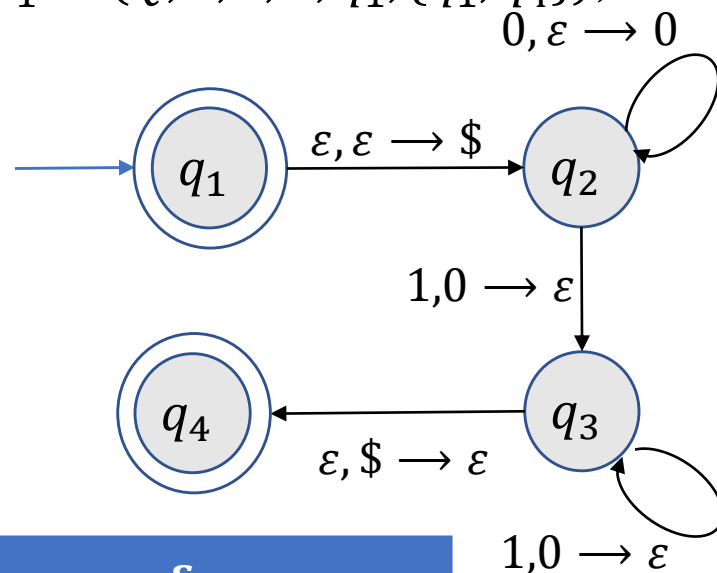


当处于状态 r_i , 读入字符 w_{i+1} , 跳转到状态 r_{i+1} 并将栈顶元素由 a 替换为 b .

下推自动机-正式定义

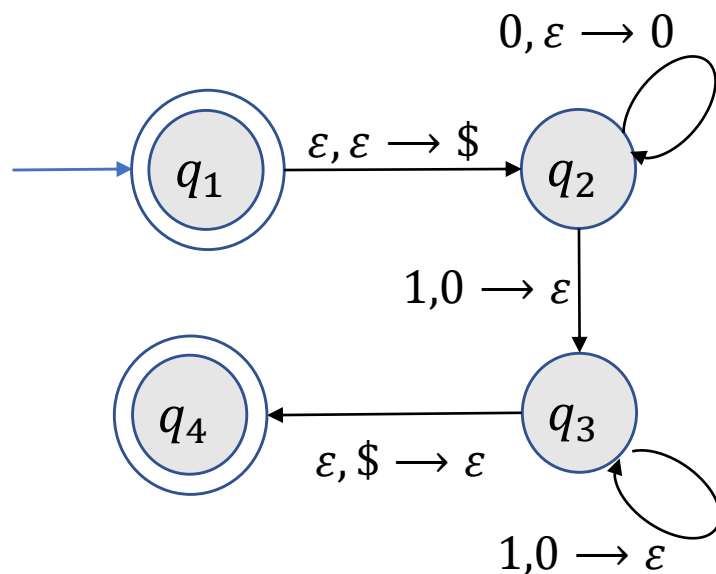
例 1: 识别 $L = \{0^k 1^k | k \geq 0\}$ 的下推自动机 $P_1 = (Q, \Sigma, \Gamma, \delta, q_1, \{q_1, q_4\})$,

- $Q = \{q_1, q_2, q_3, q_4\}$,
- $\Sigma = \{0, 1\}$,
- $\Gamma = \{0, \$\}$,
- 转移函数由下表给出:



输入	0			1			ϵ		
栈	0	\$	ϵ	0	\$	ϵ	0	\$	ϵ
q_1									$\{(q_2, \$)\}$
q_2			$\{(q_2, 0)\}$	$\{(q_3, \epsilon)\}$					
q_3				$\{(q_3, \epsilon)\}$				$\{(q_4, \epsilon)\}$	
q_4									

下推自动机-正式定义



例 1(续): 识别 $L = \{0^k 1^k | k \geq 0\}$ 的下推自动机 $P_1 = (Q, \Sigma, \Gamma, \delta, q_1, F)$ 接受 $w = 0011$ 的状态序列和栈序列为

w	ϵ	0	0	1	1	ϵ
q_1	q_2	q_2	q_2	q_3	q_3	q_4
ϵ	\$	0\$	00\$	0\$	\$	ϵ

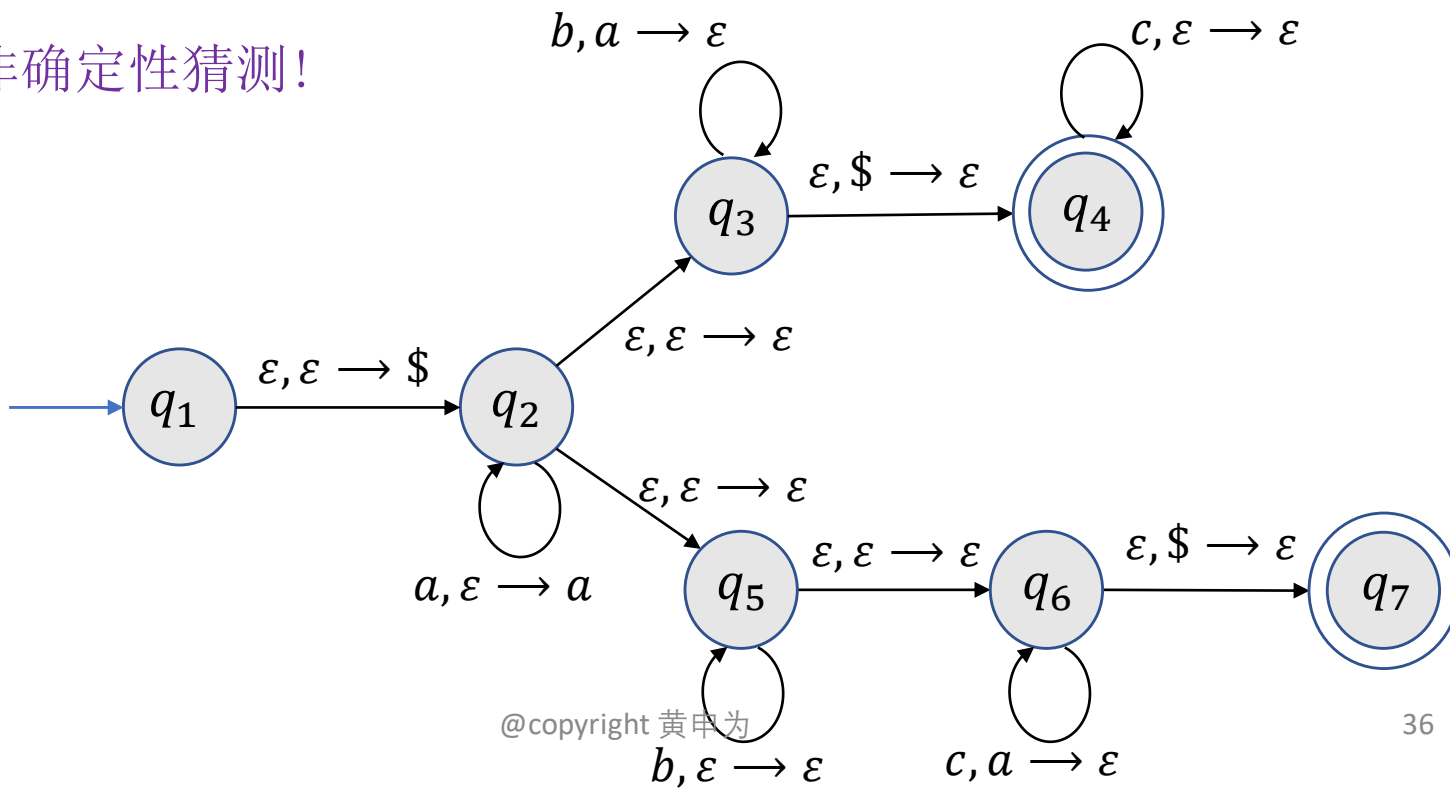
下推自动机-正式定义

例 2: 构造识别 $L = \{a^i b^j c^k \mid i, j, k \geq 0, i = j \text{ 或 } i = k\}$ 的下推自动机 P_2 .

- P_2 首先读入 a 并将 a 压入栈中;
- 当 a 读完之后需要将之与 b 的数目和 c 的数目匹配.

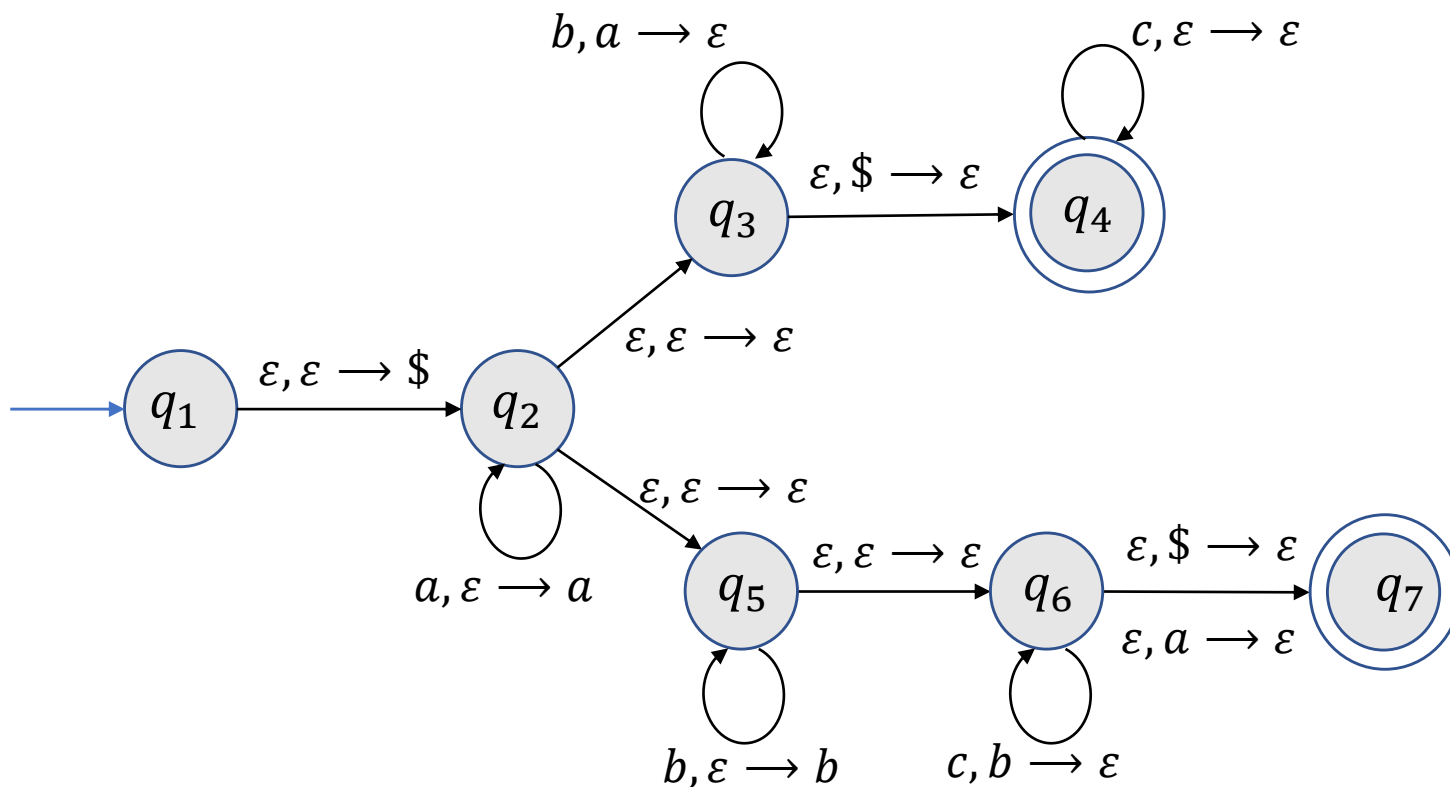
无法预先知道和 b 还是和 c 比较!

- 使用非确定性猜测!



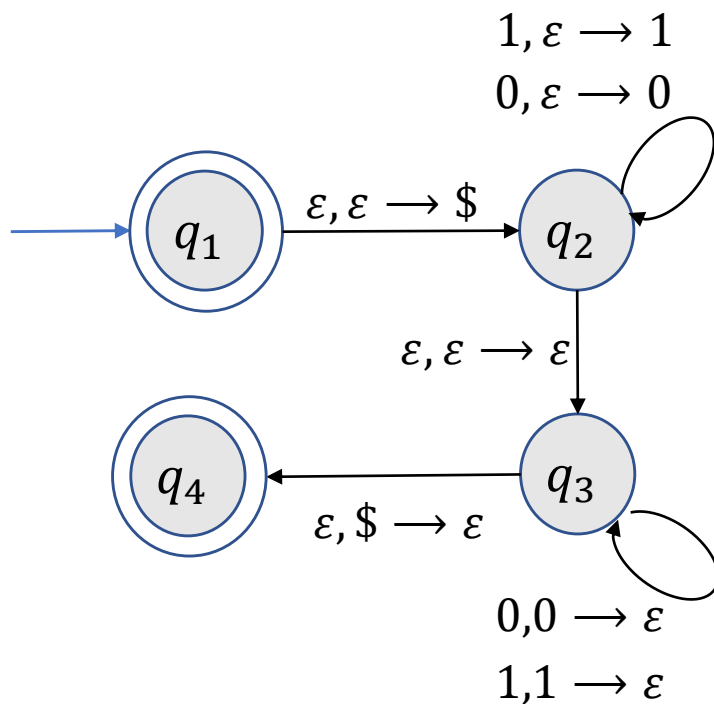
下推自动机-正式定义

练习. 构造识别 $L = \{a^i b^j c^k \mid i, j, k \geq 0, i = j \text{ 或 } j = k\}$ 的下推自动机.



下推自动机-正式定义

例 3: 构造识别 $L = \{ww^R | w \in \{0,1\}^*\}$ 的下推自动机 P_3 .



3 CFG与PDA等价

CFG与PDA等价

定理. 一个语言是上下文无关的当且仅当它能被某个下推自动机识别.

CFG与PDA等价

引理. 一个语言是上下文无关的, 则它能被某个下推自动机识别.

证明思路. 假设语言 L 能由上下文无关语法 G 生成, 需要构造一个能够识别 L 的下推自动机 P .

- P 接受字符串 w 当且仅当 w 可由 G 生成.
- 判定 G 是否生成 w 等价于是否存在一系列替换规则使得 w 可由初始变量派生.
- 难点在于判定使用哪些替换规则派生 w .



非确定性允许 P “猜测”正确的替换规则

CFG与PDA等价

引理. 一个语言是上下文无关的, 则它能被某个下推自动机识别.

证明思路. P 的非形式化描述如下:

(1) 把标记符 $\$$ 和起始变量放入栈中.

(2) 重复下述步骤:

(a) 如果栈顶是变量 A , 则非确定地选择一个关于 A 的规则并且把 A 替换成这条规则右边的字符串.

(b) 如果栈顶是终止符 a , 则读取输入中的下一个符号并且把它与 a 进行比较. 如果它们匹配, 则重复; 否则这个非确定性分支被拒绝.

(c) 如果栈顶是符号 $\$$, 则进入接受状态. 如果此刻输入已全部读完, 则接受这个输入串.

CFG与PDA等价

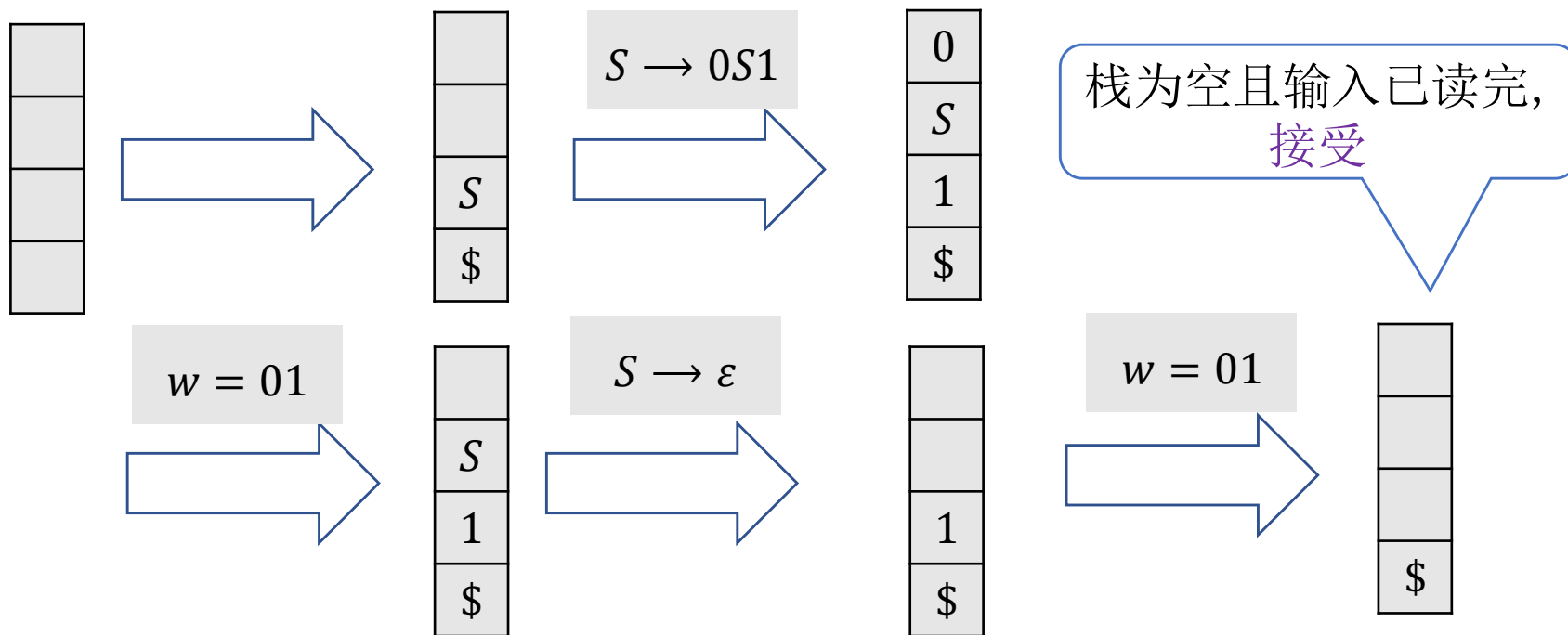
引理. 一个语言是上下文无关的, 则它能被某个下推自动机识别.

证明思路.

G_1 :

$S \rightarrow 0S1 | \varepsilon$

$w = 01$

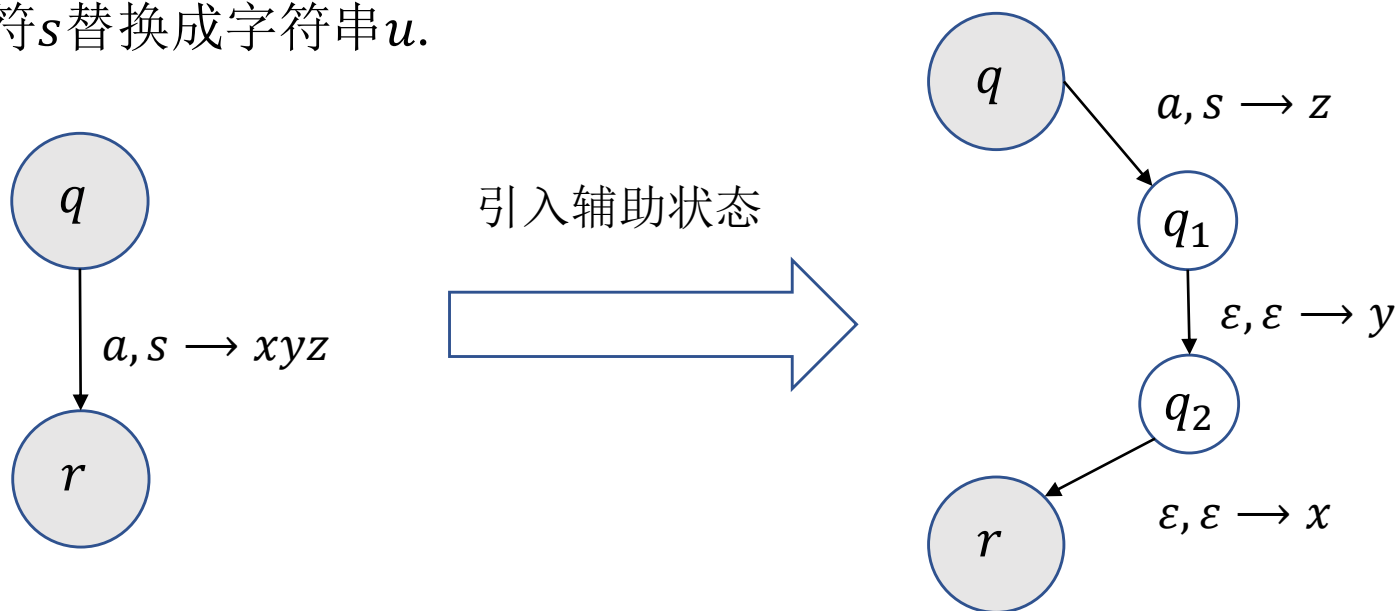


CFG与PDA等价

引理. 一个语言是上下文无关的, 则它能被某个下推自动机识别.

证明.

- 采用一种缩写记号表示转移函数, 即一步把一个字符串写入到栈中.
- $(r, u) \in \delta(q, a, s)$ 表示处于状态 q , 读入字符 a 时, 跳转到状态 r 并将栈顶字符 s 替换成字符串 u .

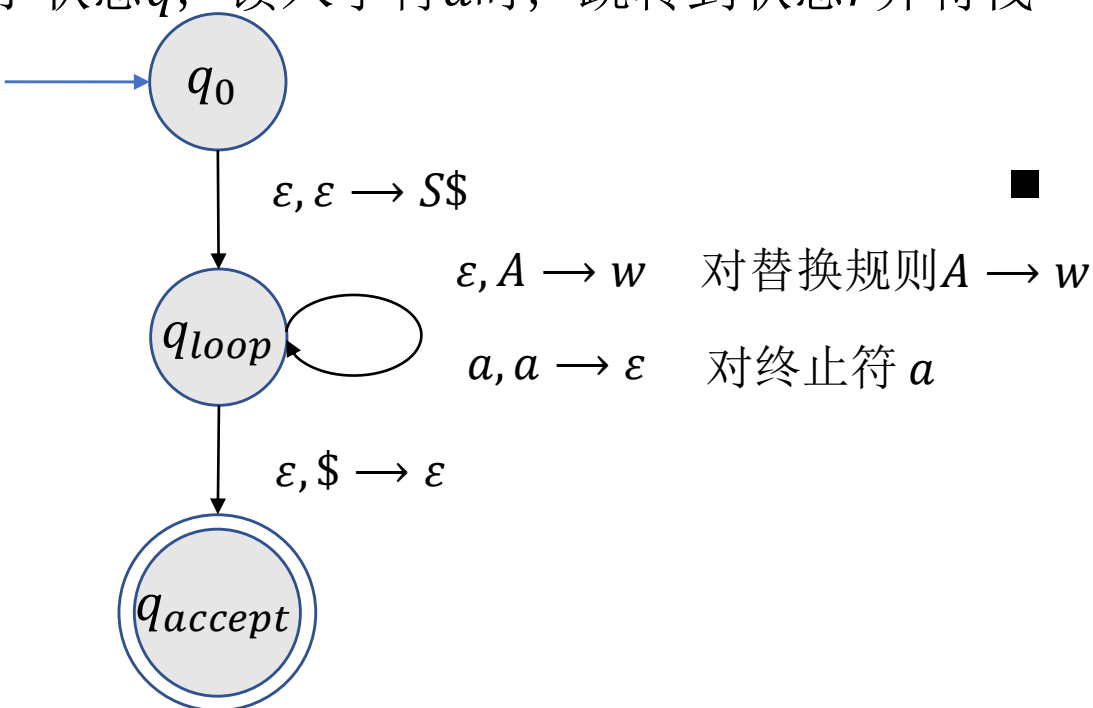


CFG与PDA等价

引理. 一个语言是上下文无关的, 则它能被某个下推自动机识别.

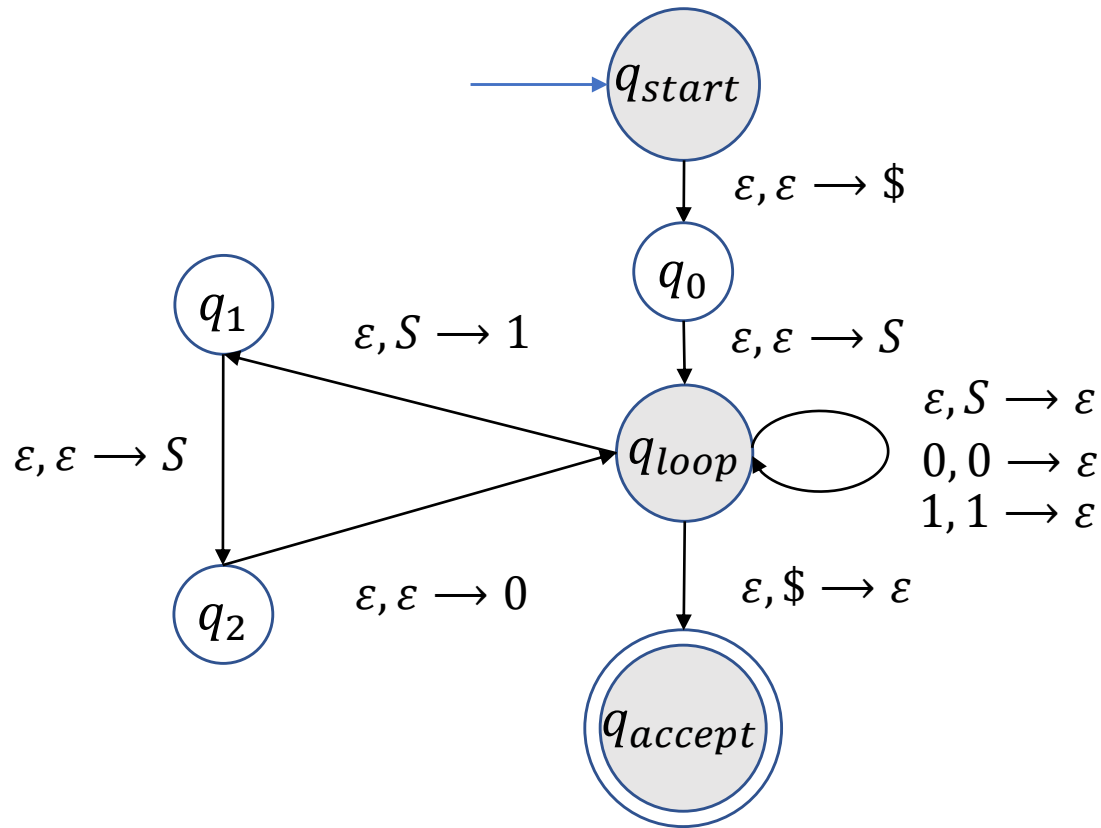
证明(续).

- 采用一种缩写记号表示转移函数, 即一步把一个字符串写入到栈中.
- $(r, u) \in \delta(q, a, s)$ 表示处于状态 q , 读入字符 a 时, 跳转到状态 r 并将栈顶字符 s 替换成字符串 u .
- 自己补充证明细节.



CFG与PDA等价

练习. 将 $G_1: S \rightarrow 0S1|\varepsilon$ 转化成一个下推自动机 P_1 .



CFG与PDA等价

引理. 一个语言若能被某个下推自动机识别, 则它是上下文无关的.

证明思路. 假设语言 L 能被PDA P 识别, 要构造上下文无关语法 G 使得字符串能由 G 生成当且仅当 P 接受该字符串.

可以假定

- P 只有一个接受状态.
- P 接受时栈为空.
- 每一个状态转移要么向栈中压入一个元素要么从栈中将顶部元素移除, 但不能同时进行.

CFG与PDA等价

给定下推自动机 P ，存在一个等价的下推自动机 P' 使得

- P' 只有一个接受状态.
- P' 接受时栈为空.
- 每一个状态转移要么向栈中压入一个元素要么从栈中将顶部元素移除，但不能同时进行.

解：

- 引入一个普通状态 q' ，将每个原来的接受状态 q 变为普通状态，并添加

$q \xrightarrow{\varepsilon, \varepsilon \rightarrow \varepsilon} q'$ 以及 $q' \xrightarrow{\varepsilon, a \rightarrow \varepsilon} q', \forall a \in \Gamma$. (q' 的作用是用来清空栈的内容)

- 引入一个新的接受状态 q_a ，添加 $q' \xrightarrow{\varepsilon, \$ \rightarrow \varepsilon} q_a$. (保证接受时栈为空)

- 若 $q \xrightarrow{a, b \rightarrow c} r$ ，则添加一个辅助状态 s ，将 $q \xrightarrow{a, b \rightarrow c} r$ 替换为

$$q \xrightarrow{a, b \rightarrow \varepsilon} s, s \xrightarrow{\varepsilon, \varepsilon \rightarrow c} r$$

CFG与PDA等价

引理. 一个语言若能被某个下推自动机识别, 则它是上下文无关的.

证明思路.

对于 P 的任何两个状态 p, q , 引入一个对应的变量 A_{pq} 使得 A_{pq} 能派生所有将 P 从处于空栈的状态 p 变到处于空栈的状态 q 的字符串.

两种可能.

- 栈在中间某个状态 r 为空.

$$A_{pq} \rightarrow A_{pr}A_{rq}$$

- 栈只有在开始和结束时为空.

$$A_{pq} \rightarrow aA_{rs}b$$

其中 a 为第一个读入的字符, b 为最后一个读入的字符, r 是紧接 p 之后的状态, s 是 q 之前的状态.

CFG与PDA等价

引理. 一个语言若能被某个下推自动机识别, 则它是上下文无关的.

证明. 给定 $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{accept}\})$, 语法 G 有变量 $A_{pq}, p, q \in Q$, 起始变量为 $A_{q_0, accept}$, 替换规则为:

- 对任何 $p, q, r, s \in Q, t \in \Gamma, a, b \in \Sigma_\varepsilon$, 若 $\delta(p, a, \varepsilon)$ 包含 (r, t) 且 $\delta(s, b, t)$ 包含 (q, ε) , 则有相应的替换规则

$$A_{pq} \rightarrow aA_{rs}b$$

- 对任何 $p, q, r \in Q$, 有相应的替换规则

$$A_{pq} \rightarrow A_{pr}A_{rq}$$

- 对任何 $p \in Q$, 有相应的替换规则

$$A_{pp} \rightarrow \varepsilon$$

CFG与PDA等价

证明(续).

A_{pq} 派生 x 当且仅当 x 能将 P 从处于空栈的状态 p 变到处于空栈的状态 q .

用归纳法证明(作为练习).

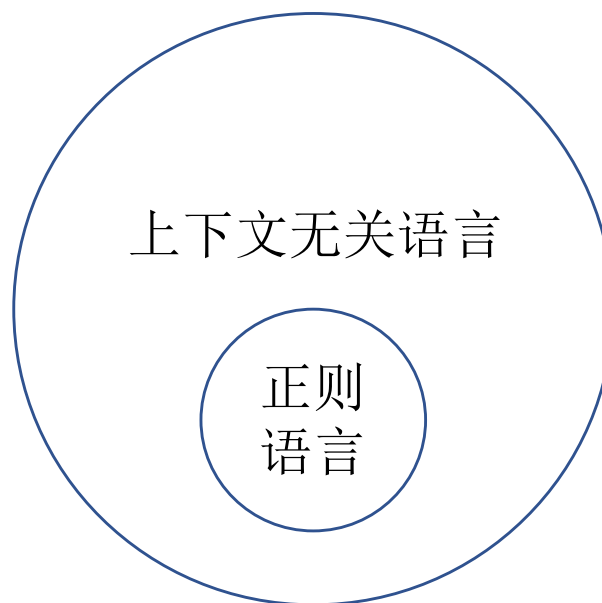


CFG与PDA等价

推论. 正则语言一定是上下文无关的.

证明.

- 任何正则语言 L 可由有非确定性限自动机识别.
- 非确定性限自动机是特殊的下推自动机.
- 故 L 是上下文无关的.



4 非上下文无关语言

非上下文无关语言

定理(上下文无关语法的泵引理). 如果 A 是上下文无关语言, 则存在正整数 p (泵长度)使得 A 中任何一个长度不小于 p 的字符串 s 都能被划分成 $s = uvxyz$ 且满足下述条件:

- (1) 对于每一个 $i \geq 0$, $uv^i xy^i z \in A$.
- (2) $|vy| > 0$.
- (3) $|vxy| \leq p$.

非上下文无关语言

定理(上下文无关语法的泵引理). 如果 A 是上下文无关语言, 则存在正整数 p (泵长度)使得 A 中任何一个长度不小于 p 的字符串 s 都能被划分成 $s = uvxyz$ 且满足下述条件:

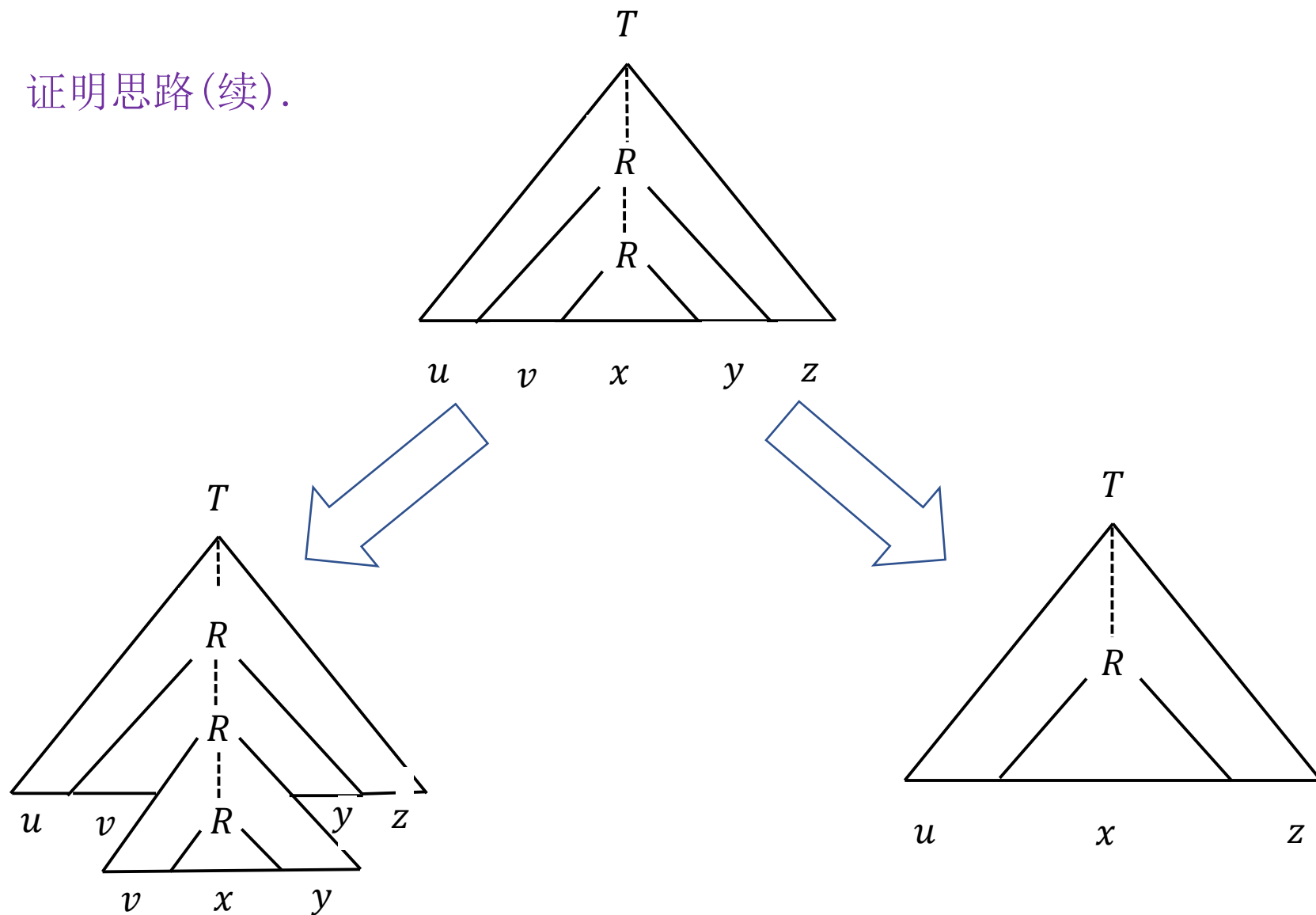
(1) 对于每一个 $i \geq 0$, $uv^i xy^i z \in A$; (2) $|vy| > 0$; (3) $|vxy| \leq p$.

证明思路. 假设 G 是生成 A 的上下文无关语法.

- 设 s 是 A 中一个“很长”的字符串. 由于 $s \in A$, 它可以用 G 派生出来, 从而有一棵解析树.
- 由于 s “很长”, s 的解析树一定“很高”, 即这棵解析树一定有一条很长的从树根的起始变元到树叶上的终结符的路径.
- 根据鸽笼原理, 在这条长路径上一定有某个变元 R 重复出现.

非上下文无关语言

证明思路(续).



非上下文无关语言

定理(上下文无关语法的泵引理). 如果 A 是上下文无关语言, 则存在正整数 p (泵长度)使得 A 中任何一个长度不小于 p 的字符串 s 都能被划分成 $s = uvxyz$ 且满足下述条件:

- (1) 对于每一个 $i \geq 0$, $uv^i xy^i z \in A$.
- (2) $|vy| > 0$.
- (3) $|vxy| \leq p$.

证明. 令 $G = (V, \Sigma, R, S)$ 是生成 A 的上下文无关语法.

假设 b 是任何规则右端字符串的最大长度.

- 解析树中每个顶点最多有 b 个孩子.
- 因此如果解析树的高度为 h , 则该树所生成的字符串长度最多为 b^h .
- 换言之, 若字符串长度至少为 $b^h + 1$, 则解析树的高度至少为 $h + 1$.

非上下文无关语言

证明(续).

假设 b 是任何规则右端字符串的最大长度.

令 $p = b^{|V|+1} + 1$. 任取 $s \in A$ 且 $|s| \geq p$.

- 令 T 是生成 s 的一棵解析树使得其在所有这样的解析树中所包含的点数最少.
- 则 T 高度至少为 $|V| + 1$, 即存在一条从起始变量到叶子的长度至少为 $|V| + 1$ 的路.
- 这条路有至少 $|V| + 1$ 个变量. 由**鸽笼原理**, 某个变量 R 出现了两次.
→ 选择这条路上最后 $|V| + 1$ 个变量中所重复出现的变量.

非上下文无关语言

证明(续).

假设 b 是任何规则右端字符串的最大长度.

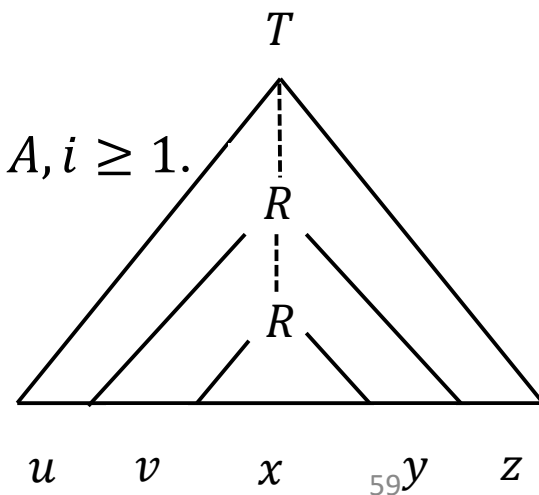
令 $p = b^{|V|+1} + 1$. 任取 $s \in A$ 且 $|s| \geq p$.

- 令 T 是生成 s 的一棵解析树使得其在所有这样的解析树中所包含的点数最少.
- 存在某条从起始变量到叶子的路, 某个变量 R 出现了两次.

由前所述,

- 将上面 R 下的子树替换下面 R 下的子树, $uv^i xy^i z \in A, i \geq 1$.
- 将下面 R 下的子树替换上面 R 下的子树, $uxz \in A$.

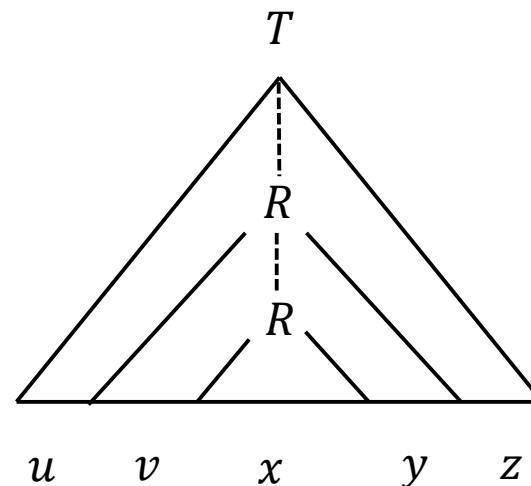
从而(1)满足.



非上下文无关语言

证明(续).

- 如果 $v = y = \varepsilon$, 则用小子树替换大子树后所得到的解析树仍生成 s , 这与 T 的选法是矛盾的. (2) 满足.



- 上面的 R 下的子树生成了 vxy , 因为两个 R 均出现在最后 $|V| + 1$ 个变量中, 这个子树的高度最多为 $|V| + 1$. 因此 $|vxy| \leq b^{|V|+1} < p$.

证毕. ■

非上下文无关语言

例 1: 证明 $B = \{a^n b^n c^n | n \geq 0\}$ 不是上下文无关语言.

证明. 假设 B 是上下文无关的, 令 p 为泵长度.

则 $s = a^p b^p c^p$ 可写成 $s = uvxyz$, 其中 v 或 y 不空.

情况 1. v 和 y 都只含一个类型的字符.

则 uv^2xy^2z 不可能有相同数量的 a, b, c .

情况 2. v 或 y 含两个类型的字符.

则 uv^2xy^2z 可能有相同数量的 a, b, c , 但顺序是打乱的.

无论哪种情况, $uv^2xy^2z \notin B$, 与泵引理矛盾. ■

非上下文无关语言

例 2: 证明 $C = \{a^i b^j c^k \mid 0 \leq i \leq j \leq k\}$ 不是上下文无关语言.

证明. 假设 C 是上下文无关的, 令 p 为泵长度.

则 $s = a^p b^p c^p$ 可写成 $s = uvxyz$, 其中 v 或 y 不空.

情况 1. v 和 y 都只含一个类型的字符. 则 a, b, c 中有一个字符 τ 既不出现在 v 中又不出现在 y 中.

1.1 $\tau = a$. 则 uxz 中 a 的数量与 s 一样, 但 b 或 c 的数量比 s 少, 故 $uxz \notin C$.

1.2 $\tau = b$. 因 $|vy| > 0$, vy 中要么含 a 要么含 c . 若 vy 中含 a , 则 uv^2xy^2z 中 a 的数量多于 b ; 若 vy 中含 c , 则 uxz 中 b 的数量多于 c . 故 $uv^2xy^2z \notin C$ 或 $uxz \notin C$.

1.3 $\tau = c$. 则 uv^2xy^2z 中 a 或 b 的数量比 c 多, 故 $uv^2xy^2z \notin C$.

情况 2. v 或 y 含两个类型的字符.

则 uv^2xy^2z 中 a, b, c 的顺序是打乱的, 故 $uv^2xy^2z \notin C$. ■

非上下文无关语言

例 3: 证明 $D = \{ww | w \in \{0,1\}^*\}$ 不是上下文无关语言.

证明. 假设 D 是上下文无关的, 令 p 为泵长度.

则 $s = 0^p 1^p 0^p 1^p$ 可写成 $s = uvxyz$, 其中 $|vxy| \leq p$.

情况 1. vxy 不经过“中间点”.

1.1 vxy 包含在前半部分中. 则 uv^2xy^2z 后一半的第一个字符必为1, 从而它不是 ww 的形式.

1.2 vxy 包含在后半部分中. 则 uv^2xy^2z 前半部分的最后一个字符必为0, 从而它不是 ww 的形式.

情况 2. vxy 经过“中间点”.

则 uxz 一定是 $0^p 1^i 0^j 1^p$ 的形式, 其中 i, j 不能同时为 p . 故 uxz 不是 ww 的形式.

无论哪种情况, 与泵引理矛盾. ■

非上下文无关语言

练习. 证明 $D = \{0^n 1^n 0^n 1^n | n \geq 0\}$ 不是上下文无关语言.

与例 3 的分析类似.

上下文无关语法总结

上下文无关语法总结

- 上下文无关语法
 - 定义
 - 设计
- 下推自动机
 - 定义: $a, b \rightarrow c$ 的解释.
 - 设计
- 上下文无关语法与下推自动机的等价性
 - CFG到PDA: 非确定性猜测替换规则 + 栈存储生成的中间字符串
 - PDA到CFG: 每一个状态对引入相应的变量
- 非上下文无关语言
 - 泵引理的证明
 - 泵引理的运用

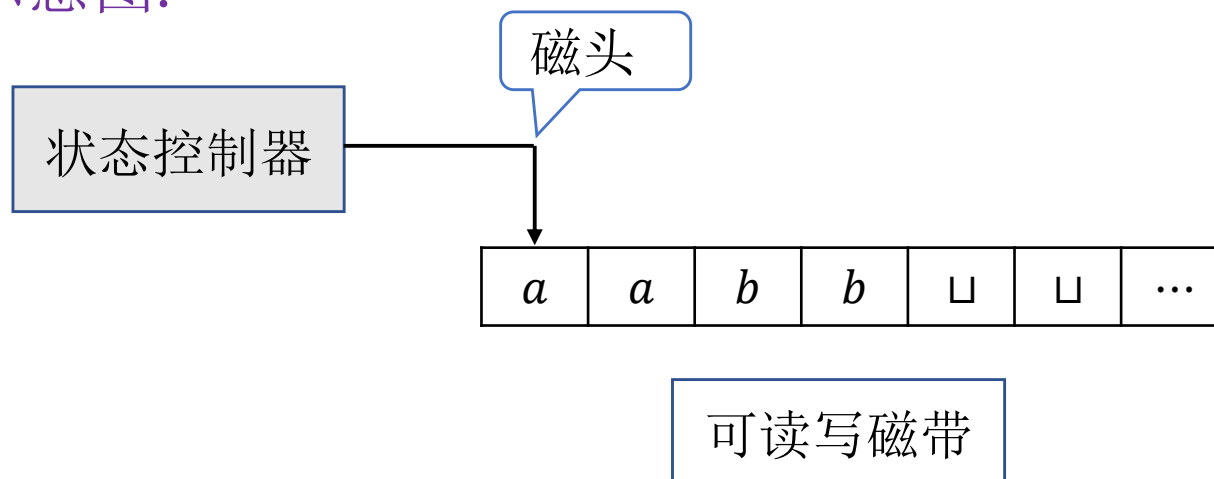
计算理论-丘奇-图灵论题

- 图灵机
- 图灵机的变种
- 算法的定义
- 描述图灵机的术语

1 图灵机

图灵机

抽象示意图.



- 磁带向右无限伸展  无限大内存
- 既能从磁带读取内容又能往磁带上写内容.
- 磁头可向左或向右移动.
- 可在任何时候接受或者拒绝.

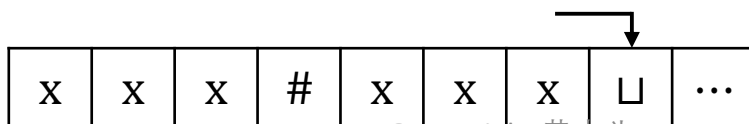
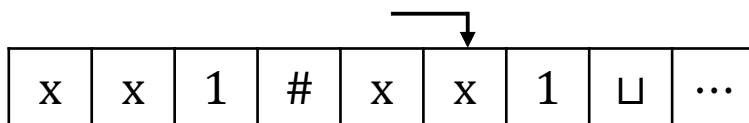
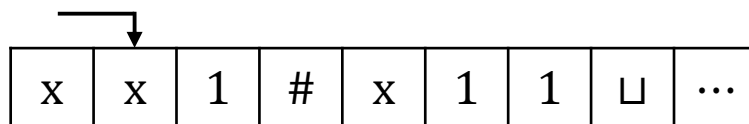
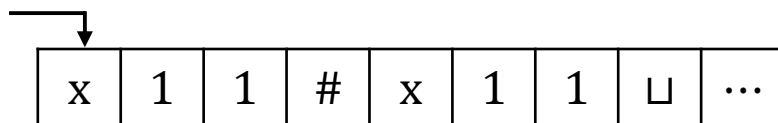
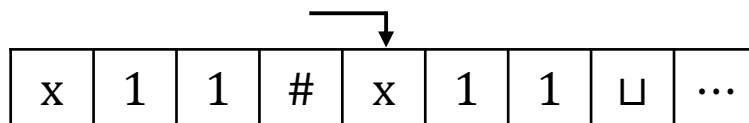
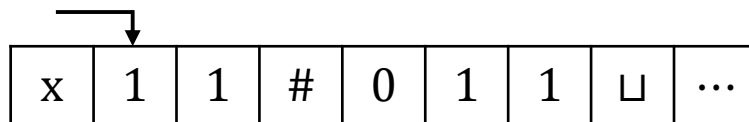
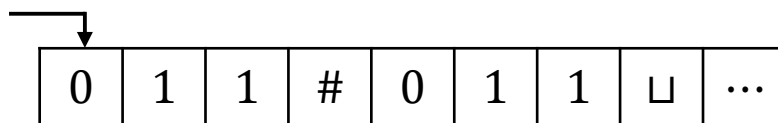
图灵机-引例

设计识别语言 $B = \{w\#w \mid w \in \{0,1\}^*\}$ 的图灵机.

$M =$ 对于输入字符串 w :

1. 在 $\#$ 两边对应的位置上来回移动, 检查这些对应位置是否包含相同的符号. 如不是或者没有 $\#$, 则拒绝.
为跟踪对应的符号, 划去所有检查过的符号.
2. 当 $\#$ 左边的所有符号都被划去时, 检查 $\#$ 右侧是否还有符号. 如果有则拒绝, 否则接受.

图灵机-引例



磁头右移

磁头左移

磁头右移

...

图灵机-正式定义

定义(图灵机). 一个图灵机是一个7-元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

其中

- Q 是有限状态集.
- Σ 是不含 $\sqcup$ 的输入字母表.
- Γ 是磁带字母表, 满足 $\sqcup \in \Gamma, \Sigma \subseteq \Gamma$.
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 是转移函数.
- q_0 是起始状态.
- q_{accept} 是接受状态.
- q_{reject} 是拒绝状态.

$$\delta(q, a) = (r, b, L)$$

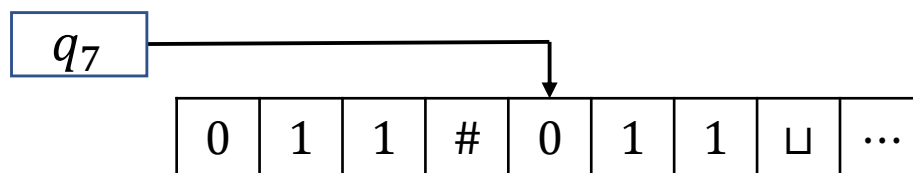
图灵机-图灵机的计算

给定图灵机 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$

- M 将输入 $w = w_1 w_2 \cdots w_n$ 存贮在磁带最左边的 n 个方格中, 其余部分填以空白符 $\sqcup$. 因为 $\sqcup \notin \Sigma$, 磁带上的第一个 $\sqcup$ 表示输入结束.
- M 开始运行后根据转移函数计算. 如果 M 试图将磁头从磁带的最左端移出, 即使转移函数指示的是 L, 磁头仍保持不动.
- 如果 M 进入接受或拒绝状态, 则停机 (halting). 否则 M 将永远运行下去 (looping).

图灵机-图灵机的计算

- 图灵机计算时需要记录3个信息：当前状态，磁头位置以及磁带内容.  配置 (configuration)
- 对于状态 q 以及 $u, v \in \Gamma^*$, uqv 表示当前状态为 q , 磁带内容为 uv , 磁头指向字符串 v 的第一个字符.
- $011\#q_7011$.



图灵机-图灵机的计算

- 如果图灵机能合法地从配置 C_1 一步进入 C_2 , 则称 C_1 产生 C_2 .
- 给定字符 $a, b, c \in \Gamma$, 字符串 $u, v \in \Gamma^*$, 状态 q_i, q_j , 配置 $uaq_i bv$,
 - 若 $\delta(q_i, b) = (q_j, c, L)$, 则 $uaq_i bv$ 产生 $uq_j acv$.
 - 若 $\delta(q_i, b) = (q_j, c, R)$, 则 $uaq_i bv$ 产生 $uacq_j v$.

思考: 若 $\delta(q_i, b) = (q_j, c, L)$, 则 $q_i bv$ 产生哪个配置?

图灵机-图灵机的计算

- 初始配置 q_0w 表示初始状态为 q_0 ，输入为 w .
- 接受(拒绝)配置是指状态为 q_{accept} (q_{reject})的配置. 接受配置和拒绝配置统称为停机配置.

图灵机 M 接受字符串 w 如果存在配置 $C_1, C_2, \dots, C_k$ 使得

- C_1 是初始配置.
- C_i 产生 C_{i+1} , $i = 1, 2, \dots, k - 1$.
- C_k 是接受配置.

给定图灵机 M , $L(M)$ 为所有被 M 接受的字符串的集合.

图灵机-正式定义

定义(图灵可识别语言). 给定语言 A , 若存在图灵机 M 使得 $A = L(M)$, 则称 A 是图灵可识别的.

一个图灵机被称为判定器(decider), 如果对于任何输入它要么接受要么拒绝.

定义(图灵可判定语言). 给定语言 A , 若存在判定器 M 使得 $A = L(M)$, 则称 A 是图灵可判定的.

- 可判定一定可识别.
- 存在可识别但不可判定语言(第4章).

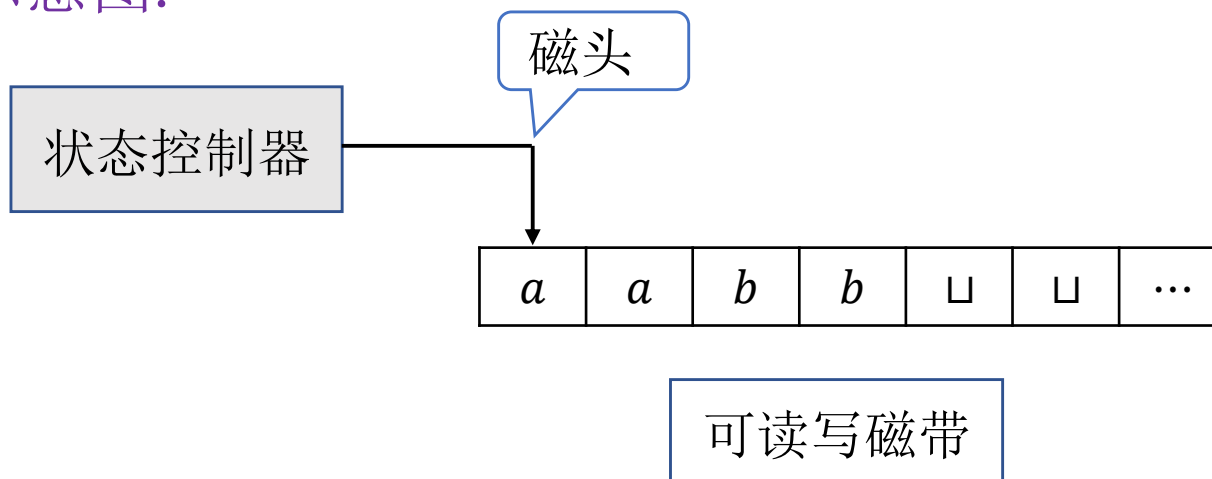
计算理论-丘奇-图灵论题

- 图灵机
- 图灵机的变种
- 算法的定义
- 描述图灵机的术语

1 图灵机

图灵机

抽象示意图.



- 磁带向右无限伸展  无限大内存
- 既能从磁带读取内容又能往磁带上写内容.
- 磁头可向左或向右移动.
- 可在任何时候接受或者拒绝.

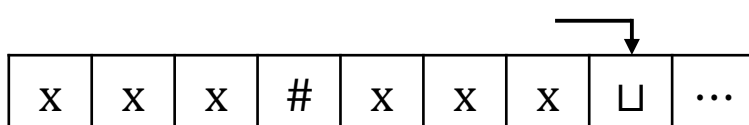
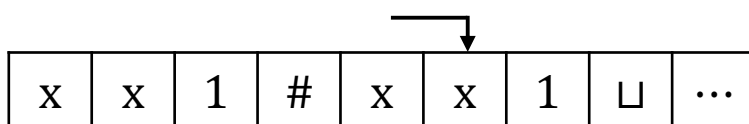
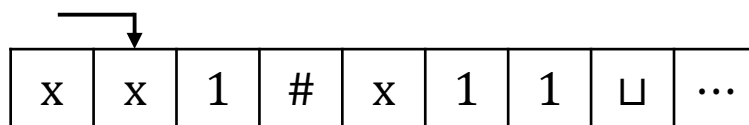
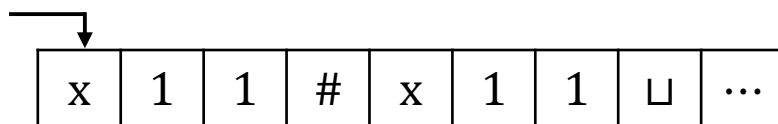
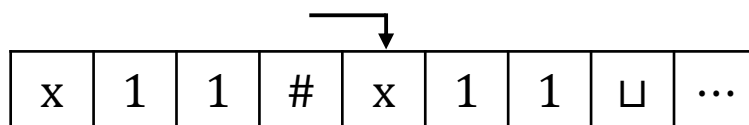
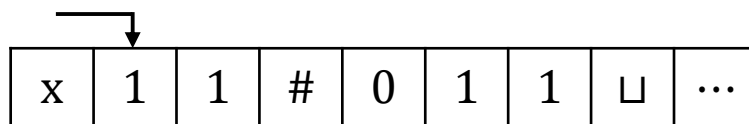
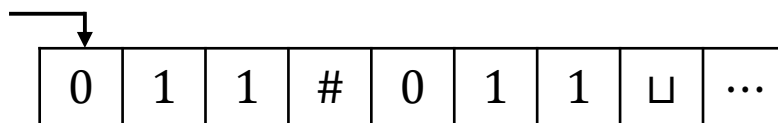
图灵机-引例

设计识别语言 $B = \{w\#w \mid w \in \{0,1\}^*\}$ 的图灵机.

$M =$ 对于输入字符串 w :

1. 在 $\#$ 两边对应的位置上来回移动, 检查这些对应位置是否包含相同的符号. 如不是或者没有 $\#$, 则拒绝. 为跟踪对应的符号, 划去所有检查过的符号.
2. 当 $\#$ 左边的所有符号都被划去时, 检查 $\#$ 右侧是否还有符号. 如果有则拒绝, 否则接受.

图灵机-引例



磁头右移

磁头左移

磁头右移

...

图灵机-正式定义

定义(图灵机). 一个图灵机是一个7-元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

其中

- Q 是有限状态集.
- Σ 是不含 $\sqcup$ 的输入字母表.
- Γ 是磁带字母表, 满足 $\sqcup \in \Gamma, \Sigma \subseteq \Gamma$.
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 是转移函数.
- q_0 是起始状态.
- q_{accept} 是接受状态.
- q_{reject} 是拒绝状态.

$$\delta(q, a) = (r, b, L)$$

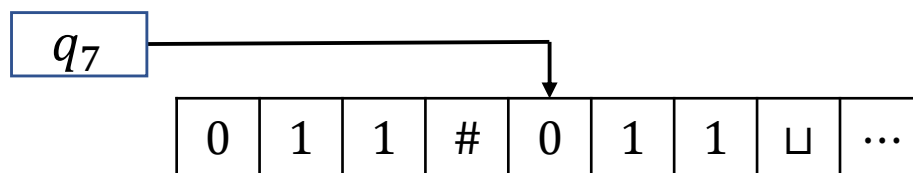
图灵机-图灵机的计算

给定图灵机 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$

- M 将输入 $w = w_1 w_2 \cdots w_n$ 存贮在磁带最左边的 n 个方格中, 其余部分填以空白符 $\sqcup$. 因为 $\sqcup \notin \Sigma$, 磁带上的第一个 $\sqcup$ 表示输入结束.
- M 开始运行后根据转移函数计算. 如果 M 试图将磁头从磁带的最左端移出, 即使转移函数指示的是 L, 磁头仍保持不动.
- 如果 M 进入接受或拒绝状态, 则停机 (halting). 否则 M 将永远运行下去 (looping).

图灵机-图灵机的计算

- 图灵机计算时需要记录3个信息：当前状态，磁头位置以及磁带内容. 配置 (configuration)
- 对于状态 q 以及 $u, v \in \Gamma^*$, uqv 表示当前状态为 q , 磁带内容为 uv , 磁头指向字符串 v 的第一个字符.
- $011\#q_7011$.



图灵机-图灵机的计算

- 如果图灵机能合法地从配置 C_1 一步进入 C_2 , 则称 C_1 产生 C_2 .
- 给定字符 $a, b, c \in \Gamma$, 字符串 $u, v \in \Gamma^*$, 状态 q_i, q_j , 配置 $uaq_i bv$,
 - 若 $\delta(q_i, b) = (q_j, c, L)$, 则 $uaq_i bv$ 产生 $uq_j acv$.
 - 若 $\delta(q_i, b) = (q_j, c, R)$, 则 $uaq_i bv$ 产生 $uacq_j v$.

思考: 若 $\delta(q_i, b) = (q_j, c, L)$, 则 $q_i bv$ 产生哪个配置?

图灵机-图灵机的计算

- 初始配置 q_0w 表示初始状态为 q_0 ，输入为 w 。
- 接受(拒绝)配置是指状态为 q_{accept} (q_{reject}) 的配置。接受配置和拒绝配置统称为停机配置。

图灵机 M 接受字符串 w 如果存在配置 $C_1, C_2, \dots, C_k$ 使得

- C_1 是初始配置。
- C_i 产生 C_{i+1} , $i = 1, 2, \dots, k - 1$ 。
- C_k 是接受配置。

给定图灵机 M , $L(M)$ 为所有被 M 接受的字符串的集合。

图灵机-正式定义

定义(图灵可识别语言). 给定语言 A , 若存在图灵机 M 使得 $A = L(M)$, 则称 A 是图灵可识别的.

一个图灵机被称为判定器(decider), 如果对任何输入它要么接受要么拒绝.

定义(图灵可判定语言). 给定语言 A , 若存在判定器 M 使得 $A = L(M)$, 则称 A 是图灵可判定的.

- 可判定一定可识别.
- 存在可识别但不可判定语言(第4章).

图灵机-正式定义

例 1: 设计判定 $A = \{0^{2^n} \mid n \geq 0\}$ 的图灵机.

M_1 = 对于输入字符串 w :

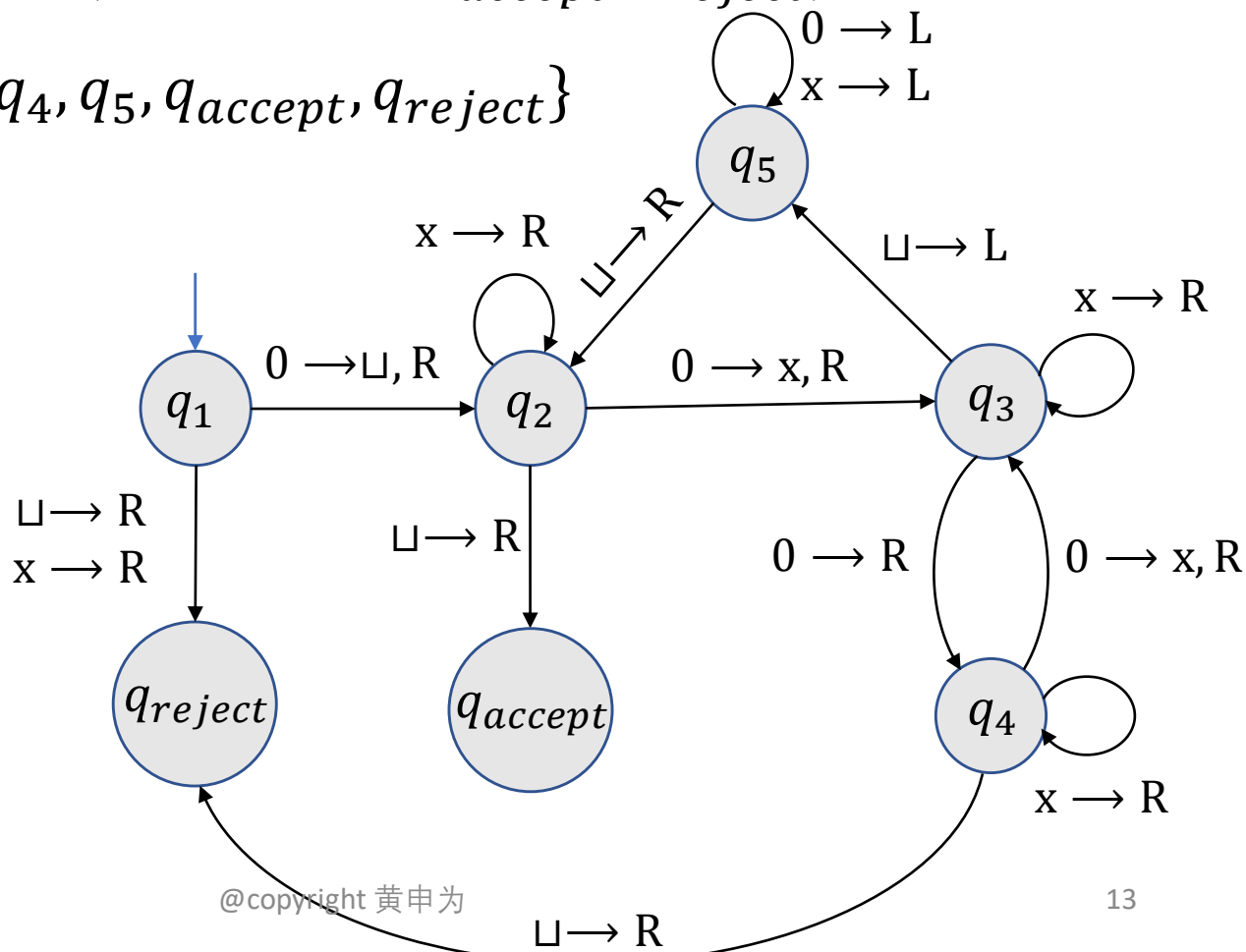
1. 从左往右扫描整个磁带, 每隔一个字符划去一个0.
2. 若第 1 步中磁带上只有1个0, 则接受; 若第 1 步中磁带上多于1个0且为奇数, 则拒绝.
3. 回到磁带最左端.
4. 回到第 1 步.

图灵机-正式定义

例 1: 设计判定 $A = \{0^{2^n} | n \geq 0\}$ 的图灵机.

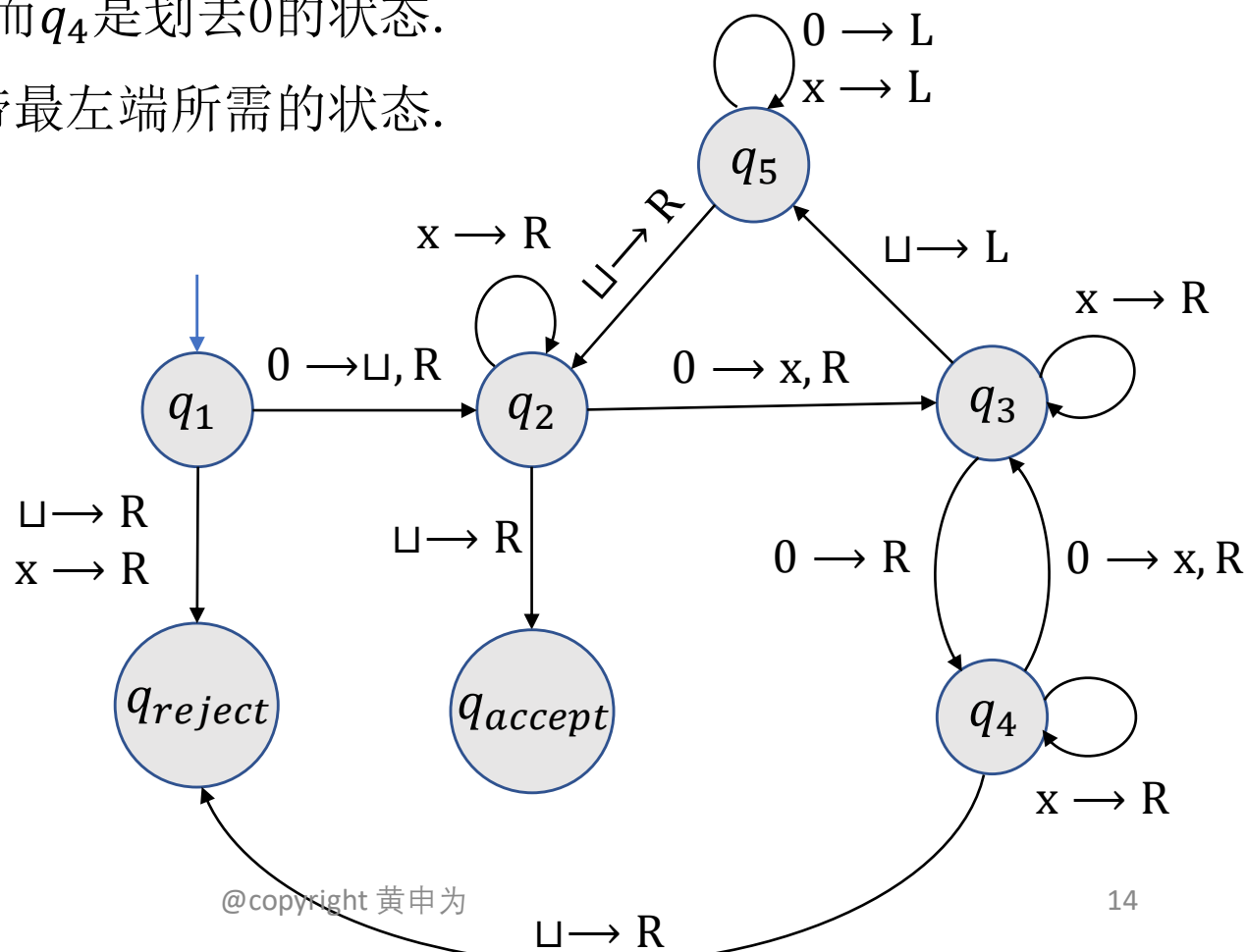
$$M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{accept}, q_{reject})$$

- $Q = \{q_1, q_2, q_3, q_4, q_5, q_{accept}, q_{reject}\}$
- $\Sigma = \{0\}$
- $\Gamma = \{0, x, \sqcup\}$
- δ 如图所示



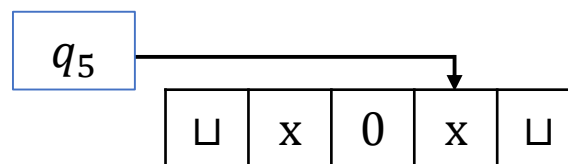
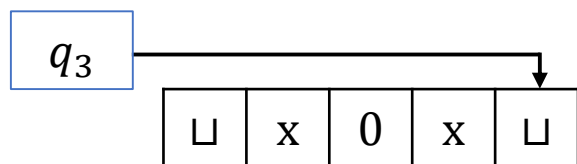
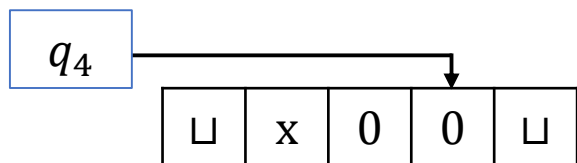
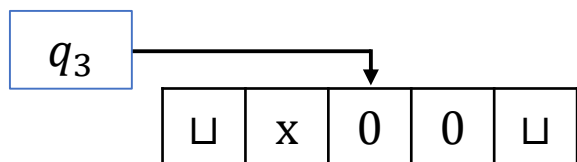
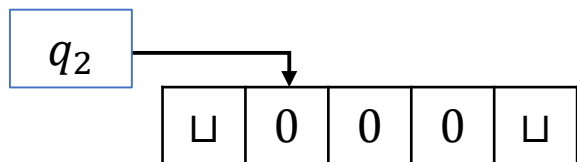
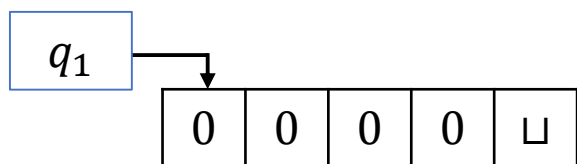
图灵机-正式定义

- q_1 是初始状态, 第一步将磁头最左端的0改写为 $\sqcup$, 来标记磁带的最左端.
- q_2 是进行划去0这个操作的初始状态.
- q_3 是跳过0的状态而 q_4 是划去0的状态.
- q_5 是磁头回到磁带最左端所需的的状态.

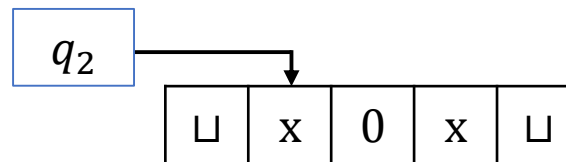
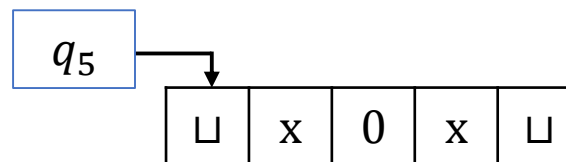


图灵机-正式定义

M_1 对于输入 $w = 0000$ 运行:

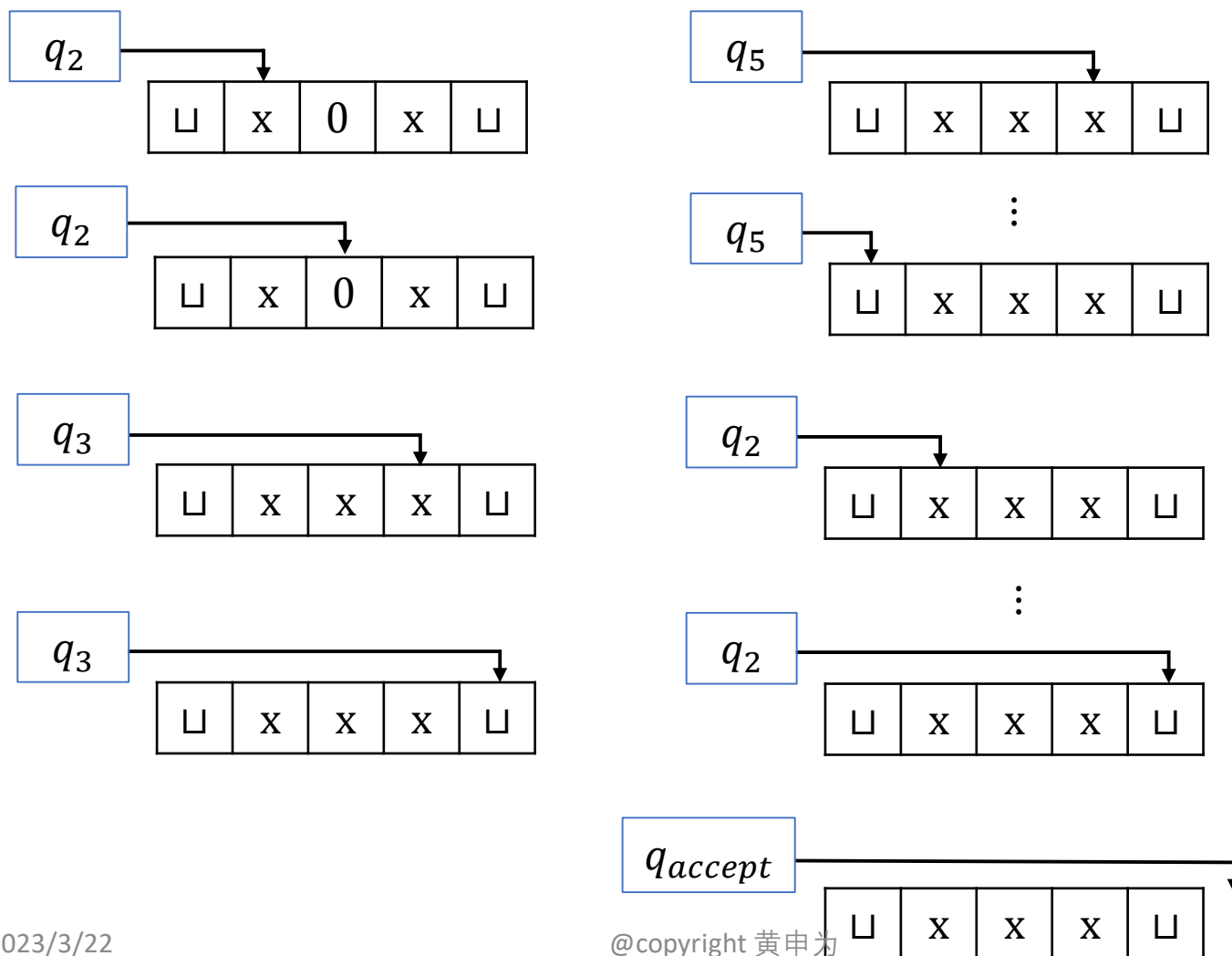


⋮



图灵机-正式定义

M_1 对于输入 $w = 0000$ 运行行为:



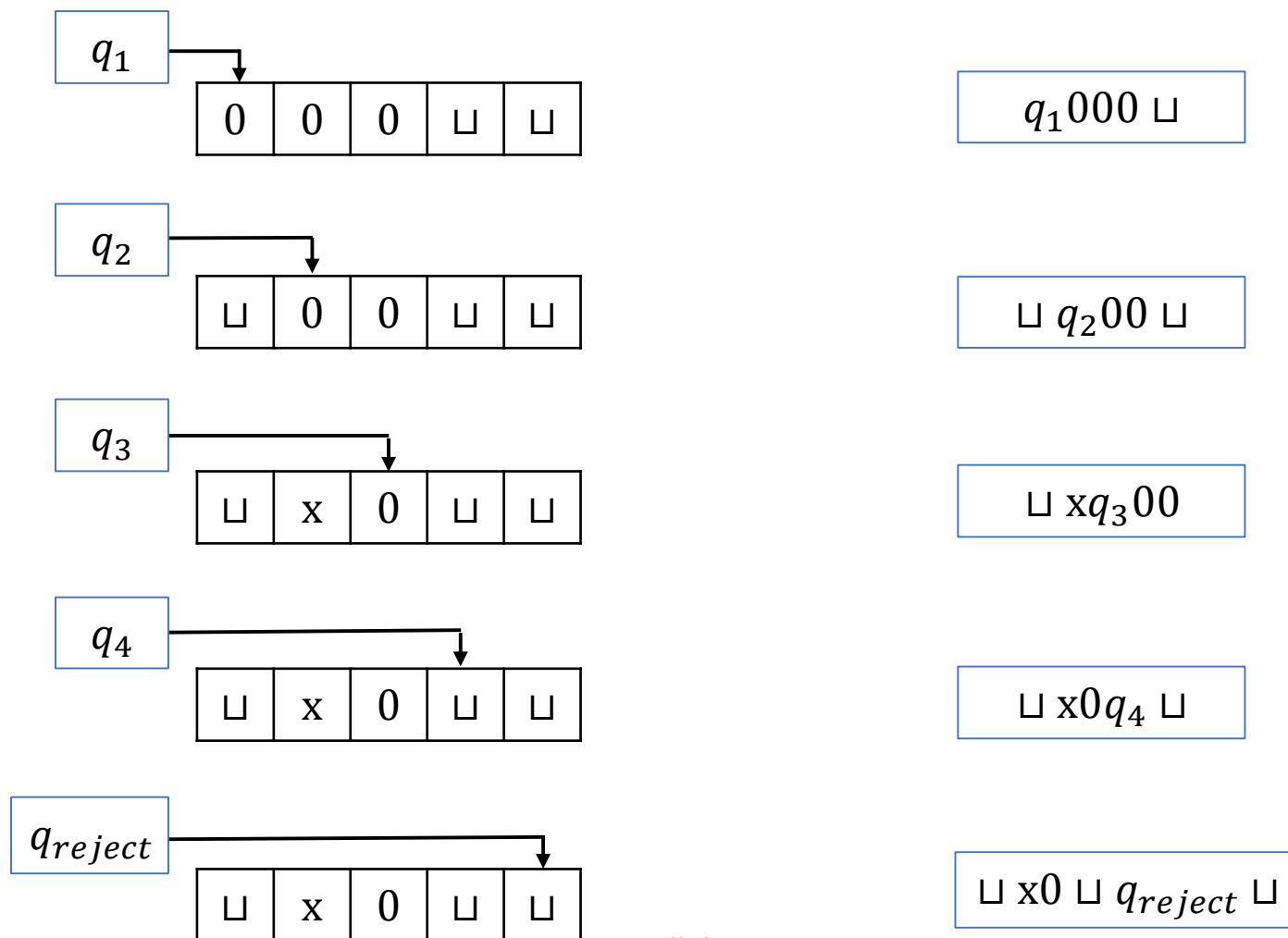
图灵机-正式定义

M_1 对于输入 $w = 0000$ 运行的配置序列为:

$q_1 0000$	$\sqcup q_2 x0x \sqcup$	$\sqcup xq_2xx \sqcup$
$\sqcup q_2 000$	$\sqcup xq_2 0x \sqcup$	$\sqcup xxq_2x \sqcup$
$\sqcup xq_3 00$	$\sqcup xxq_3x \sqcup$	$\sqcup xxxq_2 \sqcup$
$\sqcup x0q_4 0$	$\sqcup xxxq_3 \sqcup$	$\sqcup xxx \sqcup q_{accept}$
$\sqcup x0xq_3 \sqcup$	$\sqcup xxq_5x \sqcup$	
$\sqcup x0q_5x \sqcup$	$\sqcup xq_5xx \sqcup$	
$\sqcup xq_5 0x \sqcup$	$\sqcup q_5xxx \sqcup$	
$\sqcup q_5x0x \sqcup$	$q_5 \sqcup xxx \sqcup$	
$q_5 \sqcup x0x \sqcup$	$\sqcup q_2xxx \sqcup$	

图灵机-正式定义

练习. 写出 M_1 对于输入 $w = 000$ 运行过程和配置序列



图灵机-正式定义

例 2: 设计判定语言 $B = \{w\#w \mid w \in \{0,1\}^*\}$ 的图灵机 M_2 .

$M_2 =$ 对于输入字符串 w :

1. 在 $\#$ 两边对应的位置上来回移动, 检查这些对应位置是否包含相同的符号. 如不是或者没有 $\#$, 则拒绝. 为跟踪对应的符号, 划去所有检查过的符号.
2. 当 $\#$ 左边的所有符号都被划去时, 检查 $\#$ 右侧是否还有符号. 如果有则拒绝, 否则接受.

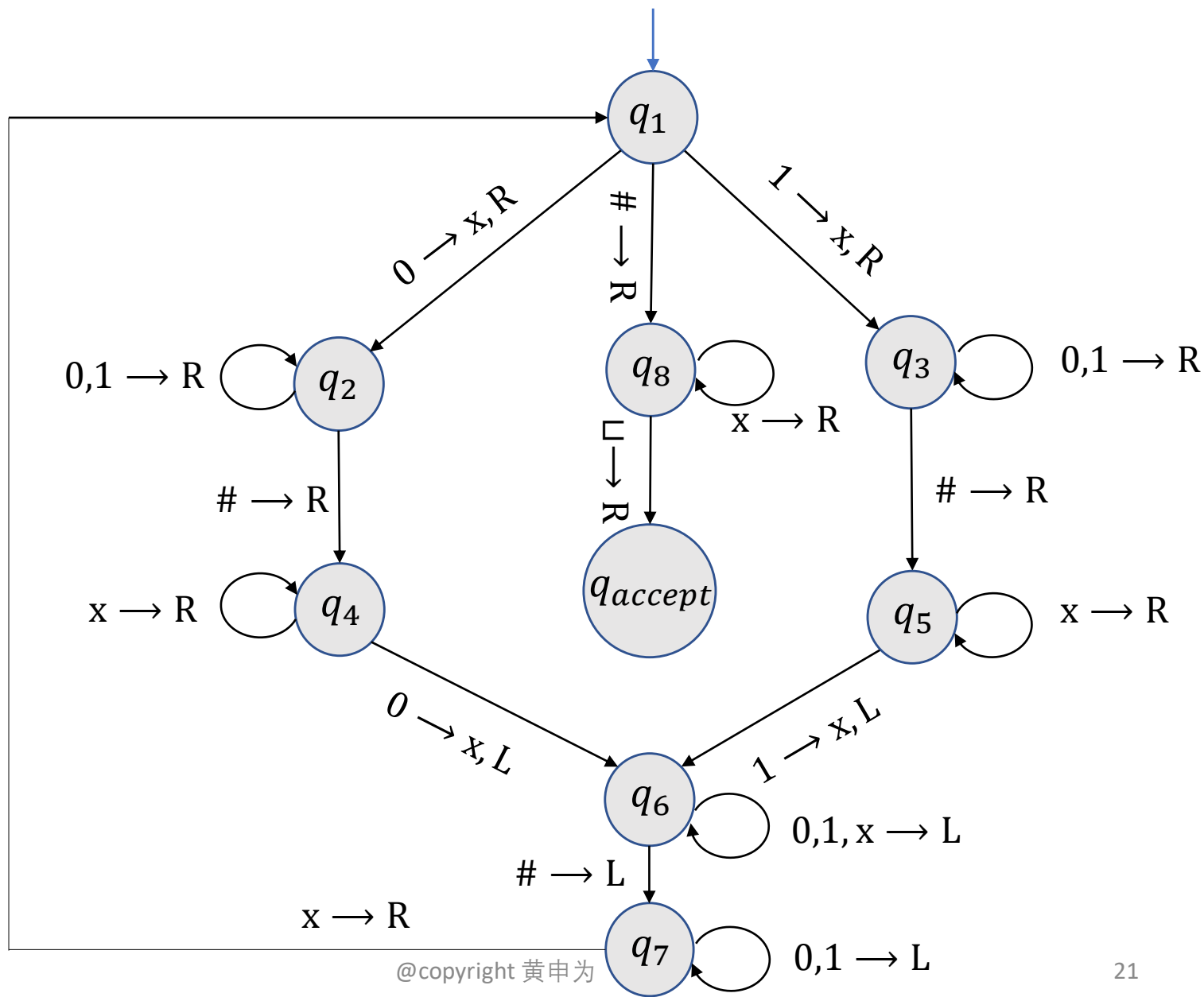
图灵机-正式定义

例 2: 设计判定语言 $B = \{w\#w \mid w \in \{0,1\}^*\}$ 的图灵机 M_2 .

$$M_2 = (Q, \Sigma, \Gamma, \delta, q_1, q_{accept}, q_{reject})$$

- $Q = \{q_1, q_2, \dots, q_8, q_{accept}, q_{reject}\}$
- $\Sigma = \{0, 1, \#\}$
- $\Gamma = \{0, 1, \#, x, \sqcup\}$
- δ 如图所示
 - q_{reject} 没有画出.
 - 若某个状态 q_i 没有一个出去的箭头上标有 a , 则表明当机器处于 q_i , 读取字符 a 时, 跳转到拒绝状态 q_{reject} .

图灵机-正式定义



图灵机-正式定义

练习. 画出 M_2 运行 $w = 0\#0$ 的流程并写出相应的配置序列.

思考: M_2 是否有可能处于状态 q_6 时读到 $\sqcup$?

图灵机-正式定义

例 3: 设计判定语言 $C = \{a^i b^j c^k \mid i \times j = k, i, j, k \geq 1\}$ 的图灵机 M_3 .

M_3 = 对于输入字符串 w :

1. 从左向右扫描输入来判定 w 是否是 $a^i b^j c^k$ 的形式. 若不是, 则拒绝; 否则回到磁带最左端.
2. 划去一个 a 然后将磁带移动到第一个 b . 来回在 b 和 c 之间游走, 划去对应位置的 b 和 c , 直到所有的 b 都划去. 若所有的 c 都划去后还有 b 没有划去, 则拒绝. 否则进入第 3 步.

$$i \times j > k$$

3. 若所有的 a 已划去,

$$i \times j < k$$

如果所有的 c 也已划去, 则接受; 否则拒绝.

否则还有没有划去的 a , 复原所有划去的 b , 重复第 2 步.

图灵机-正式定义

例 4: 设计判定语言

$$E = \{\#x_1\#x_2\cdots\#x_k \mid x_i \in \{0,1\}^* \text{ 且 } x_i \neq x_j, i \neq j\}$$

的图灵机 M_4 .

解: 机器 M_4 将 x_1 与 $x_2, \dots, x_k$ 进行比较, 然后将 x_2 与 $x_3, \dots, x_k$ 进行比较, 以此类推.

图灵机-正式定义

$M_4 =$ 对于输入字符串 w :

1. 在磁带最左端的符号上作个记号. 如果这个符号是 $\sqcup$, 则接受. 若这个符号是 $\#$, 则进入第 2 步. 否则拒绝.
2. 向右扫描下一个 $\#$ 并在此符号上作记号. 如果在遇到 $\sqcup$ 之前没有碰到 $\#$, 说明只有 x_1 , 从而接受.
3. 通过来回移动, 比较作了记号的两个 $\#$ 后面的字符串是否相等. 若相等, 则拒绝; 否则进入第 4 步.
4. 将两个带有记号的 $\#$ 中右边那个的记号移到下一个 $\#$ 上. 如果在遇到 $\sqcup$ 之前没有碰到 $\#$, 则将两个带有记号的 $\#$ 中左边那个的记号移到下一个 $\#$ 上, 并将另外一个记号移到其后面的 $\#$ 上. 如果这时找不到 $\#$ 了, 说明所有的字符串都比较过了, 从而接受. 否则回到第 3 步.

图灵机-正式定义

练习：给出判定含有连续两个0的 $\{0,1\}$ -字符串的图灵机的状态转移图以及形式化定义.

2 图灵机的变种

图灵机的变种

- 双向图灵机：磁带可以向左右两个方向无限延展.
- 多磁带图灵机：有多个磁带，每个磁带均有一个磁头.
- 非确定性图灵机：同一个配置可以同时产生多个配置，每一个这样的选择对应一个计算的分支.
- 枚举器：有“打印机”的图灵机.
-

所有这些变种均和标准图灵机等价！

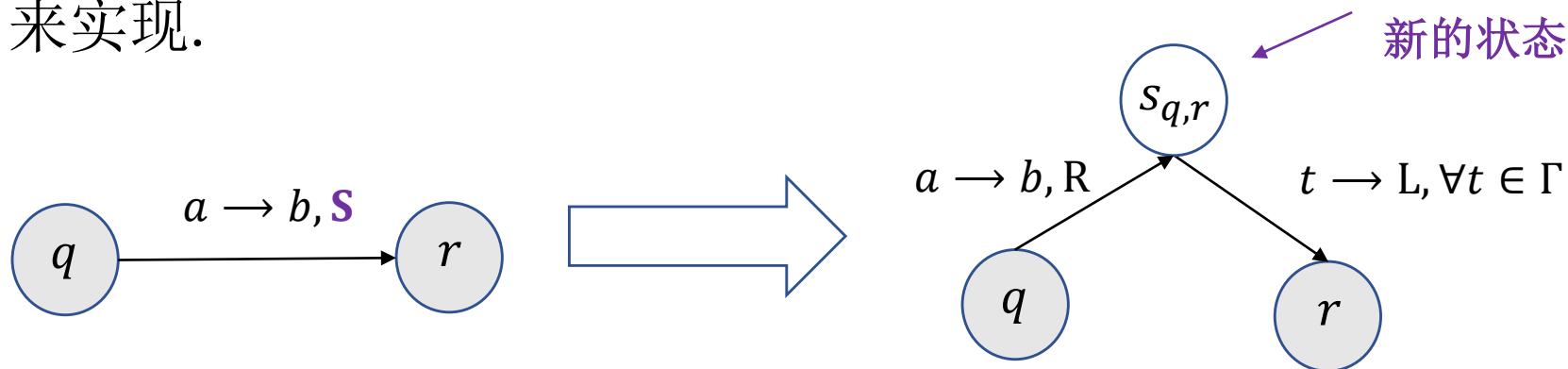
图灵机的变种

问题： 如果允许磁头保持不动，这样的图灵机是否比标准图灵机更强大？

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, \mathbf{S}\}.$$

回答： 显然不是！

每一个保持不动的动作可以通过右移一步再左移一步来实现。



思考： 最多需要引入多少个新的状态？

图灵机的变种-多磁带图灵机

定义(多磁带图灵机).

- 一个多磁带图灵机是有多个磁带的图灵机, 每个磁带有自己的磁头用于读写.
- 开始时输入出现在第一个磁带上, 其它的磁带都是空白.
- 转移函数允许多个带子同时进行读写和移动磁头:

$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$, 这里 k 是磁带个数.

$\delta(q, a_1, \dots, a_k) = (r, b_1, \dots, b_k, L, \dots, R)$ 表示当机器处于状态 q , 磁头 $1, \dots, k$ 分别读到 $a_1, \dots, a_k$, 则机器跳转到状态 r , 各磁头分别写下 $b_1, \dots, b_k$, 且按照指示移动每个磁头.

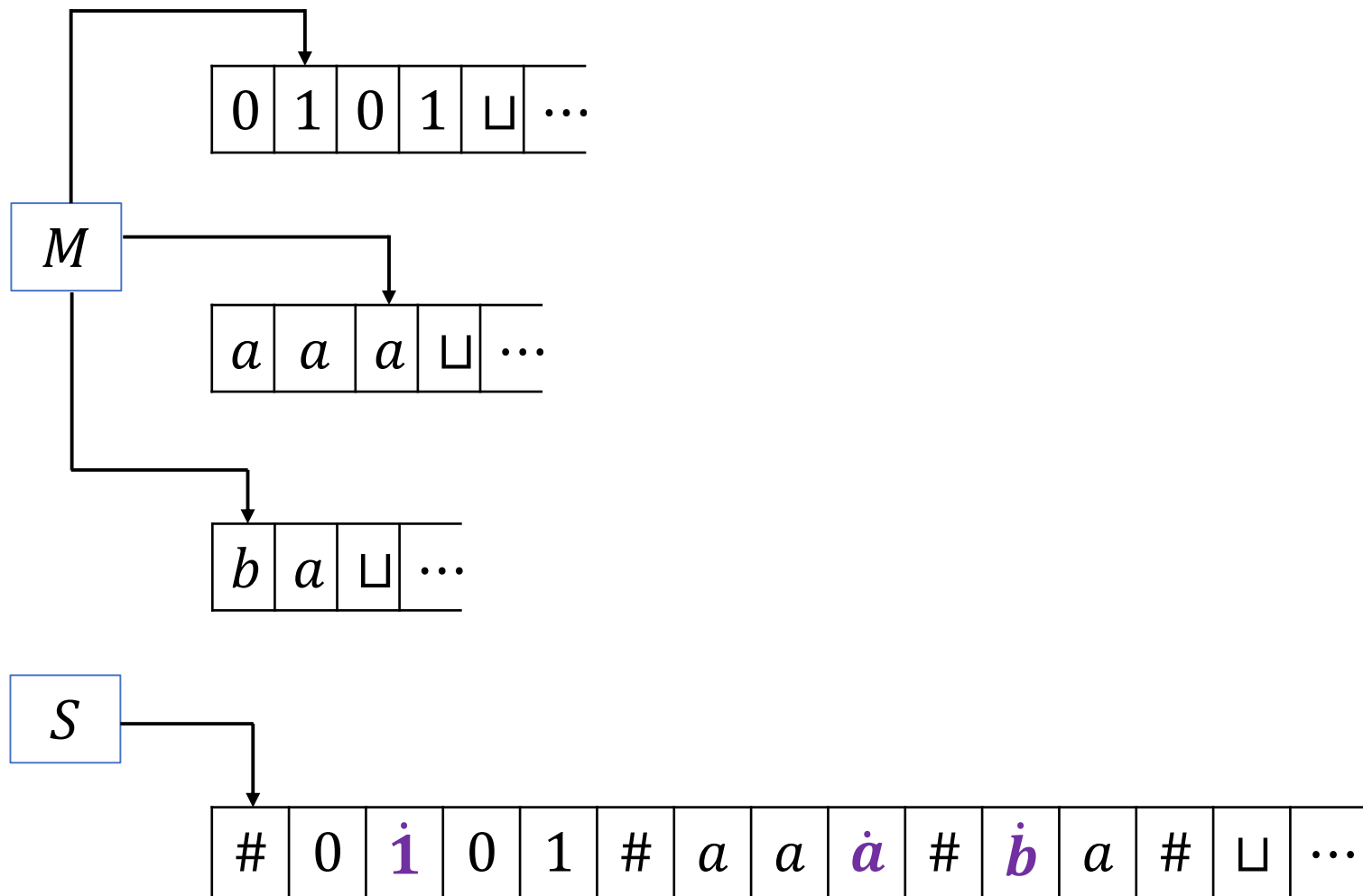
图灵机的变种-多磁带图灵机

定理 1 (多磁带图灵机). 每个多磁带图灵机有一个等价的标准图灵机.

证明. 给定多磁带图灵机 M , 需要构造一个等价的标准图灵机 S , 即如何用 S 模拟 M .

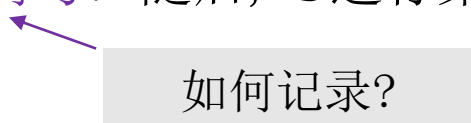
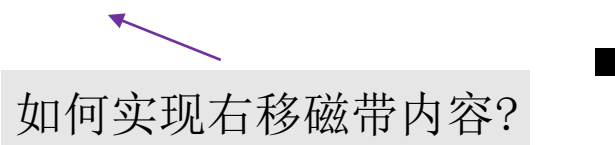
- 假设 M 有 k 个磁带. S 把这 k 个磁带的信息都存储在它的唯一磁带上并以此来模拟 k 个磁带的效果, 它用一个新的符号 $\#$ 作为定界符以分开不同磁带的內容.
- 除了磁带内容之外, S 还必须记录磁头的位置. 为此, 它在一个符号的顶上加个点以此来标记读写头在其磁带上的位置. 可以把这些带点的字符想象成虚拟磁头.

图灵机的变种-多磁带图灵机



图灵机的变种-多磁带图灵机

证明(续). $S =$ 对于输入字符串 $w_1 w_2 \cdots w_n$:

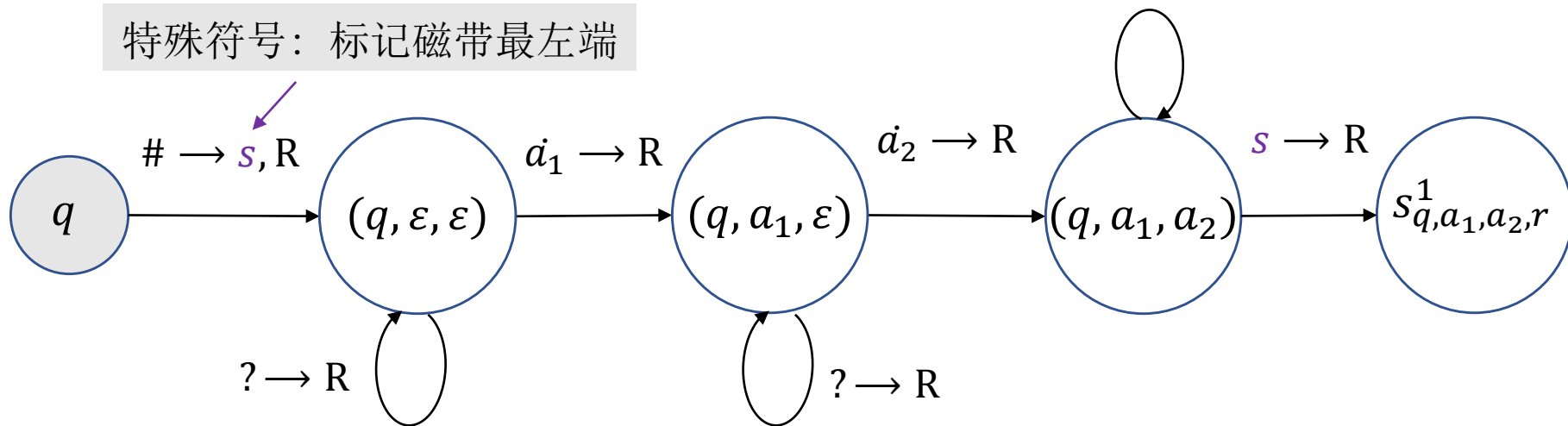
1. 初始时 S 上的内容为 $\# w_1 w_2 \cdots w_n \# \sqcup \# \sqcup \# \cdots \#$.
2. 为了模拟多带机的一步移动, S 从标记最左端的第一个 $\#$ 开始扫描至第 $(k + 1)$ 个 $\#$, 以此确定虚拟头下的符号. 随后, S 进行第二次扫描, 按照 M 所指定的方式更新磁带. 
3. 如果 S 将某个虚拟读写头向右移动到某个 $\#$ 上面, 就意味着 M 已将自己相应的读写头移动到了其所在的带子中的空白区域上, 即以前没有读过的区域上. 此时, S 在这个带方格上写下空白符并将该方格到最右端的方格中的内容都向右移动一个格. 然后再像以前一样继续模拟. 

图灵机的变种-多磁带图灵机

记录虚拟头下的符号.

- 引入辅助状态 $(q, a_1, a_2, \dots, a_k)$ 记录虚拟头下的符号, 其中 $q \in Q, a_i \in \Gamma_\varepsilon$.
- 引入辅助状态 $s_{q,a_1,\dots,a_k,r}^1$, 其中 r 是 $\delta(q, a_1, a_2, \dots, a_k)$ 中的状态分量. 这是模拟 M 转移函数的初始状态.

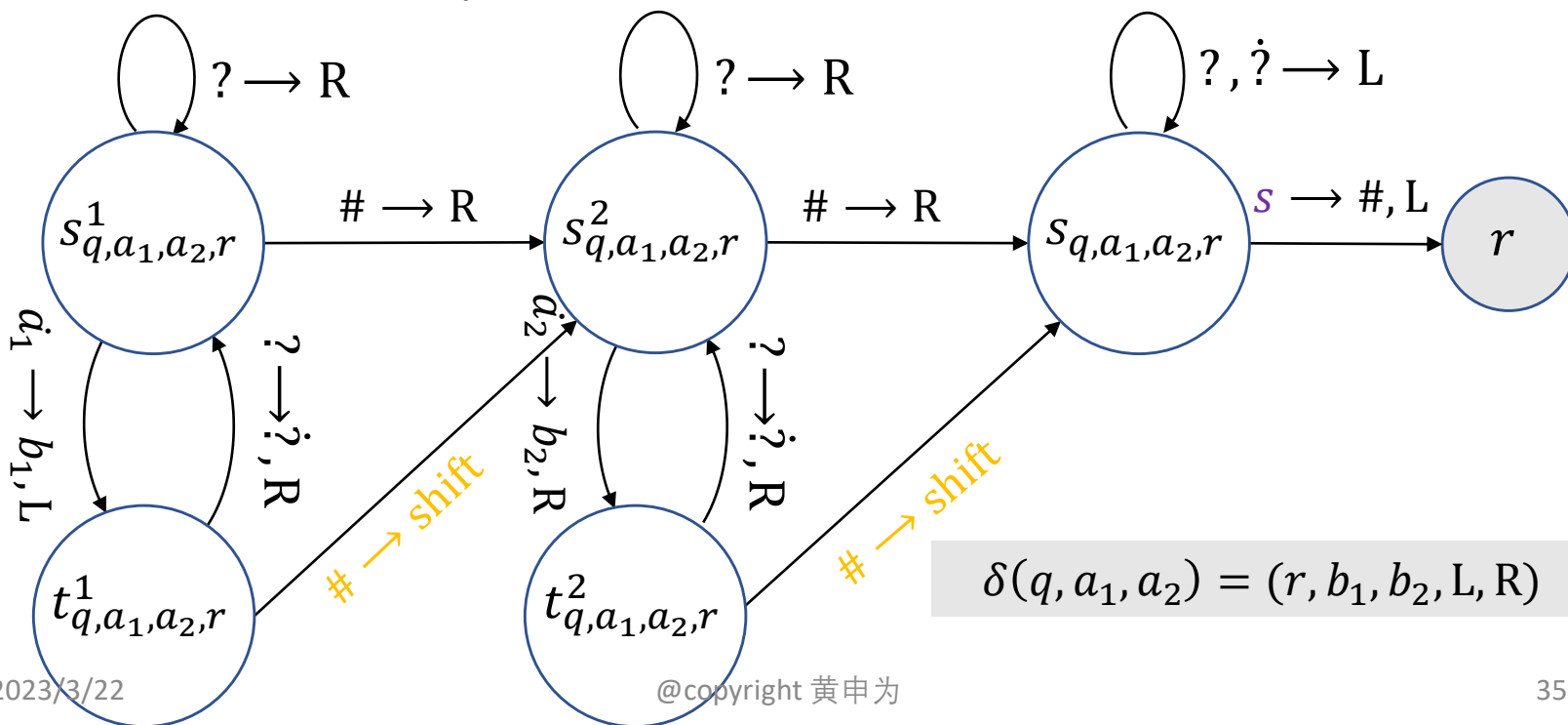
特殊符号: 标记磁带最左端



图灵机的变种-多磁带图灵机

按照 M 所指定的方式更新磁带

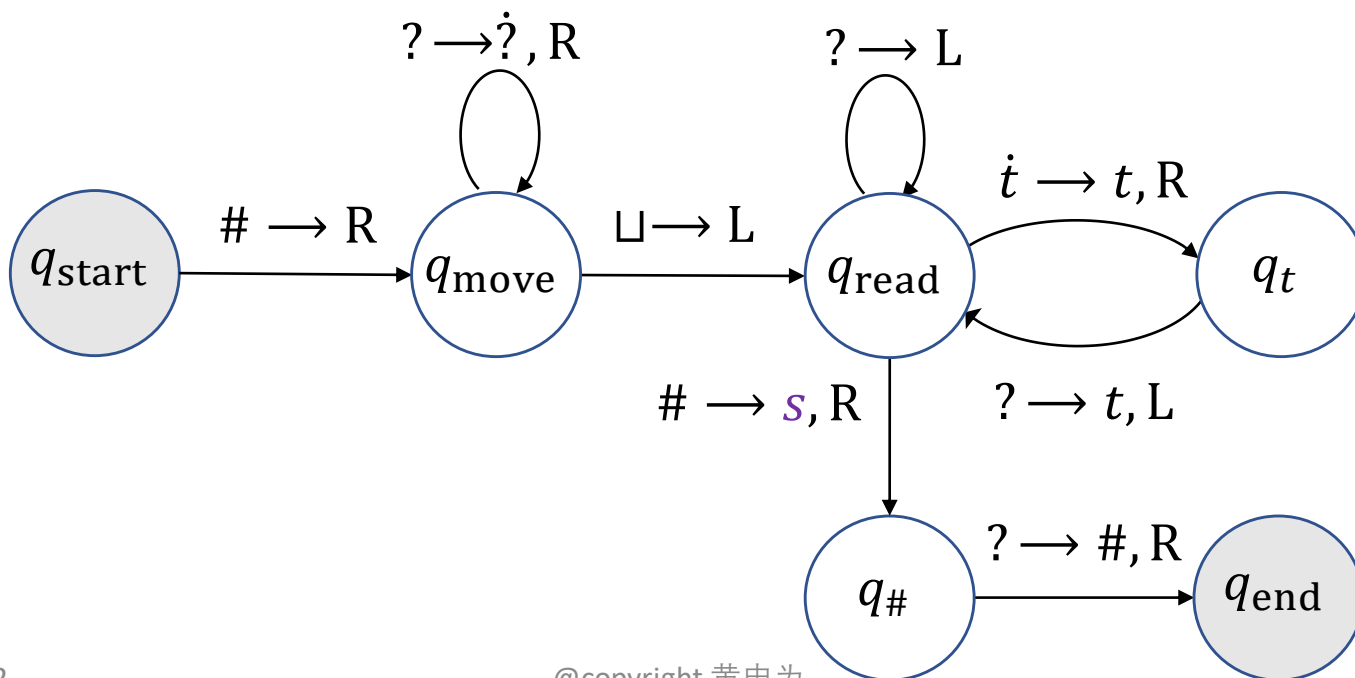
- 引入状态 $s_{q,a_1,\dots,a_k,r}^i, t_{q,a_1,\dots,a_k,r}^i$ 模拟第 i 个磁带上的读写和磁头位置变化.
- 引入状态 $s_{q,a_1,\dots,a_k,r}$ 用来回到磁带最左端.



图灵机的变种-多磁带图灵机

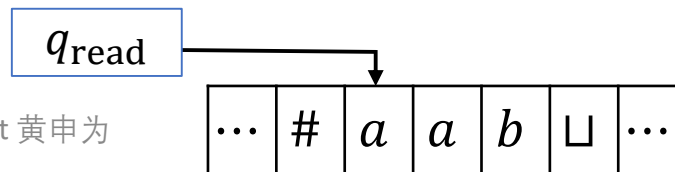
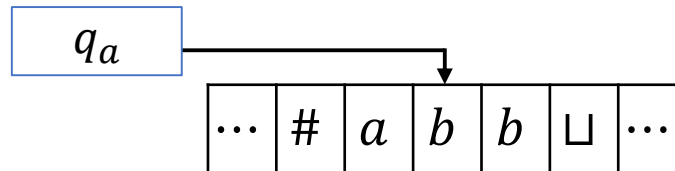
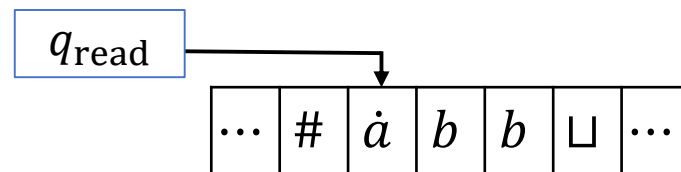
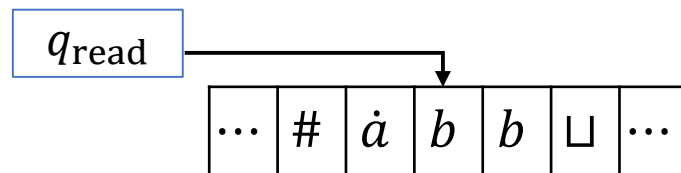
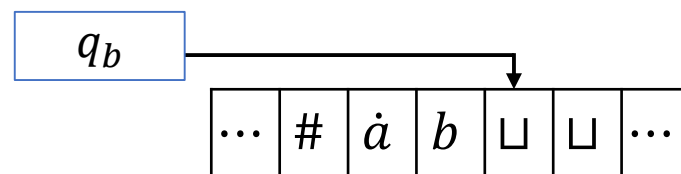
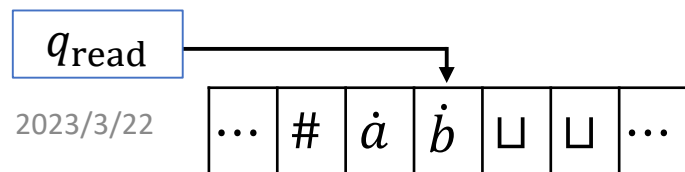
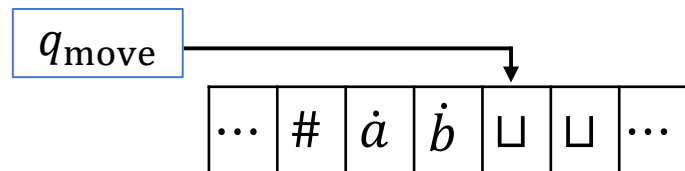
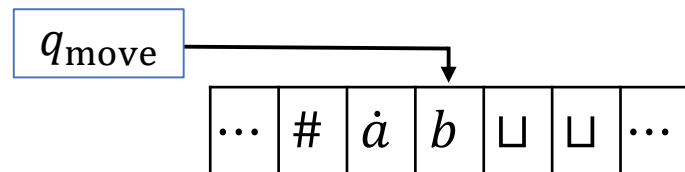
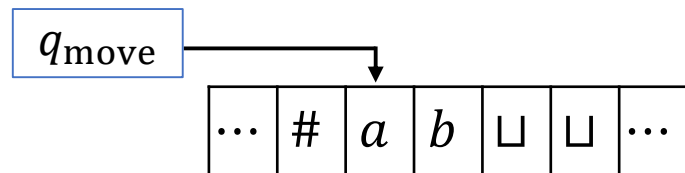
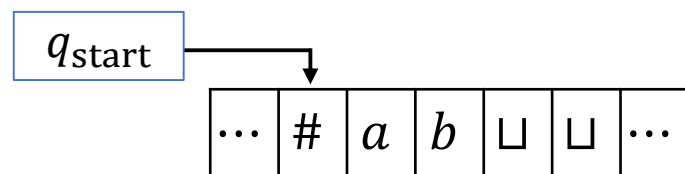
读到分隔符 $\#$ ，将其后的内容右移一格并在原来位置上写下特殊符号：

- q_{move} ：右移磁头并将字符加点直至磁带最右端.
- q_{read} ：读取当前方格字符. 若为加点字符，则表明需要将该方格字符写入右边方格，否则表明该方格字符已经写入从而左移.
- $q_t, \forall t \in \Gamma$ ：记录要写到右边方格的字符.

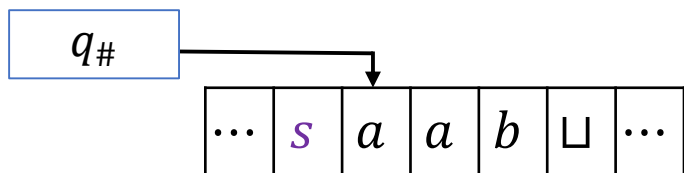
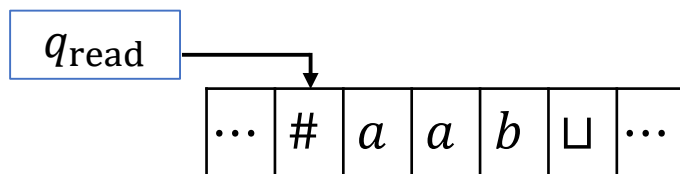
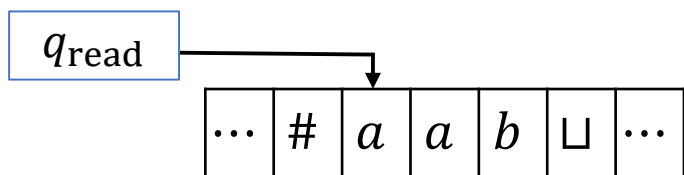


图灵机-多磁带图灵机

一个例子：



图灵机-多磁带图灵机



图灵机的变种-多磁带图灵机

推论. 一个语言是图灵可识别的当且仅当有一个多磁带图灵机识别它.

证明. 若语言 L 是图灵可识别的, 则有一个标准图灵机识别它, 而标准图灵机是一个特殊的多磁带图灵机.

反之, 若 L 能被一个多磁带图灵机识别, 则由**定理 1**, 存在一个标准图灵机识别它. ■

计算理论-可判定性

- 可判定问题
- 康托对角化方法
- 停机问题

1 可判定问题

可判定问题-与正则语言相关的问题

$$A_{\text{DFA}} = \{ \langle B, w \rangle \mid B \text{ 是一个接受字符串 } w \text{ 的 DFA} \}.$$

定理. A_{DFA} 是一个可判定语言.

证明. 只需设计一个判定 A_{DFA} 的图灵机 M 即可.

M = “对于输入 $\langle B, w \rangle$, 其中 B 是一个 DFA, w 是字符串:

1. 模拟在 B 上运行 w .
2. 如果模拟以接受状态结束, 则接受. 如果模拟以非接受状态结束, 则拒绝.”

可判定问题-与正则语言相关的问题

- **第 1 步: 检查输入** $\langle B, w \rangle$ **确实表示输入串** w **和DFA** B . B 的一个合理的表示方法是简单的列出它的五个元素 $(Q, \Sigma, \delta, q_0, F)$. 当 M 收到输入时, 首先检查它是否正确的表示了DFA B 和 w . 如果不是, 则拒绝.
- **第 2 步: M 直接执行模拟.** 用在磁带上写下信息的方法可以跟踪 B 在输入 w 上运行时的当前状态和当前位置. 运行开始时, B 的当前状态是 q_0 , 读写头的当前位置是 w 的最左端符号. 状态和位置的更新是由转移函数决定的. 当 M 处理完 w 的最后一个符号时, 如果 B 处于接收状态, 则 M **接受**这个输入; 如果不是, 则 M **拒绝**.

可判定问题-与正则语言相关的问题

$A_{\text{NFA}} = \{ \langle B, w \rangle \mid B \text{ 是一个接受字符串 } w \text{ 的 NFA} \}.$

定理. A_{NFA} 是一个可判定语言.

证明. 只需设计一个判定 A_{NFA} 的图灵机 N 即可.

$N =$ “对于输入 $\langle B, w \rangle$, 其中 B 是一个 NFA, w 是字符串:

1. 将 B 转化一个与其等价的 DFA C .
2. 用图灵机 M 运行 $\langle C, w \rangle$.
3. 如果 M 接受, 则接受. 否则拒绝.”

这里 N 使用 M 作为一个子算法. ■

可判定问题-与正则语言相关的问题

$A_{\text{REX}} = \{ \langle R, w \rangle \mid R \text{ 是一个产生字符串 } w \text{ 的正则表达式} \}.$

定理. A_{REX} 是一个可判定语言.

证明. 只需设计一个判定 A_{REX} 的图灵机 P 即可.

$P =$ “对于输入 $\langle R, w \rangle$, 其中 R 是正则表达式, w 是字符串:

1. 将 R 转化一个与其等价的NFA A .
2. 用图灵机 N 运行 $\langle A, w \rangle$.
3. 如果 N 接受, 则接受. 否则拒绝.”



可判定问题-与正则语言相关的问题

$$E_{\text{DFA}} = \{ \langle A \rangle \mid A \text{ 是 DFA 且 } L(A) = \emptyset \}.$$

定理. E_{DFA} 是一个可判定语言.

证明. 一个DFA接受某个字符串当且仅当状态图中存在从初始状态到接受状态的路径.

这是有向图的连通性问题, 因此存在图灵机 T 判定. ■

可判定问题-与正则语言相关的问题

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A \text{ 和 } B \text{ 是 DFA 且 } L(A) = L(B) \}.$$

定理. EQ_{DFA} 是一个可判定语言.

证明. 令 $L(C)$ 是 $L(A)$ 和 $L(B)$ 的对称差, 即

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (L(B) \cap \overline{L(A)}).$$

- 注意到 $L(A) = L(B) \iff L(C) = \emptyset$.
- 正则语言关于交, 并, 补都是封闭的, 故 $L(C)$ 是正则语言.

$F =$ “对于输入 $\langle A, B \rangle$, 其中 A, B 是 DFA:

1. 构造识别 $L(C)$ 的 DFA C .
2. 用图灵机 T 运行 $\langle C \rangle$.
3. 如果 T 接受, 则接受. 否则拒绝.”



可判定问题-与正则语言相关的问题

$$EQ_{\text{REX}} = \{ \langle R_1, R_2 \rangle \mid R_1 \text{ 和 } R_2 \text{ 是正则表达式且 } L(R_1) = L(R_2) \}$$

是否可判定?

可判定问题-与正则语言相关的问题

$$EQ_{\text{DFA}} = \{ \langle A, B \rangle \mid A \text{ 和 } B \text{ 是 DFA 且 } L(A) = L(B) \}.$$

定理. EQ_{DFA} 是一个可判定语言.

证明. 可以尝试所有长度不超过 $|Q_A||Q_B|$ 的字符串. 若存在某个这样的字符串使得 A, B 结果不同, 则拒绝. 否则接受.

证明: 若 $L(A) \neq L(B)$, 则必然存在长度不超过 $|Q_A||Q_B|$ 的字符串 w 使得 A, B 输出不同的结果.

可判定问题-与上下文无关语法相关的问题

$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ 是一个生成字符串 } w \text{ 的 CFG} \}.$

定理. A_{CFG} 是一个可判定语言.

证明. 若 G 是乔姆斯基范式, 则对于任何长度为 n 的 $w \in L(G)$, 生成 w 需要 $2n - 1$ 步替换.

$S =$ “对于输入 $\langle G, w \rangle$, 其中 G 是一个 CFG, w 是字符串:

1. 将 G 转化成与其等价的乔姆斯基范式.
2. 若 $n = |w| = 0$, 枚举所有长度为 1 的生成序列. 否则 $n \geq 1$, 枚举所有长度为 $2n - 1$ 的生成序列.
3. 如果任何生成序列生成 w , 则接受. 否则拒绝.”

可判定问题-与上下文无关语法相关的问题

定理. 每个上下文无关的语言都是可判定的.

证明. 令 G 是生成语言 A 的上下文无关语法, 下面设计一个判定 A 的图灵机 M_G .

回忆: S 是判定

$$A_{\text{CFG}} = \{ \langle G, w \rangle \mid G \text{ 是一个生成字符串 } w \text{ 的 CFG} \}.$$

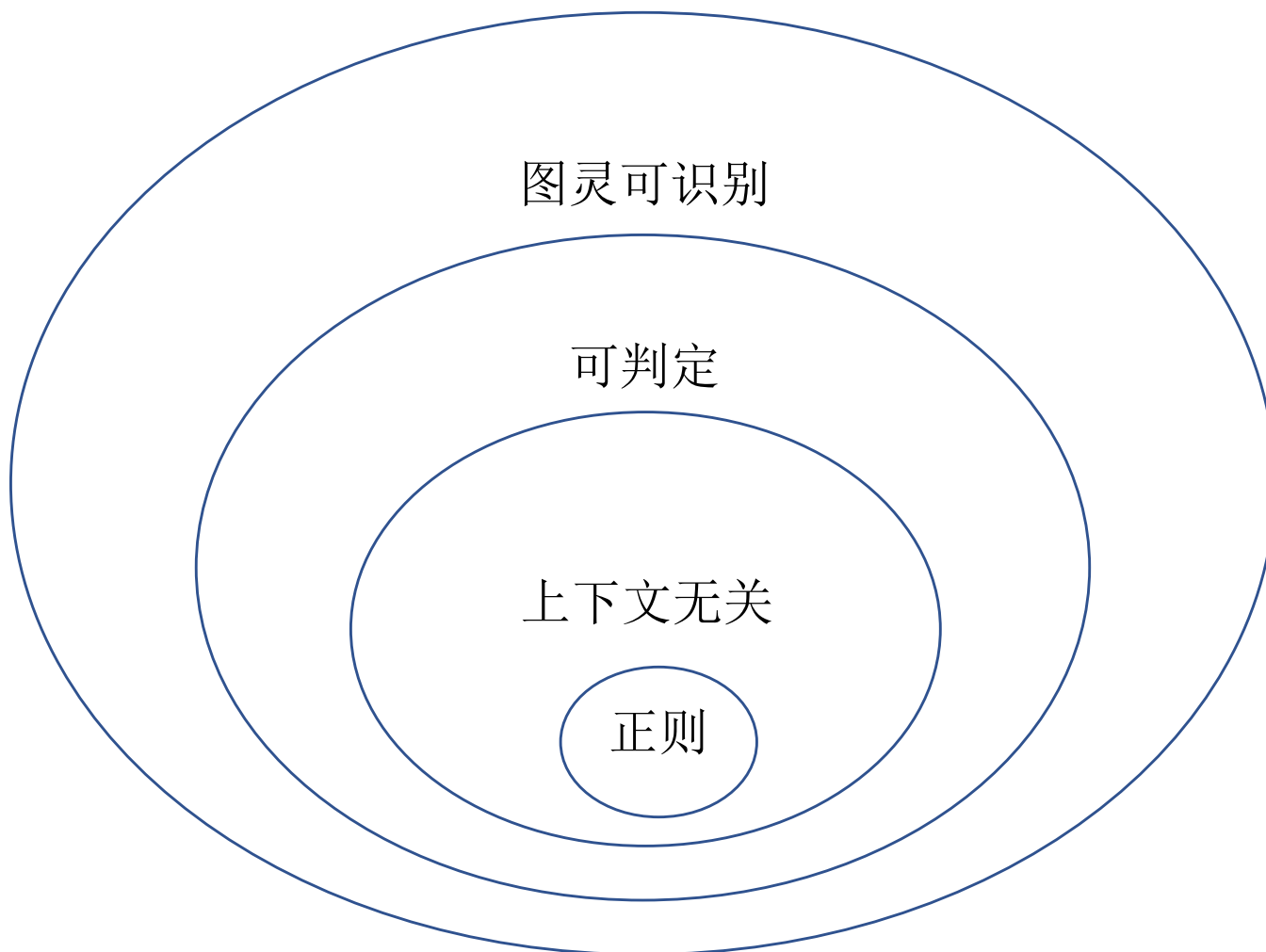
的图灵机.

$M_G =$ “对于输入 w :

1. 对于输入 $\langle G, w \rangle$ 运行 S .
2. 若 S 接受, 则 M_G 接受, 否则拒绝.”



可判定问题-与上下文无关语法相关的问题



可判定问题-与上下文无关语法相关的问题

$$E_{\text{CFG}} = \{ \langle G \rangle \mid G \text{ 是 CFG 且 } L(G) = \emptyset \}.$$

定理. E_{CFG} 是一个可判定语言.

证明. 对于每个变量, 判定该变量是否能生成一个只有终止符的字符串.

$R =$ “对于输入 $\langle G \rangle$, 其中 G 是一个 CFG:

1. 标记所有 G 的终止符.
2. 重复以下操作直到没有新的变量可以标记:

若 $A \rightarrow U_1 U_2 \cdots U_k$ 是 G 的替换规则且 $U_1, \dots, U_k$ 均已标记, 则标记变量 A .

3. 若 G 的初始变量没有标记, 则接受; 否则拒绝.”

G :

$S \rightarrow RTa$

$R \rightarrow Tb$

$T \rightarrow a$

可判定问题-与上下文无关语法相关的问题

$$EQ_{CFG} = \{ \langle G, H \rangle \mid G, H \text{ 是 CFG 且 } L(G) = L(H) \}.$$

- 利用 E_{DFA} 证明了 EQ_{DFA} 是可判定的.
- 类似的方法是否能够证明 EQ_{CFG} 是可判定的?

No! 因为上下文无关语法对交运算和补运算是不封闭的.

EQ_{CFG} 是不可判定的(第 5 章).

可判定问题

练习: 证明 $INFINITE_{DFA} = \{ \langle A \rangle \mid A \text{ 是 DFA 且 } L(A) \text{ 是无穷集} \}$ 是可判定的.

证明. 令 A 是一个DFA, k 是 A 含有的状态数量.

- 注意到 $L(A)$ 是无穷集当且仅当 $L(A)$ 接受任意长的字符串.
- 另外, 若 A 接受一个长度至少为 k 的字符串, 则泵引理^的证明告诉我们我们可以pump这个字符串, 使得 A 接受任意长的字符串.

因此, 只需判定 $L(A)$ 是否接受一个长度至少为 k 的字符串.

显然, 可以构造一个DFA D_k 使得 $L(D_k)$ 为长度至少为 k 的字符串. 从而转化为判定 $L(A) \cap L(D_k) = \emptyset$. ■

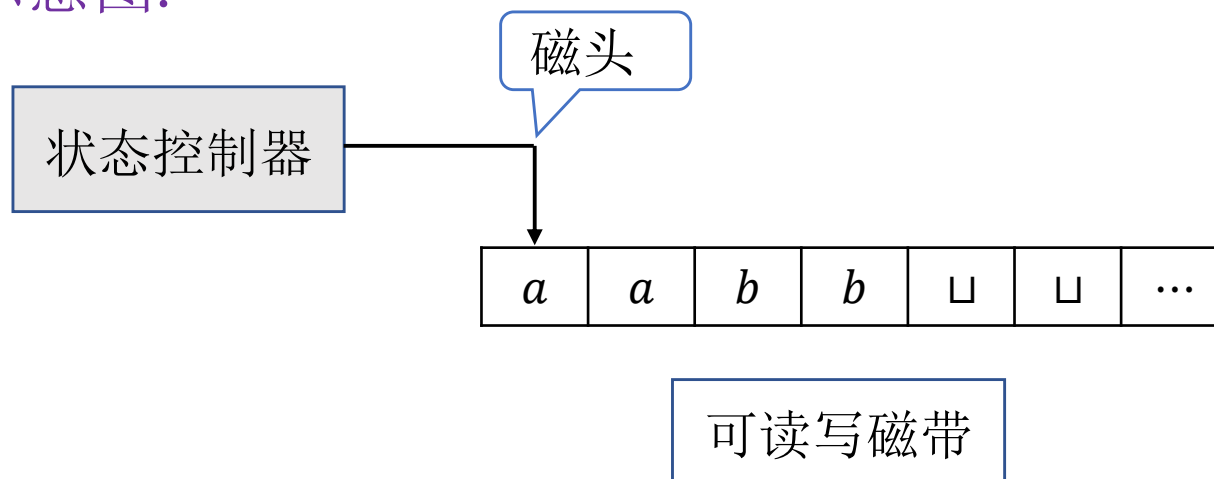
计算理论-丘奇-图灵论题

- 图灵机
- 图灵机的变种
- 算法的定义
- 描述图灵机的术语

1 图灵机

图灵机

抽象示意图.



- 磁带向右无限伸展  无限大内存
- 既能从磁带读取内容又能往磁带上写内容.
- 磁头可向左或向右移动.
- 可在任何时候接受或者拒绝.

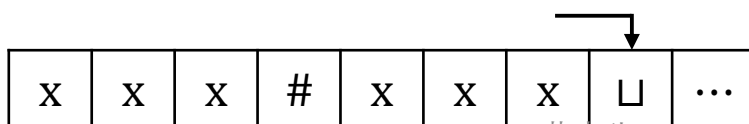
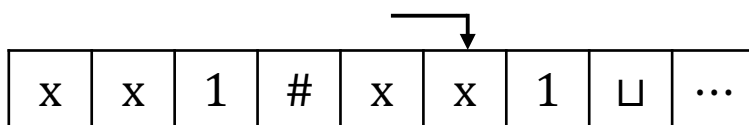
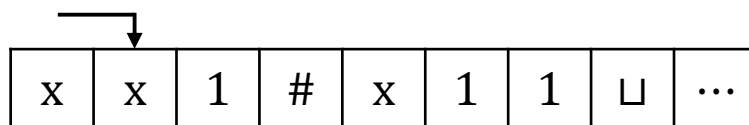
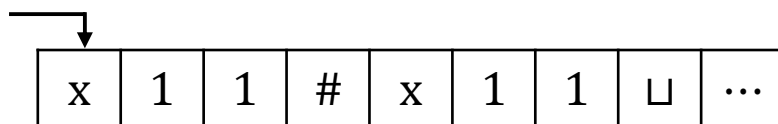
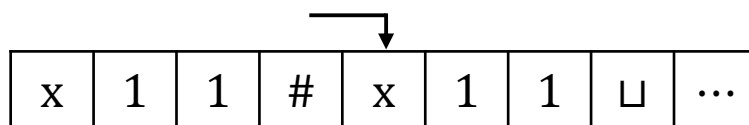
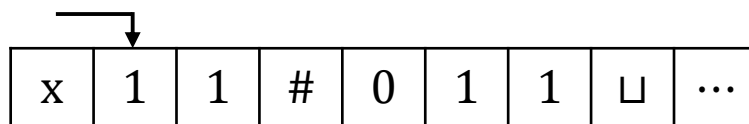
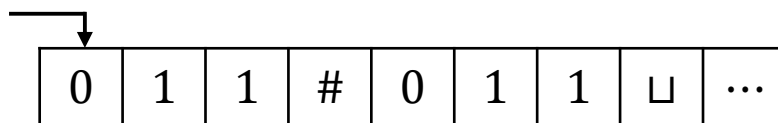
图灵机-引例

设计识别语言 $B = \{w\#w \mid w \in \{0,1\}^*\}$ 的图灵机.

$M =$ 对于输入字符串 w :

1. 在 $\#$ 两边对应的位置上来回移动, 检查这些对应位置是否包含相同的符号. 如不是或者没有 $\#$, 则拒绝. 为跟踪对应的符号, 划去所有检查过的符号.
2. 当 $\#$ 左边的所有符号都被划去时, 检查 $\#$ 右侧是否还有符号. 如果有则拒绝, 否则接受.

图灵机-引例



磁头右移

磁头左移

磁头右移

...

图灵机-正式定义

定义(图灵机). 一个图灵机是一个7-元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

其中

- Q 是有限状态集.
- Σ 是不含 $\sqcup$ 的输入字母表.
- Γ 是磁带字母表, 满足 $\sqcup \in \Gamma, \Sigma \subseteq \Gamma$.
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 是转移函数.
- q_0 是起始状态.
- q_{accept} 是接受状态.
- q_{reject} 是拒绝状态.

$$\delta(q, a) = (r, b, L)$$

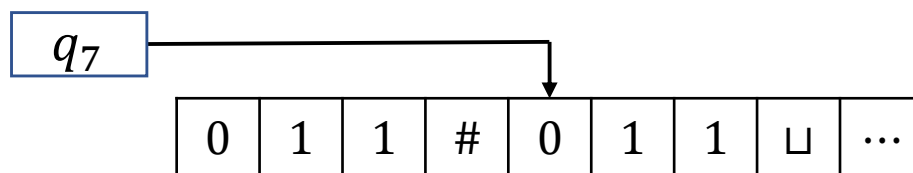
图灵机-图灵机的计算

给定图灵机 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$

- M 将输入 $w = w_1 w_2 \cdots w_n$ 存贮在磁带最左边的 n 个方格中, 其余部分填以空白符 $\sqcup$. 因为 $\sqcup \notin \Sigma$, 磁带上的第一个 $\sqcup$ 表示输入结束.
- M 开始运行后根据转移函数计算. 如果 M 试图将磁头从磁带的最左端移出, 即使转移函数指示的是 L, 磁头仍保持不动.
- 如果 M 进入接受或拒绝状态, 则停机 (halting). 否则 M 将永远运行下去 (looping).

图灵机-图灵机的计算

- 图灵机计算时需要记录3个信息：当前状态，磁头位置以及磁带内容.  配置 (configuration)
- 对于状态 q 以及 $u, v \in \Gamma^*$, uqv 表示当前状态为 q , 磁带内容为 uv , 磁头指向字符串 v 的第一个字符.
- $011\#q_7011$.



图灵机-图灵机的计算

- 如果图灵机能合法地从配置 C_1 一步进入 C_2 , 则称 C_1 产生 C_2 .
- 给定字符 $a, b, c \in \Gamma$, 字符串 $u, v \in \Gamma^*$, 状态 q_i, q_j , 配置 $uaq_i bv$,
 - 若 $\delta(q_i, b) = (q_j, c, L)$, 则 $uaq_i bv$ 产生 $uq_j acv$.
 - 若 $\delta(q_i, b) = (q_j, c, R)$, 则 $uaq_i bv$ 产生 $uacq_j v$.

思考: 若 $\delta(q_i, b) = (q_j, c, L)$, 则 $q_i bv$ 产生哪个配置?

图灵机-图灵机的计算

- 初始配置 q_0w 表示初始状态为 q_0 ，输入为 w .
- 接受(拒绝)配置是指状态为 q_{accept} (q_{reject})的配置. 接受配置和拒绝配置统称为停机配置.

图灵机 M 接受字符串 w 如果存在配置 $C_1, C_2, \dots, C_k$ 使得

- C_1 是初始配置.
- C_i 产生 C_{i+1} , $i = 1, 2, \dots, k - 1$.
- C_k 是接受配置.

给定图灵机 M , $L(M)$ 为所有被 M 接受的字符串的集合.

图灵机-正式定义

定义(图灵可识别语言). 给定语言 A , 若存在图灵机 M 使得 $A = L(M)$, 则称 A 是图灵可识别的.

一个图灵机被称为判定器(decider), 如果对任何输入它要么接受要么拒绝.

定义(图灵可判定语言). 给定语言 A , 若存在判定器 M 使得 $A = L(M)$, 则称 A 是图灵可判定的.

- 可判定一定可识别.
- 存在可识别但不可判定语言(第4章).

图灵机-正式定义

例 1: 设计判定 $A = \{0^{2^n} \mid n \geq 0\}$ 的图灵机.

M_1 = 对于输入字符串 w :

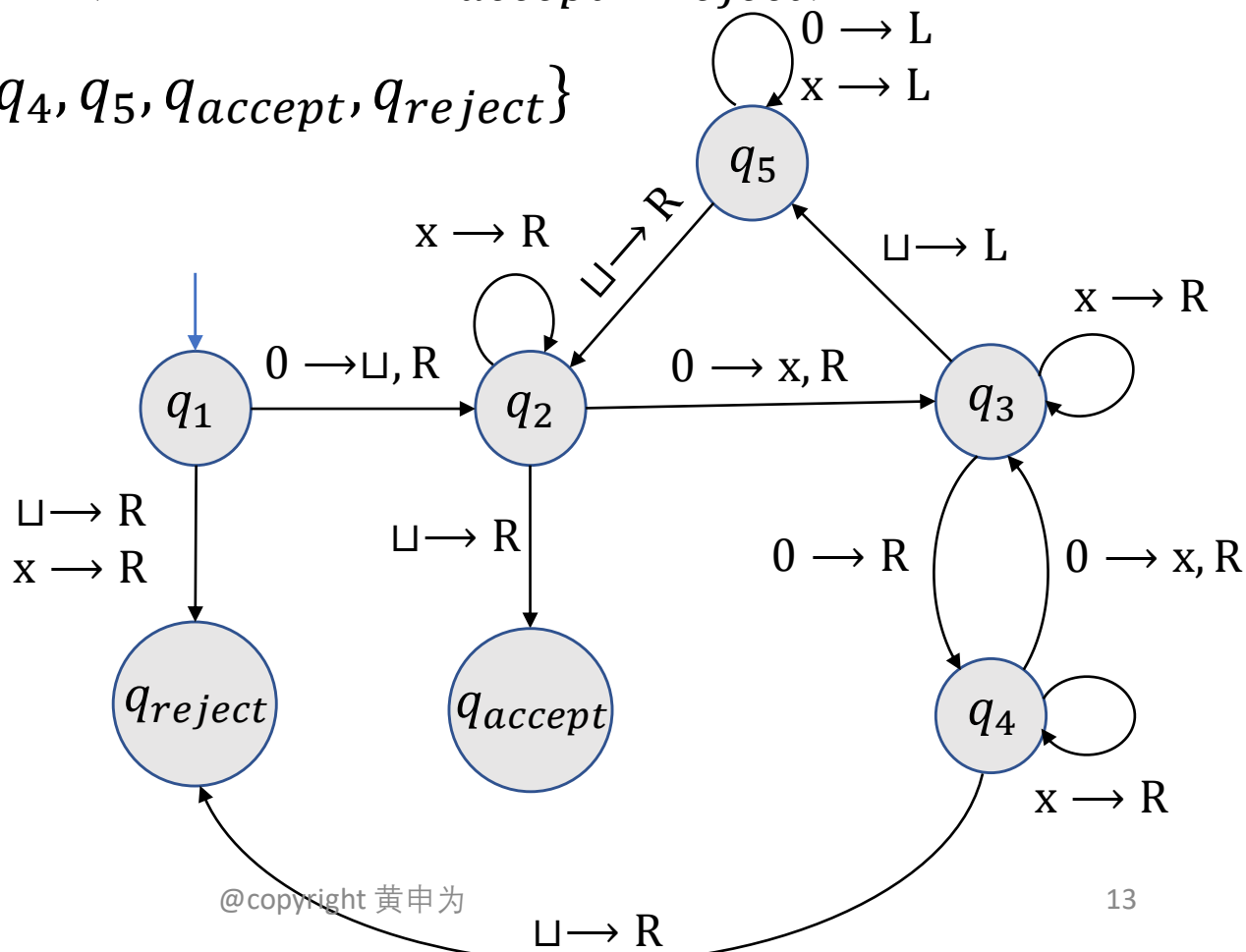
1. 从左往右扫描整个磁带, 每隔一个字符划去一个0.
2. 若第 1 步中磁带上只有1个0, 则接受; 若第 1 步中磁带上多于1个0且为奇数, 则拒绝.
3. 回到磁带最左端.
4. 回到第 1 步.

图灵机-正式定义

例 1: 设计判定 $A = \{0^{2^n} | n \geq 0\}$ 的图灵机.

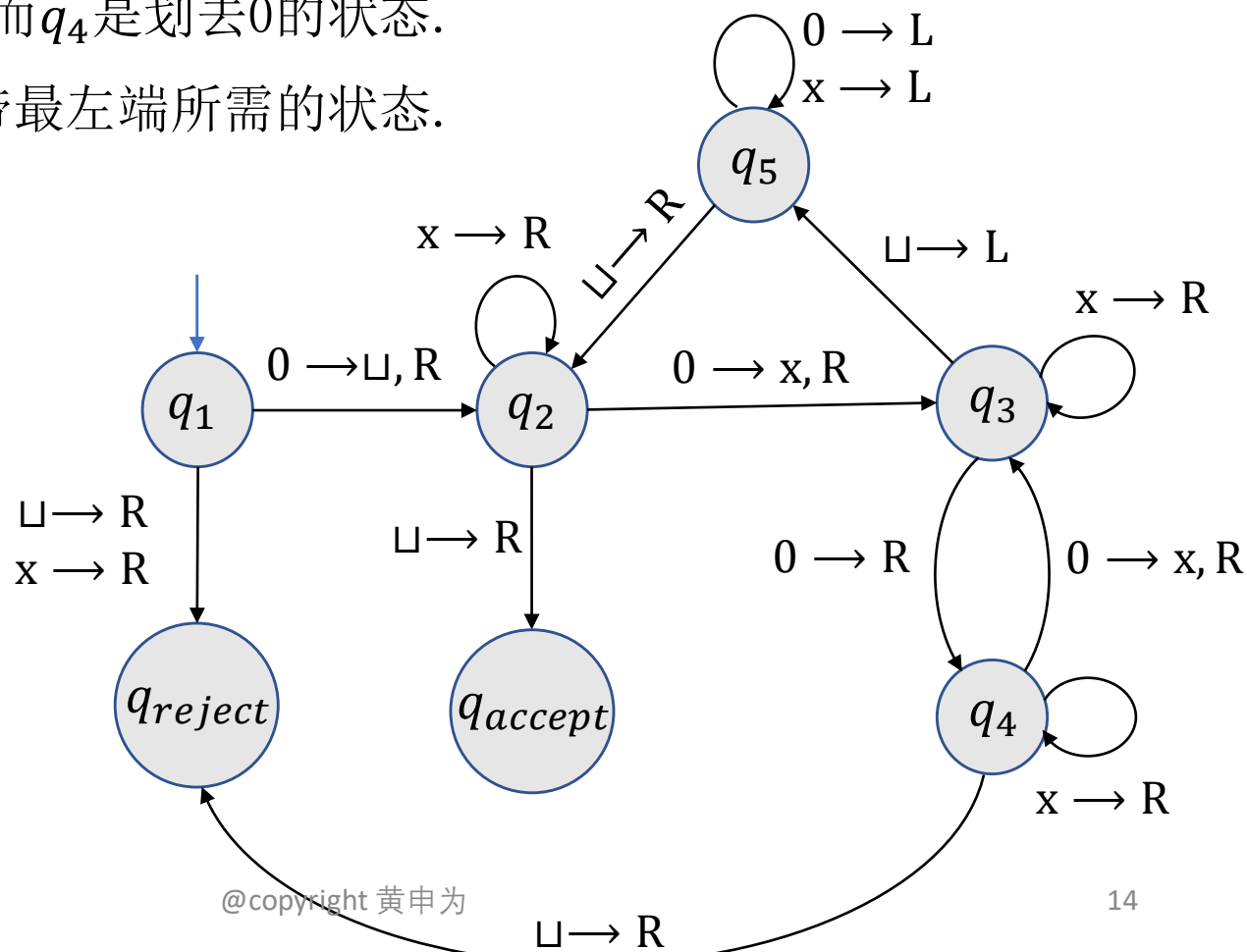
$$M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{accept}, q_{reject})$$

- $Q = \{q_1, q_2, q_3, q_4, q_5, q_{accept}, q_{reject}\}$
- $\Sigma = \{0\}$
- $\Gamma = \{0, x, \sqcup\}$
- δ 如图所示



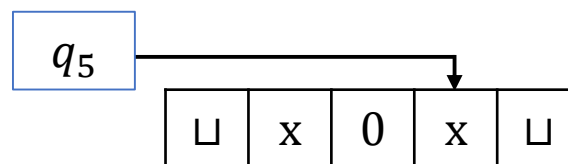
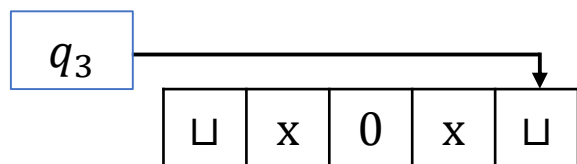
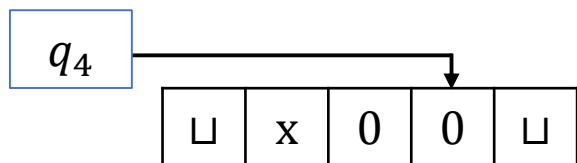
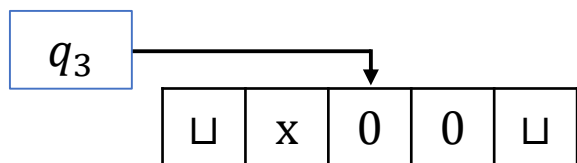
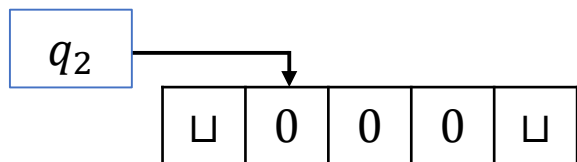
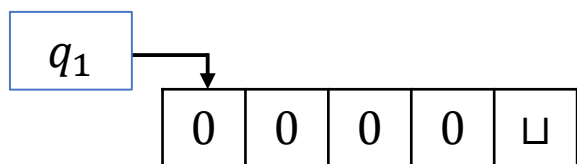
图灵机-正式定义

- q_1 是初始状态, 第一步将磁头最左端的0改写为 $\sqcup$, 来标记磁带的最左端.
- q_2 是进行划去0这个操作的初始状态.
- q_3 是跳过0的状态而 q_4 是划去0的状态.
- q_5 是磁头回到磁带最左端所需的的状态.

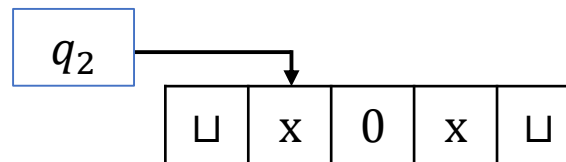
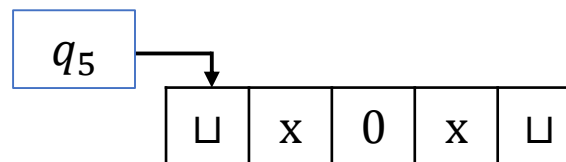


图灵机-正式定义

M_1 对于输入 $w = 0000$ 运行行为:

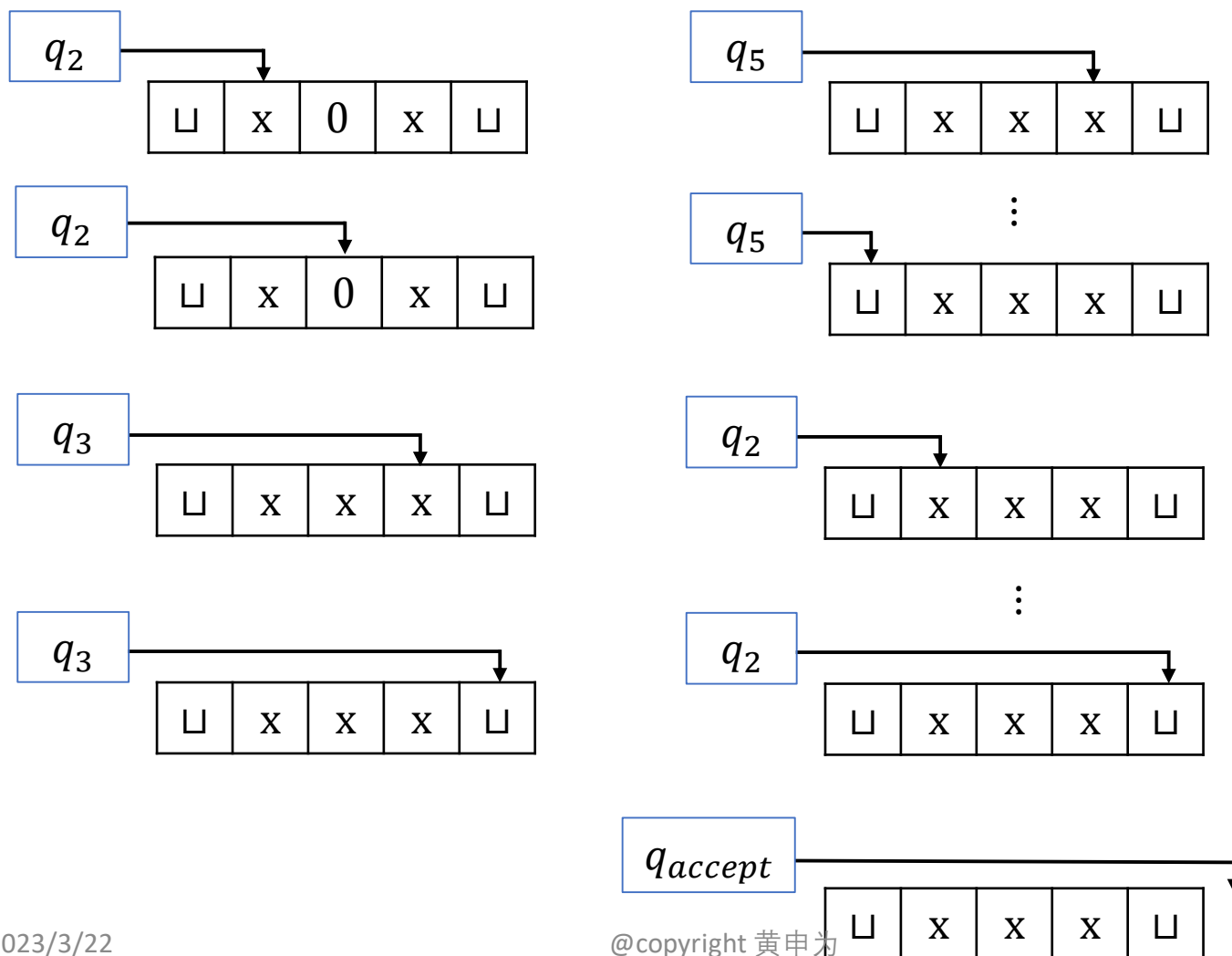


⋮



图灵机-正式定义

M_1 对于输入 $w = 0000$ 运行行为:



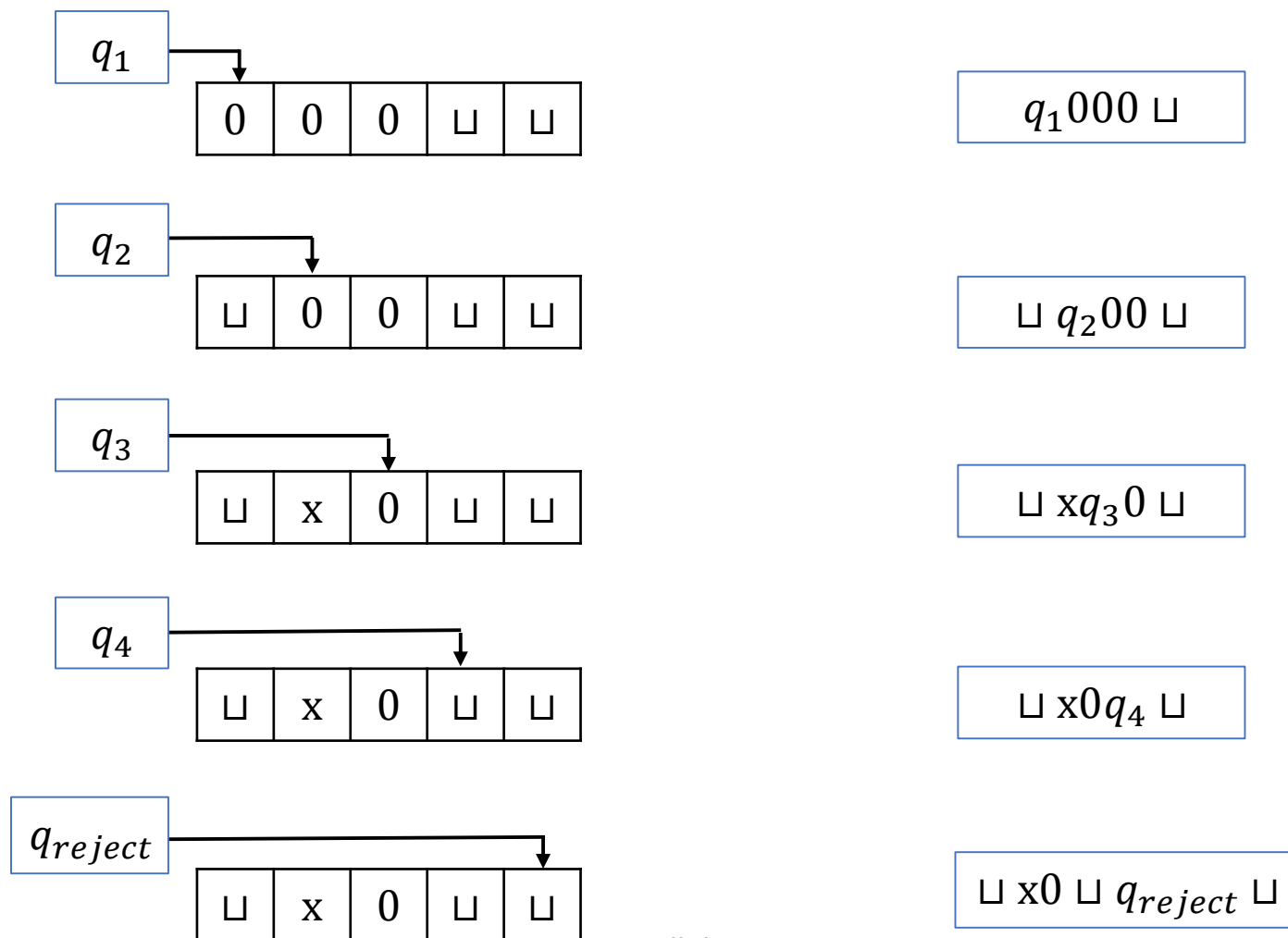
图灵机-正式定义

M_1 对于输入 $w = 0000$ 运行的配置序列为:

$q_1 0000$	$\sqcup q_2 x0x \sqcup$	$\sqcup xq_2xx \sqcup$
$\sqcup q_2 000$	$\sqcup xq_2 0x \sqcup$	$\sqcup xxq_2x \sqcup$
$\sqcup xq_3 00$	$\sqcup xxq_3x \sqcup$	$\sqcup xxxq_2 \sqcup$
$\sqcup x0q_4 0$	$\sqcup xxxq_3 \sqcup$	$\sqcup xxx \sqcup q_{accept}$
$\sqcup x0xq_3 \sqcup$	$\sqcup xxq_5x \sqcup$	
$\sqcup x0q_5x \sqcup$	$\sqcup xq_5xx \sqcup$	
$\sqcup xq_5 0x \sqcup$	$\sqcup q_5xxx \sqcup$	
$\sqcup q_5x0x \sqcup$	$q_5 \sqcup xxx \sqcup$	
$q_5 \sqcup x0x \sqcup$	$\sqcup q_2xxx \sqcup$	

图灵机-正式定义

练习. 写出 M_1 对于输入 $w = 000$ 运行过程和配置序列



图灵机-正式定义

例 2: 设计判定语言 $B = \{w\#w \mid w \in \{0,1\}^*\}$ 的图灵机 M_2 .

$M_2 =$ 对于输入字符串 w :

1. 在 $\#$ 两边对应的位置上来回移动, 检查这些对应位置是否包含相同的符号. 如不是或者没有 $\#$, 则拒绝. 为跟踪对应的符号, 划去所有检查过的符号.
2. 当 $\#$ 左边的所有符号都被划去时, 检查 $\#$ 右侧是否还有符号. 如果有则拒绝, 否则接受.

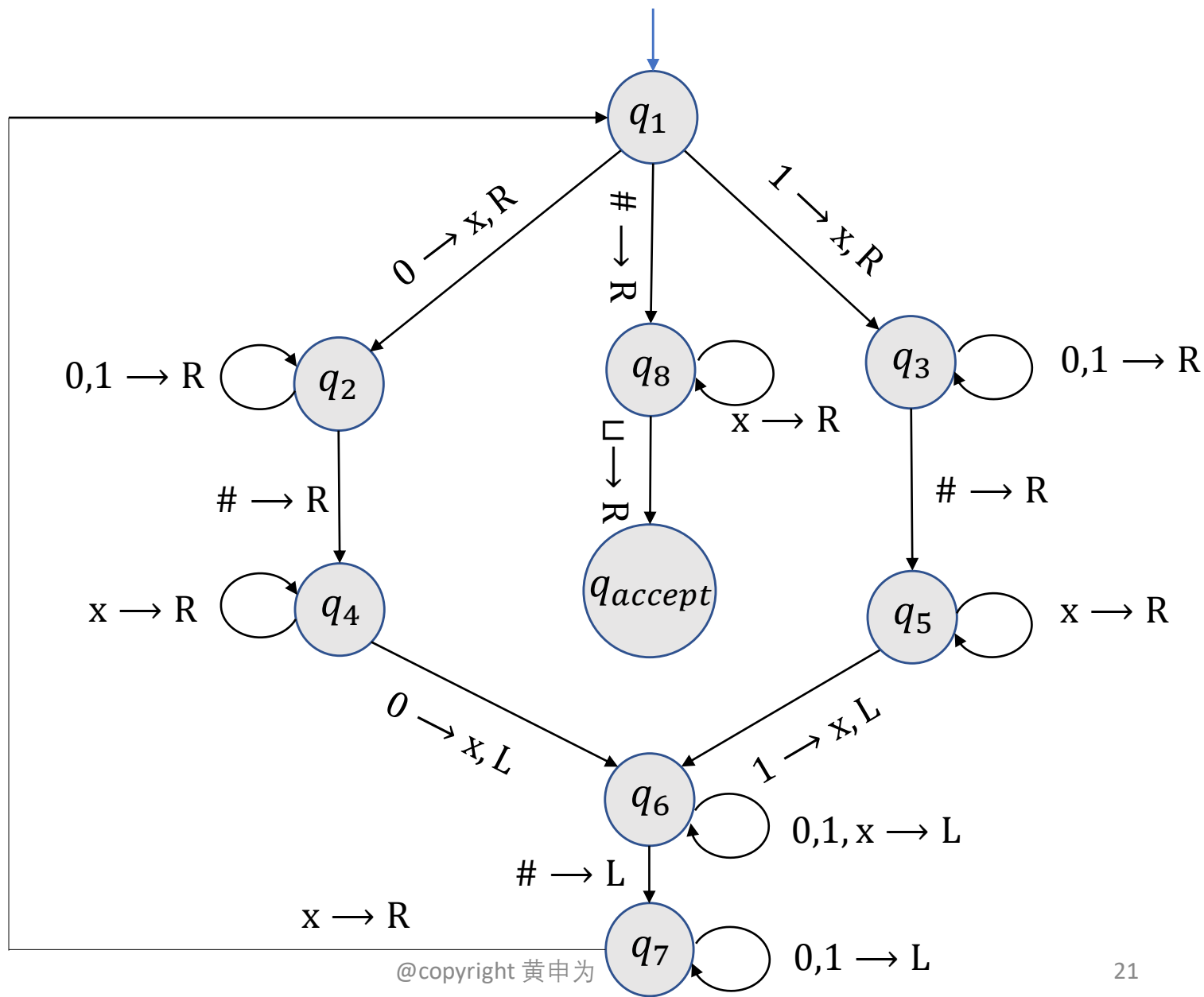
图灵机-正式定义

例 2: 设计判定语言 $B = \{w\#w \mid w \in \{0,1\}^*\}$ 的图灵机 M_2 .

$$M_2 = (Q, \Sigma, \Gamma, \delta, q_1, q_{accept}, q_{reject})$$

- $Q = \{q_1, q_2, \dots, q_8, q_{accept}, q_{reject}\}$
- $\Sigma = \{0, 1, \#\}$
- $\Gamma = \{0, 1, \#, x, \sqcup\}$
- δ 如图所示
 - q_{reject} 没有画出.
 - 若某个状态 q_i 没有一个出去的箭头上标有 a , 则表明当机器处于 q_i , 读取字符 a 时, 跳转到拒绝状态 q_{reject} .

图灵机-正式定义



图灵机-正式定义

练习. 画出 M_2 运行 $w = 0\#0$ 的流程并写出相应的配置序列.

思考: M_2 是否有可能处于状态 q_6 时读到 $\sqcup$?

图灵机-正式定义

例 3: 设计判定语言 $C = \{a^i b^j c^k \mid i \times j = k, i, j, k \geq 1\}$ 的图灵机 M_3 .

M_3 = 对于输入字符串 w :

1. 从左向右扫描输入来判定 w 是否是 $a^i b^j c^k$ 的形式. 若不是, 则拒绝; 否则回到磁带最左端.
2. 划去一个 a 然后将磁带移动到第一个 b . 来回在 b 和 c 之间游走, 划去对应位置的 b 和 c , 直到所有的 b 都划去. 若所有的 c 都划去后还有 b 没有划去, 则拒绝. 否则进入第 3 步.

$$i \times j > k$$

3. 若所有的 a 已划去,

$$i \times j < k$$

如果所有的 c 也已划去, 则接受; 否则拒绝.

否则还有没有划去的 a , 复原所有划去的 b , 重复第 2 步.

图灵机-正式定义

例 4: 设计判定语言

$$E = \{\#x_1\#x_2\cdots\#x_k \mid x_i \in \{0,1\}^* \text{ 且 } x_i \neq x_j, i \neq j\}$$

的图灵机 M_4 .

解: 机器 M_4 将 x_1 与 $x_2, \dots, x_k$ 进行比较, 然后将 x_2 与 $x_3, \dots, x_k$ 进行比较, 以此类推.

图灵机-正式定义

$M_4 =$ 对于输入字符串 w :

1. 在磁带最左端的符号上作个记号. 如果这个符号是 $\sqcup$, 则接受. 若这个符号是 $\#$, 则进入第 2 步. 否则拒绝.
2. 向右扫描下一个 $\#$ 并在此符号上作记号. 如果在遇到 $\sqcup$ 之前没有碰到 $\#$, 说明只有 x_1 , 从而接受.
3. 通过来回移动, 比较作了记号的两个 $\#$ 后面的字符串是否相等. 若相等, 则拒绝; 否则进入第 4 步.
4. 将两个带有记号的 $\#$ 中右边那个的记号移到下一个 $\#$ 上. 如果在遇到 $\sqcup$ 之前没有碰到 $\#$, 则将两个带有记号的 $\#$ 中左边那个的记号移到下一个 $\#$ 上, 并将另外一个记号移到其后面的 $\#$ 上. 如果这时找不到 $\#$ 了, 说明所有的字符串都比较过了, 从而接受. 否则回到第 3 步.

图灵机-正式定义

练习：给出判定含有连续两个0的 $\{0,1\}$ -字符串的图灵机的状态转移图以及形式化定义.

2 图灵机的变种

图灵机的变种

- 双向图灵机：磁带可以向左右两个方向无限延展.
- 多磁带图灵机：有多个磁带，每个磁带均有一个磁头.
- 非确定性图灵机：同一个配置可以同时产生多个配置，每一个这样的选择对应一个计算的分支.
- 枚举器：有“打印机”的图灵机.
-

所有这些变种均和标准图灵机等价！

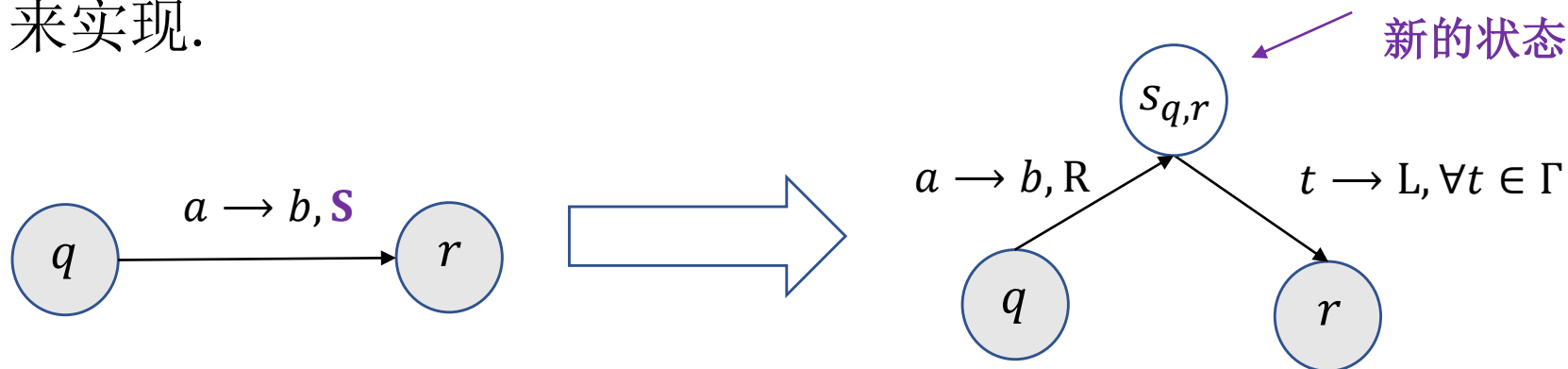
图灵机的变种

问题： 如果允许磁头保持不动，这样的图灵机是否比标准图灵机更强大？

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, \mathbf{S}\}.$$

回答： 显然不是！

每一个保持不动的动作可以通过右移一步再左移一步来实现.



思考： 最多需要引入多少个新的状态？

图灵机的变种-多磁带图灵机

定义(多磁带图灵机).

- 一个多磁带图灵机是有多个磁带的图灵机, 每个磁带有自己的磁头用于读写.
- 开始时输入出现在第一个磁带上, 其它的磁带都是空白.
- 转移函数允许多个带子同时进行读写和移动磁头:

$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$, 这里 k 是磁带个数.

$\delta(q, a_1, \dots, a_k) = (r, b_1, \dots, b_k, L, \dots, R)$ 表示当机器处于状态 q , 磁头 $1, \dots, k$ 分别读到 $a_1, \dots, a_k$, 则机器跳转到状态 r , 各磁头分别写下 $b_1, \dots, b_k$, 且按照指示移动每个磁头.

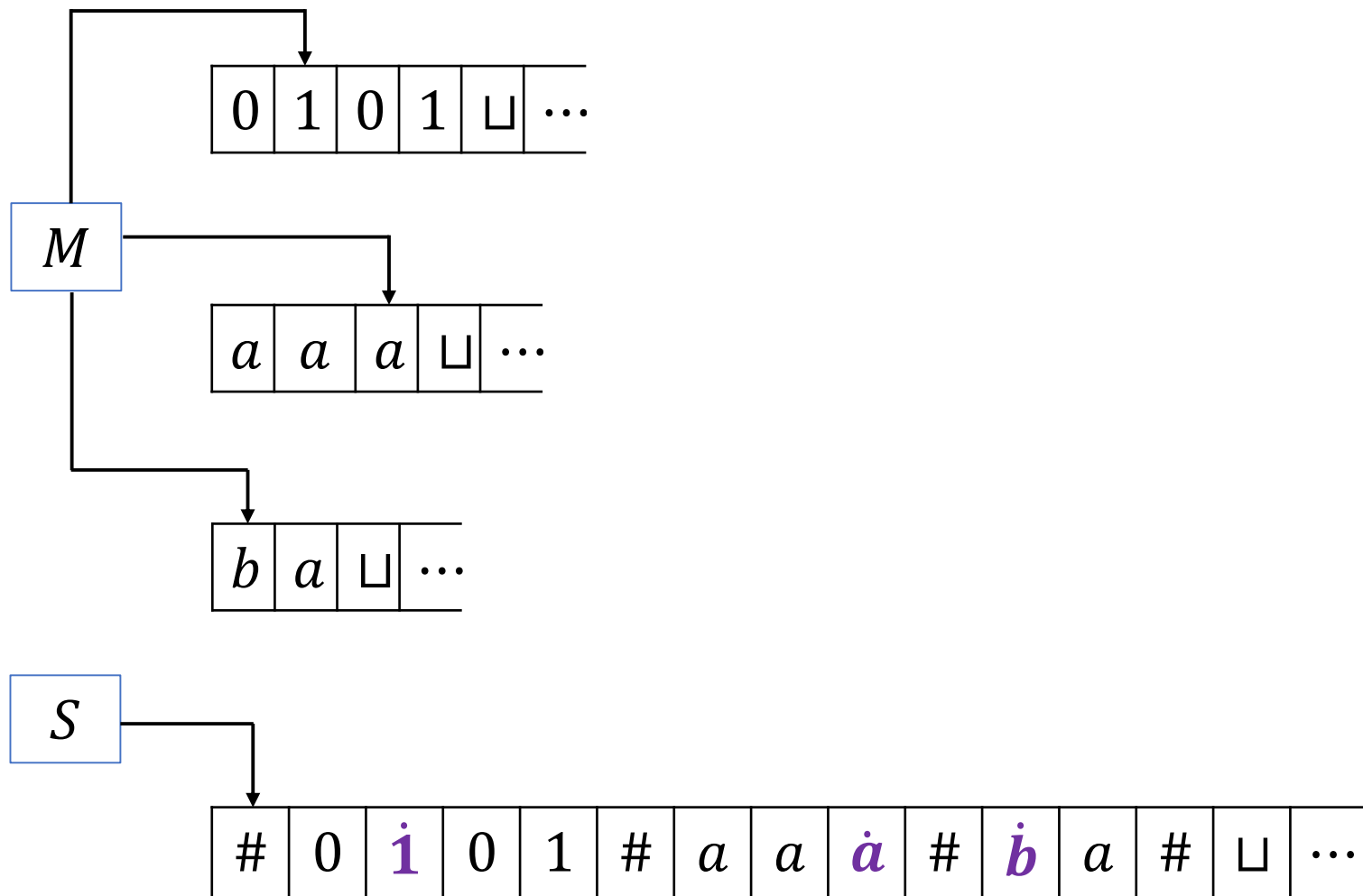
图灵机的变种-多磁带图灵机

定理 1 (多磁带图灵机). 每个多磁带图灵机有一个等价的标准图灵机.

证明. 给定多磁带图灵机 M , 需要构造一个等价的标准图灵机 S , 即如何用 S 模拟 M .

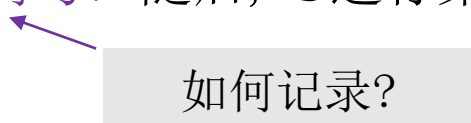
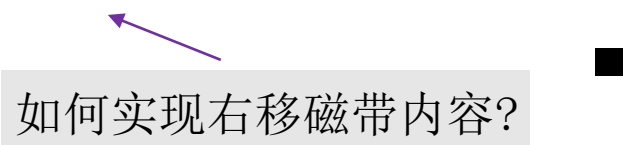
- 假设 M 有 k 个磁带. S 把这 k 个磁带的信息都存储在它的唯一磁带上并以此来模拟 k 个磁带的效果, 它用一个新的符号 $\#$ 作为定界符以分开不同磁带的內容.
- 除了磁带内容之外, S 还必须记录磁头的位置. 为此, 它在一个符号的顶上加个点以此来标记读写头在其磁带上的位置. 可以把这些带点的字符想象成虚拟磁头.

图灵机的变种-多磁带图灵机



图灵机的变种-多磁带图灵机

证明(续). $S =$ 对于输入字符串 $w_1 w_2 \cdots w_n$:

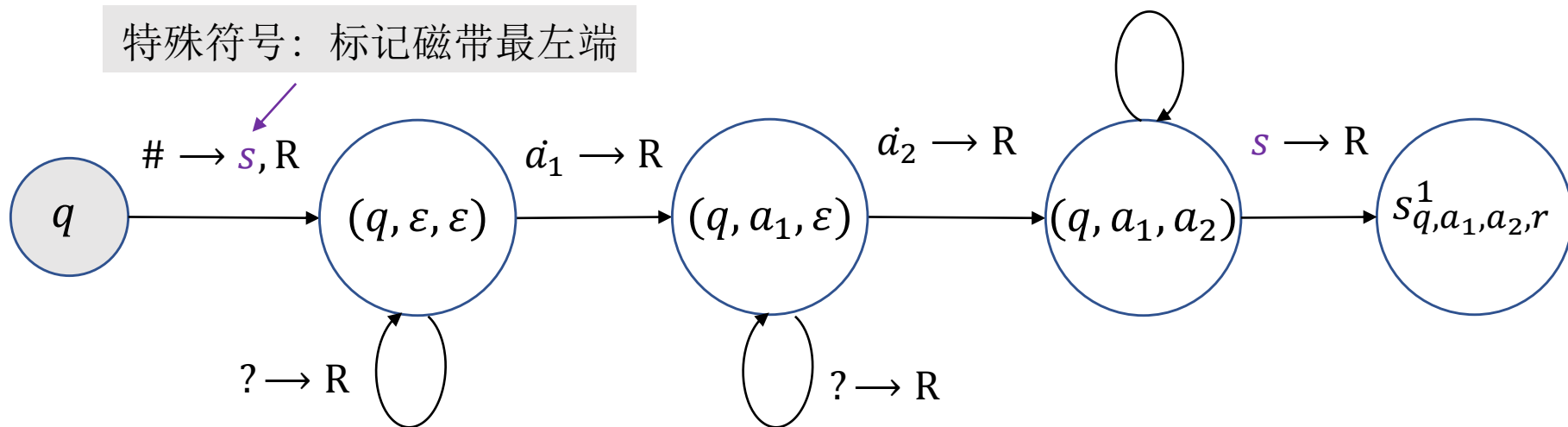
1. 初始时 S 上的内容为 $\# w_1 w_2 \cdots w_n \# \sqcup \# \sqcup \# \cdots \#$.
2. 为了模拟多带机的一步移动, S 从标记最左端的第一个 $\#$ 开始扫描至第 $(k + 1)$ 个 $\#$, 以此确定虚拟头下的符号. 随后, S 进行第二次扫描, 按照 M 所指定的方式更新磁带. 
3. 如果 S 将某个虚拟读写头向右移动到某个 $\#$ 上面, 就意味着 M 已将自己相应的读写头移动到了其所在的带子中的空白区域上, 即以前没有读过的区域上. 此时, S 在这个带方格上写下空白符并将该方格到最右端的方格中的内容都向右移动一个格. 然后再像以前一样继续模拟. 

图灵机的变种-多磁带图灵机

记录虚拟头下的符号.

- 引入辅助状态 $(q, a_1, a_2, \dots, a_k)$ 记录虚拟头下的符号, 其中 $q \in Q, a_i \in \Gamma_\varepsilon$.
- 引入辅助状态 $s_{q,a_1,\dots,a_k,r}^1$, 其中 r 是 $\delta(q, a_1, a_2, \dots, a_k)$ 中的状态分量. 这是模拟 M 转移函数的初始状态.

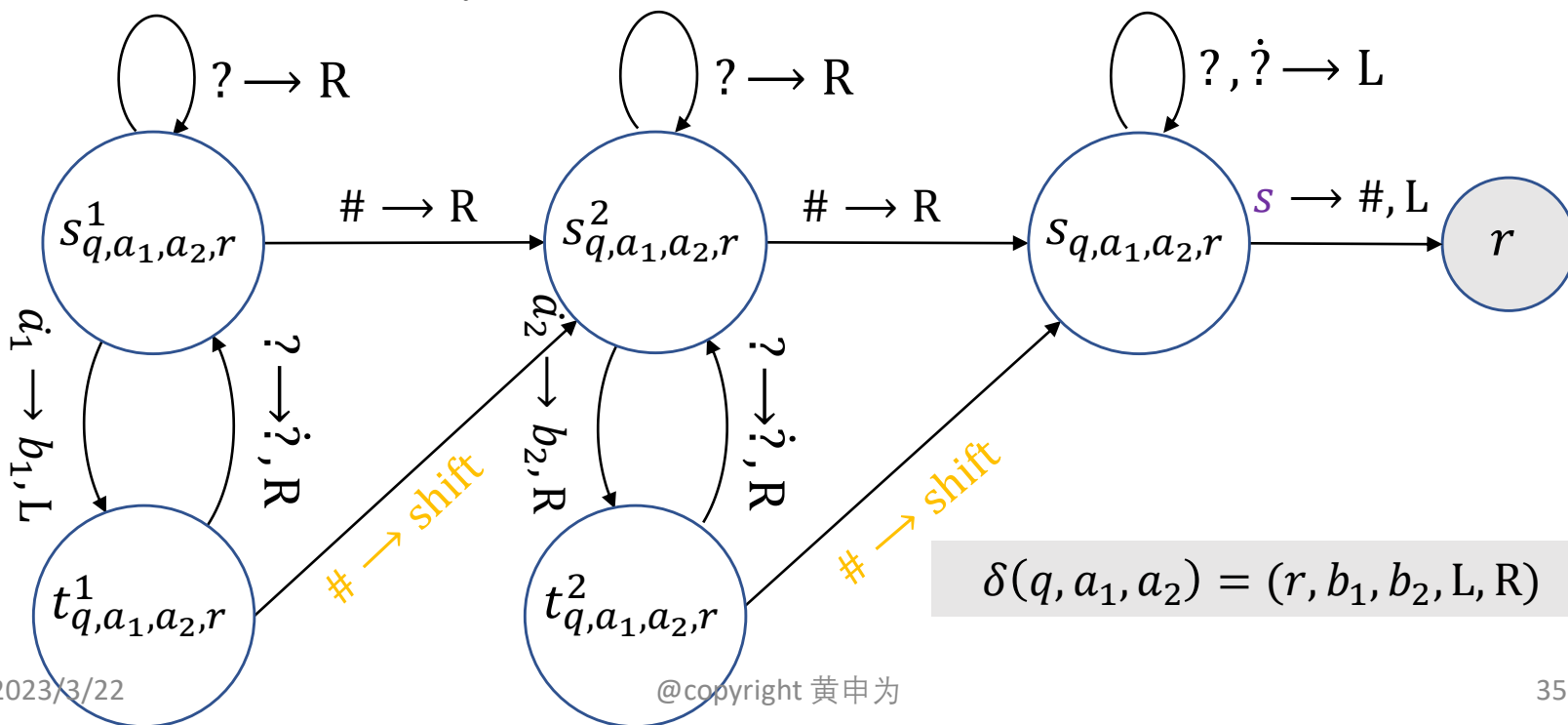
特殊符号: 标记磁带最左端



图灵机的变种-多磁带图灵机

按照 M 所指定的方式更新磁带

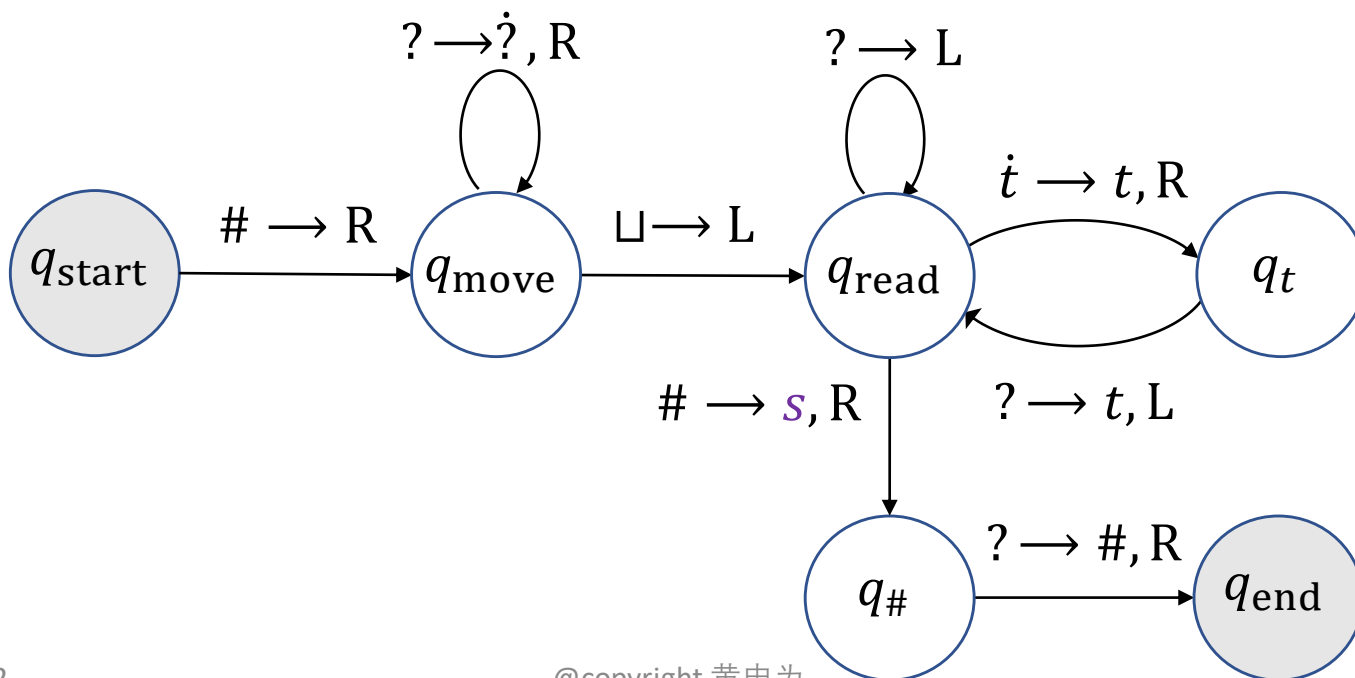
- 引入状态 $s_{q,a_1,\dots,a_k,r}^i, t_{q,a_1,\dots,a_k,r}^i$ 模拟第 i 个磁带上的读写和磁头位置变化.
- 引入状态 $s_{q,a_1,\dots,a_k,r}$ 用来回到磁带最左端.



图灵机的变种-多磁带图灵机

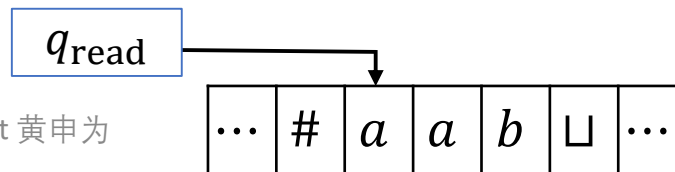
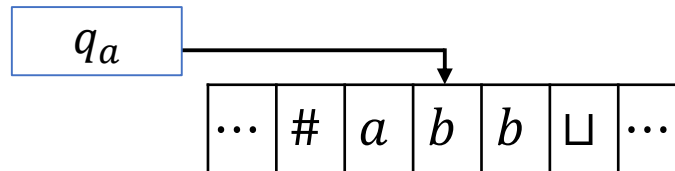
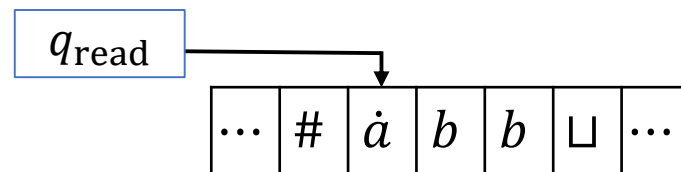
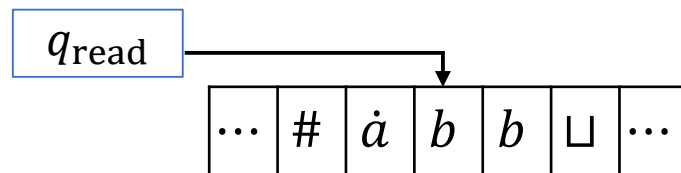
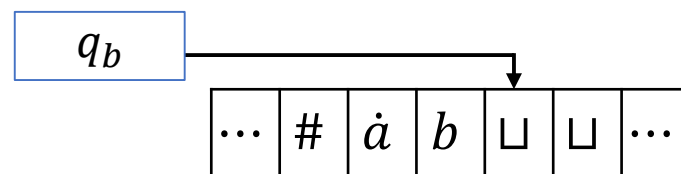
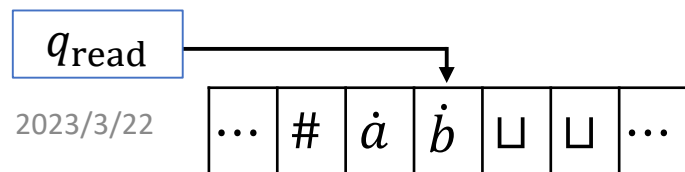
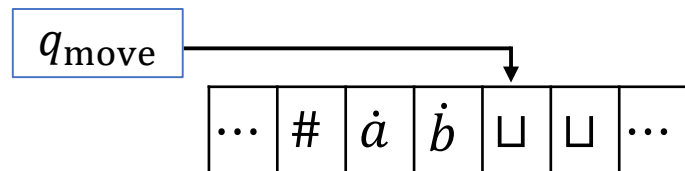
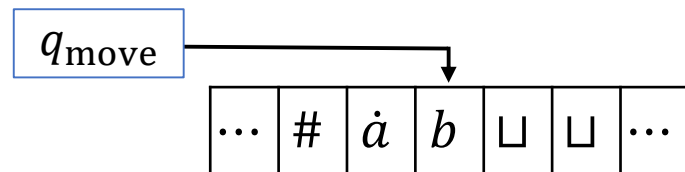
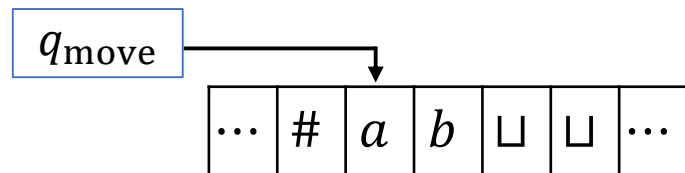
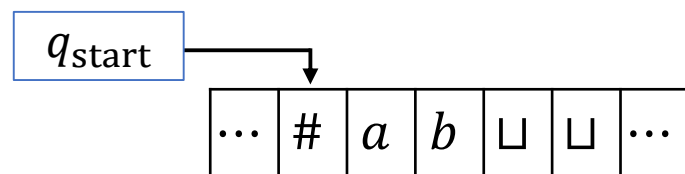
读到分隔符#, 将其后的内容右移一格并在原来位置上写下特殊符号:

- q_{move} : 右移磁头并将字符加点直至磁带最右端.
- q_{read} : 读取当前方格字符. 若为加点字符, 则表明需要将该方格字符写入右边方格, 否则表明该方格字符已经写入从而左移.
- $q_t, \forall t \in \Gamma$: 记录要写到右边方格的字符.

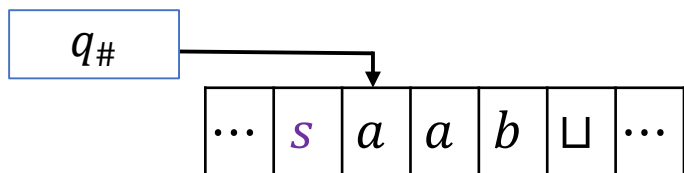
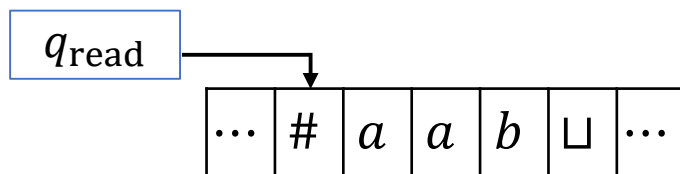
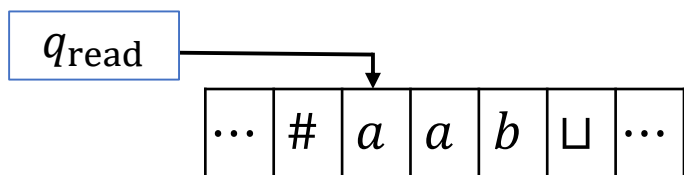


图灵机-多磁带图灵机

一个例子：



图灵机-多磁带图灵机



图灵机的变种-多磁带图灵机

推论. 一个语言是图灵可识别的当且仅当有一个多磁带图灵机识别它.

证明. 若语言 L 是图灵可识别的, 则有一个标准图灵机识别它, 而标准图灵机是一个特殊的多磁带图灵机.

反之, 若 L 能被一个多磁带图灵机识别, 则由**定理 1**, 存在一个标准图灵机识别它. ■

图灵机的变种-非确定性图灵机

定义(非确定性图灵机). 非确定性图灵机在计算的过程中, 机器可以在多种可能性动作中选择一种继续进行. 它的转移函数具有如下形式:

$$\delta: Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\}).$$

其计算是一棵树, 不同分支对应着机器的不同计算分支. 若存在一个计算分支进入接受状态, 则该图灵机接受输入字符串.

图灵机的变种-非确定性图灵机

定理. 每个非确定性图灵机都有一个等价的标准图灵机.

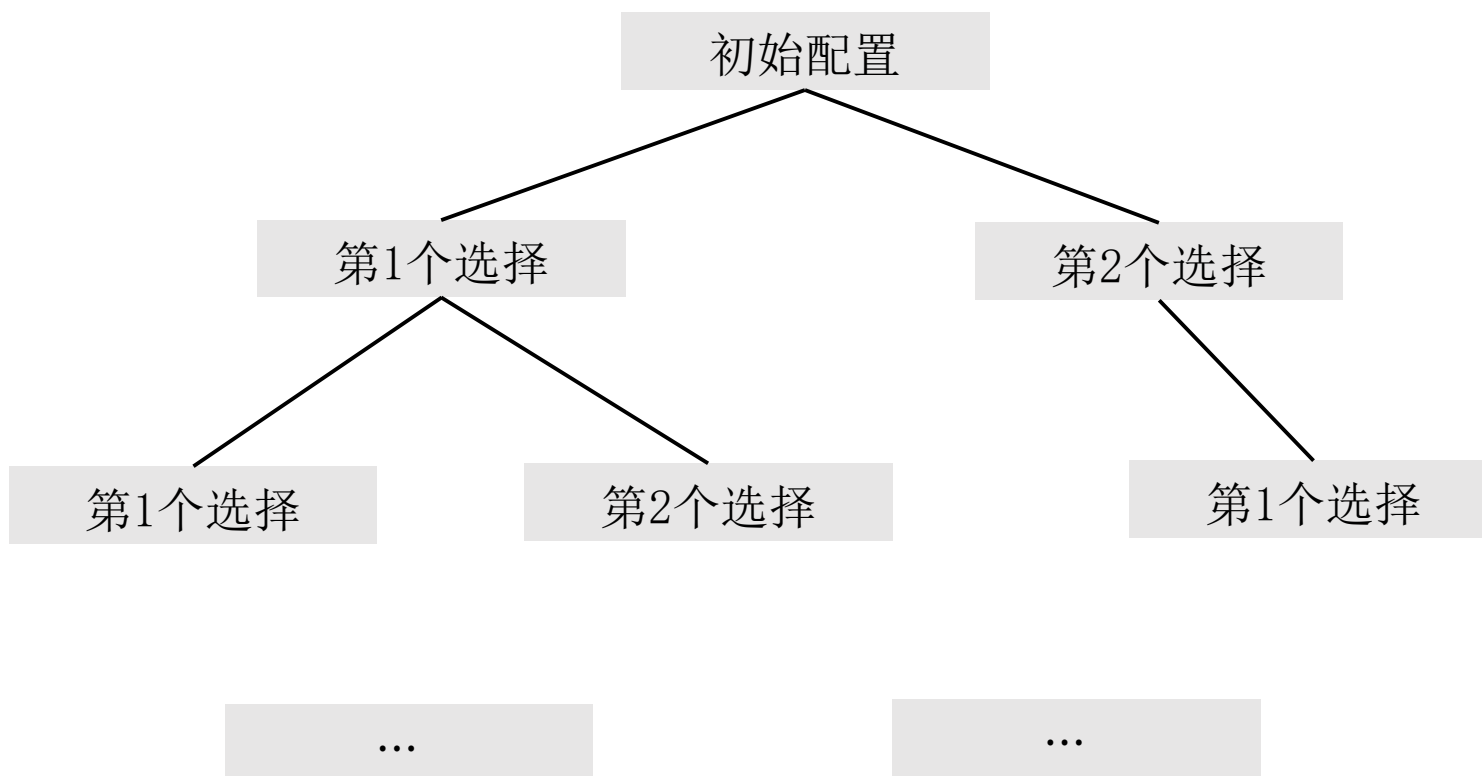
证明思路. 用确定性图灵机 D 来模拟非确定性图灵机 N 的计算: 让 D 尝试 N 的非确定性计算的所有分支. 若 D 能在某个分支中到达接受状态, 则接受; 否则 D 的模拟将永不终止.

将 N 在输入 w 的计算看作一棵树, 树的每个分支代表非确定性的一个计算分支, 结点是 N 的一个配置, 根是起始配置. D 在这个树上搜索接受配置. 我们采用“**广度优先**”策略搜索整棵树, 这个策略是: 在搜索一个深度内的所有分支之后, 再去搜索下一个深度内的所有分支. 此方法能保证 D 可以访问树的所有结点, 直到它遇到接受配置.

图灵机的变种-非确定性图灵机

定理. 每个非确定性图灵机都有一个等价的标准图灵机.

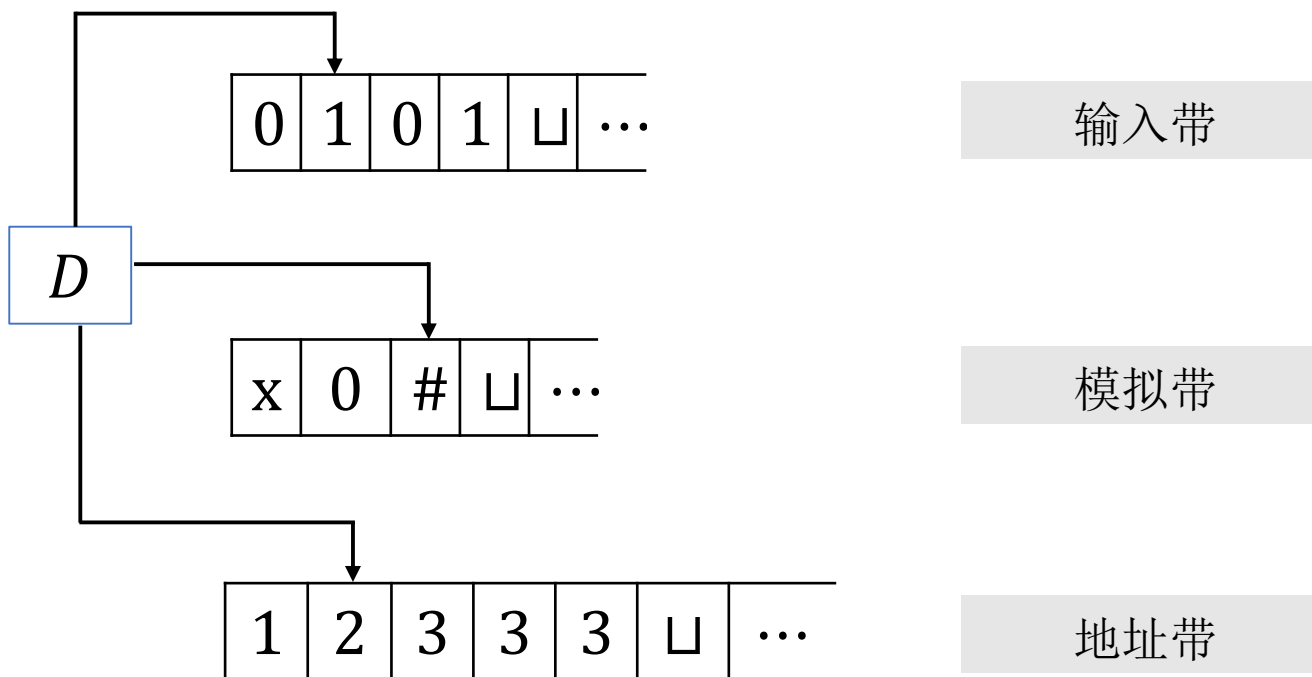
证明思路. 以 $b = 2$ 为例.



图灵机的变种-非确定性图灵机

定理. 每个非确定性图灵机都有一个等价的标准图灵机.

证明. 模拟非确定性图灵机 N 的图灵机 D 有三个磁带：磁带1包含输入字符串且不再改变；磁带2存放 N 的磁带中的内容，此内容对应 N 的某个计算分支；磁带3记录 D 在 N 的非确定性计算树中的位置.



图灵机的变种-非确定性图灵机

证明(续). 首先考虑地址带上表示的数据.

- N 的每个配置确定一个集合, 它是由此配置可能转移到的下一个配置组成, 这些下一个配置是由 N 的转移函数指定的. N 的非确定性计算中的每个结点最多有 b 个子结点, 其中 b 是上述集合中最大的集合所含的元素个数. 对树的每个结点可以给它分配一个地址, 它是字母表 $\Sigma_b = \{1, 2, \dots, b\}$ 上的一个字符串.
- 例如把地址231分配给按以下方式到达的结点: 从根出发走到它的第二个子结点, 再由此走到第三个子结点, 最后由此走到第一个子结点. 如果一个配置所能有的选择太少, 则一个符号可能不对应任何选择. 此种情况下, 地址将无效, 不对应任何结点.

地址带上包含 Σ_b 上的一个字符串, 它代表 N 的计算树中如下分支: 起点是根, 终点是此串表示的地址所对应的结点, 除非这个地址是无效的. 空串是树根的地址.

图灵机的变种-非确定性图灵机

证明(续). D 的描述如下.

1. 开始时第一个磁带包含输入 w , 第二和第三个磁带都是空的.
 2. 把第一个磁带的内容复制到第二个磁带上.
 3. 用模拟带去模拟 N 在输入 w 上的某个计算分支. 在 N 的每一步动作之前, 查询第三个带子上的下一个数字, 以决定在 N 的转移函数所允许的选择中如何作选择. 如果第三个带子上没有符号剩下或这个非确定性的选择是无效的, 则放弃这个分支, 转到第 4 步. 如果遇到拒绝配置也转到第 4 步. 如果遇到接受配置, 则接受这个输入.
 4. 在地址带上用字典序下的下一个字符串代替原来的字符串, 转到第 2 步.
- 

图灵机的变种-非确定性图灵机

推论. 一个语言是图灵可识别的当且仅当存在非确定性图灵机识别它. ■

若NTM N 对所有输入的所有计算分支都停机, 则称 N 是一个判定器.

推论. 一个语言是图灵可判定的当且仅当存在非确定性图灵机判定它.

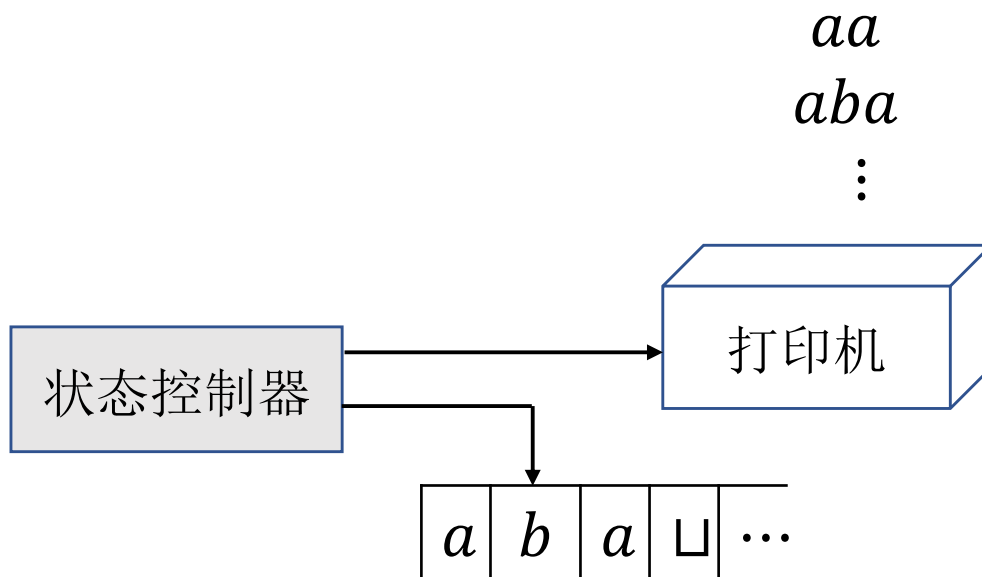
证明. 若 N 是一个判定器, 则每个计算分支要么拒绝要么接受. D 必然能够完整地模拟每个计算分支, 从而 D 也是一个判定器. ■

图灵机的变种-枚举器

定义(枚举器).

- 枚举器是带有“打印机”的图灵机，图灵机把打印机当作输出设备，从而可以打印字符串。每当图灵机想在打印序列中增加一个字符串时，就把此字符串送到打印机。
- 枚举器以空白输入带开始运行，如果不停机它可能会打印出个无限序列的字符串。**枚举器 E 所枚举的语言是最终打印出的字符串的集合。**而且 E 可能以任意顺序生成这个语言中的串，甚至还会有重复。

图灵机的变种-枚举器



图灵机的变种-枚举器

定理(枚举器). 一个语言是图灵可识别的当且仅当存在某个枚举器能枚举它.

证明.

充分性. 假设存在枚举器 E 枚举语言 A , 构造图灵机 M 如下:

$M =$ “对于输入字符串 w :

1. 运行 E . 每一次 E 打印一个字符串, 将其与 w 比较.
2. 如果 w 出现在 E 的打印列表中, 则接受.”

显然 M 接受的字符串恰为 E 打印的字符串.

图灵机的变种-枚举器

定理(枚举器). 一个语言是图灵可识别的当且仅当存在某个枚举器能枚举它.

证明(续).

必要性. 假设存在图灵机 M 识别语言 A , 构造枚举器 E 如下. 假设 $s_1, s_2, s_3, \dots$ 是 Σ^* 中的所有字符串.

First try.

$E =$ “忽略输入:

是否可行?

1. 用 M 依次运行 $s_1, s_2, s_3, \dots$
2. 若 M 接受某个字符串 s_i , 则打印 s_i .”

若 M 运行 s_1 不停机, 则 E 打印不了任何字符串. 😞

图灵机的变种-枚举器

证明(续).

必要性. 假设存在图灵机 M 识别语言 A , 构造枚举器 E 如下. 假设 $s_1, s_2, s_3, \dots$ 是 Σ^* 中的所有字符串.

Second try.

$E =$ “忽略输入:

1. 对 $i = 1, 2, 3, \dots$ 重复如下步骤.
2. 对 $s_1, s_2, \dots, s_i$ 中的每一个, M 以其作为输入运行 i 步.
3. 如果 M 在某个时刻接受某个字符串 s_j , 则打印 s_j .”

显然 E 打印的字符串为 M 接受的字符串. ■

图灵机的变种-枚举器

思考：若 M 接受 s_i 和 s_j ($i < j$)， E 是否一定先打印 s_i 后打印 s_j ？

回答：不一定。若接受 s_j 所需要的步数小于接受 s_i 所需要的步数，则 E 可能先打印 s_j ，后打印 s_i 。

图灵机的变种-总结

- 这里所有讲过的图灵机的变种都与标准图灵机等价.
- 文献中还有许多不同于图灵机的计算模型，它们都有一个共同的特点：有无限大的内存.
- 在一些合理的假设下(比如单一计算步骤只能完成有限量的工作)，所有这些模型都和图灵机等价.



尽管Java和C语法不同，但某一个算法如果能用Java实现，那也能用C实现，反之亦然.

3 算法的定义

算法的定义

非形式地说，**算法**是为实现某个任务而构造的简单指令集。
用日常用语来说，算法有时称为**过程**或**处方**。算法在数学中也起着重要的作用，古代数学文献中包含了各种各样任务的算法描述，如寻找素数和最大公因子。当代数学中更是充满了算法。

算法的定义-希尔伯特第10问题

希尔伯特第10问题：设计一个算法来判定一个多项式是否有整数根.

- 一个**多项式**是一些**项**的和，其中每个项都是一个常数和一些变元的乘积，此常数叫**系数**.
 - $6x^3yz^2$ 是一个项，其系数是6.
 - $6x^3yz^2 + 3xy^2 - x^3 - 10$ 是一个多项式，它有四个项和三个变元 x, y, z .
- 多项式的**根**是对它的变元的一个赋值使此多项式的值为0. 若所有变元的值都为整数，则称这个根是一个**整数根**.

算法的定义-希尔伯特第10问题

希尔伯特第10问题：设计一个算法来判定一个多项式是否有整数根.

- 算法的直观观念不足以回答该问题：证明某个问题不存在任何算法，必须有一个算法的形式化定义.
- 图灵和丘奇在1936年分别利用图灵机和 λ -微分给出了算法的形式化定义. 这两个定义是等价的.

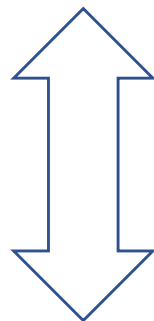
算法的直观观念 等于 图灵机算法



图灵-丘奇论题

算法的定义-希尔伯特第10问题

希尔伯特第10问题: 设计一个算法来判定一个多项式是否有整数根.



$D = \{p \mid p \text{ 是一个有整数根的多项式}\}$ 是否是可判定的?

定理(Matijasevič 1970). D 是不可判定的, 即不存在任何算法判定一个多项式是否有整数根.

算法的定义-希尔伯特第10问题

尽管 $D = \{p | p \text{ 是一个由整数根的多项式}\}$ 是不可判定的，但它是可识别的。

为方便起见，考虑单变量多项式的整数根问题

$$D_1 = \{p | p \text{ 是一个有整数根的单变量多项式}\}.$$

识别 D_1 的图灵机 M_1 如下：

$M_1 =$ “输入是关于变量 x 的多项式 p ：

1. 将 x 相继设置为 $0, 1, -1, 2, -2, \dots$ ，并计算 $p(x)$ 。若对 x 的某个赋值， $p(x) = 0$ ，则接受。”

算法的定义-希尔伯特第10问题

说明. 可以证明若多项式

$$p = c_1x^n + c_2x^{n-1} + \cdots + c_nx + c_{n+1}$$

有根 x_0 , 则

$$|x_0| < (n+1) \frac{c_{\max}}{c_1}, \text{ 其中 } c_{\max} = \max_{1 \leq i \leq n+1} |c_i|.$$

- 从而可以将 M_1 变成一个判定器.
- Matijasevič证明了对于多元多项式计算这样的上界是不可能的!

4 描述图灵机的术语

描述图灵机的术语

三种不同程度的描述.

- **形式化描述**: 给出图灵机的每个组成部分, 包括状态集和转移函数.

例: 判定 $A = \{0^{2^n} | n \geq 0\}$ 的图灵机.

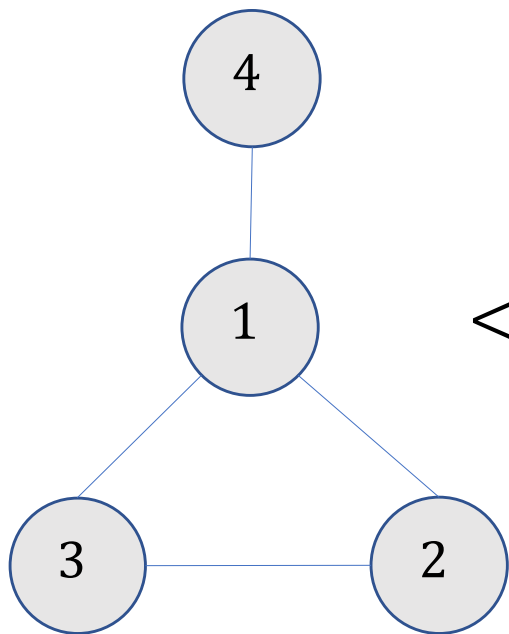
- **实现描述**: 用平直语言描述图灵机是如何移动磁头以及更改磁带内容, 但不给出状态和转移函数.

例: 判定语言 $C = \{a^i b^j c^k | i \times j = k, i, j, k \geq 1\}$ 的图灵机 M_3 .

- **上层描述**: 用平直的语言描述算法, 不涉及如何移动磁头以及更改磁带内容.

描述图灵机的术语

- 图灵机的输入总是一个字符串，如果输入的对象不是字符串，首先需要通过某种编码方式将对象用字符串表示。
- 给定对象 O ，用 $\langle O \rangle$ 表示 O 编码之后的字符串。如果输入有多个对象 $O_1, O_2, \dots, O_k$ ，用 $\langle O_1, O_2, \dots, O_k \rangle$ 表示 $O_1, O_2, \dots, O_k$ 。



$$\langle G \rangle = (1,2,3,4)((1,2), (2,3), (3,1), (1,4))$$

描述图灵机的术语

上层描述：用平直的语言描述算法，不涉及如何移动磁头以及更改磁带内容.

例：令 $A = \{ \langle G \rangle \mid G \text{ 是一个无向连通图} \}$. 给出一个判定语言 A 的图灵机 M .

解： $M =$ “输入 $\langle G \rangle$:

1. 选择 G 的第一个顶点并标记该顶点.
2. 重复以下操作直到没有新的顶点可以标记:

对每个 G 的顶点, 若它和某个已经标记的顶点有边相连, 则标记该顶点.

3. 扫描 G 的所有顶点. 若所有顶点均已标记, 则接受; 否则拒绝.”

描述图灵机的术语

练习. 给出判定图是否连通的图灵机的实现描述.

描述图灵机的术语

练习. 给出判定图是否连通的图灵机的实现描述.

$M =$ “输入 $\langle G \rangle$:

1. 选择 G 的第一个顶点并标记该顶点.
2. 重复以下操作直到没有新的顶点可以标记:

 选择一个已标记的顶点 v , 从左往右扫描磁带对每个 G 的未被划去的顶点, 若它和 v 有边相连, 则标记该顶点.

 划去 v 并将磁带回到最左端.

3. 扫描 G 的所有顶点. 若所有顶点均被划去, 则接受; 否则拒绝.”

丘奇-图灵论题总结

丘奇-图灵论题总结

- 图灵机
- 图灵机的变种
 - 多带机
 - 非确定性
 - 枚举器
- 算法的定义
 - 丘奇-图灵论题
 - 希尔伯特第10问题
- 描述图灵机的术语
 - 形式化描述
 - 实现描述
 - 上层描述

习题

1. 根据图灵机的定义回答下列问题.

- 图灵机能否在磁带上写下 $\sqcup$?
- 磁带字符表 Γ 和输入字符表 Σ 有可能相等吗?
- 图灵机的磁头在连续两个配置中是否可能处于同一个位置?
- 图灵机能否只含有一个状态?

2. 解释为什么下面的描述不是一个合法的图灵机.

M_{bad} = 输入为一个关于 $x_1, \dots, x_k$ 的多项式 p :

1. 尝试所有 $x_1, \dots, x_k$ 的整数取值.
2. 对所有这样的取值计算 p 的值.
3. 若 p 在任何变量的取值时为0, 则接受; 否则拒绝.

习题

3. 给出判定 $\{w \mid w \text{ 包含相同数量的0和1}\}$ 的图灵机的实现描述.
4. 一个有左重置的图灵机和标准的图灵机类似, 但它的转移函数有如下形式 $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, \text{RESET}\}$. 如果 $\delta(q, a) = (r, b, \text{RESET})$, 则机器处于状态 q 时读入 a , 将当前位置字符改写为 b 跳转到状态 r , 并将磁头重置到磁带最左端. 证明有左重置的图灵机等价于标准图灵机.
5. 证明一个语言是可判定的当且仅当存在枚举器能够以字典序枚举该语言.

计算理论-可判定性

- 可判定问题
- 康托对角化方法
- 停机问题

1 可判定问题

可判定问题-与正则语言相关的问题

$$A_{\text{DFA}} = \{ \langle B, w \rangle \mid B \text{ 是一个接受字符串 } w \text{ 的 DFA} \}.$$

定理. A_{DFA} 是一个可判定语言.

证明. 只需设计一个判定 A_{DFA} 的图灵机 M 即可.

M = “对于输入 $\langle B, w \rangle$, 其中 B 是一个 DFA, w 是字符串:

1. 模拟在 B 上运行 w .
2. 如果模拟以接受状态结束, 则接受. 如果模拟以非接受状态结束, 则拒绝.”

可判定问题-与正则语言相关的问题

- **第 1 步:** 检查输入 $\langle B, w \rangle$ 确实表示输入串 w 和 DFA B . B 的一个合理的表示方法是简单的列出它的五个元素 $(Q, \Sigma, \delta, q_0, F)$. 当 M 收到输入时, 首先检查它是否正确的表示了 DFA B 和 w . 如果不是, 则拒绝.
- **第 2 步:** M 直接执行模拟. 用在磁带上写下信息的方法可以跟踪 B 在输入 w 上运行时的当前状态和当前位置. 运行开始时, B 的当前状态是 q_0 , 读写头的当前位置是 w 的最左端符号. 状态和位置的更新是由转移函数决定的. 当 M 处理完 w 的最后一个符号时, 如果 B 处于接收状态, 则 M 接受这个输入; 如果不是, 则 M 拒绝.

可判定问题-与正则语言相关的问题

$A_{\text{NFA}} = \{ \langle B, w \rangle \mid B \text{ 是一个接受字符串 } w \text{ 的 NFA} \}.$

定理. A_{NFA} 是一个可判定语言.

证明. 只需设计一个判定 A_{NFA} 的图灵机 N 即可.

$N =$ “对于输入 $\langle B, w \rangle$, 其中 B 是一个 NFA, w 是字符串:

1. 将 B 转化一个与其等价的 DFA C .
2. 用图灵机 M 运行 $\langle C, w \rangle$.
3. 如果 M 接受, 则接受. 否则拒绝.”

这里 N 使用 M 作为一个子算法. ■

可判定问题-与正则语言相关的问题

$A_{\text{REX}} = \{ \langle R, w \rangle \mid R \text{ 是一个产生字符串 } w \text{ 的正则表达式} \}.$

定理. A_{REX} 是一个可判定语言.

证明. 只需设计一个判定 A_{REX} 的图灵机 P 即可.

$P =$ “对于输入 $\langle R, w \rangle$, 其中 R 是正则表达式, w 是字符串:

1. 将 R 转化一个与其等价的NFA A .
2. 用图灵机 N 运行 $\langle A, w \rangle$.
3. 如果 N 接受, 则接受. 否则拒绝.”



可判定问题-与正则语言相关的问题

$$E_{\text{DFA}} = \{ \langle A \rangle \mid A \text{ 是 DFA 且 } L(A) = \emptyset \}.$$

定理. E_{DFA} 是一个可判定语言.

证明. 一个DFA接受某个字符串当且仅当状态图中存在从初始状态到接受状态的路径.

这是有向图的连通性问题, 因此存在图灵机 T 判定. ■

可判定问题-与正则语言相关的问题

$$EQ_{\text{DFA}} = \{ \langle A, B \rangle \mid A \text{ 和 } B \text{ 是 DFA 且 } L(A) = L(B) \}.$$

定理. EQ_{DFA} 是一个可判定语言.

证明. 令 $L(C)$ 是 $L(A)$ 和 $L(B)$ 的对称差, 即

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (L(B) \cap \overline{L(A)}).$$

- 注意到 $L(A) = L(B) \iff L(C) = \emptyset$.
- 正则语言关于交, 并, 补都是封闭的, 故 $L(C)$ 是正则语言.

$F =$ “对于输入 $\langle A, B \rangle$, 其中 A, B 是 DFA:

1. 构造识别 $L(C)$ 的 DFA C .
2. 用图灵机 T 运行 $\langle C \rangle$.
3. 如果 T 接受, 则接受. 否则拒绝.”



可判定问题-与正则语言相关的问题

$$EQ_{\text{REX}} = \{ \langle R_1, R_2 \rangle \mid R_1 \text{ 和 } R_2 \text{ 是正则表达式且 } L(R_1) = L(R_2) \}$$

是否可判定?

可判定问题-与正则语言相关的问题

$$EQ_{\text{DFA}} = \{ \langle A, B \rangle \mid A \text{ 和 } B \text{ 是 DFA 且 } L(A) = L(B) \}.$$

定理. EQ_{DFA} 是一个可判定语言.

证明. 可以尝试所有长度不超过 $|Q_A||Q_B|$ 的字符串. 若存在某个这样的字符串使得 A, B 结果不同, 则拒绝. 否则接受.

证明: 若 $L(A) \neq L(B)$, 则必然存在长度不超过 $|Q_A||Q_B|$ 的字符串 w 使得 A, B 输出不同的结果.

可判定问题-与上下文无关语法相关的问题

$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ 是一个生成字符串 } w \text{ 的 CFG} \}.$

定理. A_{CFG} 是一个可判定语言.

证明. 若 G 是乔姆斯基范式, 则对于任何长度为 n 的 $w \in L(G)$, 生成 w 需要 $2n - 1$ 步替换.

$S =$ “对于输入 $\langle G, w \rangle$, 其中 G 是一个 CFG, w 是字符串:

1. 将 G 转化成与其等价的乔姆斯基范式.
2. 若 $n = |w| = 0$, 枚举所有长度为 1 的生成序列. 否则 $n \geq 1$, 枚举所有长度为 $2n - 1$ 的生成序列.
3. 如果任何生成序列生成 w , 则接受. 否则拒绝.”

可判定问题-与上下文无关语法相关的问题

定理. 每个上下文无关的语言都是可判定的.

证明. 令 G 是生成语言 A 的上下文无关语法, 下面设计一个判定 A 的图灵机 M_G .

回忆: S 是判定

$$A_{\text{CFG}} = \{ \langle G, w \rangle \mid G \text{ 是一个生成字符串 } w \text{ 的 CFG} \}.$$

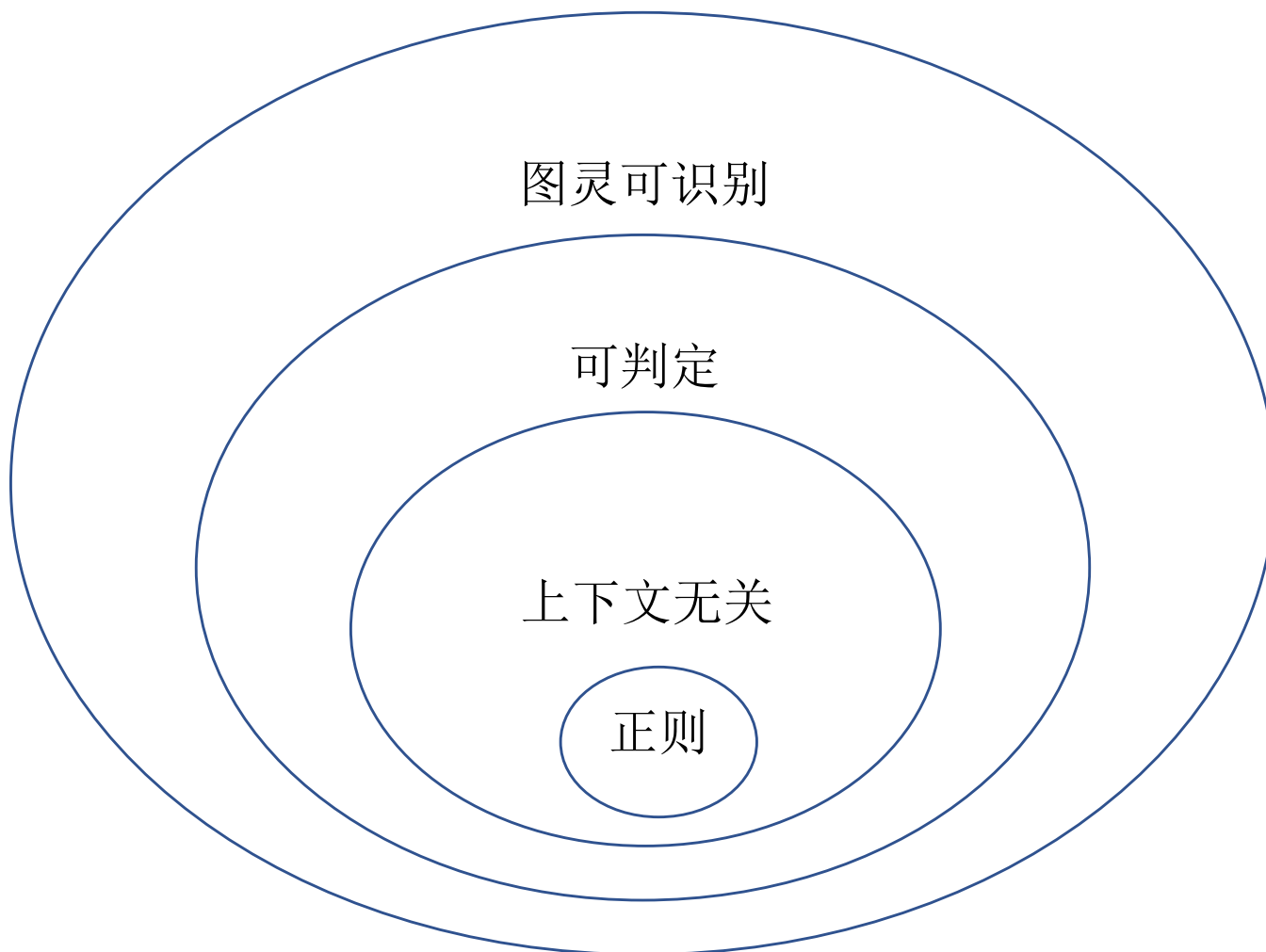
的图灵机.

M_G = “对于输入 w :

1. 对于输入 $\langle G, w \rangle$ 运行 S .
2. 若 S 接受, 则 M_G 接受, 否则拒绝.”



可判定问题-与上下文无关语法相关的问题



可判定问题-与上下文无关语法相关的问题

$$E_{\text{CFG}} = \{ \langle G \rangle \mid G \text{ 是 CFG 且 } L(G) = \emptyset \}.$$

定理. E_{CFG} 是一个可判定语言.

证明. 对于每个变量, 判定该变量是否能生成一个只有终止符的字符串.

$R =$ “对于输入 $\langle G \rangle$, 其中 G 是一个 CFG:

1. 标记所有 G 的终止符.
2. 重复以下操作直到没有新的变量可以标记:

若 $A \rightarrow U_1 U_2 \cdots U_k$ 是 G 的替换规则且 $U_1, \dots, U_k$ 均已标记, 则标记变量 A .

3. 若 G 的初始变量没有标记, 则接受; 否则拒绝.”

G :

$S \rightarrow RTa$

$R \rightarrow Tb$

$T \rightarrow a$

可判定问题-与上下文无关语法相关的问题

$$EQ_{CFG} = \{ \langle G, H \rangle \mid G, H \text{ 是 CFG 且 } L(G) = L(H) \}.$$

- 利用 E_{DFA} 证明了 EQ_{DFA} 是可判定的.
- 类似的方法是否能够证明 EQ_{CFG} 是可判定的?

No! 因为上下文无关语法对交运算和补运算是不封闭的.

EQ_{CFG} 是不可判定的(第 5 章).

可判定问题

练习: 证明 $INFINITE_{DFA} = \{ \langle A \rangle \mid A \text{ 是 DFA 且 } L(A) \text{ 是无穷集} \}$ 是可判定的.

证明. 令 A 是一个DFA, k 是 A 含有的状态数量.

- 注意到 $L(A)$ 是无穷集当且仅当 $L(A)$ 接受任意长的字符串.
- 另外, 若 A 接受一个长度至少为 k 的字符串, 则泵引理^的证明告诉我们我们可以pump这个字符串, 使得 A 接受任意长的字符串.

因此, 只需判定 $L(A)$ 是否接受一个长度至少为 k 的字符串.

显然, 可以构造一个DFA D_k 使得 $L(D_k)$ 为长度至少为 k 的字符串. 从而转化为判定 $L(A) \cap L(D_k) = \emptyset$. ■

2 康托对角化方法

康托对角化方法

- 比较有限集合 A 和 B 的大小: 通过数集中的元素.
- 若 A, B 是无穷集合, 如何比较大小?
 - 德国数学家康托通过映射给出了一个非常优雅的理论.
 - 直观上, 如果 A, B 中的元素能够两两配对, 则 A, B 大小相同.

康托对角化方法

定义(单射, 满射, 双射). 设 $f: A \rightarrow B$ 是一个映射.

- 若任何 $a_1, a_2 \in A$ 且 $a_1 \neq a_2$, 都有 $f(a_1) \neq f(a_2)$, 则称 f 为单射. (不同的元素有不同的像)
- 若任给 $b \in B$, 都存在 $a \in A$ 使得 $b = f(a)$, 则称 f 为满射.
- 若 f 既是单射又是满射, 则称 f 为双射(一一对应).

康托对角化方法

若存在 A 到 B 的双射，则称 A 与 B 有相同的大小.

定义(可数集). 如果一个集合 A 是有限的或者与 $\mathbb{N}$ 有相同的大小，则称 A 是可数的.

例 1: 正偶数集合 $\mathbb{E} = \{2, 4, 6, \dots\}$ 是可数的.

解: $f(n) = 2n$ 是 $\mathbb{N}$ 到 $\mathbb{E}$ 的双射.



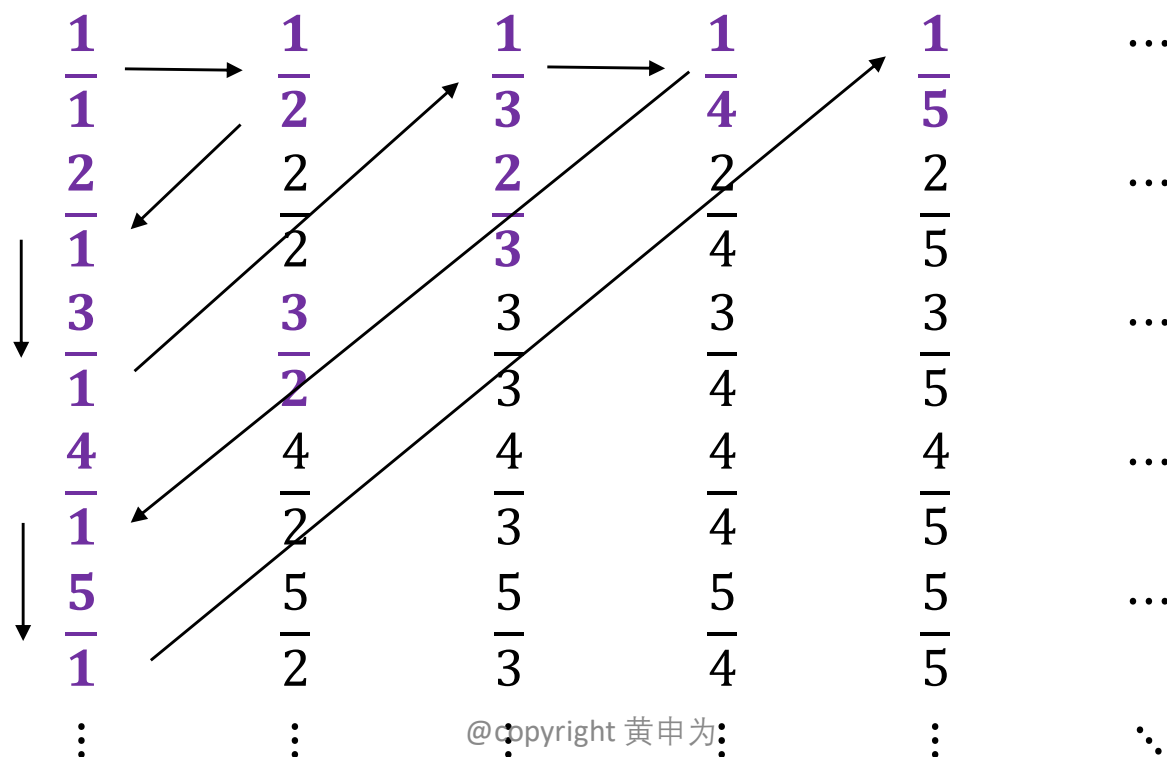
康托对角化方法

练习. 证明 $A = \{2, 3, 4, \dots\}$ 是可数的.

康托对角化方法

定理. 正有理数集 $\mathbb{Q}^+ = \left\{ \frac{m}{n} \mid m, n \in \mathbb{N} \right\}$ 是可数的.

证明. 将 $\mathbb{Q}^+$ 中所有元素列为 $a_1, a_2, a_3, \dots$, 则将 a_i 与 i 配对表明 $\mathbb{Q}^+$ 是可数的.



康托对角化方法

练习. 证明可数个可数集合的并仍然是可数集.

康托对角化方法

问题：是否所有的无穷集都是可数的？

定理. 实数集 $\mathbb{R}$ 不是可数的.

证明. 只需证明 $[0,1]$ 是不可数的即可.

- 假设 $[0,1]$ 是可数的，即可以将 $[0,1]$ 中所有实数列为 $r_1, r_2, \dots, r_n, \dots$
- 根据实数定义，每个 r_i 都有一个唯一的表示

$$r_i = 0.d_{i1}d_{i2} \cdots d_{in} \cdots$$

- 现构造一个新的实数 $r = 0.d_1d_2 \cdots d_n \cdots \in [0,1]$ 如下：

$$d_i = \begin{cases} 5 & d_{ii} \neq 5 \\ 4 & d_{ii} = 5 \end{cases}$$

- 则对任何 i ， r 与 r_i 在第 i 个位置上的数不相同，从而 $r \neq r_i$.
- 这与 $r_1, r_2, \dots, r_n, \dots$ 是所有 $[0,1]$ 中的实数矛盾. ■

康托对角化方法

								r
r_1	0.	<u>1</u>	4	1	5	9	...	5
r_2	0.	5	<u>5</u>	5	5	5	...	4
r_3	0.	1	2	<u>3</u>	4	5	...	5
r_4	0.	4	1	6	<u>0</u>	8	...	5
r_5	0.	5	9	7	3	<u>4</u>	...	5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$

康托对角化方法

练习. 证明所有无穷 $\{0,1\}$ -序列的集合是不可数的.

康托对角化方法

推论. 存在图灵不可识别的语言.

证明.

- 所有图灵机是可数集.
- 所有语言的集合是不可数的.
 - 所有语言和无穷 $\{0,1\}$ -序列存在一一对应关系.
 - 所有的无穷 $\{0,1\}$ -序列是不可数的.
- 而每个图灵机只能识别一个语言, 从而推论得证. ■

康托对角化方法

扩展. 一个实数 a 称为代数数, 如果存在有理系数多项式 $p(x)$ 使得 $p(a) = 0$. 否则称 a 为超越数.

例.

- $\sqrt{2}$ 是代数数, 因为它是 $x^2 - 2 = 0$ 的根.
- 而 π 和 e 是超越数.

定理. 代数数是可数的, 从而超越数是不可数的.

说明. 这个定理表明和超越数相比, 代数数只是沧海一粟. 但是要给出一个具体的超越数却极其困难 (尝试证明 π 是超越数!). 在草堆中找一根草竟然是如此困难!

3 停机问题

停机问题

令 $A_{TM} = \{ \langle M, w \rangle \mid M \text{ 是接受 } w \text{ 的图灵机} \}$.

观察. A_{TM} 是图灵可识别的.

证明. 我们给出下列识别 A_{TM} 的图灵机.

$U =$ “对于输入 $\langle M, w \rangle$, 其中 M 是一个图灵机, w 是字符串:

1. 模拟在 M 上运行 w .
2. 若 M 进入接受状态, 则接受; 若 M 进入拒绝状态, 则拒绝.”

注意若 M 运行 w 不停机, 则 U 对于输入 $\langle M, w \rangle$ 也不停机. 所以 U 是一个识别器. ■

停机问题

定理. A_{TM} 是不可判定的.

证明. 我们用反证法.

- 假设存在 A_{TM} 的判定器 H . 则

$$H(< M, w >) = \begin{cases} \text{接受} & \text{若 } M \text{ 接受 } w \\ \text{拒绝} & M \text{ 不接受 } w \end{cases}$$

停机问题

证明(续).

- 现构造一个新的图灵机 D , 该图灵机的输入是一个图灵机 M , 并调用 H 以获得 M 运行自己的描述 $\langle M \rangle$ 时的接受情况, 从而输出相反的结果.

$D =$ “对于输入 $\langle M \rangle$, 其中 M 是一个图灵机:

1. 对 $\langle M, \langle M \rangle \rangle$ 运行 H .
2. 输出与 H 相反的结果, 即若 H 接受, 则拒绝; 若 H 拒绝, 则接受.”

说明. “在一个机器上运行它自己的描述” 类似以一个程序本身作为输入来运行这个程序(比如编译器是翻译其他程序的程序), 是有意义的.

停机问题

证明(续). 总而言之,

$$D(< M >) = \begin{cases} \text{接受} & \text{若 } M \text{ 不接受 } < M > \\ \text{拒绝} & \text{若 } M \text{ 接受 } < M > \end{cases}.$$

- 现在考察如果让 D 运行自己的描述 $< D >$ 会是什么结果?

根据 D 的构造有,

$$D(< D >) = \begin{cases} \text{接受} & \text{若 } D \text{ 不接受 } < D > \\ \text{拒绝} & \text{若 } D \text{ 接受 } < D > \end{cases},$$

无论那种情况都是一个矛盾. ■

停机问题

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	接受		接受	接受	...
M_2	接受	接受	接受	接受	...
M_3	接受	接受			...
M_4					...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

图灵机 M_i 运行 $\langle M_j \rangle$ 的情况

停机问题

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	接受	拒绝	接受	接受	...
M_2	接受	接受	接受	接受	...
M_3	接受	接受	拒绝	拒绝	...
M_4	拒绝	拒绝	拒绝	拒绝	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

图灵机 H 运行 $\langle M_i, \langle M_j \rangle \rangle$ 的情况

停机问题

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$
M_1	<u>接受</u>	拒绝	接受	接受	...	
M_2	接受	<u>接受</u>	接受	接受	...	
M_3	接受	接受	<u>拒绝</u>	拒绝	...	
M_4	拒绝	拒绝	拒绝	<u>拒绝</u>	...	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	
D	拒绝	拒绝	接受	接受	...	<u>?</u>

图灵机 D 反转对角线上的结果

一个图灵不可识别问题

定理. 一个语言是可判定的当且仅当它和它的补都是图灵可识别的.

推论. $\overline{A_{\text{TM}}}$ 不是图灵可识别的.

证明. 因为 A_{TM} 是图灵可识别且不可判定, 故推论由上述定理可得. ■

一个图灵不可识别问题

定理. 一个语言是可判定的当且仅当它和它的补都是图灵可识别的.

证明.

必要性. 假设 A 是可判定的.

- 则 $\bar{A}$ 是可判定的.
- 一个可判定的语言必然是图灵可识别的.

从而必要性得证.

一个图灵不可识别问题

证明(续).

充分性. 假设 $A, \bar{A}$ 是可识别的.

- 令 M_1, M_2 分别是识别 $A, \bar{A}$ 的图灵机.
- 构造判定 A 的图灵机 M :

M = “对于输入 w :

1. 平行地在 M_1 和 M_2 上运行 w .
2. 如果 M_1 接受, 则接受; 如果 M_2 接受, 则拒绝.”

则 M 判定 A (为什么?).



一个图灵不可识别问题

图灵可识别性对于哪些操作不是封闭的？

- 并
- 交
- 补
- 连接
- 星号

可判定性总结

可判定性总结

- 可判定问题
 - 与正则语言相关的问题
 - 与上下文语法相关的问题
- 康托对角化方法
 - 可数集
 - 不可数集
- 停机问题

计算理论-可规约性

- 语言理论中的不可判定问题
- 通过计算历史规约
- 映射规约

语言理论中的不可判定问题

归约旨在将一个问题转化为另一个问题, 且使得可以用第二个问题的解来求解第一个问题. 在日常生活中, 虽然不这样称呼, 但时常会遇见可归约性问题.

- 例如, 在一个新城市中认路, 如果有一张地图就将在城市认路问题归约为得到地图问题.
- 数学上的例子更多.
 - 计算长方形的面积规约到计算边长.
 - A_{NFA} 规约到 A_{DFA} .

1 语言理论中的不可判定问题

语言理论中的不可判定问题

$HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ 是图灵机且对于输入 } w \text{ 停机} \}.$

定理. $HALT_{TM}$ 是不可判定的.

证明. 假设 $HALT_{TM}$ 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

- 给定输入 $\langle M, w \rangle$, 使用 R 可判定 M 对于 w 是否停机. 若不停机, 则 $\langle M, w \rangle \notin A_{TM}$, 故 S 拒绝.
- 否则 M 对于 w 停机. 从而 S 模拟在 M 上运行 w .

语言理论中的不可判定问题

定理. $HALT_{TM}$ 是不可判定的.

证明(续). 假设 $HALT_{TM}$ 有一个判定器 R . 则利用可以构造判定 A_{TM} 的判定器 S .

$S =$ “对于输入 $\langle M, w \rangle$, 其中 M 为图灵机, w 为字符串:

1. 对于输入 $\langle M, w \rangle$ 运行 R .
2. 如果 R 拒绝, 则拒绝.
3. 如果 R 接受, 则模拟在 M 上运行 w . 若 M 接受, 则接受;
否则 M 拒绝, 则拒绝.”

显然, S 判定 A_{TM} .

这与 A_{TM} 是不可判定的矛盾, 故 R 不能存在. ■

语言理论中的不可判定问题

$$E_{\text{TM}} = \{ \langle M \rangle \mid M \text{ 是图灵机且 } L(M) = \emptyset \}.$$

定理. E_{TM} 是不可判定的.

证明. 假设 E_{TM} 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

First Try. 对输入 $\langle M, w \rangle$, 在 R 上运行 $\langle M \rangle$.

- 若 R 接受, 则 $L(M) = \emptyset$, 从而 M 不接受 w .
- 若 R 拒绝, 则 $L(M) \neq \emptyset$, 可知 M 必然接受某个字符串, 但是没有办法知道这个字符串是不是 w .

语言理论中的不可判定问题

定理. E_{TM} 是不可判定的.

证明(续).

Second Try. 修改 M 使得除 w 外所有字符串都拒绝.

$M_w =$ “对于输入 x :

1. 若 $x \neq w$, 则拒绝.
2. 若 $x = w$, 则模拟在 M 上运行 w . 如果 M 接受, 则 M_w 接受.”

显然, M 接受 w 当且仅当 $L(M_w) \neq \emptyset$.

语言理论中的不可判定问题

定理. E_{TM} 是不可判定的.

证明(续). 假设 E_{TM} 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

$S =$ “对于输入 $\langle M, w \rangle$, 其中 M 为图灵机, w 为字符串:

1. 按上述方式构造图灵机 M_w .
2. 在 R 上运行 $\langle M_w \rangle$.
3. 若 R 接受, 则 S 拒绝; 若 R 拒绝, 则 S 接受.”

这与 A_{TM} 是不可判定的矛盾, 故 R 不能存在. ■

语言理论中的不可判定问题

$EQ_{TM} = \{ \langle M_1, M_2 \rangle \mid M_1, M_2 \text{ 是图灵机且 } L(M_1) = L(M_2) \}.$

定理. EQ_{TM} 是不可判定的.

证明. 假设 EQ_{TM} 有一个判定器 R . 则利用 R 可以构造判定 E_{TM} 的判定器 S .

$S =$ “对于输入 $\langle M \rangle$, 其中 M 为图灵机:

1. 令 M_{reject} 是拒绝所有字符串的图灵机.
2. 在 R 上运行 $\langle M, M_{\text{reject}} \rangle$.
3. 若 R 接受, 则 S 接受; 若 R 拒绝, 则 S 拒绝.”

显然 S 判定 E_{TM} , 这是一个矛盾. ■

语言理论中的不可判定问题

$REGULAR_{TM} = \{ \langle M \rangle \mid M \text{ 是图灵机且 } L(M) \text{ 是正则语言} \}.$

定理. $REGULAR_{TM}$ 是不可判定的.

证明. 假设 $REGULAR_{TM}$ 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

给定输入 $\langle M, w \rangle$, 构造一个图灵机 M_w 使得

M 接受 w 当且仅当 $L(M_w)$ 是正则语言.

- M_w 接受非正则语言 $\{0^n 1^n \mid n \geq 0\}$ 的所有字符串.
- 若 M 接受 w , 则 M_w 接受任何其他字符串.

$M_w =$ “对于输入 x :

1. 若 $x = 0^n 1^n$, 则接受.
2. 若 $x \neq 0^n 1^n$, 则模拟在 M 上运行 w . 如果 M 接受, 则 M_w 接受.”

语言理论中的不可判定问题

定理. $REGULAR_{TM}$ 是不可判定的.

证明(续). 假设 $REGULAR_{TM}$ 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

$S =$ “对于输入 $\langle M, w \rangle$, 其中 M 为图灵机, w 为字符串:

1. 按上述方式构造图灵机 M_w .
2. 在 R 上运行 $\langle M_w \rangle$.
3. 若 R 接受, 则 S 接受; 若 R 拒绝, 则 S 拒绝.”

显然, S 判定 A_{TM} , 这是一个矛盾. ■

语言理论中的不可判定问题

练习. 考虑下面的问题: 给定一个2个磁带的图灵机 M 以及字符串 w , M 运行 w 时是否在它的第2个磁带上写下一个非空白符? 给出该问题的语言描述并证明这个问题是不可判定的.

证明. 给定 A_{TM} 的实例 $\langle M, w \rangle$, 令 M' 是如下图灵机:

$M' =$ “ 输入 x :

1. 在第1个磁带上模拟 M 运行 x .
2. 若 M 进入接受状态, 则在 M' 的第2个磁带上写下一个非空白符号.”

则 M 接受 w 当且仅当 M' 运行 w 在第2个磁带上写下非空白符.

计算理论-可规约性

- 语言理论中的不可判定问题
- 通过计算历史规约
- 映射规约

语言理论中的不可判定问题

归约旨在将一个问题转化为另一个问题, 且使得可以用第二个问题的解来求解第一个问题. 在日常生活中, 虽然不这样称呼, 但时常会遇见可归约性问题.

- 例如, 在一个新城市中认路, 如果有一张地图就将在城市认路问题归约为得到地图问题.
- 数学上的例子更多.
 - 计算长方形的面积规约到计算边长.
 - A_{NFA} 规约到 A_{DFA} .

1 语言理论中的不可判定问题

语言理论中的不可判定问题

$HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ 是图灵机且对于输入 } w \text{ 停机} \}.$

定理. $HALT_{TM}$ 是不可判定的.

证明. 假设 $HALT_{TM}$ 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

- 给定输入 $\langle M, w \rangle$, 使用 R 可判定 M 对于 w 是否停机. 若不停机, 则 $\langle M, w \rangle \notin A_{TM}$, 故 S 拒绝.
- 否则 M 对于 w 停机. 从而 S 模拟在 M 上运行 w .

语言理论中的不可判定问题

定理. $HALT_{TM}$ 是不可判定的.

证明(续). 假设 $HALT_{TM}$ 有一个判定器 R . 则利用可以构造判定 A_{TM} 的判定器 S .

$S =$ “对于输入 $\langle M, w \rangle$, 其中 M 为图灵机, w 为字符串:

1. 对于输入 $\langle M, w \rangle$ 运行 R .
2. 如果 R 拒绝, 则拒绝.
3. 如果 R 接受, 则模拟在 M 上运行 w . 若 M 接受, 则接受;
否则 M 拒绝, 则拒绝.”

显然, S 判定 A_{TM} .

这与 A_{TM} 是不可判定的矛盾, 故 R 不能存在. ■

语言理论中的不可判定问题

$$E_{\text{TM}} = \{ \langle M \rangle \mid M \text{ 是图灵机且 } L(M) = \emptyset \}.$$

定理. E_{TM} 是不可判定的.

证明. 假设 E_{TM} 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

First Try. 对输入 $\langle M, w \rangle$, 在 R 上运行 $\langle M \rangle$.

- 若 R 接受, 则 $L(M) = \emptyset$, 从而 M 不接受 w .
- 若 R 拒绝, 则 $L(M) \neq \emptyset$, 可知 M 必然接受某个字符串, 但是没有办法知道这个字符串是不是 w .

语言理论中的不可判定问题

定理. E_{TM} 是不可判定的.

证明(续).

Second Try. 修改 M 使得除 w 外所有字符串都拒绝.

$M_w =$ “对于输入 x :

1. 若 $x \neq w$, 则拒绝.
2. 若 $x = w$, 则模拟在 M 上运行 w . 如果 M 接受, 则 M_w 接受.”

显然, M 接受 w 当且仅当 $L(M_w) \neq \emptyset$.

语言理论中的不可判定问题

定理. E_{TM} 是不可判定的.

证明(续). 假设 E_{TM} 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

$S =$ “对于输入 $\langle M, w \rangle$, 其中 M 为图灵机, w 为字符串:

1. 按上述方式构造图灵机 M_w .
2. 在 R 上运行 $\langle M_w \rangle$.
3. 若 R 接受, 则 S 拒绝; 若 R 拒绝, 则 S 接受.”

这与 A_{TM} 是不可判定的矛盾, 故 R 不能存在. ■

语言理论中的不可判定问题

$EQ_{TM} = \{ \langle M_1, M_2 \rangle \mid M_1, M_2 \text{ 是图灵机且 } L(M_1) = L(M_2) \}.$

定理. EQ_{TM} 是不可判定的.

证明. 假设 EQ_{TM} 有一个判定器 R . 则利用 R 可以构造判定 E_{TM} 的判定器 S .

$S =$ “对于输入 $\langle M \rangle$, 其中 M 为图灵机:

1. 令 M_{reject} 是拒绝所有字符串的图灵机.
2. 在 R 上运行 $\langle M, M_{\text{reject}} \rangle$.
3. 若 R 接受, 则 S 接受; 若 R 拒绝, 则 S 拒绝.”

显然 S 判定 E_{TM} , 这是一个矛盾. ■

语言理论中的不可判定问题

$REGULAR_{TM} = \{ \langle M \rangle \mid M \text{ 是图灵机且 } L(M) \text{ 是正则语言} \}.$

定理. $REGULAR_{TM}$ 是不可判定的.

证明. 假设 $REGULAR_{TM}$ 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

给定输入 $\langle M, w \rangle$, 构造一个图灵机 M_w 使得

M 接受 w 当且仅当 $L(M_w)$ 是正则语言.

- M_w 接受非正则语言 $\{0^n 1^n \mid n \geq 0\}$ 的所有字符串.
- 若 M 接受 w , 则 M_w 接受任何其他字符串.

$M_w =$ “对于输入 x :

1. 若 $x = 0^n 1^n$, 则接受.
2. 若 $x \neq 0^n 1^n$, 则模拟在 M 上运行 w . 如果 M 接受, 则 M_w 接受.”

语言理论中的不可判定问题

定理. $REGULAR_{TM}$ 是不可判定的.

证明(续). 假设 $REGULAR_{TM}$ 有一个判定器 R . 则利用 R 可以构造判定 A_{TM} 的判定器 S .

$S =$ “对于输入 $\langle M, w \rangle$, 其中 M 为图灵机, w 为字符串:

1. 按上述方式构造图灵机 M_w .
2. 在 R 上运行 $\langle M_w \rangle$.
3. 若 R 接受, 则 S 接受; 若 R 拒绝, 则 S 拒绝.”

显然, S 判定 A_{TM} , 这是一个矛盾. ■

语言理论中的不可判定问题

练习. 考虑下面的问题: 给定一个2个磁带的图灵机 M 以及字符串 w , M 运行 w 时是否在它的第2个磁带上写下一个非空白符? 给出该问题的语言描述并证明这个问题是不可判定的.

证明. 给定 A_{TM} 的实例 $\langle M, w \rangle$, 令 M' 是如下图灵机:

M' = “输入 x :

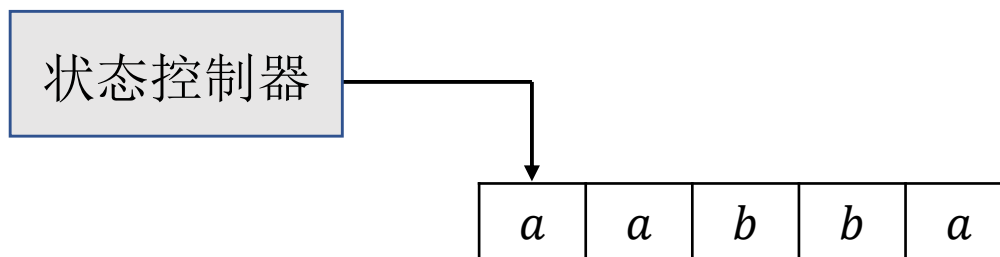
1. 在第1个磁带上模拟 M 运行 x .
2. 若 M 进入接受状态, 则在 M' 的第2个磁带上写下一个非空白符号.”

则 M 接受 w 当且仅当 M' 运行 w 在第2个磁带上写下非空白符.

2 通过计算历史规约

线性有界自动机

定义(线性有界自动机). 线性有界自动机(LBA)是一种受到限制的图灵机, 它不允许其读写头离开包含输入的磁带区域. 如果此机器试图将它的读写头移出输入的两个端点, 则读写头就保持在原地不动. 这与普通图灵机的读写头不会离开磁带的左端点的方式一样.



说明. 判定 $A_{\text{DFA}}, A_{\text{CFG}}, E_{\text{DFA}}, E_{\text{CFG}}$ 的图灵机是线性有界自动机.

线性有界自动机

引理. 设 M 是线性有界自动机, 且 M 有 q 个状态, g 个磁带字符. 则对一个输入规模为 n 的字符串, M 有 qng^n 个不同的配置.

证明. 一个配置由三个部分组成, 即所处状态, 磁带内容和磁头位置.

- 所处状态有 q 个选择.
- 磁带内容有 g^n 个不同的可能.
- 磁头位置有 n 个选择.

故配置的个数为 qng^n .



线性有界自动机

令 $A_{\text{LBA}} = \{ \langle M, w \rangle \mid M \text{ 是 LBA 且 } M \text{ 接受 } w \}$

定理. A_{LBA} 是可判定的.

证明. 考虑下面的图灵机

$L =$ “对于输入 $\langle M, w \rangle$:

1. 模拟 M 运行 w qng^n 步或者直到停机.
2. 若 M 停机, 则输出 M 的结果; 否则拒绝.”

这里的关键是能够判定 M 什么时候不停机.

由上页的引理, 若 M 运行 w qng^n 步后仍不停机, 则由鸽笼原理, M 必定在两个不同的时刻进入了同一个配置, 从而 M 对 w 会无限运行下去. 这时候拒绝 w . ■

通过计算历史规约- E_{LBA}

$$E_{\text{LBA}} = \{ \langle M \rangle \mid M \text{ 是 LBA 且 } L(M) = \emptyset \}.$$

计算历史

定义(接受和拒绝计算历史). 令 M 是一个图灵机, w 是一个字符串. 在 M 上运行 w 的一个接受(拒绝)计算历史是一个序列 $C_1, C_2, \dots, C_\ell$, 其中

- C_1 是初始配置;
- C_ℓ 是接受(拒绝)配置;
- C_i 是由 C_{i-1} 根据 M 的转移规则而产生的.

通过计算历史规约- E_{LBA}

令 $E_{\text{LBA}} = \{ \langle M \rangle \mid M \text{ 是 LBA 且 } L(M) = \emptyset \}$.

定理. E_{LBA} 是不可判定的.

证明思路: 通过 A_{TM} 规约.

- 给定 A_{TM} 的输入 $\langle M, w \rangle$, 若 M 接受 w , 则存在一个接受计算历史.
- 所以如果能构造一个 LBA B 使得 $L(B)$ 恰好为 M 接受 w 的计算历史, 则通过判定 $L(B)$ 是否为空, 就能给出 M 是否接受 w 的答案: 若 $L(B) = \emptyset$, 则 M 不接受 w ; 若 $L(B) \neq \emptyset$, 则 M 接受 w .

因此, 只需要从 $\langle M, w \rangle$ 出发构造这样一个 LBA B 即可.

通过计算历史规约- E_{LBA}

定理. E_{LBA} 是不可判定的.

证明思路(续): 为了方便叙述, 假设 B 的输入具有如下形式:

$$\# \underbrace{\quad}_{C_1} \# \underbrace{\quad}_{C_2} \# \underbrace{\quad}_{C_3} \# \cdots \# \underbrace{\quad}_{C_\ell} \#$$

其含义是每两个 $\#$ 之间的字符串代表一个配置.

给定输入 x , B 是如下方式工作:

- 首先 B 检查 C_1 是否为 M 运行 w 的初始配置, 即检查 C_1 是否为 $q_0 w_1 \dots w_n$.
- 其次 B 检查 C_ℓ 是否为接受配置, 即检查 C_ℓ 中的状态字符是否为 q_{accept} .

通过计算历史规约- E_{LBA}

定理. E_{LBA} 是不可判定的.

证明思路(续): 给定输入 x , B 是如下方式工作:

- 首先 B 检查 C_1 是否为 M 运行 w 的初始配置, 即检查 C_1 是否为 $q_0 w_1 \dots w_n$.
- 其次 B 检查 C_ℓ 是否为接受配置, 即检查 C_ℓ 中的状态字符是否为 q_{accept} .
- 对于每两个相邻的字符串 C_i 和 C_{i+1} , 检查 C_{i+1} 是否由 C_i 通过 M 的转移规则得到.

为了检查这个条件, 磁头需要在 C_i 和 C_{i+1} 之间来回移动, 为了记录当前位置, 在来回移动的时候对当前位置中的符号加点.

通过计算历史规约- E_{LBA}

令 $E_{\text{LBA}} = \{ \langle M \rangle \mid M \text{ 是 LBA 且 } L(M) = \emptyset \}$.

定理. E_{LBA} 是不可判定的.

证明: 假设存在判定 E_{LBA} 的判定器 R . 可以构造判定 A_{TM} 的判定器 S :

$S =$ “对于输入 $\langle M, w \rangle$:

1. 从 M, w 出发构造 **LBA** $B_{M,w}$.
2. 在 R 上运行 $\langle B_{M,w} \rangle$.
3. 若 R 拒绝, 则**接受**; 否则**拒绝**.”



通过计算历史规约- ALL_{CFG}

令 $ALL_{CFG} = \{ \langle G \rangle \mid G \text{ 是 CFG 且 } L(G) = \Sigma^* \}$.

定理. ALL_{CFG} 是不可判定的.

证明思路: 通过 A_{TM} 规约.

- 给定 A_{TM} 的输入 $\langle M, w \rangle$, 构造一个 CFG G 使得 $L(G) = \Sigma^*$ 当且仅当 M 不接受 w .
- 要满足这个条件, G 会生成所有代表非接受计算历史的字符串.

通过计算历史规约- ALL_{CFG}

定理. ALL_{CFG} 是不可判定的.

证明: 假设存在判定 ALL_{CFG} 的判定器.

- 给定 A_{TM} 的输入 $\langle M, w \rangle$, 构造一个CFG G 使得 $L(G) = \Sigma^*$ 当且仅当 M 不接受 w .
- 因为CFG与PDA等价, 构造接受所有非接受计算历史的PDA P .
一个计算历史 $C_1, \dots, C_\ell$ 是非接受的, 则必须违反三个条件之一:
 - C_1 不是初始配置;
 - C_ℓ 中的状态不是接受状态;
 - C_{i+1} 不是由 C_i 通过 M 的转移函数得到的.

通过计算历史规约- ALL_{CFG}

证明(续): 构造接受代表非接受计算历史的所有字符串的PDA P .

一个计算历史 $C_1, \dots, C_\ell$ 是非接受的, 则必须违反三个条件之一:

- C_1 不是初始配置;
- C_ℓ 中的状态不是接受状态;
- C_{i+1} 不是由 C_i 通过 M 的转移函数得到的.

P 按照如下方式工作: 非确定性猜测违反哪一个条件, 即有一个分支去验证相应的条件是不满足的.

- 前两个条件容易判定.
- 要检验第3个条件, 需要将 C_i 入栈, 当读到 C_{i+1} 时与栈内的字符比较.

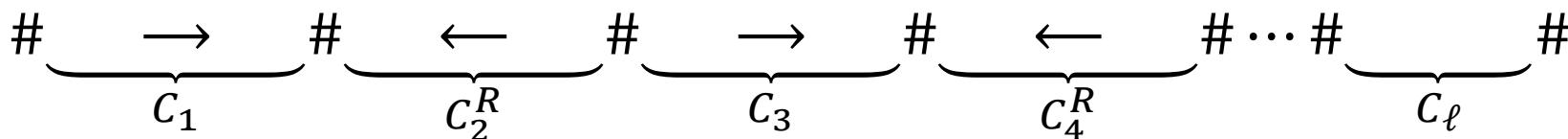
通过计算历史规约- ALL_{CFG}

证明(续):

- 要检验第3个条件, 需要将 C_i 入栈, 当读到 C_{i+1} 时与栈内的字符比较.

问题: C_i 字符出栈时的顺序和原来是相反的, 无法与 C_{i+1} 对应的字符比较.

解决: 将偶数位置上的字符串按相反的顺序排列. ■



通过计算历史规约- ALL_{CFG}

ALL_{CFG} 是不可判定的证明为什么对 E_{CFG} 无效?

3 映射规约

可计算函数

定义(可计算函数). 一个函数 $f: \Sigma^* \rightarrow \Sigma^*$ 是可计算的如果存在图灵机 M 使得对于每个输入字符串 w , M 对于 w 停机且停机时磁带上的字符串恰为 $f(w)$.

例 1: 通常的加, 减, 乘, 除运算都是可计算函数.

习题

计算 $f(x) = x + 1, x \in \mathbb{Z}_{\geq 0}$.

解：假定 x 以二进制表示.

$M =$ 对于输入字符串 x :

1. 若第1个字符为 $\sqcup$ (不是一个合法的输入), 则拒绝. 将输入右移一格(子程序)并在第1个格子上写下 $\sqcup$ (标记磁带最左端).
2. 向右扫描至输入最右端.
3. 向左扫描. 若当前字符为 0, 则改写为 1 后接受. 否则将当前字符改写为 0 并继续左移.
4. 若当前字符为 $\sqcup$ (回到了磁带最左端), 则改写为 1 (进位) 并接受.

可计算函数

例 2: 可计算函数可以是机器描述之间的变换.

如果 $w = \langle M \rangle$ 是图灵机的编码, 可以有一个可计算函数 f , 以 w 为输入且返回一个图灵机的描述 $\langle M' \rangle$: M' 是一个与 M 识别相同语言的机器, 但 M' 从不试图将它的读写头移出磁带的左端点.

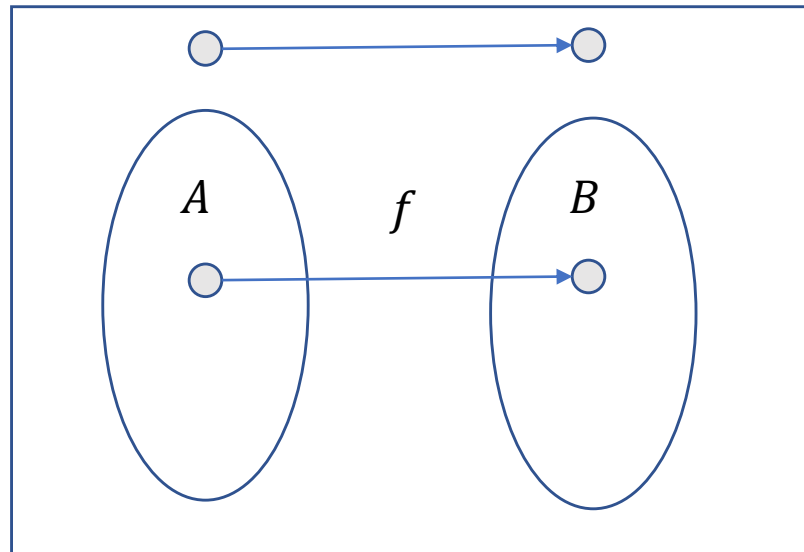
- 函数 f 通过在 M 的描述中加入一些状态来完成这个任务.
- 如果 w 不是图灵机的合法编码, f 就返回 ε .

映射规约

定义(映射规约). 语言 A 映射规约到 B , 记为 $A \leq_m B$, 如果存在可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$ 使得对任何 $w \in \Sigma^*$,

$$w \in A \Leftrightarrow f(w) \in B.$$

函数 f 称为从 A 到 B 的一个规约.



映射规约

例 1: $A_{\text{TM}} \leq_m \overline{E_{\text{TM}}}$.

解: 给定 $\langle M, w \rangle$, f 将 $\langle M, w \rangle$ 映射为 $\langle M_w \rangle$, 其中 M_w 是对于任何输入模拟 M 运行 w 的图灵机.

例 2: $E_{\text{TM}} \leq_m EQ_{\text{TM}}$.

解: 给定图灵机的描述 $\langle M \rangle$, f 将 $\langle M \rangle$ 映射为 $\langle M, M_{\text{reject}} \rangle$, 其中 M_{reject} 是拒绝所有输入的图灵机.

映射规约

例 3: $HALT_{TM}$ 是不可判定的证明不是映射规约.

解: 证明利用 $HALT_{TM}$ 的判定器去判定 A_{TM} , 但是并没有将 A_{TM} 的实例转化为 $HALT_{TM}$ 的实例.

映射规约

定理. 若 $A \leq_m B$ 且 B 是可判定的, 则 A 是可判定的.

证明. 令 M 是判定 B 的图灵机, f 是从 A 到 B 的映射规约.

下面是判定 A 的图灵机 N .

$N =$ “对于输入 w :

1. 计算 $f(w)$.
2. 在 M 上运行 $f(w)$. 输出 M 的回答.”

因为 f 是从 A 到 B 的映射规约, $w \in A \iff f(w) \in B$.

若 M 接受, 则 $w \in A$. 否则 M 拒绝, $w \notin A$. 故 N 判定 A . ■

推论. 若 $A \leq_m B$ 且 A 是不可判定的, 则 B 也不可判定.

映射规约

定理. 若 $A \leq_m B$ 且 B 是图灵可识别的, 则 A 是图灵可识别的.

推论. 若 $A \leq_m B$ 且 A 不是图灵可识别的, 则 B 也不是图灵可识别.

映射规约

若 $A \leq_m B$, 下面哪个是正确的?

- $B \leq_m A$

- $\bar{A} \leq_m \bar{B}$

映射规约

证明图灵不可识别的策略.

- 已知 $\overline{A_{\text{TM}}}$ 不是图灵可识别的.
- 若能证明 $\overline{A_{\text{TM}}} \leq_m B$, 则 B 不是图灵可识别的.
- 因为 $A \leq_m B \iff \bar{A} \leq_m \bar{B}$, 故只需证明 $A_{\text{TM}} \leq_m \bar{B}$.

映射规约

定理. EQ_{TM} 既不是图灵可识别的, 也不是补图灵可识别的.

证明. 首先证明 EQ_{TM} 不是图灵可识别的.

我们证明 $A_{TM} \leq_m \overline{EQ_{TM}}$. 下面给出这样的规约 f .

$F =$ “对于输入 $\langle M, w \rangle$:

1. 定义如下图灵机 M_1, M_2 .

$M_1 =$ “对于输入 x :

(1) 拒绝.”

$M_2 =$ “对于输入 x :

(1) 模拟 M 运行 w . 若 M 接受, 则接受.”

2. 输出 $\langle M_1, M_2 \rangle$.”

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow L(M_1) \neq L(M_2)$$

映射规约

定理. EQ_{TM} 既不是图灵可识别的, 也不是补图灵可识别的.

证明(续). 其次证明 $\overline{EQ_{TM}}$ 不是图灵可识别的.

我们证明 $A_{TM} \leq_m EQ_{TM}$.

$F =$ “对于输入 $\langle M, w \rangle$:

1. 定义如下图灵机 M_1, M_2 .

$M_1 =$ “对于输入 x :

(1) ~~拒绝.~~”

接受

$M_2 =$ “对于输入 x :

(1) 模拟 M 运行 w . 若 M 接受, 则接受.”

2. 输出 $\langle M_1, M_2 \rangle$.”

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow L(M_1) = L(M_2)$$



可规约性总结

可规约性总结

- 语言理论中的不可判定问题
 - $HALT_{TM}$
 - E_{TM} 和 EQ_{TM}
 - $REGULAR_{TM}$
- 通过计算历史规约
 - E_{LBA}
 - ALL_{CFG}
- 映射规约

计算理论-时间复杂度

- 时间复杂度介绍
- 大O符号
- P类
- NP类
- NP-完全问题
- 更多NP-完全问题

1 时间复杂度

渐进记号

定义 (O). 令 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$. 称 $f(n) = O(g(n))$ 如果存在正整数 c, n_0 使得对任何整数 $n \geq n_0$

$$f(n) \leq cg(n).$$

此时, 称 $g(n)$ 是 $f(n)$ 的一个渐进上界.

例. $f(n) = 5n^3 + 2n^2 + n + 1$.

- $f(n) = O(n^3)$.
- $f(n) \neq O(n^2)$.

渐进记号

定义 (o). 令 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$. 称 $f(n) = o(g(n))$ 如果

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

即对任何实数 $c > 0$, 存在 n_0 使得对任何 $n \geq n_0$

$$f(n) \leq cg(n).$$

例.

- $n^2 = o(n^3)$.
- $\log n = o(n)$.

渐进记号

练习.

- 证明 $p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 = O(n^k)$.
- 若 $f(n) = O(g(n))$, 是否有 $2^{f(n)} = O(2^{g(n)})$?

时间复杂度

定义(时间复杂度). 设 M 是对于所有输入都停机的确定性图灵机. M 的运行时间或时间复杂度是一个函数 $f: \mathbb{N} \rightarrow \mathbb{N}$, 其中 $f(n)$ 是对于输入规模为 n 的字符串的最大运行步数.

若 $f(n)$ 是 M 的运行时间, 则称 M 是 $f(n)$ 时间的图灵机.

时间复杂度

例. 识别 $A = \{0^n 1^n | n \geq 0\}$ 的图灵机.

M_1 = “输入 w :

1. 扫描磁带, 若在1的右边发现0, 则拒绝.

2. 若磁带上还有0和1, 重复下列步骤

扫描磁带, 划去一个0和一个1.

3. 若所有1划去后仍然还有0或者所有0划去后仍然有1, 则拒绝; 否则磁带上所有的0和1都被划去, 从而接受.”

时间复杂度

例. 识别 $A = \{0^n 1^n | n \geq 0\}$ 的图灵机.

时间复杂度

- 第1阶段从左向右扫描磁带直到输入末端需要 n 步, 返回磁带最左端也需要 n 步.
- 第2阶段是一个循环, 每一次循环扫描磁带划去一个0和1.
 - 每次划去2个字符, 因此循环执行的次数最多为 $n/2$.
 - 每次执行循环需要扫描磁带, 需要 $O(n)$ 步.
- 第3阶段需要扫描磁带一次, 需要 $O(n)$ 步.

总步数为 $O(n) + O(n^2) + O(n) = O(n^2)$.

时间复杂度

是否有更快的算法?

- 每次扫描只划去2个字符, 因此需要扫描 $n/2$ 次.
- 每次划去一半的字符, 则只需要 $\log n$ 次扫描.



$O(n \log n)$

M_2 = “输入 w :

1. 扫描磁带, 若在1的右边发现0, 则拒绝.
2. 若磁带上还有0和1, 重复下列步骤
3. 扫描磁带并计算剩余的0和1的总数. 若为奇数, 则拒绝.
4. 扫描磁带, 从第一个0开始每隔一个0, 划去一个0. 同样, 从第一个1开始每隔一个1, 划去一个1.
5. 若所有1划去后仍然还有0或者所有0划去后仍然有1, 则拒绝; 否则磁带上所有的0和1都被划去, 从而接受.”

时间复杂度

是否有更快的算法?

- No! 在单带图灵机模型下
- Yes! 在两带图灵机模型下.

$M_3 =$ “输入 w :

1. 扫描磁带, 若在1的右边发现0, 则拒绝.
2. 扫描磁带1, 直到第一个1. 同时, 将所有的0都复制到磁带2.
3. 扫描磁带1, 并划去当前的1. 对每个划去的1, 在磁带2上划去一个0. 若在所有的1划去前, 磁带2上的0都被划去了, 则拒绝.
4. 若还有0没有被划去, 则拒绝; 否则接受.”



$O(n)$

时间复杂度

- 可计算理论：所有的模型都是等价的.
- 复杂性理论：时间复杂度取决于所考虑的模型！

问题：不同计算模型之间的时间复杂度有什么联系？

不同模型之间时间复杂度的关系

定理(多带机). 令 $t(n) \geq n$ 是一个函数. 则每个 $t(n)$ 时间的多带机都等价于一个 $O(t^2(n))$ 时间的标准图灵机.

证明思路. 在第 3 章中, 我们证明了每个多带图灵机都等价于一个单带图灵机, 也就是可以用一个单带图灵机模拟多带图灵机的运行.

因此我们只需要去分析模拟多带图灵机的每一步需要花多少时间即可.

不同模型之间时间复杂度的关系

定理(多带机). 令 $t(n) \geq n$ 是一个函数. 则每个 $t(n)$ 时间的多带机都等价于一个 $O(t^2(n))$ 时间的标准图灵机.

证明. 令 M 是一个有 k (常数)个磁带的图灵机且运行时间为 $t(n)$. 我们构造一个单带图灵机 S , 其运行时间为 $O(t^2(n))$.

- $t(n) \geq n$ 是一个合理的假设, 因为读取输入至少需要 n 步.
- 我们证明模拟多带图灵机的每一步需要 $O(t(n))$ 步.



运行时间: $O(n) + t(n)O(t(n)) = O(t^2(n))$.

不同模型之间时间复杂度的关系

证明(续). 令 M 是一个有 k (常数) 个磁带的图灵机且运行时间为 $t(n)$. 我们将构造一个单带图灵机 S , 其运行时间为 $O(t^2(n))$.

回顾 S 是如何模拟 M 的.

- S 把 M 所有磁带上的内容都复制到自己的磁带上, 用加点的方式来记录 M 每个磁带磁头的位置.
- 每一步模拟, S 需要从左至右扫描自己的磁带两次:
 - 第1次扫描确定虚拟磁头的位置以及磁头所指方格的内容.
 - 第2次扫描按照 M 的转移方式更新方格内容并移动磁头. 若 M 的某个磁头移动到了空格的地方, 则 S 需要将其相应的磁带内容右移动一格.

时间分析.

- 模拟所需的时间正比于 S 的磁带上内容的长度.
- 因为 M 运行 $t(n)$ 步中止, 每一步最多向右移动一格, 故 M 每个磁带所使用的格子数至多为 $t(n)$. 从而 S 的磁带内容长度为 $O(t(n))$. ■

不同模型之间时间复杂度的关系

定义(非确定性图灵机的运行时间). 令 N 是一个非确定性图灵机且对所有输入都停机. 称 N 的运行时间为 $f(n)$, 如果对于输入规模为 n 的字符串, N 的任何计算分支在 $f(n)$ 步之后停机.

不同模型之间时间复杂度的关系

定理 (非确定性图灵机). 令 $t(n) \geq n$ 是一个函数. 则每个 $t(n)$ 时间的非确定性图灵机都等价于一个 $2^{O(t(n))}$ 时间的标准图灵机.

证明. 令 N 是一个非确定性图灵机, 运行时间为 $t(n)$. 定理 3.16 表明存在一个三带图灵机 D 与 N 等价. 下面分析 D 的运行时间.

- 假设根据 N 的转移规则, 每一步转移最多产生 b 个不同的可能. 则 N 的计算树 (nondeterministic computational tree) 中每个顶点的孩子个数至多为 b .
- 根据假设, N 的每个计算分支至多有 $t(n)$ 个顶点.
- 故 N 的计算树最多有 $b^{t(n)}$ 个叶子 (计算分支).
- 因为 N 的每个计算分支花费 $t(n)$ 时间, D 的运行时间为 $O(t(n)b^{t(n)}) = 2^{O(t(n))}$.
- 根据前一页的定理, D 有一个等价的单带图灵机 M , 其运行时间为 $(2^{O(t(n))})^2 = 2^{O(2t(n))} = 2^{O(t(n))}$. ■

2 P类

P类

定义(P类). **P**是标准图灵机在多项式时间内可判定的语言类. 换言之,

$$\mathbf{P} = \bigcup_k \text{TIME}(n^k),$$

其中 $\text{TIME}(t(n))$ 表示所有在 $O(t(n))$ 内可判定的语言集合.

为什么多项式时间?

- 所有“合理”的确定型计算模型都是多项式等价的, 即**P**在所有这些模型下不变. 故这个类的定义不依赖于所使用的计算模型. 比如多带图灵机和标准图灵机是等价的.
- 从实用的观点看, **P**中的问题“大致”对应实际中可以解决的问题. 比如排序可以在 $O(n \log n)$ 时间内解决.

P类

为什么多项式时间？

- $O(n^{100})$ 的算法当然不是一个实际可行的算法，但是在理论上称这种算法是“有效的”。
- 这种做法背后的哲学是：许多困难的问题目前已知的算法只有暴力搜索，而暴力搜索通常需要指数时间。如果想设计出比暴力搜索在渐进意义下更快速的算法，必须有非平凡的算法思想和洞察，从而促进对这些困难问题的理解。

如果能够给出一个计算图中最大团的 $O(n^{10^{10}})$ 的算法，那将是计算机中最重大的突破(这就证明了 $P = NP!$)。

约定

- 下面所有图灵机的描述都使用高层描述.
- 图的输入规模为图的顶点个数.

PATH

$PATH = \{ \langle G, s, t \rangle \mid G \text{ 是一个有向图且有一条从 } s \text{ 到 } t \text{ 的有向路} \}.$

定理. $PATH \in P.$

证明. 我们给出一个多项式时间算法.

First try: 暴力算法.

- 一条路径是一个顶点序列 $v_1, v_2, \dots, v_n$, 其中 n 是 G 的顶点个数.
- 对每个位置有 n 个选择, 故这样的序列个数为 n^n .

因此, 暴力搜索需要指数时间.

PATH

定理. $PATH \in P$.

证明. Second try: 从 s 出发依次标记到 s 距离为 $0, 1, \dots, n - 1$ 的顶点.

$M =$ “输入 $\langle G, s, t \rangle$:

1. 标记 s .
2. 重复下面步骤直到不再有顶点被标记.

扫描 G 的所有边. 如果找到边 (a, b) 使得 a 已标记而 b 还未标记, 则标记 b .

3. 若 t 被标记, 则接受; 否则拒绝.”

步骤 2 最多执行 n 次循环, 每次需要 $O(m)$ 时间. 所以用到的总步数最多是 $O(nm)$, 是 G 的规模的多项式. ■

RELPRIME

RELPRIME = $\{ \langle x, y \rangle \mid x \text{ 和 } y \text{ 互素的正整数} \}$.

定理. **RELPRIME** $\in$ P.

证明. 欧几里得的辗转相除法.

$x \bmod y$ 表示 x 除以 y 后的余数.

$E =$ “输入 $\langle x, y \rangle$, 其中 x, y 是以二进制表示的正整数:

1. 重复下面步骤直到 $y = 0$.
2. 令 $x \leftarrow x \bmod y$.
3. 交换 x 和 y .
4. 若 $x = 1$, 则接受; 否则拒绝.”

正确性: 基础数论.

RELPRIME

证明(续).

$E =$ “输入 $\langle x, y \rangle$, 其中 x, y 是以二进制表示的正整数:

1. 重复下面步骤直到 $y = 0$.
2. 令 $x \leftarrow x \bmod y$.
3. 交换 x 和 y .
4. 若 $x = 1$, 则接受; 否则拒绝.”

计算 $x \bmod y$ 是
多项式时间的!

时间分析.

- 除了第一次外, 每次执行步骤 2, x 的值至少减半.
- 每执行步骤 2 两次后, x 和 y 的值均减少至少一半.

输入规模的
多项式!

步骤 2 和 步骤 3 执行的次数为 $\min\{2\log x, 2\log y\}$. ■

RELPRIME

如何修改上述算法给出 x, y 的最大公约数?

3 NP类

NP

- 许多问题存在多项式时间算法.
- 但仍有成千上万的问题还不知道是否有多项式时间算法.

或许我们还没有发现巧妙的思路

或许这些问题根本不存在多项式时间算法

- 让人沮丧的是到目前为止无法回答到底这些问题是否存在多项式时间算法. 😞
- 但是, 我们可以证明这些问题都是多项式时间等价的.

多项式验证器

$HAMPATH = \{ \langle G, s, t \rangle \mid G \text{ 是一个有向图} \\ \text{且有一条从 } s \text{ 到 } t \text{ 的哈密尔顿路} \}.$

- 暴力搜索需要指数时间.
- 不知道是否有多项式时间算法.
- 虽然不知道如何判定 G 是否存在一条从 s 到 t 的哈密尔顿路, 但是如果已经发现了这样一条路径, 我们很容易说服别人 G 有一条哈密尔顿路: 只需要展示已经发现的这条路径即可.

多项式验证器

$$COMPOSITE = \{x \in \mathbb{N} | x = pq, p, q > 1\}.$$

- 暴力搜索需要指数时间.
 - 容易验证答案：验证 pq 等于 x .
 - 在2002年，该问题被证明属于P.
- **2006 Gödel Prize**: The result obtained by Agrawal, Kayal, and Saxena can be seen as a crowning achievement of a long algorithmic and mathematical quest.

多项式验证器

定义(多项式验证器). 语言 A 的验证器是一个算法 V 使得

$$A = \{w \mid \text{对某个字符串 } c, V \text{ 接受 } \langle w, c \rangle\}.$$

一个多项式时间验证器是指运行时间是 $|w|$ 的多项式的验证器.

一个语言 A 是多项式可验证的, 如果它有一个多项式时间验证器.

理解.

- 验证器利用额外的信息(字符串 c)来验证字符串 w 是 A 的成员.

该信息称为 A 的成员资格证书(certificate)或证据(proof).

- 若 V 是多项式验证器, 则 $|c| \leq p(|w|)$, 其中 p 是一个多项式.

多项式验证器

例 1. 下面是 $HAMPATH$ 的一个多项式验证器.

$\langle G, s, t \rangle \in HAMPATH$ 的证据是一条从 s 到 t 的哈密尔顿路.

$V =$ “输入 $\langle w, c \rangle$:

1. 检查 c 是否代表一个有 n 个顶点的顶点序列 $p_1, \dots, p_n$. 若不是, 则拒绝.
2. 若 $p_1, \dots, p_n$ 中有重复的数字, 拒绝.
3. 检查是否有 $s = p_1, t = p_n$. 若不是, 则拒绝.
4. 对所有 $1 \leq i \leq n - 1$, 检查是否有某个 (p_i, p_{i+1}) 不是边. 若有, 则拒绝, 否则接受.”

多项式验证器

例 2. 下面是 $COMPOSITE$ 的一个多项式验证器.

$x \in COMPOSITE$ 的证据是 x 的非平凡因子 p, q .

$V =$ “输入 $\langle x, c \rangle$:

1. 检查 c 是否代表两个正整数 $p, q \leq x$. 若不是, 则拒绝.
2. 若 $x \neq pq$, 拒绝. 否则接受.”

多项式验证器

定义(团). 图 G 的一个团是一个子图, 其中该子图中的任何两个顶点都相邻. 包含 k 个顶点的团称为 k -团.

练习. 给出

$$CLIQUE = \{ \langle G, k \rangle \mid G \text{ 是有一个 } k - \text{ 团的图} \}$$

的一个多项式验证器.

NP类

定义(NP). NP是具有多项式时间验证器的语言类.

例. *HAMPATH*, *COMPOSITE*, *CLIQUE*都属于NP.

练习. 给出一个属于NP的问题.

NP类

定义(NP). NP是具有多项式时间验证器的语言类.

NP是非确定性多项式时间 (nondeterministic polynomial time) 的缩写. 在NP中的问题通常被称为NP-问题.

下面证明NP可以等价地用非确定性图灵机来定义.

NP类

$HAMPATH = \{ \langle G, s, t \rangle \mid G \text{ 是一个有向图} \\ \text{且有一条从 } s \text{ 到 } t \text{ 的哈密尔顿路} \}.$

判定 $HAMPATH$ 的非确定性图灵机.

$N_1 =$ “输入 $\langle G, s, t \rangle$, 其中 G 是一个有向图, $s, t \in V(G)$:

1. 写下数字 $p_1, \dots, p_n$, 其中 $n = |V(G)|$. 每个数字都是随机的从 $[n]$ 中选取的.
2. 记 $c = p_1, \dots, p_n$. 调用验证器 $V \langle G, s, t, c \rangle$, 输出 V 的答案.”

N_1 的每一步都能够在多项式时间内完成.

NP类

定理(NP的等价刻画). 一个语言属于NP当且仅当存在某个多项式时间地非确定性图灵机判定它.

证明思路.

- 非确定性图灵机通过“猜测”证据来模拟多项式验证器.
- 多项式验证器使用接受计算分支作为证据.

NP类

定理(等价刻画). 一个语言属于NP当且仅当存在某个多项式时间地非确定性图灵机判定它.

证明.

必要性. 假设 $A \in \text{NP}$. 令 V 是 A 的一个多项式时间验证器并假设对于输入长度为 k 的字符串, V 在 n^k 步后中止.

构造如下的非确定性图灵机 N :

$N =$ “输入长度为 n 的字符串 w :

1. 非确定性的“猜测”一个长不超过 n^k 的字符串 c .
2. 对 $\langle w, c \rangle$ 运行 V .
3. 若 V 接受, 接受; 否则拒绝.”

NP类

定理(等价刻画). 一个语言属于NP当且仅当存在某个多项式时间地非确定性图灵机判定它.

证明(续).

充分性. 假设 A 被一个非确定性图灵机 N 所判定.

构造如下的多项式时间验证器 V :

$V =$ “输入 $\langle w, c \rangle$:

1. 模拟 N 运行 w , 把 c 的每个字符看作是对每一步非确定性选择的描述.
2. 若 N 的该计算分支接受, 则接受; 否则拒绝.”

类似于多带机模拟NTM中的地址带

练习. 补充证明细节.

NP类

定义.

$\text{NTIME}(t(n)) = \{L \mid L \text{ 能够被一个 } O(t(n)) \text{ 时间的非确定性图灵机判定}\}.$

推论.

$$\mathbf{NP} = \bigcup_k \mathbf{NTIME}(n^k).$$

NP类

定理(团). $CLIQUE \in NP$.

证明. 多项式验证器.

$V =$ “输入 $\langle G, k \rangle, c \rangle$:

1. 检查 c 是否是 G 的大小为 k 的子集.
2. 检查 c 中的任何两个顶点是否相邻.
3. 若上面两步都通过, 则接受; 否则拒绝.”

非确定性图灵机.

$N =$ “输入 $\langle G, k \rangle$:

1. 非确定性选择大小为 k 的顶点子集 c .
 2. 检查 c 中的任何两个顶点是否相邻. 若回答是肯定的, 则接受; 否则拒绝.”
- 

NP类

子集和问题 (*SUBSET – SUM*)

输入：一组正整数 $S = \{w_1, w_2, \dots, w_n\}$ 和一个正整数 W .

问题：是否存在 S 的子集 S' 使得 $\sum_{w \in S'} w = W$?

SUBSET – SUM

$= \{ \langle S, W \rangle \mid S = \{w_1, w_2, \dots, w_n\} \text{ 且存在 } S' \subseteq S \text{ 使得 } \sum_{w \in S'} w = W \}.$

例. $\langle \{4, 11, 16, 21, 27\}, 25 \rangle \in \text{SUBSET – SUM}.$

NP类

定理. $SUBSET - SUM \in NP$.

证明. 多项式验证器.

$V =$ “输入 $\langle \langle S, W \rangle, c \rangle$:

1. 检查 S 是否包含 c 中所有的数字. 若不是, 则拒绝.
2. 检查 c 中的所有数字的和是否为 W . 若是则接受; 否则拒绝.”

非确定性图灵机.

$N =$ “输入 $\langle S, W \rangle$:

1. 非确定性选择 S 的子集 c .
2. 若 c 中的所有数字的和为 W , 则接受; 否则拒绝.”



NP类

练习. 证明 $VERTEX - COVER \in NP$.

定义(顶点覆盖). 图 G 的一个顶点覆盖是一个顶点子集 $S \subseteq V(G)$ 使得 G 的任何一条边的至少一个端点在 S 中.

$VERTEX - COVER =$

$\{ \langle G, k \rangle \mid G \text{ 是一个图且有一个大小不超过 } k \text{ 的点覆盖} \}.$

P vs NP

- P是所有可以在多项式时间内判定的语言类.
- NP是所有正确答案可以在多项式时间内验证的语言类.

定理. $P \subseteq NP$.

证明. 设 $A \in P$. 则 A 被一个多项式时间标准图灵机 M 所判定. 构造 A 的多项式验证器如下:

$V =$ “输入 $\langle w, c \rangle$:

1. 忽略 c , 在 M 上运行 w .
2. 若 M 接受, 则接受; 否则拒绝.”

若 $w \notin A$, 则对任何 c , V 拒绝 $\langle w, c \rangle$, 故 V 拒绝 w .

若 $w \in A$, 则对任何 c , V 接受 $\langle w, c \rangle$, 故 V 接受 w . ■

P vs NP

问题： $P = NP$?

即每个正确答案能被快速验证的问题是否也能快速求解?

直觉告诉我们答案应该是否定的.

猜想： $P \neq NP$.

- 计算机领域最重要的难题，被美国凯莱数学会列为千禧年七大数学难题之一.
- 奖金 \$1,000,000.

计算理论-时间复杂度

- 时间复杂度介绍
- O 表示法
- P类
- NP类
- NP-完全问题
- 更多NP-完全问题

1 时间复杂度

渐进记号

定义 (O). 令 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$. 称 $f(n) = O(g(n))$ 如果存在正整数 c, n_0 使得对任何整数 $n \geq n_0$

$$f(n) \leq cg(n).$$

此时, 称 $g(n)$ 是 $f(n)$ 的一个渐进上界.

例. $f(n) = 5n^3 + 2n^2 + n + 1$.

- $f(n) = O(n^3)$.
- $f(n) \neq O(n^2)$.

渐进记号

定义 (o). 令 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$. 称 $f(n) = o(g(n))$ 如果

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

即对任何实数 $c > 0$, 存在 n_0 使得对任何 $n \geq n_0$

$$f(n) \leq cg(n).$$

例.

- $n^2 = o(n^3)$.
- $\log n = o(n)$.

渐进记号

练习.

- 证明 $p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 = O(n^k)$.
- 若 $f(n) = O(g(n))$, 是否有 $2^{f(n)} = O(2^{g(n)})$?

时间复杂度

定义(时间复杂度). 设 M 是对于所有输入都停机的确定性图灵机. M 的运行时间或时间复杂度是一个函数 $f: \mathbb{N} \rightarrow \mathbb{N}$, 其中 $f(n)$ 是对于输入规模为 n 的字符串的最大运行步数.

若 $f(n)$ 是 M 的运行时间, 则称 M 是 $f(n)$ 时间的图灵机.

时间复杂度

例. 识别 $A = \{0^n 1^n | n \geq 0\}$ 的图灵机.

M_1 = “输入 w :

1. 扫描磁带, 若在1的右边发现0, 则拒绝.
2. 若磁带上还有0和1, 重复下列步骤
扫描磁带, 划去一个0和一个1.
3. 若所有1划去后仍然还有0或者所有0划去后仍然有1, 则拒绝; 否则磁带上所有的0和1都被划去, 从而接受.”

时间复杂度

例. 识别 $A = \{0^n 1^n | n \geq 0\}$ 的图灵机.

时间复杂度

- 第1阶段从左向右扫描磁带直到输入末端需要 n 步, 返回磁带最左端也需要 n 步.
- 第2阶段是一个循环, 每一次循环扫描磁带划去一个0和1.
 - 每次划去2个字符, 因此循环执行的次数最多为 $n/2$.
 - 每次执行循环需要扫描磁带, 需要 $O(n)$ 步.
- 第3阶段需要扫描磁带一次, 需要 $O(n)$ 步.

总步数为 $O(n) + O(n^2) + O(n) = O(n^2)$.

时间复杂度

是否有更快的算法?

- 每次扫描只划去2个字符, 因此需要扫描 $n/2$ 次.
- 每次划去一半的字符, 则只需要 $\log n$ 次扫描.



$O(n \log n)$

M_2 = “输入 w :

1. 扫描磁带, 若在1的右边发现0, 则拒绝.
2. 若磁带上还有0和1, 重复下列步骤
3. 扫描磁带并计算剩余的0和1的总数. 若为奇数, 则拒绝.
4. 扫描磁带, 从第一个0开始每隔一个0, 划去一个0. 同样, 从第一个1开始每隔一个1, 划去一个1.
5. 若所有1划去后仍然还有0或者所有0划去后仍然有1, 则拒绝; 否则磁带上所有的0和1都被划去, 从而接受.”

时间复杂度

是否有更快的算法?

- No! 在单带图灵机模型下
- Yes! 在两带图灵机模型下.

$M_3 =$ “输入 w :

1. 扫描磁带, 若在1的右边发现0, 则拒绝.
2. 扫描磁带1, 直到第一个1. 同时, 将所有的0都复制到磁带2.
3. 扫描磁带1, 并划去当前的1. 对每个划去的1, 在磁带2上划去一个0. 若在所有的1划去前, 磁带2上的0都被划去了, 则拒绝.
4. 若还有0没有被划去, 则拒绝; 否则接受.”



$O(n)$

时间复杂度

- 可计算理论：所有的模型都是等价的.
- 复杂性理论：时间复杂度取决于所考虑的模型！

问题：不同计算模型之间的时间复杂度有什么联系？

不同模型之间时间复杂度的关系

定理(多带机). 令 $t(n) \geq n$ 是一个函数. 则每个 $t(n)$ 时间的多带机都等价于一个 $O(t^2(n))$ 时间的标准图灵机.

证明思路. 在第 3 章中, 我们证明了每个多带图灵机都等价于一个单带图灵机, 也就是可以用一个单带图灵机模拟多带图灵机的运行.

因此我们只需要去分析模拟多带图灵机的每一步需要花多少时间即可.

不同模型之间时间复杂度的关系

定理(多带机). 令 $t(n) \geq n$ 是一个函数. 则每个 $t(n)$ 时间的多带机都等价于一个 $O(t^2(n))$ 时间的标准图灵机.

证明. 令 M 是一个有 k (常数)个磁带的图灵机且运行时间为 $t(n)$. 我们构造一个单带图灵机 S , 其运行时间为 $O(t^2(n))$.

- $t(n) \geq n$ 是一个合理的假设, 因为读取输入至少需要 n 步.
- 我们证明模拟多带图灵机的每一步需要 $O(t(n))$ 步.



运行时间: $O(n) + t(n)O(t(n)) = O(t^2(n))$.

不同模型之间时间复杂度的关系

证明(续). 令 M 是一个有 k (常数) 个磁带的图灵机且运行时间为 $t(n)$. 我们将构造一个单带图灵机 S , 其运行时间为 $O(t^2(n))$.

回顾 S 是如何模拟 M 的.

- S 把 M 所有磁带上的内容都复制到自己的磁带上, 用加点的方式来记录 M 每个磁带磁头的位置.
- 每一步模拟, S 需要从左至右扫描自己的磁带两次:
 - 第1次扫描确定虚拟磁头的位置以及磁头所指方格的内容.
 - 第2次扫描按照 M 的转移方式更新方格内容并移动磁头. 若 M 的某个磁头移动到了空格的地方, 则 S 需要将其相应的磁带内容右移动一格.

时间分析.

- 模拟所需的时间正比于 S 的磁带上内容的长度.
- 因为 M 运行 $t(n)$ 步中止, 每一步最多向右移动一格, 故 M 每个磁带所使用的格子数至多为 $t(n)$. 从而 S 的磁带内容长度为 $O(t(n))$. ■

不同模型之间时间复杂度的关系

定义(非确定性图灵机的运行时间). 令 N 是一个非确定性图灵机且对所有输入都停机. 称 N 的运行时间为 $f(n)$, 如果对于输入规模为 n 的字符串, N 的任何计算分支在 $f(n)$ 步之后停机.

不同模型之间时间复杂度的关系

定理 (非确定性图灵机). 令 $t(n) \geq n$ 是一个函数. 则每个 $t(n)$ 时间的非确定性图灵机都等价于一个 $2^{O(t(n))}$ 时间的标准图灵机.

证明. 令 N 是一个非确定性图灵机, 运行时间为 $t(n)$. 定理 3.16 表明存在一个三带图灵机 D 与 N 等价. 下面分析 D 的运行时间.

- 假设根据 N 的转移规则, 每一步转移最多产生 b 个不同的可能. 则 N 的计算树 (nondeterministic computational tree) 中每个顶点的孩子个数至多为 b .
- 根据假设, N 的每个计算分支至多有 $t(n)$ 个顶点.
- 故 N 的计算树最多有 $b^{t(n)}$ 个叶子 (计算分支).
- 因为 N 的每个计算分支花费 $t(n)$ 时间, D 的运行时间为 $O(t(n)b^{t(n)}) = 2^{O(t(n))}$.
- 根据前一页的定理, D 有一个等价的单带图灵机 M , 其运行时间为 $(2^{O(t(n))})^2 = 2^{O(2t(n))} = 2^{O(t(n))}$. ■

2 P类

P类

定义(P类). **P**是标准图灵机在多项式时间内可判定的语言类. 换言之,

$$\mathbf{P} = \bigcup_k \text{TIME}(n^k),$$

其中 $\text{TIME}(t(n))$ 表示所有在 $O(t(n))$ 内可判定的语言集合.

为什么多项式时间?

- 所有“合理”的确定型计算模型都是多项式等价的, 即**P**在所有这些模型下不变. 故这个类的定义不依赖于所使用的计算模型. 比如多带图灵机和标准图灵机是等价的.
- 从实用的观点看, **P**中的问题“大致”对应实际中可以解决的问题. 比如排序可以在 $O(n \log n)$ 时间内解决.

P类

为什么多项式时间？

- $O(n^{100})$ 的算法当然不是一个实际可行的算法，但是在理论上称这种算法是“有效的”。
- 这种做法背后的哲学是：许多困难的问题目前已知的算法只有暴力搜索，而暴力搜索通常需要指数时间。如果想设计出比暴力搜索在渐进意义下更快速的算法，必须有非平凡的算法思想和洞察，从而促进对这些困难问题的理解。

如果能够给出一个计算图中最大团的 $O(n^{10^{10}})$ 的算法，那将是计算机中最重大的突破(这就证明了 $P = NP!$)。

约定

- 下面所有图灵机的描述都使用高层描述.
- 图的输入规模为图的顶点个数.

PATH

$PATH = \{ \langle G, s, t \rangle \mid G \text{ 是一个有向图且有一条从 } s \text{ 到 } t \text{ 的有向路} \}.$

定理. $PATH \in P.$

证明. 我们给出一个多项式时间算法.

First try: 暴力算法.

- 一条路径是一个顶点序列 $v_1, v_2, \dots, v_n$, 其中 n 是 G 的顶点个数.
- 对每个位置有 n 个选择, 故这样的序列个数为 n^n .

因此, 暴力搜索需要指数时间.

PATH

定理. $PATH \in P$.

证明. Second try: 从 s 出发依次标记到 s 距离为 $0, 1, \dots, n - 1$ 的顶点.

$M =$ “输入 $\langle G, s, t \rangle$:

1. 标记 s .
2. 重复下面步骤直到不再有顶点被标记.

扫描 G 的所有边. 如果找到边 (a, b) 使得 a 已标记而 b 还未标记, 则标记 b .

3. 若 t 被标记, 则接受; 否则拒绝.”

步骤 2 最多执行 n 次循环, 每次需要 $O(m)$ 时间. 所以用到的总步数最多是 $O(nm)$, 是 G 的规模的多项式. ■

RELPRIME

RELPRIME = $\{ \langle x, y \rangle \mid x \text{ 和 } y \text{ 互素的正整数} \}$.

定理. **RELPRIME** $\in$ P.

证明. 欧几里得的辗转相除法.

$x \bmod y$ 表示 x 除以 y 后的余数.

$E =$ “输入 $\langle x, y \rangle$, 其中 x, y 是以二进制表示的正整数:

1. 重复下面步骤直到 $y = 0$.
2. 令 $x \leftarrow x \bmod y$.
3. 交换 x 和 y .
4. 若 $x = 1$, 则接受; 否则拒绝.”

正确性: 基础数论.

RELPRIME

证明(续).

$E =$ “输入 $\langle x, y \rangle$, 其中 x, y 是以二进制表示的正整数:

1. 重复下面步骤直到 $y = 0$.
2. 令 $x \leftarrow x \bmod y$.
3. 交换 x 和 y .
4. 若 $x = 1$, 则接受; 否则拒绝.”

计算 $x \bmod y$ 是
多项式时间的!

时间分析.

- 除了第一次外, 每次执行步骤 2, x 的值至少减半.
- 每执行步骤 2 两次后, x 和 y 的值均减少至少一半.

输入规模的
多项式!

步骤 2 和 步骤 3 执行的次数为 $\min\{2\log x, 2\log y\}$. ■

RELPRIME

如何修改上述算法给出 x, y 的最大公约数?

3 NP类

NP

- 许多问题存在多项式时间算法.
- 但仍有成千上万的问题还不知道是否有多项式时间算法.

或许我们还没有发现巧妙的思路

或许这些问题根本不存在多项式时间算法

- 让人沮丧的是到目前为止无法回答到底这些问题是否存在多项式时间算法. 😞
- 但是, 我们可以证明这些问题都是多项式时间等价的.

多项式验证器

$HAMPATH = \{ \langle G, s, t \rangle \mid G \text{ 是一个有向图} \\ \text{且有一条从 } s \text{ 到 } t \text{ 的哈密尔顿路} \}.$

- 暴力搜索需要指数时间.
- 不知道是否有多项式时间算法.
- 虽然不知道如何判定 G 是否存在一条从 s 到 t 的哈密尔顿路, 但是如果已经发现了这样一条路径, 我们很容易说服别人 G 有一条哈密尔顿路: 只需要展示已经发现的这条路径即可.

多项式验证器

$$COMPOSITE = \{x \in \mathbb{N} | x = pq, p, q > 1\}.$$

- 暴力搜索需要指数时间.
 - 容易验证答案：验证 pq 等于 x .
 - 在2002年，该问题被证明属于P.
- **2006 Gödel Prize**: The result obtained by Agrawal, Kayal, and Saxena can be seen as a crowning achievement of a long algorithmic and mathematical quest.

多项式验证器

定义(多项式验证器). 语言 A 的验证器是一个算法 V 使得

$$A = \{w \mid \text{对某个字符串 } c, V \text{ 接受 } \langle w, c \rangle\}.$$

一个多项式时间验证器是指运行时间是 $|w|$ 的多项式的验证器.

一个语言 A 是多项式可验证的, 如果它有一个多项式时间验证器.

理解.

- 验证器利用额外的信息(字符串 c)来验证字符串 w 是 A 的成员.

该信息称为 A 的成员资格证书(certificate)或证据(proof).

- 若 V 是多项式验证器, 则 $|c| \leq p(|w|)$, 其中 p 是一个多项式.

多项式验证器

例 1. 下面是 *HAMPATH* 的一个多项式验证器.

$\langle G, s, t \rangle \in \text{HAMPATH}$ 的证据是一条从 s 到 t 的哈密尔顿路.

$V =$ “输入 $\langle w, c \rangle$:

1. 检查 c 是否代表一个有 n 个顶点的顶点序列 $p_1, \dots, p_n$. 若不是, 则拒绝.
2. 若 $p_1, \dots, p_n$ 中有重复的数字, 拒绝.
3. 检查是否有 $s = p_1, t = p_n$. 若不是, 则拒绝.
4. 对所有 $1 \leq i \leq n - 1$, 检查是否有某个 (p_i, p_{i+1}) 不是边. 若有, 则拒绝, 否则接受.”

多项式验证器

例 2. 下面是 $COMPOSITE$ 的一个多项式验证器.

$x \in COMPOSITE$ 的证据是 x 的非平凡因子 p, q .

$V =$ “输入 $\langle x, c \rangle$:

1. 检查 c 是否代表两个正整数 $p, q \leq x$. 若不是, 则拒绝.
2. 若 $x \neq pq$, 拒绝. 否则接受.”

多项式验证器

定义(团). 图 G 的一个团是一个子图, 其中该子图中的任何两个顶点都相邻. 包含 k 个顶点的团称为 k -团.

练习. 给出

$$CLIQUE = \{ \langle G, k \rangle \mid G \text{ 是有一个 } k - \text{ 团的图} \}$$

的一个多项式验证器.

NP类

定义(NP). NP是具有多项式时间验证器的语言类.

例. *HAMPATH*, *COMPOSITE*, *CLIQUE*都属于NP.

练习. 给出一个属于NP的问题.

NP类

定义(NP). NP是具有多项式时间验证器的语言类.

NP是非确定性多项式时间 (nondeterministic polynomial time) 的缩写. 在NP中的问题通常被称为NP-问题.

下面证明NP可以等价地用非确定性图灵机来定义.

NP类

$HAMPATH = \{ \langle G, s, t \rangle \mid G \text{ 是一个有向图} \\ \text{且有一条从 } s \text{ 到 } t \text{ 的哈密尔顿路} \}.$

判定 $HAMPATH$ 的非确定性图灵机.

$N_1 =$ “输入 $\langle G, s, t \rangle$, 其中 G 是一个有向图, $s, t \in V(G)$:

1. 写下数字 $p_1, \dots, p_n$, 其中 $n = |V(G)|$. 每个数字都是随机的从 $[n]$ 中选取的.
2. 记 $c = p_1, \dots, p_n$. 调用验证器 $V \langle G, s, t, c \rangle$, 输出 V 的答案.”

N_1 的每一步都能够能够在多项式时间内完成.

NP类

定理(NP的等价刻画). 一个语言属于NP当且仅当存在某个多项式时间地非确定性图灵机判定它.

证明思路.

- 非确定性图灵机通过“猜测”证据来模拟多项式验证器.
- 多项式验证器使用接受计算分支作为证据.

NP类

定理(等价刻画). 一个语言属于NP当且仅当存在某个多项式时间地非确定性图灵机判定它.

证明.

必要性. 假设 $A \in \text{NP}$. 令 V 是 A 的一个多项式时间验证器并假设对于输入长度为 k 的字符串, V 在 n^k 步后中止.

构造如下的非确定性图灵机 N :

$N =$ “输入长度为 n 的字符串 w :

1. 非确定性的“猜测”一个长不超过 n^k 的字符串 c .
2. 对 $\langle w, c \rangle$ 运行 V .
3. 若 V 接受, 接受; 否则拒绝.”

NP类

定理(等价刻画). 一个语言属于NP当且仅当存在某个多项式时间地非确定性图灵机判定它.

证明(续).

充分性. 假设 A 被一个非确定性图灵机 N 所判定.

构造如下的多项式时间验证器 V :

$V =$ “输入 $\langle w, c \rangle$:

1. 模拟 N 运行 w , 把 c 的每个字符看作是对每一步非确定性选择的描述.
2. 若 N 的该计算分支接受, 则接受; 否则拒绝.”

类似于多带机模拟NTM中的地址带

练习. 补充证明细节.

NP类

定义.

$\text{NTIME}(t(n)) = \{L \mid L \text{ 能够被一个 } O(t(n)) \text{ 时间的非确定性图灵机判定}\}.$

推论.

$$\mathbf{NP} = \bigcup_k \mathbf{NTIME}(n^k).$$

NP类

定理(团). $CLIQUE \in NP$.

证明. 多项式验证器.

$V =$ “输入 $\langle \langle G, k \rangle, c \rangle$:

1. 检查 c 是否是 G 的大小为 k 的子集.
2. 检查 c 中的任何两个顶点是否相邻.
3. 若上面两步都通过, 则接受; 否则拒绝.”

非确定性图灵机.

$N =$ “输入 $\langle G, k \rangle$:

1. 非确定性选择大小为 k 的顶点子集 c .
 2. 检查 c 中的任何两个顶点是否相邻. 若回答是肯定的, 则接受; 否则拒绝.”
- 

NP类

子集和问题 (*SUBSET – SUM*)

输入：一组正整数 $S = \{w_1, w_2, \dots, w_n\}$ 和一个正整数 W .

问题：是否存在 S 的子集 S' 使得 $\sum_{w \in S'} w = W$?

SUBSET – SUM

$= \{ \langle S, W \rangle \mid S = \{w_1, w_2, \dots, w_n\} \text{ 且存在 } S' \subseteq S \text{ 使得 } \sum_{w \in S'} w = W \}.$

例. $\langle \{4, 11, 16, 21, 27\}, 25 \rangle \in \text{SUBSET – SUM}.$

NP类

定理. $SUBSET - SUM \in NP$.

证明. 多项式验证器.

$V =$ “输入 $\langle \langle S, W \rangle, c \rangle$:

1. 检查 S 是否包含 c 中所有的数字. 若不是, 则拒绝.
2. 检查 c 中的所有数字的和是否为 W . 若是则接受; 否则拒绝.”

非确定性图灵机.

$N =$ “输入 $\langle S, W \rangle$:

1. 非确定性选择 S 的子集 c .
2. 若 c 中的所有数字的和为 W , 则接受; 否则拒绝.”



NP类

练习. 证明 $VERTEX - COVER \in NP$.

定义(顶点覆盖). 图 G 的一个顶点覆盖是一个顶点子集 $S \subseteq V(G)$ 使得 G 的任何一条边的至少一个端点在 S 中.

$VERTEX - COVER =$

$\{ \langle G, k \rangle \mid G \text{ 是一个图且有一个大小不超过 } k \text{ 的点覆盖} \}.$

P vs NP

- P是所有可以在多项式时间内判定的语言类.
- NP是所有正确答案可以在多项式时间内验证的语言类.

定理. $P \subseteq NP$.

证明. 设 $A \in P$. 则 A 被一个多项式时间标准图灵机 M 所判定. 构造 A 的多项式验证器如下:

$V =$ “输入 $\langle w, c \rangle$:

1. 忽略 c , 在 M 上运行 w .
2. 若 M 接受, 则接受; 否则拒绝.”

若 $w \notin A$, 则对任何 c , V 拒绝 $\langle w, c \rangle$, 故 V 拒绝 w .

若 $w \in A$, 则对任何 c , V 接受 $\langle w, c \rangle$, 故 V 接受 w . ■

P vs NP

问题： $P = NP$?

即每个正确答案能被快速验证的问题是否也能快速求解?

直觉告诉我们答案应该是否定的.

猜想： **$P \neq NP$** .

- 计算机领域最重要的难题，被美国凯莱数学会列为千禧年七大数学难题之一.
- 奖金 \$1,000,000.

4 NP-完全问题

多项式时间规约

定义. 一个函数 $f: \Sigma^* \rightarrow \Sigma^*$ 是多项式时间可计算的, 如果存在多项式时间图灵机 M 使得对任何输入 w , M 输出 $f(w)$.

定义(多项式时间规约). 语言 A 是多项式时间规约到语言 B , 如果存在多项式时间可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$ 使得对任何 w ,

$$w \in A \Leftrightarrow f(w) \in B.$$

函数 f 称为从 A 到 B 的多项式时间规约.

记号: $A \leq_p B$.

多项式时间规约

定理. 若 $A \leq_p B$ 且 $B \in P$, 则 $A \in P$.

证明. 令 M 是判定 B 的多项式时间图灵机, f 是 A 到 B 的多项式时间规约. 下面给出一个判定 A 的多项式时间图灵机.

$N =$ “输入 w :

1. 计算 $f(w)$.
2. 在 M 上运行 $f(w)$. 若 M 接受, 则接受; 否则拒绝.”

时间复杂度.

- $f(w)$ 可在 $|w|$ 的多项式时间内计算.
- 而 M 运行 $f(w)$ 是关于 $|f(w)|$ 的多项式.



多项式时间规约

定理. 若 $A \leq_p B$ 且 $B \in P$, 则 $A \in P$.

直观理解. 若 $A \leq_p B$, B 比 A 困难: 求解 B 的算法足以求解 A .

推论. 若 $A \leq_p B$ 且 $A \notin P$, 则 $B \notin P$.

NP-完全问题

定义(NP-完全问题). 语言 B 是NP-完全的, 如果

- $B \in \text{NP}$.
- 每一个NP-问题 A 都能多项式时间规约到 B .

理解. 定义中的第二条是说 B 比NP中的任何问题都困难, 所以NP-完全问题是NP中“最困难”的问题.

定理. 若 B 是NP-完全问题且 $B \in \text{P}$, 则 $\text{P} = \text{NP}$.

证明. 多项式时间规约的性质. 

NP-完全问题

是否存在NP-完全问题？

Cook-Levin定理. ***SAT***是NP-完全的.

SAT

- 布尔变量是取值要么为1 (TRUE) 要么为0 (FALSE) 的变量.
- 一个布尔公式是涉及布尔变量和布尔运算符 $\vee, \wedge, \neg$ 的表达式.

$$\phi = (\bar{x} \wedge y) \vee (x \wedge \bar{z})$$

- 一个布尔公式 ϕ 是可适定的 (satisfiable), 如果存在一组对其变量的赋值 (真值分配) 使得该公式的值为1.

$$x = 0, y = 1, z = 0 \text{ 适定 } \phi$$

- 可适定性问题.

$$SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可适定的布尔公式} \}.$$

CNF – SAT

- 变量 x 或它的否定 $\bar{x}$ 称为文字 (literal).
- 若干文字的析取称为一个句子 (clause).

$$(x_1 \vee \bar{x}_2 \vee \bar{x}_3 \vee x_4)$$

- 若一个布尔公式是若干句子的合取, 则该公式被称为是合取范式 (conjunction normal form).

$$(x_1 \vee \bar{x}_2 \vee \bar{x}_3 \vee x_4) \wedge (x_3 \vee \bar{x}_5 \vee x_6) \wedge (x_3 \vee \bar{x}_6)$$

- $CNF - SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可适定的合取范式} \}$.

3SAT

- 一个合取范式是3正则的，如果它的每个句子含有恰好3个文字.

$$(x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_4} \vee x_5)$$

- $3SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可适定的3正则合取范式} \}.$

SAT

练习.

- $(x \vee y) \wedge (x \vee \bar{y}) \wedge (\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y})$ 可适定?
- $\phi = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_2 \vee x_3 \vee \bar{x}_4)$ 属于 3SAT?

Cook-Levin定理

Cook-Levin定理. ***SAT***是NP-完全的.

证明思路. 需要证明NP类中的任何问题 A 可以多项式规约到 SAT .

- 给定输入 w , 在多项式时间内构造一个布尔公式 ϕ , 该公式模拟在判定 A 的非确定性图灵机 N 运行 w .
- 要求: N 接受 $w \iff \phi$ 是可适定的.
- 细节是魔鬼! (The devil is in the details!)

Cook-Levin定理

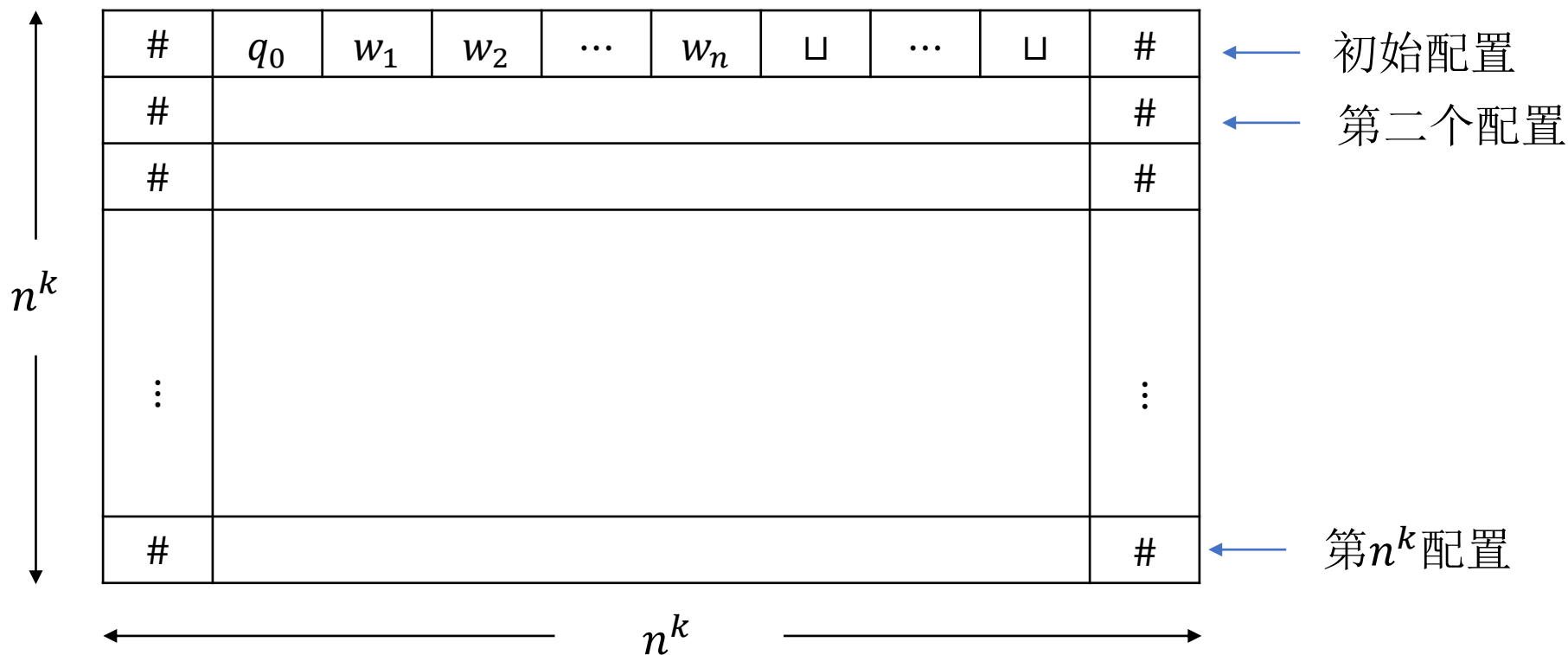
Cook-Levin定理. **SAT**是NP-完全的.

证明. 证 $\forall A \in \text{NP}, A \leq_p \text{SAT}$.

令 N 是判定 A 的非确定性图灵机, 其对于输入规模为 n 的输入, 在 n^k 时间内停机, 其中 k 是某个常数.

- 给定输入 w , N 关于 w 的计算表是指一个 $n^k \times n^k$ 的表格, 其每行都是某个计算分支的配置.
- 为了方便, 假定每行的第一个和最后一个字符都是#.

Cook-Levin定理



- 若某行是一个接受配置， 则称计算表是接受的。
- 存在 N 的计算分支接受 $w \iff$ 存在一个接受的计算表。

Cook-Levin定理

证明(续). 现构造布尔公式 ϕ .

- 对任何 $1 \leq i, j \leq n^k$ 以及 $s \in C \stackrel{\text{def}}{=} Q \cup \Gamma \cup \{\#\}$, 引入变量 $x_{i,j,s}$, 该变量的意思是

$$x_{i,j,s} = 1 \iff (i,j)\text{-单元中的字符是}s.$$

- $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.
 - ϕ_{cell} : 每个单元格出现且只出现一个字符.
 - ϕ_{start} : 计算表的第一行是初始配置.
 - ϕ_{move} : 计算表的后一行都是从前一行根据转移函数一步得到.
 - ϕ_{accept} : 计算表中包含接受配置.

Cook-Levin定理

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s, t \in C, s \neq t} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right]$$

(i, j) -单元至少出现一个字符

(i, j) -单元最多出现一个字符

Cook-Levin定理

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

$\phi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge$ ← 初始状态为 q_0

$x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \cdots \wedge x_{1,n+2,w_n}$ ← 输入为 w

$x_{1,n+3,\sqcup} \wedge \cdots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#}$ ← 用 $\sqcup$ 补齐

Cook-Levin定理

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}$$



计算表中出现接受状态

Cook-Levin定理

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

ϕ_{move} : 计算表的后一行都是从前一行根据转移函数一步得到的配置.

#	q_0	w_1	w_2	$\cdots$	w_n	$\sqcup$	$\cdots$	$\sqcup$	#						
#									#						
#									#						
$\vdots$	<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table> 窗口														$\vdots$
#									#						

Cook-Levin定理

称一个 2×3 的窗口是合法的, 如果它不违反 N 的转移函数所规定的行为.

例. $\delta(q_1, a) = \{(q_1, b, R)\}$, $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$.

a	q_1	b
q_2	a	c

a	q_1	b
a	a	q_2

a	a	q_1
a	a	b

$\#$	b	a
$\#$	b	a

a	b	a
a	b	q_2

b	b	b
c	b	b

合法窗口

Cook-Levin定理

称一个 2×3 的窗口是合法的, 如果它不违反 N 的转移函数所规定的行为.

例. $\delta(q_1, a) = \{(q_1, b, R)\}$, $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$.

a	b	a
a	a	a

a	q_1	b
q_1	a	a

b	q_1	b
q_2	b	q_2

不合法窗口

Cook-Levin定理

断言(局部合法推出全局合法). 如果计算表的第一行是初始配置且每个窗口都是合法的, 则计算表的每一行都是由上一行根据 N 的转移函数经过一个转移得到的.

证明. 每个窗口可由上方中间单元格所定义, 该单元格的字符不可能是#.

- 若该单元格本身不是状态符且两边的字符也不是状态符, 则该窗口下方中间单元格中的字符必然与该单元格的字符相同.
- 若某个窗口上方中包含状态符号, 则该窗口下方的3个字符必然根据 N 的转移函数进行更改. ■

Cook-Levin定理

#	a	a	q	b	c	#

Cook-Levin定理

#	a	a	q	b	c	#
#						#

Cook-Levin定理

#	<i>a</i>	<i>a</i>	<i>q</i>	<i>b</i>	<i>c</i>	#
#						#

Cook-Levin定理

#	<i>a</i>	<i>a</i>	<i>q</i>	<i>b</i>	<i>c</i>	#
#	<i>a</i>					#

Cook-Levin定理

#	a	a	q	b	c	#
#	a					#

Cook-Levin定理

#	<i>a</i>	<i>a</i>	<i>q</i>	<i>b</i>	<i>c</i>	#
#	<i>a</i>				<i>c</i>	#

Cook-Levin定理

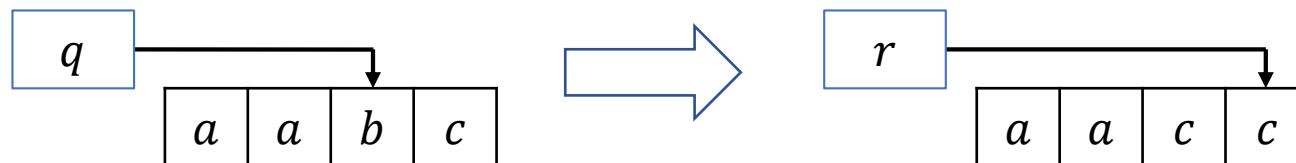
#	<i>a</i>	<i>a</i>	<i>q</i>	<i>b</i>	<i>c</i>	#
#	<i>a</i>				<i>c</i>	#

$$\delta(q, b) = (r, c, R)$$

Cook-Levin定理

#	<i>a</i>	<i>a</i>	<i>q</i>	<i>b</i>	<i>c</i>	#
#	<i>a</i>	<i>a</i>	<i>c</i>	<i>r</i>	<i>c</i>	#

$$\delta(q, b) = (r, c, R)$$



Cook-Levin定理

Cook-Levin定理. **SAT**是NP-完全的.

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

$$\phi_{\text{move}} = \bigwedge_{1 \leq i < n^k, 1 \leq j < n^k} (i, j) - \text{窗口是合法的}$$

其中 (i, j) - 窗口是合法的可以表示为

$$\bigvee_{\substack{a_1, \dots, a_6 \text{ 是} \\ \text{合法窗口}}} (x_{i, j-1, a_1} \wedge x_{i, j, a_2} \wedge x_{i, j+1, a_3} \wedge x_{i+1, j-1, a_4} \wedge x_{i+1, j, a_5} \wedge x_{i+1, j+1, a_6})$$

	$j-1$	j	$j+1$
i	a_1	a_2	a_3
$i+1$	a_4	a_5	a_6

Cook-Levin定理

Cook-Levin定理. **SAT**是NP-完全的.

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

- 已证明 N 接受 $w \iff \phi$ 是可适定的.
- 时间复杂度.

常数

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s, t \in C, s \neq t} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right]$$

n^{2k}

Cook-Levin定理

Cook-Levin定理. **SAT**是NP-完全的.

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

- 已证明 N 接受 $w \iff \phi$ 是可适定的.

- 时间复杂度.

- $\phi_{\text{cell}}: O(n^{2k})$.

n^k

$$\phi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge$$

$$x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \cdots \wedge x_{1,n+2,w_n}$$

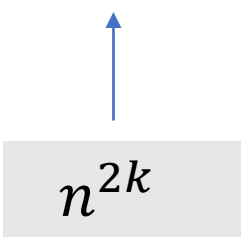
$$x_{1,n+3,\sqcup} \wedge \cdots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#}$$

Cook-Levin定理

Cook-Levin定理. **SAT**是NP-完全的.

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

- 已证明 N 接受 $w \iff \phi$ 是可适定的.
- 时间复杂度.
 - $\phi_{\text{cell}}: O(n^{2k})$.
 - $\phi_{\text{start}}: O(n^k)$.

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}$$


A diagram illustrating the complexity of the ϕ_{accept} formula. A gray rectangular box at the bottom contains the expression n^{2k} . A blue arrow points vertically upwards from this box to the summation index $1 \leq i, j \leq n^k$ in the equation above.

Cook-Levin定理

Cook-Levin定理. **SAT**是NP-完全的.

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

- 已证明 N 接受 $w \iff \phi$ 是可适定的.

- 时间复杂度.

- $\phi_{\text{cell}}: O(n^{2k})$.

- $\phi_{\text{start}}: O(n^k)$.

- $\phi_{\text{accept}}: O(n^{2k})$.

$$\phi_{\text{move}} = \bigwedge_{1 \leq i < n^k, 1 < j < n^k} \text{常数} \quad (i, j) - \text{窗口是合法的}$$

$\uparrow$
 n^{2k}

Cook-Levin定理

Cook-Levin定理. **SAT**是NP-完全的.

证明(续). $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$.

- 已证明 N 接受 $w \iff \phi$ 是可适定的.
- 时间复杂度.
 - $\phi_{\text{cell}}: O(n^{2k})$.
 - $\phi_{\text{start}}: O(n^k)$.
 - $\phi_{\text{accept}}: O(n^{2k})$.
 - $\phi_{\text{move}}: O(n^{2k})$.



多项式时间规约

定理得证.



Cook-Levin定理

思考. 如何将证明中的 ϕ 替换成一个等价的多项式大小的CNF?

推论. ***CNF – SAT***是NP-完全的.

证明. 利用De Morgan's Law

$$(x \wedge y) \vee z = (x \vee z) \wedge (y \vee z)$$

可将 ϕ_{move} 转换成一个CNF. ■

NP-完全问题

思考. 若选择 2×2 的窗口, 则断言是不正确的(给出反例).